THE OURONET AUDIT BOOK

Version 3.1 · built 2026-11-02 · commit d5a00faa+dirty · generated by REPL/tools/_auditbook.py

A single consolidated account of every audit performed on Ouronet: the per-module rounds, the main-work round, and the red team — what was looked for, what was found, what was done about it, and what proves the fix is still there.

This file is GENERATED. Edit the chapter sources under AUDIT-BOOK/, then rebuild with python3 REPL/tools/_auditbook.py --docx. Editing this file directly will be overwritten, and --check fails the gate if it drifts from its sources.

Changelog

Version 3.1 — 2026-11-01

- NEW MODULE AUDITED: AQP-EVCT – reward eviction, 08b_EVCT.pact, written 2026-10-29. Lets an FVT owner collect a reward ON BEHALF OF a beneficiary who has not signed, running the SAME XI_CollectDrip/XI_CollectPay phase functions a self-collect runs rather than a copy, so the beneficiary receives exactly what they would have collected themselves. Already wired into TS02-C3 Talos (gasless admin twins included) and Deploy/PureV18.
- FINDING, not fixed: AQP-EVCT's own three capability bodies (EVCT|C>EVICT-MULTI/-SINGLE/-FULL-EVICT) prove the TX-SIGNER owns the FVT, but do not themselves bind the evictor parameter to that owner – read in isolation that looks like a decorative executor. The binding exists one hop downstream, inside 05_FVT.pact's FVT|XE>EVICT-REWARD capability (UEV_ExecutorIzFvtOwner), reached because every custody-moving leg forwards through it and a Pact transaction is atomic – so it is functionally safe. But no @doc in AQP-EVCT names that route, and no red-team case signs as the real owner while naming a DIFFERENT, wrong-but-owned account as evictor to prove the binding actually refuses. Recorded as the adversarial test still to write.
- CITIZEN BOOTSTRAP EXTENDED: AQP-BOOT (2_CITIZEN/5_VaultsMinter) gained four late one-shot steps (7b/8b/9b/12b, dated 2026-10-08) finishing three entities – NosferatuDracula, WonderCoach, StoicPower – the original Step 7/8/9/12 ladder never reached, so their pools stayed employed but unstakeable. Citizen modules sit outside the patron/executor canon by owner ruling (2026-09-26); the four new steps take a single patron, matching all fourteen existing siblings in the file.
- SCOPE: a discovered, UNAUDITED drift in 04_TS02-C3.pact – its sweep-completion heading from 2026-09-22 satisfies the audit-delta guard's filename match while covering none of the 40 signature changes and 21 new entrypoints added to that file since (an AQP-POOL vacate-finalization family, stake-beneficiary reassignment, score revocation, among others). Recorded here rather than left for the guard's leniency to hide silently; not yet audited, queued as the next pass.

Version 3.0 — 2026-09-22

- THE v2 AUDIT RE-ISSUE. This edition is the audit re-verified after the patron/executor/executee canon sweep re-signed the client surface of every sovereign module. 694 entrypoint signatures changed across 58 files, so EVERY adversarial call site in the previous edition's attack register has MOVED. An attack that still passes without being re-pointed is passing on an ARITY ERROR rather than on the guard it names – a short modref call partially applies into a closure and the expect-failure around it goes green for the wrong reason. That happened six times during the round, twice inside expect-failures that had looked green for weeks. Part V is the new material.
- SWEEP COMPLETE: 776 of 776 A_/C_ entrypoints take patron first, executor second and executee third where one exists, across 46 modules, and EVERY executor is proven – directly, by a forward, or by a route the function's own @doc names. Position is canon, not merely presence, because Pact arguments are

positional: correcting a signature rewrites every call site that reaches it, and three single turns rewrote 172, 86 and 31 of them.

- THE PLAN WAS WRONG BY 63% ON ITS FIRST DAY. The refactor was scoped against a tool whose entrypoint filter was anchored wrongly and could not match a name containing a bar – so every Talos entrypoint, all 482 of them, was invisible. It reported 302 done / 474 remaining against an actual 776, and Talos is the only supported client path in this system. Found by a SECOND tool written to re-derive the same worklist by another route.
- SECURITY: ANK|C>REVOKE-BOOST-CLASS validated only that the BoostClass was empty and active – it checked NO account. Any account reachable through Talos could revoke any empty BoostClass that was not theirs. The owner field and its enforcement both already existed, added earlier on the ATTACH path; the REVOKE path was never carried over. A partially-applied fix is the most convincing kind of absence.
- DEFECT (attribution): four 00_Demipad admin operations – register asset, define price, toggle open-for-business, toggle retrieval – ran under an admin keyset and took NO account argument, so the events they emitted named nobody. Measured with the new guard disabled, A_DefinePrice SUCCEEDED and returned a price-updated message: an administrator could change a launchpad price and have the event name somebody else. An admin keyset says MAY, never WHO. Pinned by a rollback-tx, because the subject mutates when the guard is absent.
- DEFECT (published price): two repurpose entrypoints each bound a whole duplicated cost model and read none of it. The price sheet derives a price from the tier legs a body MENTIONS, and the dead block mentioned BOTH tiers where the live code charges one – so the sheet quoted a floor of 5 for an operation whose floor is 2 or 3. Dead code is usually harmless because nothing reads it; a generator reads everything.
- DEFECT (published price): a price CLASS rested on a hyphenated word in a comment. The sheet's 'does this scale with input size' test matched the literal phrase per-nonce and runs over text including @doc prose – so two functions with an identical price shape were classified differently because one doc says 'per-nonce' and the other says 'per-fragment'. Replaced with structural patterns; tally moved 185 exact / 135 floor to 183 / 137.
- DEFECT (published price): DOCUMENTING A FUNCTION RAISED ITS PRICE. The deterrence worksheet counts cost legs from source text and counted them inside ;; comments and @doc strings – 24 phantom cross-module hops in @doc plus 25 in comments, inflating 29 published rows. Found because eight functions gained an @doc explaining how their executor is proven. The rule now written into the generator: a cost leg must be read from a FORM, and a form only exists in code.
- INSTRUMENT: _executorenforced.py is now GATE-FATAL over the whole tree (766 proven, 0 UNPROVEN). It had been run with a –swept filter derived from the worklist's ticks, and the tick parser required the index column to be a NUMBER – one module carries an em-dash there, because its signatures arrived by cascade rather than by a turn of its own. So the filter judged 45 modules while the worklist said 46 and reported a confident 0 UNPROVEN while eight executors sat unproven for two days. All eight were genuinely proven once traced; the finding is that a verified zero was being reported over a set that silently excluded a module.
- MEASUREMENT: the authorisation surface HEADLINE went the wrong way and the round was still a net gain. Entrypoints reaching no ownership enforce rose 291 -> 337, because a separate piece of work added the same admin pair (P|A_SetIMP / P|A_RemoveIMP) to 58 modules and those are policy functions, canon-exempt and account-ownership-free by construction. Diffed per entrypoint instead: of the 1,075 present in both measurements, 60 GAINED an ownership gate and 0 lost one. A total is a summary of two populations, and a summary cannot tell you that one of them changed.
- ARCHITECTURE: six entrypoints were deliberately given NO executor, with the four-question test that settles it recorded beside them – is any account on the path checked, is the op idempotent truth-restoration, who pays, and would a signature requirement remove a legitimate path. The two different rationales (permissionless repair vs. authority-is-a-guard) are kept visibly separate rather than blurred into one register.

Version 2.0 — 2026-09-20

- PATRON/EXECUTOR SEPARATION, the whole AQP family. 52 of the 89 patron-taking C_/A_ entrypoints now name the account that ACTS separately from the account that PAYS. Before this, an entrypoint took a patron and the actor was either that same account or a value derived inside a capability and never named – so 'who is doing this?' had no answer at the call site, and several functions answered it WRONGLY. Every edit keeps the existing derived ownership gate and adds the executor check BESIDE it; the derived check was never replaced, which is the single edit that would have turned this into an authorisation hole.
- SECURITY: the two DPNF anchor paths (ANK|C>ISSUE-DPNF, -DPNF-SET) enforced NO ownership at all, while their DPTF sibling ran CAP_TF|Owner and their DPSF sibling ran CAP_OwnerOrCreator. UEV_AssetAnchorCap caps an asset at 49 anchors, so a stranger could mint 49 anchors against a collection he did not own, exhaust the cap PERMANENTLY, and own every boost class created along the way. It was being exercised: closing the gate immediately failed two fixtures and the BOOT itself, all anchoring collections owned by KST.LUMY while signing only as the admin.
- SECURITY: C_IssueSingleScoreModel and C_CombineTripletScoreModel enforced only 'the patron account exists' – nothing asserted the caller owned it. Not exploitable, because the Talos wrappers do reach a patron ownership enforce and P|UEV_IMC makes Talos the only reachable path; the hole was one layer of defence deep, not zero. Closed anyway, and closing it is what now lets a sponsor pay for someone else's model. C_IssueMultipletFamily had the same shape and the same fix.
- DEFECT (state-corrupting): C_AdmitAgency passed patron as the OPERATOR in four places, one of them WRITTEN TO STATE as DSA|Agency.operator-konto. A sponsored agency open would have recorded the sponsor – or, under the gasless patron, DALOS|SC_NAME – as the agency's operator permanently. Every other defect in this release is recoverable by calling again with the right account; this one wrote the wrong owner into a row.
- DEFECT: the boost-class owner added in 1.1 was being filled with the PATRON at all four inline-creation sites, so under a gasless patron every inline boost class would have been owned by DALOS|SC_NAME rather than its creator – silently, since nothing reads it back at issuance.
- DEFECT: six DSA entrypoints ran (enforce (= patron fvt-owner)), forcing the GAS PAYER to BE the FVT owner. No sponsor could pay for a vault owner's operation and the gasless patron was unusable across that entire surface.
- DEFECT: MTX|2|C_Inject hardcoded XB_FvtInject patron patron, so the documented spike fallback for the Stage Two daily emission could only run when the gas payer was also the token source – which is precisely the shape it exists to rescue. The minter's own @doc asserted the fallback was available; that assertion was false.
- INSTRUMENT: ARCHITECTURE/AUTH-SURFACE.md records, per entrypoint, every ownership enforce reachable transitively through its capability chain – 1,104 entrypoints, 813 enforcing. _authsurface.py – check is FATAL in the gate and fails if any entrypoint's set shrinks. As far as we can tell this is the first time the system's authorisation surface has been written down. It reported 'none weakened, N strengthened' on every commit in this release, which is how the closed holes were counted rather than claimed.
- The Stage Two daily emission (AA_OuroMinterStageTwo) was measured for the first time: ~940,000 gas, roughly HALF a 2M block, against Stage One's ~135,000. The figure is a FLOOR, not a ceiling – four of its six legs scale with staker count. Pinned as a band rather than a number, because a regression that silently skips legs makes the emission CHEAPER and without a lower bound that reads as an improvement.

Version 1.1 — 2026-09-19

- AQP schemas hoisted into ONE interface, AcquisitionSchemasV1: 74 declarations -> 63 (11 exact duplicates collapsed, 0 conflicts). A schema living in a module cannot appear in that module's interface, so 106 functions returning one were unreachable through a modref; that count is now 0. It is also what CAUSED the duplication – RPS declared none of its 10 UDC_ constructors, so FVT copied the shapes.

- RPS/FVT deduplicated: 20 byte-identical function bodies removed or turned into the documented #75 facades. 05_FVT.pact 3,978 -> 3,495 lines.
- ALL AQP interfaces are V1, plus the three never-deployed Talos interfaces (TalosStageTwo_ClientThree/DemiPad, CitizenLaunchpadTalos). The 2026-09-18 pass had bumped six AQP interfaces V2->V3 as 'live + 1' when none of them is live.
- DEFECT: the FVT admission rule was INVERTED – vault admitted SF/NF and treasury admitted OF. Corrected to vault=TF/OF, treasury=SF/NF. AQP-BOOT Step 8 was issuing four Treasury-named entities at class 1, which only worked BECAUSE the rule was inverted.
- DEFECT: the pure Bloodshed score was employed by a pool but admitted to no FVT, so every stake on DHBloodshed aborted. Fixed by adding the fifth treasury it never had.
- SECURITY: ANK|BoostClass had no owner field at all. Attaching an anchor to an EXISTING class validated only 'active' and 'a slot is free', so anyone holding any anchorable asset could attach it to another vault's class and grant their holders a boost inside that vault's scoring. Added class-owner + enforcement.
- ARCHITECTURE: a score's boost class is now carried by the score MODEL and applied at issue, so a delegation vault's admin defines how scores behave and an agency cannot decline or redirect it. Previously it was settable only after issue, by the score's owner – the agency operator.
- Deploy/ is now gate-enforced byte-for-byte against the sovereign sources (_deploybundle.py –check). It was the ONE generated artefact the gate did not diff.
- FINDING, not fixed: the triplet quality-split LANES read the SILVER score's base only, so a heterogeneous split routes every staker on one row. Recorded in the Custodians design notes.
- FINDING, not fixed: royalty disposal is not isolated per vault – an inject into one vault funded another vault's disposal, discovered when it flipped a red-team expect-failure into a success.

Version 1.0 — 2026-09-18

- First consolidated edition: per-module rounds, the main-work round, and the red team.

Contents

#	Chapter	Page
1	How to read this book	6
2	The system under audit	8
3	How Ouronet is tested	18
4	Part I — The module audits	24
5	DALOS — accounts, guards, governance	28
6	ATS — autostake pools	49
7	SWP — the automated market maker	67
8	DPDC — collectables	92
9	DEMIPAD — the launchpad	108
10	AQP — acquisition pools	120
11	Part II — The main-work round	141
12	The cost-reader and preview surfaces	147
13	Re-pricing IGNIS	155
14	Module splits under the deploy ceiling	163
15	The test architecture	169
16	Part III — The red team	182
17	How the round was designed	186
18	Method: building an attack that proves something	190

#	Chapter	Page
19	The ownership-gate programme	193
20	The defects, and what was done	198
21	The full attack register	202
22	Part IV — The instruments	225
23	Part V — The canon sweep	230
24	The canon, and the four shapes a proof can take	233
25	What the sweep found	236
26	Method: how the round was run	240
27	Appendix A — Reproducing everything	244
28	Appendix B — Verification state	247

1. How to read this book

This is a single consolidated record of every audit performed on Ouronet: four rounds of work, carried out over several months, against a virtual blockchain of ninety-three Pact modules.

It is written to be read by three different people, and it is worth saying at the outset which one you are.

If you are evaluating the system. Read Chapter 2 for what Ouronet is and Chapter 3 for how it was verified, then Appendix B (Chapter 28) for the verification state — what is proven, what is bounded, and what is explicitly not claimed. Then read Chapter 20, which is the defect record. Those three chapters are the answer to "should I trust this", including the parts of the answer that are "not entirely".

If you are auditing it yourself, or auditing this audit. Chapter 18 is the methodological core: it is about the five ways a security test can look like a proof and be worthless, all five of which this programme committed before it learned to detect them. Chapter 27 tells you how to re-run everything here from a clean checkout. Chapter 21 is the full attack register, generated from the attacks themselves.

If you are going to work on this codebase. Read Chapter 2, then Part I, which is the per-module audit and doubles as the most honest description of how each subsystem actually behaves. Chapter 22 is about the instruments — and about the fifteen defects found *in the instruments*, which is the chapter most likely to change how you think about your own tooling.

Whichever you are, read Chapter 25 if you have read an earlier edition of this book. The canon sweep changed **694 entrypoint signatures across 58 files**, so every adversarial call site in the attack register has moved, and an attack that still passes without being re-pointed is passing on an arity error rather than on the guard it names.

What this book claims, and in what voice

Three commitments shape every chapter, and they are worth stating because they are unusual.

Findings are reported with their measurements, not their suspicions. Where a defect was found, the exploit was executed and its effect recorded *before* the fix was written. This is why figures in this book are specific — 2,919.77 IGNIS committed before a gate refused, a 10.5× discrepancy between two doors to the same operation, a 38.65% mispricing. Three findings in this programme would have been published with fabricated magnitudes without that rule, and one would have been published as a defect that did not exist.

The programme's own errors are in the book. Not in an appendix of lessons learned — in the chapters, at the point where they happened. A measurement that was wrong four times says so and explains each wrong version. A figure published without being measured says so. An instrument that reported a tidier number than the truth says which direction it was tidier in. This is not self-flagellation; it is the only way a reader can calibrate how much to trust the figures that were *not* found to be wrong. An audit document with no errors in it is either extraordinary or incomplete, and the reader has no way to tell which.

Negative results are labelled as negative results. An attack that was refused is reported as refused — which is weaker evidence than a fix, not stronger. A gate that has never been observed to turn anybody away is reported as never observed, not as "presumed working". A bound is presented as a bound. Where the honest answer is "nothing is known about this from the outside", the book says that.

How to read a figure in this book

Every quantitative claim here is re-derivable, and most are generated rather than typed. The headline tables are checked by `REPL/tools/_booktables.py`, which fails the build if a table stops summing to its own total. The attack register in Chapter 21 is emitted directly from the attack files. This document itself is assembled by `REPL/tools/_auditbook.py` and a `--check` run fails the build if the assembled file drifts from its chapter sources.

That machinery exists because of a specific, repeated failure: **three times in this programme, a published figure went stale while every document agreed with every other document.** Cross-checking documents against each other detects nothing, because they were all copied from the same original. Only re-derivation from the tree detects it.

So: where a number in this book looks important, Chapter 27 tells you how to reproduce it. If it does not reproduce, the book is wrong and the tree is right.

Structure

- **Chapter 2–Chapter 3** — orientation: what the system is, and the test plant that verified it. Every later chapter cites these.
- **Part I** (Chapters 4–10) — the per-module audits. Six subsystems, read line by line.
- **Part II** (Chapters 11–15) — the main-work round: the cost-preview surface, the IGNIS re-pricing, the module-size question, and the architecture of the test suite itself.
- **Part III** (Chapters 16–21) — the red team. How the round was designed, the method, the ownership-gate programme, the defects, and the complete register.
- **Part IV** (Chapter 22) — the instruments, and what was wrong with them.
- **Part V** (Chapters 23–26) — the patron/executor/executee canon sweep: the round that re-signed the client surface of every module so the system can say WHO performed an operation, and the three live defects that asking the question exposed.
- **Appendices** (Chapters 27–28) — reproduction, and the verification state.

A reader going front to back will find Part I heavy. Part I is a reference; Parts II and III are arguments. Reading Part III first and returning to Part I for the subsystem detail is a reasonable route, and the cross-references are written to support it.

---

2. The system under audit

What Ouronet is

Ouronet is a virtual blockchain written entirely in Pact, Kadena's smart-contract language, and deployed on StoaChain — a Chainweb-class stack with its own native coin and namespace economy — under the namespace ouronet-ns. Earlier testing lived under Kadena's free namespace; the whole tree was migrated.

"Virtual blockchain" is the load-bearing phrase and it is not decoration. Ouronet is not a dapp that holds balances in a table. It re-implements, inside Pact, the things a chain normally provides to the contracts running on it, and then provides those things to its own contracts:

chain concern	what StoaChain supplies	what Ouronet supplies for itself
accounts	k: principal accounts in the root coin module	DALOS|AccountTable — an Ouronet account with a sovereign, a governor guard, a smart-contract flag and payability rules
native balance	coin	DALOS|StoaLedger, the per-account mirror of native STOA
gas	real gas, metered by the node, paid in native coin	**IGNIS**, a virtual gas unit pegged at 1 IGNIS = 1 US/EUR cent
where gas is held	the payer's coin account	the ignis field **on the Ouronet account row itself** — IGNIS is not a separate fungible
who pays real gas	whoever signs	the Ouronet gas station: DALOS's GAS_PAYER capability, allowance const DALOS|GAS-BUDGET = 2,000,000 gas
token standards	fungible-v2	DPTF, DPOF, DPDC (see next section)
transaction entry	any module a signer names	**Talos only** (see *The layer cake*)

Read the last two rows together and the shape of the system falls out. A transaction that takes a Talos path and is accepted by the gas station has its **real** gas paid by Ouronet, and in exchange Ouronet charges its own **virtual** gas, IGNIS, at rates it sets itself. Real gas and virtual gas are two separate meters running at once, and essentially all of the economics — and most of the audit surface — live on the second one.

Two precisions, because this is the kind of summary that invites overstatement. First, "the user spends no StoaChain gas" holds only for sponsored paths: the gas station pays **only** for paths defined in Talos, and its GAS_PAYER capability is a whitelist over the transaction's top-level forms, not a blanket subsidy. A transaction it does not match is paid for by whoever signed it. Second, DALOS|GAS-BUDGET is a **per-transaction allowance**, not a pot that drains: the gate is gas-price × gas-limit ≤ GAS-BUDGET × ceiling, enforced against the protocol minimum read live from coin on every call, so the allowance does not decay as the network's minimum price rises.

That allowance is 2,000,000 gas, which is also StoaChain's **block** gas limit — so a sponsored transaction may in principle consume a whole block. The two numbers are stated separately here because no source states that they were set equal deliberately, and treating a coincidence as a design intent is how a reader ends up reasoning from a rule that does not exist.

The system deploys in stages, because it has to (see *Interfaces and the cascade rule*):

- **Stage 1** — identity and gas (DALOS, IGNIS), fungibles (DPTF, DPOF, TFT), autostake (ATS, ATSU), the swapper family (SWP, SWPI, SWPL, SWPLC, SWPU, SWPT, MTX-SWP), VST, ELITE, BRD, CODEX, PYTHIA, OUROBOROS, LIQUID.

- **Stage 2** — collectables (the DPDC family), the DEMIPAD launchpad core, and acquisition pools (AQP-POOL, AQP-ANK, AQP-SCORE, RPS, AQP-FVT, AQP-VCT, MTX-AQP, AQP-DSA).
- **Stage Z** — read-only and deployer modules (DPL-UR, EXPLORER, DSP), last, as they read everything.

Development is REPL-first: nothing is deployed to prove it works, it is loaded into staged .repl harnesses first, and those harnesses are the largest single artefact in the repository. Two version records exist and deliberately disagree — LIVE-INTERFACE-VERSIONS.md records what is **deployed on chain**, the tree records what is **developed**, and the first is never refreshed from the second because being the thing the second is compared against is its whole purpose.

The asset taxonomy

Ouronet defines four asset families plus one legacy one. The distinctions are not marketing categories: each family is a different set of tables with a different row key, and the row key tells you what "one unit" of the thing is.

family	module(s)	the table that defines a unit	row key	one unit is
true fungible (DPTF)	DPTF	DPTF|BalanceTable	<id> + <account>	a decimal balance
ortofungible (DPOF)	DPOF	DPOF|T|Nonces	<id> + <nonce>	a nonce: one holder, a supply, an immutable metadata chain
semi-fungible (DPSF)	DPDC family	DPSF|T|Nonces + DPSF|T|AccountSupplies	<id>+<nonce>, and <account>+<id>+<nonce>	an edition, held by many accounts in parts
non-fungible (DPNF)	DPDC family	DPNF|T|Nonces + DPNF|T|AccountSupplies	same shape	a unique item
meta-fungible (DPMF)	DPMF	—	—	legacy; deployed for provenance, never called

Collectables is the umbrella term for semi- plus non-fungible, and DPDC is where they live — one module owning both the DPSF and DPNF table sets, with the rest of the family (DPDC-C, DPDC-I, DPDC-R, DPDC-MNG, DPDC-T, DPDC-S, DPDC-F, DPDC-N, DPDC-UDC, EQUITY) routing every write through it.

Ortofungible is the term a new reader will not recognise, and it is worth being exact. It is a fungible token whose supply is partitioned into *nonces* — numbered batches — each carrying its own metadata. The schema says it plainly:

```
(defschema DPOF|NonceElement
  holder:string               ;;Stores the <OuronetAccount> holding the nonce - mutable
  id:string                   ;;ID of the Ortofungible - immutable.
  value:integer               ;;Stores the Nonce value itself - immutable.
  supply:decimal              ;;Nonce Supply - mutable
  meta-data-chain:[object]    ;;Stores Nonce Metadata - immutable
)
```

A nonce has exactly one holder and a divisible supply, so an ortofungible sits between the two familiar extremes: unlike a true fungible, units in different nonces are not interchangeable, because the nonces carry different metadata; unlike an NFT, a nonce is not one indivisible item, it has an amount. The name changed from "MetaFungible" to keep the live path (DPOF) distinct from the legacy migration semantics (DPMF).

The difference between DPOF and DPDC is visible in the table above: DPOF has no AccountSupplies table, because a nonce is **single-held**; DPDC has one, keyed by account *and* nonce, because an edition is held in parts by many accounts. That one table is the whole ortofungible/collectable distinction in structural form.

On top of the assets sit the DeFi primitives, built out of these families rather than beside them: **ATS** autostake pairs and **SWP** liquidity pools (both from true fungibles, with ortofungibles able to interact with SWP), **AQP** acquisition pools (earning pools with anchors, scores and reward-per-share accounting), **VST** vesting, and **DEMIPAD**, the launchpad whose per-asset sales were deliberately pushed out into citizen modules.

Sovereign and citizen

Every module in the tree is one of two classes, and the tree says which by path. **Sovereign** (1_SOVEREIGN/) modules are the canonical Ouronet architecture, maintained by the project. They hold the business logic, own the tables, and define the capability gates. Talos — the orchestration and gas boundary — is sovereign.

Citizen (2_CITIZEN/) modules are extensions that anyone may write. They call into sovereign public APIs and nothing else. They do not add capabilities to the core surface, and their client functions carry no UEV_IMC — the sovereign inter-module gate, which belongs to Talos and core paths. The citizen modules in the tree (AOZ, the Bloodshed/Nosferatu/Bunnies/Vaults minters, the CADUCEUS bridge, the five launchpad sales, DPL-UR, EXPLORER, DSP+) are Admin-authored but structurally citizen — the standing proof that the boundary is usable from outside.

What the boundary buys an auditor is a decidable question. “Can this module write to that table?” has a mechanical answer: only through a sovereign entrypoint, and only if that entrypoint’s gate lets it. That gate is the policy layer — every module opens with two policy tables, P|T and P|MT, before any business logic, holding the guards through which modules authorise each other. The inter-module check is P|UEV_IMC, which enforces that any guard returned by P|UR_IMP is satisfied; P|UR_IMP defaults to the module’s own SECURE capability guard when nothing has been registered, so an unregistered policy list is satisfiable only from inside the module and the gate answers identically before and after first registration. That default was added in September 2026, after the bare read was found to raise a raw table error rather than refuse cleanly.

Two documented places bend the sovereign/citizen line, and both matter later in this book. **There is a citizen Talos**: 2_CITIZEN/7_Launchpad/99_TS02-CPAD.pact carries the per-sale user wrappers for the five launchpad sales and is the sole gas-funded path for them, while its sovereign counterpart 1_SOVEREIGN/STAGE_02/3_Talos/05_TS02-DPAD.pact carries the DEMIPAD|* ops — one letter apart, and the project’s own documentation had the roles swapped until 2026-09-17. And **the citizen minters invert the client direction**; see the end of the next section.

The layer cake

Sovereign code is three layers: **Utilities → Core → Talos**.

Utilities exist only in Stage 1 — thirteen modules (U|CT, U|G, U|ST, U|RS, U|LST, U|INT, U|DEC, U|DALOS, U|ATS, U|DPTF, U|VST, U|SWP, U|BFS) of small pure helpers. They exist because of Kadena’s roughly 150k deploy-size cap: core modules could not hold their own UC_* compute inline and stay under it. Two kinds live there — genuinely generic compute (U|LST strings, U|INT integers, U|G guards, U|BFS search) and module-family-tied compute centralised once per family because splitting it further would blow the budget (U|SWP, shared across the six SWP modules). **Stage 2 dropped the pattern entirely**: it deploys under roughly a 2,000,000 gas ceiling, so every Stage 2 module keeps its UC_* inline. There is no U|DPDC and no U|AQP, and proposing one would reintroduce a pattern the project deliberately left behind.

Core is the business logic — DALOS, IGNIS, DPTF, DPOF, ATS, VST, the SWP family, DPDC, DEMIPAD, the AQP family. Each starts with its policy tables, then schemas, tables and constants, then capabilities, then functions.

Talos is orchestration, and it is the part an auditor should internalise first. Talos modules sequence A_ and C_ calls across core modules into curated flows. Three consequences follow, and they are the reason Talos is a choke point rather than a convenience layer:

- **Talos is the only supported client path.** A core C_ is blocked from being invoked inside its own module by design; clients reach it through Talos.
- **The gas station pays execution only for paths defined in Talos.** Ad-hoc A_ calls straight into a core module remain possible for maintenance, but they fall outside the gas-station envelope.
- **Talos is the only place that collects IGNIS after a C_.** Because the core cannot call its own C_, a Talos sequence can force "execute C_, then collect" and a client cannot skip paying while still using the blessed path.

The operative rule for anyone adding code: a new A_, C_ or protected X* on a core module is not finished until it is wired into the appropriate Talos module. Until then it has no client semantics and no gas semantics.

For an auditor this inverts the usual problem. The client-reachable attack surface is not "every public function in 88 modules" — it is the Talos entrypoint list, which is enumerable. Eleven files carry it (six Stage 1 sovereign, four Stage 2 sovereign, one citizen) and define **413** client and **73** admin entrypoints between them. Conversely, a surface not in Talos is not reachable by a client, and a finding about such a surface needs to say how a client would get to it.

Every Talos client op passes the same door first: the P|TS capability, which reads DALOS::UR_GAP and refuses while the global administrative pause is online — *"While Global Administrative Pause is online, no client Functions can be executed"* — and then composes P|TALOS-SUMMONER.

The documented exception. In the citizen minters NOSFERATU and KBunnies the direction is inverted: their C_Spawn / C_Fix call *into* Talos and return the wrapper's string, after Talos has already collected. So A_Step01 → C_Spawn is a citizen module calling its own C_, which the sovereign rule forbids — and it is sound here precisely because there is no cumulator at that level to mishandle. The conformance tooling splits these out as self-C-call-citizen observations bounded to those two files. The bound is the point: a citizen C_ that *does* return an OutputCumulator is an ordinary billing shape A and the rule applies to it normally.

IGNIS: virtual gas on top of real gas

IGNIS is the virtual-chain gas collector, its unit pegged hard at **1 IGNIS** = **1 US/EUR cent**, so a 5000 deterrence is $50. Balances are not a fungible — they are the ignis field on the DALOS account row, so charging gas is an account-table update.

The OutputCumulator

The object that carries a charge through an operation is the OutputCumulator, defined in IGNIS:

```
(defschema OutputCumulator
  cumulator-chain:[object{ModularCumulator}]
  output:list
)
(defschema ModularCumulator
  ignis:decimal
  interactor:string
)
```

It is a **list of legs**, each naming an amount and the interactor it is attributable to. A core C_ builds one as it works, concatenating the legs its XI_/XE_/XB_ writers produce, and returns it. Talos hands the finished object to the collector with the transaction's patron:

```
(ref-IGNIS::C_Collect patron
  (ref-TFT::C_Transfer id sender receiver transfer-amount method)
)
```

C_Collect compresses the chain per interactor, primes it against the patron, and debits. The patron is the account that pays; a smart account may not be one, with a single hard-coded exception for the admin gasless path (the Ouroboros daily minter) — a restriction that was only a caller-side convention until a red-team finding in September 2026 made it an enforcement.

This is what the "C_ must not be invoked inside its own module" rule protects. A sovereign C_ builds a cumulator that only Talos may collect. A self-call can drop it or double it. For sovereign modules the count of such calls must stay at zero.

The cost model

```
IGNIS charged = deter (IG|DETER) + the op's own compute (IG|COMPONENTS)    ;; UC_IgnisPrice
STOA  charged = dollars(deter) / stoa_price          — ISSUE ops only      ;; UC_StoaPrice
```

Four constant maps in 1_SOVEREIGN/STAGE_01/2_Core/02_IGNIS.pact hold every price: IG|DETER (54 per-op deterrence keys — the business decision), IG|COMPONENTS (~394 generated per-op compute costs, calibrated against measured gas), IG|WEIGHTS (14 primitives the components are computed from), IG|LEGS (22 per-write and per-item unit costs charged inside the writers).

Four properties of that model are easy to get wrong and are worth stating flatly:

- **Deterrence is additive, not the total.** "A $50 function" means its deterrence is 5000; it still pays its ordinary compute on top. usage = 1 is the neutral element — an op with no deterrence still pays its full computation.
- **Every price is denominated in dollars** and converted to STOA at the oracle price. No STOA quantity is hard-coded. The one deliberate exception is CODEX::UC_StoicTagStoaFee — 1 STOA per glyph, fixed in STOA units, non-discountable, by owner ruling.
- **A tier value that is multiplied by a count is a unit, not a price.** Migrating such a site to deter + components replaces a unit with a whole-op price and then multiplies it by the item count — a compounding error far worse than the flat tier it replaced.
- **Quoted prices are full prices.** Discounts apply on top (IGNIS up to 49% via Elite tiers, STOA up to 24.5%); PYTHIA tolls, StoicTag registration and some asymmetric-liquidity legs are flagged non-discountable in code.

Every cost-emitting op has a URCi_* reader: pure, read-only, no enforce, returning the cost cumulator. It is called **inside the execution path for billing and served to the UI for preview**, so the two cannot drift. A *leaf* URCi prices one writer; a *composer* URCi concatenates leaves into the op's total, and the INFO_ preview calls the composer. A URCi lives in the same module as the function it prices — if a module gets too big the module is split, not its cost functions. The tree holds **595** URCi_ and **623** INFO_ definitions.

The six billing shapes

The description above — core C_ returns the cumulator, Talos collects — is the common case but not the only correct one. A full trace of all 38 non-cumulator C_s on 2026-09-13 found six legitimate shapes:

shape	where the billing happens	example
A	core C_ returns the cumulator; Talos passes it to IGNIS::C_Collect	most ops
B	Talos wrapper builds the cumulator from a URCi_* and collects	DALOS::C_RotateGuard → TS01-C1
C	STOA-priced — wrapper calls STOA|C_Collect*, no IGNIS at all	DALOS::C_DeploySmartAccount
D	billed **in the core** — the C_ itself ends on STOA|C_CollectWT	SWPLC::C_UpgradeBrandingLPs
E	**defpact step** — the C_ is only a starter; a later step bills	the 8 MTX-SWP pool/liquidity ops
F	**nested Talos** — the core calls another Talos client that collects	DEMIPAD::C_Transmit* → DPTF|C_Transfer

Plus the **primitives** — the IGNIS collectors, XB_MoveDalosFuel / XB_Collect* / XE_Collect* (until 2026-09-20 C_TransferDalosFuel and the STOA|C_Collect* family) — which *are* the collectors and cannot collect from themselves. The practical rule that falls out, and which cost two rounds of wrong conclusions before it was written down: **to decide whether an op charges, follow the Talos wrapper's IGNIS::C_Collect argument, never the core C_'s return type.** A core C_ returning a string says nothing.

One deliberately free op exists. TS01-C4::PYTHIA|C_Link takes no patron and collects nothing while its three siblings all charge. It is safe because it is bounded, not because it is cheap: linking requires two deployed Apollo halves at 500 native STOA each, and counterparts are never cleared (revoke only deactivates), so it is one-shot per pair, forever. If counterparts ever become clearable, the op stops being safe.

The function prefix system

Every function name in sovereign Pact begins with a prefix that is a contract about what the function may do. The prefixes are the vocabulary the rest of this book uses. They split into **unprotected** (callable without capabilities — safe by construction) and **protected** (locked inside their module; not the public integrator surface).

Unprotected:

Prefix	Meaning
UC_*	Pure compute on arguments only — **no table reads, no enforce**. First under FUNCTIONS.
UCv_*	UC_ whose enforce is **intrinsic to its own computation** (a shape/domain guard on the computation itself), not business validation.
UR_*	Table reads. **No raw read on domain tables outside UR_*.** Per-field UR_* take table keys, not row objects.
URC_*	Read + derive. **No enforce** (validation lives in UEV_* / defcap). May call UR / UC / other URC.
URCv_*	URC_ whose enforce is **intrinsic to its own computation** — same v role as UCv_. Use it when the guard is unavoidable in the derivation itself and relocating it would duplicate the identical check at every real call site; use a UEV_* / defcap when the check is a business rule.
UEV_*	Read + enforce. Failure aborts the tx. Unprotected.
UDC_*	Data construction — named constructors for objects; prefer over ad-hoc object{} literals.
CAP_*	Ouronet account-ownership enforcement (UEV-like but specifically tied to account ownership).

Protected:

Prefix	Meaning
A_* / AA_*	Admin-key mutations. Doubled AA_ = **heavy**: a heavy scan (URH_*/URHC_*/URD_*) is reached **somewhere in the whole execution tree, at any depth** (transitive).
C_* / CC_*	Client entry for citizen modules. Builds IGNIS cumulators and returns OutputCumulator. **Cannot be invoked from its own module** — clients reach it via Talos. Doubled CC_ = **heavy** (a URH_*/URHC_*/URD_* scan is

Prefix	Meaning
	reached anywhere in its execution tree, at any depth).
Cp_* / CCp_* / Ap_* / AAp_*	**Hydra** multi-transaction recipes (contrast defpact = one ordered continuation). Cp_/Ap_ carry NO heavy read; CCp_/AAp_ still do. Anatomy: URH_* preflight → slices/pages → optional C_/CC_ begin/finalize.
XI_*	Internal-only protected (this module).
XE_*	For external modules only (forward-module entrypoints).
XB_*	Both internal and external.

The letters compose rather than enumerate: a **doubled** initial letter means heavy (a table scan reached transitively), a trailing **v** that the function's own enforce is intrinsic to its computation, an **i** (as in URCi_) that it returns a cost cumulator, a **p** a Hydra recipe. The AQP modules additionally use W_ write helpers (WI_ insert, WU_ update, WW_ upsert) and UCk_ key constructors, a rollout scoped to 1_SOVEREIGN/STAGE_02/2_Core/03_AQP/.

Two shapes wear the p, and only one is parallel. A *fed-slice* takes an explicit slice object and is order-independent and parallel-safe (Cp_WipeSlice, in DPOF and DPDC-MNG). A *cursor pager* takes a size, computes its own window from stored progress, and is strictly sequential (CCp_SweepRecomputeChunk, in FVT). Check which one you have before firing N at once.

The intended decomposition of a client operation, which later chapters assume: the **client defcap** holds **all** authorisation and validation (CAP_EnforceAccountOwnership, UEV_*, UEV_Fee, table reads); **XI_* / XB_*** do persisted writes under require-capability on a SECURE-composing cap, must **not** enforce or call UEV_*, and end on the insert/update/write with no trailing true and no cumulator return; **XE_*** starts with UEV_IMC, then with-capability, then writes only; and **C_*** is wiring and billing — UEV_IMC, with-capability, the X* calls, optional STOA collection, then the cumulator.

Counted with grep -rhoP '^\s*\(defun\s+\Q<prefix>\E' across 1_SOVEREIGN/ and 2_CITIZEN/ (declarations and implementations together): UR_ 1,405 · URC_ 655 · C_ 640 · INFO_ 623 · URCi_ 595 · UEV_ 528 · XI_ 412 · UC_ 405 · XE_ 334 · UDC_ 304 · A_ 188 · URH_ 118 · XB_ 62 · WU_ 62 · CAP_ 35 · UCv_ 34 · CC_ 32 · URCv_ 28 · WW_ 28 · URHC_ 20 · WI_ 20 · CCp_ 18 · Cp_ 4 · AA_ 3 · Ap_ 1.

Capability bands C1–C4

Capabilities in every module are grouped into four bands, in this order: **C1** — trivial, "always true"-style capability roots; **C2** — simple capabilities that do **not** compose other capabilities; **C3** — ownership-related patterns; **C4** — composite capabilities (compose-capability).

That ordering sits inside the canonical module section order — **(1)** schemas, tables and constants (labelled {1}, {2}, {3} in sources), **(2)** capabilities by band, **(3)** functions with true UC_* compute helpers first; reference layout 0_Sample/C0s__01_01_ModuleSample.pact. Within FUNCTIONS the order runs UC_ → UCK_ → UR_ → UDC_ → W_ → URD_ → URC_ → URDC_ → UEV_ → C_ → A_ → X*; UR_ blocks follow the order the schemas are declared in, and within a block mirror the field order of the defschema.

Two conventions inside a defcap produce most of the findings in Part III. **Boolean checks are combined by count**: one condition is a plain enforce; two use (enforce (and p q) "msg"); three or more use (enforce (fold (and) true [p q r ...]) "msg"). CAP_*, UEV_* and UEV_Fee stay as separate calls *before* the combined boolean enforce, because they are not plain booleans. The fold form is not stylistic: (and a b c d) raises *"Attempted to apply a closure to too many arguments"*.

Authorisation precedes validation — an owner ruling of 2026-09-14. When a defcap both authorises (composes a GOV|*_ADMIN capability or equivalent) and validates (business enforces), the authorisation goes first: before any

business rule, and before the let that derives the data those rules read. The reason is testability, and it is the origin of a term this book uses often.

A **shadowed gate** is an authorisation check that never runs, because an earlier check refuses first in every reachable state. GOV|WIPE_ALL-TREASURY-DEBT had the order reversed:

```
(enforce (< treasury-supply 0.0) "Cannot Wipe Positive Treasury Balance")   ;; business
(compose-capability (GOV|DPTF_ADMIN))                                       ;; authorisation
```

A solvent treasury is the normal state, so every attempt — admin or stranger — was turned away by the business rule and the admin gate was never reached. A red-team test could report "a non-admin was refused", be telling the truth, and prove nothing: **had GOV|DPTF_ADMIN been deleted from that capability, the test would still have passed.** From outside, a shadowed gate is indistinguishable from an absent one.

Two qualifications this book needs the reader to carry:

- **The 2026-09-14 sweep covered the admin band only.** It fixed 18 sites matching "a business enforce before a compose-capability of a GOV|*_ADMIN". Re-scanning all 989 defcaps on 2026-09-16 found two more admin-band sites (GOV|GAP and GOV|MIGRATE, both in DALOS — the ruling's own module) and, reading "or equivalent" to include the CAP_* ownership gates, **62 sites across 23 files** the sweep's definition never covered.
- **Those 62 are not to be fixed by reordering.** Ordering is a proxy for testability and can expose only one of two state-dependent guards at a time — some existing tests *depend* on the current order to reach an argument guard without a signature. A fixture that satisfies the first guard exposes both, and cannot introduce an authorisation hole, which a reorder demonstrably can. Relatedly: an authorisation form inside an if/and/or/cond branch is *conditional* by design, and hoisting the bare compose-capability out of the branch silently changes who may call the function.

Interfaces and the cascade rule

Kadena's roughly 150k deploy-size cap forces a strict deploy order: a module may only call into modules already deployed. Everything in this section is downstream of that one constraint.

Cross-module calls use module references and ::, not module.function:

```
(let ((ref-DALOS:module{OuronetDalosV2} DALOS))
  (ref-DALOS::UR_GAP))
```

The type is the *interface*, not the module, so only the used interface members are relevant to typing and coupling. Consequently **interfaces carry nearly the whole public API** — 87 interfaces are declared across 1_SOVEREIGN/ and 2_CITIZEN/, containing 3,177 defun declarations against 5,451 implementations in module bodies. In most files the interface and the module that implements it now live in the same .pact file, the interface first; the 0_Interfaces/ directories have been reduced to short index comments.

Interface names always end in a version suffix, and each revision advances it by exactly one. SwapperIssueV2 → SwapperIssueV3, never V2 → V4.

The cascade rule: when interface **B** is superseded by **B′**, every interface **A** that *names* B in its surface is stale and must become **A′** with the new reference. "Names" covers both module{B} in a type position and qualified row types such as object{B.SomeSchema}. Every consumer — module, Talos client, citizen bridge, sample, REPL — updates in lockstep, and implementing modules implements only the **latest** version of each family.

A worked example from the current tree: on chain SWP implements SwapperV3; in the tree it implements SwapperV4. That one bump names the interface in **16 files** and **241** type positions — the SWP family itself (SWPI, SWPL, SWPLC, SWPU, MTX-SWP, several also using object{SwapperV4.PoolTokens} and object{SwapperV4.FeeSplit}), three Talos modules (TS01-A, TS01-C3, TS01-P), the read layer INFO-ONE+, four Stage 2 AQP modules (AQP-SCORE, AQP-POOL, RPS, AQP-FVT) that reference pools they do not own, and two citizen modules (DPL-UR, EXPLORER). And because those consumers are themselves interfaced, their own interfaces bumped in lockstep: SwapperIssueV3 → V4, SwapperMtxV3

→ V4, SwapperUsageV2 → V3, SwapperLiquidityV1 → V2, SwapTracerV1 → V3. A one-line change to a schema in SwapperV3 is never a one-line change.

Three narrower rules follow from the same mechanics:

- **Same-interface object types** are written unqualified: inside SwapperV4, write object{PoolTokens}, not object{SwapperV4.PoolTokens}. Reserve the qualified form for schemas owned by a *different* interface.
- **Interface object-return rule**: if a function would return object{Schema} where Schema is defined in the *implementing module*, remove it from the interface — the interface loads before the module's schemas exist. Ouronet keeps schemas in modules, so such functions stay module-only (this is why AQP-ANK and AQP-SCORE have module-only readers).
- **Version policy**: new work stays on V1 until first mainnet deployment, and bumps only afterwards if a post-deploy adjustment forces it. Until then V1 code is edited freely — which is why the tree holds one V1, sixty V2s, fourteen V3s, five V4s, four higher numbers on long-lived families (DeployerReadsV13, TalosStageOne_ClientFourV8, DemiourgosPactMetaFungibleV7, PythiaV5) and three unversioned citizen interfaces (AgeOfZalmoxis, Bloodshed, Dispenser).

The scale of the thing

Every figure below was measured on 2026-09-18 against the working tree, each taken twice by two different commands. Where the project's own tooling reports a different number for the same quantity, both are given with the reason.

Pact source (1_SOVEREIGN/ and 2_CITIZEN/ only — the Stoa/Kadena sandboxes and 0_Sample/ are excluded):

	count	how
.pact files	**93**	find 1_SOVEREIGN 2_CITIZEN -name '*.pact' | wc -l (74 sovereign, 19 citizen)
lines of Pact	**116,040**	find … -exec cat {} + | wc -l, cross-checked with xargs wc -l
(module …) declarations	**89**	grep -rhoP '^\s*\(module\s+\S+'; **88 distinct names** — CADUCEUS, a bridge scaffold, declares two draft module blocks in one file
(interface …) declarations	**87**	grep -rhoP '^\s*\(interface\s+\S+'; all 87 names distinct, spread over 77 files
defun	**8,628**	of which **5,451** are implementations inside module bodies and **3,177** are interface declarations
defcap	**989**	988 in module bodies, 1 in an interface — matches the 989 figure the project's own capability sweep used
defschema	**217**	grep -rhoP '^\s*\(defschema\s+\S+'
deftable	**231**	grep -rhoP '^\s*\(deftable\s+\S+'
defconst	**903**	grep -rhcP '^\s*\(defconst\s' summed
defpact	**6**	the multi-step continuations, as distinct from the Hydra p recipes

The anchored regex (^\s*\(defun) and an unanchored one differ by exactly one for defun and one for defcap — two definitions not starting their line; the anchored figures are used above. The largest modules are AQP/04_RPS.pact (5,621 lines), AQP/02_SCORE.pact (4,283), Z_Reads/02_INFO-ONE+.pact (4,199), AQP/05_FVT.pact (3,977) and AQP/06_VCT.pact (3,508): the acquisition-pool family dominates, which is why it gets its own chapter.

The Talos surface, counted inside module bodies only, is client/admin TS01-A 0/31, TS01-C1 62/3, TS01-C2 77/3, TS01-C3 34/3, TS01-P 8/3, TS01-C4 8/9, TS02-C1 66/3, TS02-C2 60/3, TS02-C3 81/5, TS02-DPAD 10/7 and the citizen TS02-CPAD 7/3 — **413 client and 73 admin across 12,611 lines**. TS01-A is the admin Talos and carries no client op at all. The pricing documentation counts **452** Talos client entrypoints by its own census, **442** of them carrying a price row and 5 not (3 admin ops free by rule, 2 billing-shape-B wrappers); the difference from 413 is a counting definition, not a discrepancy — that census resolves Talos→Talos delegation and includes entrypoints this chapter's prefix-anchored count does not.

The test harness (REPL/, excluding REPL/archive/, as the project's own scale tool does):

	count	how
.repl files	**207**	find REPL -name '*.repl' -not -path '*/archive/*' | wc -l (275 including the 68 archived)
lines	**137,733**	the project's _scale_report.py. A raw cat | wc -l gives 137,712; the 21-line gap is **21 files with no trailing newline**, counted directly, which wc -l does not count
distinct assertions written	**5,873**	project tool, counted from source
assertions executed per full gate run	**25,035**	project tool, from gate output — 20,042 expect and 4,993 expect-failure

The last two rows differ by about 4× because shared files run once per entrypoint that loads them. **Quote the distinct figure for "how many tests exist"; the executed figure answers "how much ran".** A raw source count of (expect/(expect-failure forms returns 5,529, lower than 5,873 because the project's counter also resolves assertions generated inside map/fold constructs.

The gate that has to be green is python3 REPL/tools/_gate.py. The fast path, REPL/Z.repl, is deliberately not the gate: it skips Stage_01/[6.1]_Cumulator.repl and its 75 leg-level pricing assertions, runs issuance-only DPTF and SWP variants, and drops the Stage-1 scenario tail, though it still runs 106 pricing assertions of its own. Any change to pricing, STOA collection or IGNIS billing is verified with the gate, not with Z.repl.

3. How Ouronet is tested

Everything asserted in this book was established by running something. This chapter is about the machine that does the running — its size, how it is organised, why a full verification takes five minutes instead of eighty, and what the procedure actually is. It is placed before the audit chapters because every one of them cites it.

The corpus, measured

.repl files, live	**207**
.repl files, archived	68
total lines of REPL, live	**137,733**
code	103,405
comment	32,219
blank	2,109
sandbox bootstrap .repl (outside REPL/)	16 files, 567 lines
analysis tools (REPL/tools/*.py)	53 files, 12,099 lines

Lines by area:

area	lines	what lives there
modules/	44,304	one tester per module — the per-module functional suites
Stage_02/	42,605	Stage 2 deploys and the scenario suites (DPDC, DemiPad, AQP, Talos)
Stage_01/	31,242	Stage 1 deploys and scenarios (DALOS, DPTF, DPOF, ATS, VST, SWP)
Kursan/	9,333	finding-verification harnesses — one per audit finding that needed a standing proof
RedTeam/	5,067	the attack files
(root)	4,414	the stage loaders, Z.repl, ZALL.repl, the golden-path drivers
fixtures/	768	shared minimal fixtures (mock-tf, mock-of, mock-pair, …)

The archived 68 are kept for provenance and are excluded from every count by the measuring tool — named as an exclusion rather than silently dropped, because a denominator that quietly shrinks is how three of this programme's own instruments came to report tidier numbers than the truth.

For scale, the code under test is **93 Pact modules**. The suite is therefore larger than a one-to-one relationship with the subject; roughly speaking, there is more test than there is system.

Two assertion counts, and which one to quote

This distinction matters enough that the generated statistics file leads with it, because quoting the wrong one overstates the suite by more than four times.

distinct assertions written	**5,873**
assertions executed per full gate run	**25,035**
positive (expect)	20,042
negative (expect-failure)	4,993

Executed exceeds *distinct* because a shared file runs once for **every entrypoint that loads it** — 92 entrypoints reach 321 distinct files, and the overlap is deliberate: a Stage-1 deploy chain is a prerequisite for dozens of suites and is re-executed by each.

Quote 5,873 for "how many tests exist". Quote 25,035 for "how much ran". This book uses 25,035 when describing a gate run and says so each time.

The **negative** count is the one to look at for a security audit. Roughly one assertion in five asserts that an operation was *refused*, and of those, **1,296 check the refusal's message** rather than accepting any error at all. That last figure is the load-bearing one: an expect-failure that accepts any error cannot distinguish the gate under test from an unrelated check that happened to fire first, and in a language with eager let bindings, an unrelated check firing first is the normal case, not the exotic one. Exactly **one** message-free negative assertion remains in the tree, and it sits in an ungated scratch file.

One runner, and why that is a rule

> **RULE 4 — there is exactly ONE authoritative runner: python3 REPL/tools/_gate.py.**

This is stated as a rule because it was once violated. A second runner existed — regressions/run.sh — which ran a set of files the gate excluded. Two runners with different inclusion lists means the honest answer to "is the suite green?" is "which one did you run?", and the Kursan/ harnesses sat in exactly that gap: **343 assertions**, including two suites of 68 and 69, excluded wholesale from the gate as "one-off finding-verification harnesses" while the second runner's own manifest listed them as runnable. They are gate entrypoints now and run.sh is deleted.

The gate resolves its own paths, so it runs from any directory:

```
python3 REPL/tools/_gate.py        # the gate. ~5-7 minutes.
python3 REPL/tools/_gate.py -j 8    # fewer workers
python3 REPL/tools/_gate.py --audit-only   # orphan check only, run nothing
```

It also resolves the pact binary itself, with a PACT= environment override, because a PATH missing ~/.local/bin is a common way for the gate to "fail" for reasons that have nothing to do with the suite.

Two faster paths exist and neither is the gate:

- cd REPL && pact Z.repl — the fast pipeline. It **deliberately skips** [6.1]_Cumulator.repl and the Stage-1 scenario tail. It does run 106 pricing assertions, but the 75 it skips are the **leg-level** ones, and on one occasion a leg-level assertion was the only thing in the suite that caught a preview leg-split which every total-level assertion passed straight through.
- cd REPL && pact ZALL.repl — every suite, but serially, in one process, as a single gate entrypoint. It is the largest single entrypoint and therefore sets the floor described below.

Why it runs in parallel, and why that is safe

Verified with strace: a pact <file>.repl run opens ZERO files for writing.

That single observation is what the whole execution model rests on. Each run is a separate OS process holding its own **in-memory** database. Source files are read-only and shared. There is no shared mutable state for concurrent runs to corrupt, no lock to take, no temp directory to collide in, and no ordering requirement between entrypoints.

The consequence is that parallelism here requires **no support from Pact and no test-framework infrastructure**. The operating system provides it. The gate uses a thread pool of os.cpu_count() workers dispatching subprocesses; the equivalent with no Python at all is:

```
cd REPL && ls modules/*.repl | xargs -P 16 -I{} pact {}
```

The only shared resources are CPU, RAM (a run peaks well under 1 GB, against 62 GB available) and the read-only tree.

This is worth stating plainly because “run the tests in parallel” is usually a project with a database-isolation problem attached. Here it is free, and it is free because of a property that was *measured* rather than assumed — the strace run is the reason the claim is in this book at all.

The arithmetic of the speedup

From one green run, **2026-09-20**, on 16 cores. These are a dated SNAPSHOT rather than live figures, deliberately: the paragraphs below build an argument on them — the efficiency, the scheduling floor, the 24-second tail — and a number that silently moved would leave the reasoning attached to it pointing at nothing.

entrypoints	92
distinct files reached	321
serial cost (sum of all entrypoint times)	**4,700.7 s** ≈ 78 min
wall time	**318.5 s** ≈ 5.3 min
speedup	**14.8**× on 16 workers

A 14.8× speedup on 16 workers is 92% efficiency. The residual 8% is scheduling tail: towards the end of a run the remaining long entrypoints cannot fill 16 workers, so cores idle. It is **not** a single dominant file — see the next section, where that turns out to matter.

Why more cores would not help

The testers are independent, which is what gives the parallelism. But **one file can only use one core**, so a single long entrypoint is a hard floor:

```
wall time = max( longest single entrypoint , total work / cores )
```

For the run above: total work / 16 = **294 s**, and the longest entrypoint — ZALL.repl — is **226.8 s**. The work bound is the larger of the two, so this suite is currently **work-bound, not file-bound**: the formula’s floor is 294 s and the measured wall was 318.5 s, the 24 s difference being the scheduling tail.

RE-MEASURED 2026-09-23, and the model held. A later green run on the same 16 cores: **5,206 s** serial, **362 s** wall, **14.4**×. The work grew by 11% (the canon sweep and the Stage Two suites added assertions and two entrypoints), the wall time grew by 14%, and the efficiency moved from 92% to 90% — all of it consistent with a longer tail rather than a changed model. Two runs three days apart, same shape. That is the only evidence worth having that the arithmetic above describes the system rather than one lucky afternoon.

That is the healthy state and it is worth being precise about, because the intuitive story — “one huge file dominates” — is what RULE 1 exists to prevent and is **not** what the numbers say here. The practical consequence is the opposite of the intuitive one: adding cores *would* help this suite today, up to the point where the work bound drops below ZALL’s 226.8 s. That happens at about 21 workers. Past 21, extra cores buy nothing until ZALL is split.

workers	work bound (4,700.7 / n)	longest entrypoint	⇒ floor
8	587.6 s	226.8 s	**587.6 s**
16	293.8 s	226.8 s	**293.8 s** *(measured*

workers	work bound (4,700.7 / n)	longest entrypoint	⇒ floor
			wall 318.5 s)
21	223.8 s	226.8 s	**226.8 s** ← crossover
32	146.9 s	226.8 s	**226.8 s**

> **RULE 1 — balance the split: aim for `longest_entrypoint ≈ total_work / cores`.**
>
> Not a fixed minute count — the target moves as the suite grows. Splitting a file already below the work bound wastes effort; leaving one far above it wastes hardware. **Whatever the longest entrypoint costs is what the whole suite costs.**

This is the reason the suite is organised as 92 entrypoints rather than one. The structure is not aesthetic; it is the direct consequence of that formula. When a suite grows past the bound, the fix is to split the entrypoint, and the rule tells you which one and by how much.

The half of the gate that matters more

The gate has two jobs, and running the tests is the first one.

> **Job 2: prove that no asserting file is orphaned.**

Every `.repl` carrying assertions must be reachable from a gate entrypoint, or be explicitly and justifiably excluded in a list that states its reason. Anything else fails the gate.

This exists because it was missing. Four `deb-staleness-*` driver files were archived on an "assertion-free" heuristic — true of the drivers themselves, false of what they **load**. They were the only path to an entire family of suites, and moving them also broke their relative `(load …)` paths, so they could not have run even where they were put. **About 125 assertions silently left the suite, and the ledger did not notice, because a ledger counts files and not execution.**

An assertion nothing executes is not coverage. It is decoration that reads as coverage — to a tracker, to a reviewer, and to the person who wrote it. The orphan check is the only thing in this system that can tell the difference, and it is why `--audit-only` exists as a mode: you can ask that question in seconds without running anything.

Exclusions carry their reason in the source, and a **wrong** reason is treated as worse than none. One exclusion note read "fails inside its own probe module — same known breakage as the sibling", which was never checked and is not what happens: the file dies at its very first `load`, before any Ouronet code runs, because it was moved into a subdirectory and its relative paths were never fixed. A wrong exclusion reason sends the next person to debug a module that is never reached.

Fixtures: mock by default

> **RULE 2 — a module tester builds the SMALLEST fixture that exercises its logic.**

Shared minimal fixtures live in `REPL/fixtures/`: `mock-tf`, `mock-of`, `mock-sft`, `mock-nft`, `mock-pair`, `mock-pool`, `mock-accounts`. A large, live-shaped fixture is used only when the test's subject **is** scale or the live definition — set composition, fragments, make/break, gas-ladder probes — and the file header must say so.

The cost of getting this wrong is direct: one populate step spends 28 seconds minting a full live-shaped collection. That collection is correct for the tests that need a real set definition and wrong as a default, because nothing about burning a single token needs 500 nonces. Under RULE 1, a 28-second fixture in the wrong file is 28 seconds added to the floor of the entire suite.

Assertion discipline

Five rules govern what an assertion must do to count. Each was written after a violation.

> **RULE 3 — an `enforce` you cannot write a failing test for is DEAD CODE. Delete it.**

This is the rule that turns "we should test the guards" into a finite, checkable task. It is measured by `_enforce_coverage.py`, and the live unpinned count is **0 of 790**.

> **RULE 8 — an `expect-failure` MUST assert the expected error message.**

Without the message, the assertion proves only that *something* refused. With eager `let` bindings that something is routinely a hard read several lines above the guard under test. This is the single most important rule in the suite and the one this book returns to most often.

> **RULE 9 — an `expect-failure` on a state-mutating operation needs its own `(rollback-tx)`.**

`expect-failure` does **not** roll back REPL writes. An attack that expects a refusal, gets one, and does not roll back leaves partial state behind for whatever runs next. This was learned by turning the gate red: one attack used `commit-tx` and consumed a fixture another attack depended on.

> **RULE 10 — a commented-out invocation is not coverage. Measure with comments stripped.**

> **RULE 11 — coverage counted is not coverage GATED. Measure both.**

RULE 11 is the subtlest and it caught its own author: a coverage figure of 448/448 was published when the *gated* figure — how much of that coverage a red build would actually protect — was 439/448. A test that exists but that no gate entrypoint reaches is RULE 11's failure mode, and the orphan check above is its enforcement.

The checks that are not tests

A tree can pass all 25,035 assertions and still fail the gate. These static checks are **fatal**:

check	what it enforces
orphan check	every asserting file is reachable from an entrypoint
`_pricesync --check`	the generated price sheet matches the price tables in source
`_figuresync --check`	the figures in the architecture documents match the tree
`_toolpaths --check`	every hard-coded path literal in every tool still resolves — across **all four** tool directories, plus a discovery pass that reports any directory of `.py` files the list does not cover
`_redteam`	every attack header parses; the register matches the defect ledger
`_modref --check`	no module-reference call names a member that exists nowhere
`_booktables --check`	this book's headline tables sum to their totals, and its defects chapter names every attack the register marks fixed
`_auditbook --check`	this book is not stale against its chapter sources

`_toolpaths` earns its place: a directory move once killed eleven tools that died at **import**, so every check that consumed their output silently stopped checking anything. A tool that fails to load produces no output, and no output is indistinguishable from clean output to anything downstream.

The procedure, end to end

1. **Change something.**
2. **Run the affected module tester** — cd REPL && pact modules/ATS.repl. Seconds to a minute.
3. **If the change touches pricing, STOA collection or IGNIS billing, do not trust Z.repl.** Run the gate. A quarter of the pricing assertions — every leg-level one — are outside the fast path.
4. **Run the gate before committing.** Five to seven minutes.
5. **Compare the assertion count against the previous green run.** This is the step most easily skipped and the most informative. A refactor that is supposed to be behaviour-preserving must leave the count *exactly* unchanged; the seven-interface version bump touching 3,333 references across 116 files landed at 25,035, the same figure as before it, and that identity is the evidence the bump changed names and nothing else. A count that moved when it should not have is a finding.
6. **If a fix closes a finding, the finding gets a named assertion tag** that goes red if the fix is reverted. RULE 7: a red-team finding closes with a regression test named for the finding, not with a note.

What the suite does not do

- **It is not a fuzzer.** Every input is chosen. Adversarial cases are *derived from the source* — enumerate the enforceS, write the input that trips each — rather than discovered by search.
- **It does not test the deployed chain.** Everything runs against sandboxes that reproduce StoaChain's genesis ordering. The suite proves the tree behaves; it cannot prove the chain holds the tree.
- **It does not measure gas against a real node.** IGNIS is the virtual gas and is fully modelled and asserted; the underlying StoaChain gas is not what these assertions are about.
- **Coverage is entrypoint coverage, not branch coverage.** The worklists in Chapter 28 count functions, previews, enforce sites and gates — each driven to zero — but no claim is made that every branch of every function has been executed.

---

4. Part I — The module audits

Six per-module audit rounds, run 2026-08 through 2026-09, before the main-work round of Part II. Source material: the .../Audit/* trees, **31,438 lines across 50 files** (31,346 in the 49 .md files). *Re-measured 2026-09-18; this read 31,225. Three commits on 2026-09-17 edited files inside those trees, one of them the AQP design-document correction described below.* **Every chapter carries a verification pass dated 2026-09-17** — each fix recorded as FIXED was re-checked against *current* source, not taken from the audit's own word.

#	module	what it is	findings	source
1	**DALOS**	accounts, guards, governance, the gas station	**88**	Audit/module-audits/DALOS/
2	**ATS**	autostake pools — staking, indices, recovery	**35**	Audit/module-audits/ATS/
3	**SWP**	the AMM — pools, routing, swaps, liquidity	**83**	Audit/module-audits/SWP/
4	**DPDC**	collectables — collections, nonces, sets, fragments	**58**	Audit/module-audits/DPDC/
5	**DEMIPAD**	the launchpad — asset sales, custody, the ICO vault	**17**	Audit/module-audits/DEMIPAD/
6	**AQP**	acquisition pools — anchors, scores, rewards, vaults	**33**	Audit/module-audits/AQP/
			314	

How these audits were run

All six followed the same shape, and the shape is worth stating because it determines what the numbers in the table above mean.

Scope was fixed in writing before the reading began. Each audit opens by naming its modules, its utilities, the Talos wiring that exposes them, and the interfaces involved. ATS, for instance, scoped two core modules, two utility modules, four Talos modules and six interfaces. A finding outside that scope was recorded but not counted, which is why the per-module totals are comparable to each other.

Two rounds, with an owner decision between them. Round I raised findings and stopped. It did not fix anything. The output was a ranked issue list, which the owner reviewed and ruled on — the ROUND-01-OWNER-FEEDBACK.md in each tree is that ruling. Round II then implemented the fixes the owner accepted. The separation matters: an auditor who fixes as they read inevitably stops raising the findings that are inconvenient to fix, and the ruling step forces every finding to be adjudicated on the record rather than quietly dropped.

A finding has one of four dispositions, and all four are counted in the totals:

disposition	meaning
fixed and proven	repaired, with a REPL regression test that goes red if

disposition	meaning
	the repair is reverted
closed as not-a-bug	investigated and found to be intentional design, with the reasoning recorded
deferred	confirmed real, deliberately not fixed now, with the reason and the trigger stated
open	none. Every audit closed with zero

ATS is representative: 35 raised, 19 fixed and proven, 13 closed as intentional, 3 deferred — two to a planned module rehaul and one tracked as a hard prerequisite for deployment. **A third of the findings in these audits were not defects**, and reporting them anyway is deliberate. A finding closed as intentional is a piece of design rationale that now exists in writing, and the next reader who has the same suspicion can stop in one minute instead of one day.

Severity is on the id, not in a column. The id schemes differ by module — SWP writes `#65bL` and `#32bM`, AQP writes `C1` / `H4` / `S4` — but in all of them the letter is the severity class, so a finding cannot be referred to without its severity coming along. The inconsistency between schemes is a real wart, and it is what makes the two count discrepancies below unresolvable by pattern matching.

Each audit named its priority target before starting. ATS's was whether removing and re-adding a reward token preserves accounting for stakers whose positions predate the change. It does not, in three independently confirmed ways sharing one root cause. Naming the target in advance is what separates an audit from a reading: it creates something the audit can *fail* to find, and therefore something its silence would mean.

Then, months later, every FIXED was re-checked. That pass is the next section, and it is the part of Part I with the most to say — because re-checking an audit's own word against current source is not a formality, and this one found four different ways that word had become unreliable.

Independent corroboration of the counts (2026-09-17)

The per-module totals above were re-derived from the audit trees by a **different method** than the chapters used — counting distinct finding ids in the source `.md` files rather than reading the trackers:

module	chapter	independent count	
DALOS	88	**88**	exact
ATS	35	**35**	exact
DPDC	58	**58**	exact
DEMIPAD	17	**17**	exact
SWP	83	82	within 1
AQP	33	36	differs by 3

The two that differ are the two with irregular id schemes — SWP uses letter-suffixed ids (`#65bL`, `#32bM`) and AQP uses a bare class-letter form (`C1`, `H4`, `S4`) that a pattern also matches in prose. **The independent method is cruder than the chapters', so the chapters' figures stand**; this is corroboration, not a correction, and the two gaps are recorded rather than reconciled away.

> An attempt to audit the chapters' findings *tables* row-by-row was abandoned: the six chapters use different table shapes, and a single regex produced obvious nonsense on three of them (0 rows where there are dozens). A measurement that cannot be trusted is not reported as a number — which is the book's third rule applied to the book's own audit.

REPL/tools/_booktables.py gates only what it can check **exactly**: that the two headline tables sum to their own totals, and that Part III's total matches the attack register in the tree.

What the verification pass found

The headline is a positive one

Of the hundreds of fixes recorded as FIXED, essentially all are present in current source — and that held through four intervening whole-tree rewrites: an interface version cascade, the Part II pricing rehaul, a syntax-convention sweep, and a module split forced by the deploy-size ceiling. Three fixes were briefly invisible to a search on the audit's own wording because they had been **renamed, not deleted**.

The mechanism that made this checkable at all is a convention: fixes carry a source comment naming the finding (DPDC Audit #NN). There are **51** such markers in DPDC — and **zero** in one module, which is the one module where verification had to fall back to behaviour and where the single unresolved question remains.

> **Corrected 2026-09-18.** This also said *"and 30 in AQP"*. The DPDC figure re-derives exactly; the AQP one does not, under any marker convention found in the tree — there is no AQP Audit #NN form, and a case-insensitive audit over 03_AQP/0*.pact totals 25. The figure is withdrawn rather than replaced, because what it was counting is not recoverable. AQP's fixes *are* annotated (audit finding #15M / M6, L7 #19, and so on), just not to one pattern.

One fix was genuinely gone

The SWP audit's M14 archived two frozen historical interfaces. A later automated sweep deleted them, because **zero references is the defining property of a deliberately-frozen archive** and the tool's heuristic was "zero references means dead". Investigated in full and closed as **superseded, not reopened**: a dated convention amendment had retired the frozen-copy practice the same day, and a restored archive cannot be both loadable and historical under this codebase's cascade rule.

Two verdicts rested on premises that had been retracted

- **SWP M11/M12** were closed as DESIGN on the grounds that a module family had *"zero client wiring — unreachable"*. That premise was retracted three weeks earlier and the verdicts were never revisited. Reopened during this work, proven **live by execution**, and fixed. Chapter 20.
- **DALOS H14** fixed a constant chosen as *"mainnet's approximate KDA/USD price"*. After a protocol-wide rename that constant now serves as the **STOA**/USD price, unchanged. Nothing in the tree asserts the two coincide. Flagged, not fixed.

And a class of fix that was present but unwitnessed

Eleven fixes — including **two** critical-ranked findings, ATS C2 and DALOS C3 — were present in source with **nothing in the running suite that would go red if they were reverted**. Their proofs were written into scratch harnesses that now sit in an archive directory the gate excludes by name.

> **Corrected 2026-09-18.** This said *"Around ten fixes — including one critical-ranked finding"*, while the table below lists eleven rows and marks two of them critical.
>
> SWP is the near-exception: eight of its nine proof tags survive, **because they were written into the canonical suite files rather than into scratch harnesses.** That is a filing decision, not a rigour one, and it is the single largest difference in durability between these six rounds. The ninth, SWP|TX 015b, is the counter-example — it lives in a canonical suite file that _gate.py excludes by name, so being filed well is necessary and not sufficient (Chapter 7).

All eleven are now closed, as this book was assembled:

finding	what the witness had to do
ATS C2 (critical)	drive the **all-zero** object the removed gate *rejects*, and show it is reshaped anyway
ATS #6H	lock a pool, then show the **same owner** is refused on both setters
ATS #32N	drive a cull from an account that has a ledger row but nothing ripe — one with no row dies earlier, elsewhere
ATS #5C	none of the usual owner's pools has a Hot-RBT; the fixture that does is owned by a third account, which makes the refusal *more* legible
DALOS C3 (critical)	hand a duplicated **nonce** list to all three capabilities — the message was already asserted, at other call sites
DALOS H3	two **distinct** smart-account interactors, or compression merges the legs and the fix's branch is never entered
DALOS M5	a batch whose first leg removes the collateral the second leg's overdraft depends on
DALOS M6	the return value cannot change, so the witness is **gas**: the reader must cost no more than its pure-read sibling
DALOS M7	the source said it was unreachable; measurement showed a **partial** shadow, and the guard was hoisted
DALOS M1	read the ledger table that no test had ever read
DALOS N2	drive the **Talos wrapper**, not the core — the existing test pinned a different guard entirely

> Not one of them was closed by a straightforward test. Every case needed a specific input, fixture or measurement that distinguishes the fixed code from the reverted code — and in four of them a first attempt passed while proving nothing. That is the argument for the book's second rule: *every fix names the assertion that would go red if it were reverted.* Without it, a fix and a claim look identical six weeks later.

Two audits' own arithmetic did not add up

One tracker states *"FIXED: 19"* and enumerates 18, with a compensating error elsewhere so that the **total reconciles**. A total that reconciles is precisely why nobody re-counted the parts.

And two design documents understate the shipped code

Both in the AQP tree. One declares an operational constraint — an anchor *"locked forever"* until an unwind is built — that has not applied since the unwind shipped, through two client doors. **A reader consulting it would plan around a permanent lock that does not exist.** Corrected, with the original preserved.

> Checking each identifier **by exact name** mattered here: the first pass flagged four stale claims in those documents and only **two** survived, because two had been matched against similarly-named functions that do exist but are different mechanisms.

---

5. DALOS — accounts, guards, governance

Source tree: Audit/module-audits/DALOS/ (3,759 lines across 5 files). Audit ran 2026-08-22 → 2026-08-29. This chapter was written 2026-09-17 and re-checks every recorded fix against the tree as it stands today.

Evidence labels used in this chapter

label	means
[V-cmd]	established by running a read-only command (grep, sed, git log, git show) and reading its output
[V-read]	established by reading the cited source and judging what it does
[INFERRED]	reasoned from other facts; not directly observed
[REPORTED]	quoted from the audit trail, a commit message or DEFECT-LEDGER.md; **not** re-established here

Nothing in this chapter was established by running the test suite. The suite and the gate were off-limits to this pass (they are contended and slow). Every claim of the form *"this was reproduced live"* is **[REPORTED]** — it is the audit's own record of an execution it performed, not an execution performed for this chapter. Claims about what the source *says* today are **[V-cmd]** or **[V-read]**, and those are the ones this chapter is actually staking its reputation on.

1.1 What the module does

"DALOS" names this audit but not its scope. The scope is **everything in Stage 1 that the sibling ATS and SWP audits did not claim** — twenty-two modules and thirteen utility libraries, organised by the audit into ten clusters. The name comes from its flagship, 01_DALOS.pact.

DALOS itself is the account layer. Every actor on Ouronet — a person, a pool, a launchpad, a bridge — is a *Ouronet account*: a string with a glyph prefix (Ѻ. for a standard account, Σ. for a *smart* account governed by a module rather than a key), a guard, a linked StoaChain address, and a sovereign. DALOS owns the table that maps all of that, the reverse index from chain address back to account, the Elite-tier record that decides how far a user may overdraw, and the gas-station capability (GAS_PAYER) that lets the protocol pay for its own users' transactions. If DALOS is wrong, identity is wrong, and every authorisation decision downstream of identity inherits the error.

IGNIS is the meter. Ouronet is a virtual blockchain: it charges its own gas, denominated in IGNIS, on top of whatever StoaChain charges in native gas. Client functions across the whole codebase each return an OutputCumulator — a little ledger of *(who gets paid, how much)* — and Talos, the orchestration layer, concatenates them and calls IGNIS::C_Collect exactly once per transaction. C_Collect compresses the chain, splits fees between interactors and the treasury, and debits the patron. Everything the protocol earns flows through this one function.

The rest is the token estate. DPTF (true fungibles) and TFT (their transfer layer) run the plain token surface: mint, burn, transfer, freeze, the fee machinery, and the OURO overdraft — a credit line whose size is a function of how much Elite-Auryn an account holds. DPOF (orto-fungibles) runs the nonce-carrying, metadata-rich variant, where a single token id holds many numbered positions with independent supplies. VST locks tokens into vesting, sleeping, hibernating and reserved forms. LIQUID and OUROBOROS bridge to native STOA and mint/burn the OURO gas token. CODEX and PYTHIA handle on-chain naming and the oracle/dirty-read relay. BRD and ELITE handle branding and tiering. Thirteen U_* libraries carry the shared maths.

If any of this breaks, the failure is not contained. A DPOF nonce that can be double-counted inflates an account's total supply, which feeds ELITE's tier calculation, which sets the OURO overdraft ceiling in TFT — a metadata bug becomes a credit-line bug three modules away. That is not a hypothetical: it is finding C3, below, and it is exactly the path it took.

1.2 How it was audited

The audit ran in the same shape as its siblings (ATS, SWP, AQP) and borrowed their discipline explicitly.

Round I — ten parallel lenses, read-only. The scope was enumerated from MODULE-INDEX.md and direct file reads rather than guessed, and split into ten clusters: (1) DALOS/IGNIS/fuel core, (2) legacy/branding/elite, (3) true fungibles, (4) orto-fungibles, (5) vesting/liquid/ouroboros plus their Talos wiring, (6) identity/info, (7) oracle/codex plus Talos Client-Four, (8) utility maths, (9) Talos Admin and Client-One wiring, (10) interface cascade review. One deep-read auditor per cluster, run in parallel, each briefed to load SKILL.md and StoicSyntax.md first, to work read-only, and — the instruction that matters — **to assume nothing is correct despite being live on mainnet**. Each finding had to carry a severity, an exact file:line, a failure scenario, and a CONFIRMED-or-PLAUSIBLE confidence tag. [REPORTED — DALOS/README.md § Method]

Round I produced **5 CRITICAL, 19 HIGH, 27 MEDIUM, 34 LOW**, compiled into ROUND-01-FINDINGS.md (full write-ups, frozen) and ISSUES-RANKED.md (a flat #1C → #85L ranking). Three further findings (N1–N3) were discovered later, during fix work, and appended rather than renumbered. [V-cmd — the counts and the #1C–#85L range are read directly from ISSUES-RANKED.md]

Round I — owner feedback, one finding at a time. Findings were presented to the owner sequentially in ranked order. The audit wrote itself a HARD RULE for this, because it had already broken it: *no finding is settled until the verdict is written into all four places in the same turn* — ROUND-01-OWNER-FEEDBACK.md, the README tracker, the ISSUES-RANKED.md annotation, and (if code changed) a numbered entry in ROUND-02-FIXES.md.

The audit also logged its own bookkeeping failure in public: while presenting findings one at a time it **skipped #11H–#14H and mislabelled H10 as #11H for several turns** before catching it — the precise error class the SWP audit's HARD RULE had been written to prevent. All four affected files were corrected before work continued. [REPORTED — DALOS/README.md, "Numbering-mixup note"]

Round II — 31 sequential fixes. Each fix got its own numbered entry with a diff, a rationale, and a REPL proof. The methodology was adversarial and the owner enforced it: *reproduce the bug live before accepting the finding; apply the fix; re-run the identical harness; then run the full Z.repl regression.* Mid-audit the owner made this a standing instruction — **"everytime you give me a bug i want you to confirm it to me in repl"** — and from #25M onward every finding was re-verified live before a recommendation was even offered. [REPORTED — ROUND-02-FIXES.md Fix #21]

Two details of the method are worth recording because they are unusual and because they worked:

- **Reverts were done by hand, never by git.** The audit explicitly refused git stash/reset for pre-fix reproduction, citing a confirmed data-loss incident in a shared worktree. Where a fix had to be temporarily removed to prove the bug, the lines were commented out by hand and a grep for the marker string confirmed none survived. [REPORTED — ROUND-02-FIXES.md Fix #3 step 3-4]
- **The live chain was queried before deciding a fix's shape.** For #30M the audit did not guess whether a legacy backfill was still needed; it read all 18 deployed DPTF tokens off StoaChain via PYTHIA's dirty-read relay, found zero gaps, and only then deleted the backfill. [REPORTED — ROUND-02-FIXES.md Fix #25]

Round III was never run on this branch. The cycle table lists ROUND-03-REVERIFY.md as *"not yet created"*, and it still does not exist. [V-cmd] What re-verification exists happened later and elsewhere: the Part II/Part III programme, and this chapter.

Was the audit thin anywhere? Yes, in one place and by its own admission. Cluster 2 (DPMF/BRD/ELITE) had **no dedicated REPL file at all** — it was "grepped… covered incidentally". Clusters 5, 6 and 7 leaned on REPL suites that were commented out of the default pipeline at the time. The audit converted these into findings (#38M, #41M, #47M–#49M, #55L, #59L, #60L, #64L, #79L) rather than footnotes, which is the right call, but it means the CONFIRMED/PLAUSIBLE tags on those clusters rest more on reading than on execution.

1.3 The findings

88 findings: #1C–#85L plus N1–N3. Severity is **as recorded by the audit**, not re-graded here. The *Evidence today* column is what this chapter checked, on 2026-09-17, against current source.

A note on names before the table: **a great many identifiers have changed since the audit.** The StoicSyntax refactor (Part II) renamed UEV_IMC → P|UEV_IMC, URC_Parent → URCv_Parent, A_*/C_* heavy variants to AA_*/CC_*, UR|KDA-PID → UR_STOA-PID|Price, and bumped almost every interface. The INFO-ONE+/INFO-ZERO modules were moved out of 2_Core/ into STAGE_01/Z_Reads/ and their function family was re-shaped from MODULE|INFO_Name to INFO_MODULE|Name. Where a fix is recorded against a name that no longer exists, this chapter says so explicitly rather than reporting it missing.

CRITICAL

id	summary	severity	verdict	evidence today
C1 #1C	DPMF has no create-table calls anywhere — every function fails	CRIT	**NOT A BUG**	Technical claim stands; DPMF is the retired MetaFungible module, kept for history. 2_Core/00_DPMF.pact still present, still tableless. [V-cmd]
C2 #2C	DPOF::C_MoveCreateRole never revoked the create role from the previous holder — every past holder kept mint access forever	CRIT	**FIXED**	06_DPOF.pact:3088-3095 — XI_SwitchCreateRole now runs before XI_UpdateVerum4, with a comment naming #2C. **Witnessed**: REPL/modules/DPOF.repl:272 <<DPOF-MCR>>, a two-move test whose last assertion (:329) is the revoke. [V-read]
C3 #3C	No nonce-uniqueness check on DPOF batch ops — supply inflation *and* negative nonce supply	CRIT	**FIXED**	06_DPOF.pact:908, 1046, 1080 — ref-U|LST::UEV_IzUnique on C>DEBIT, C>TRANSFER, C>BULK-TRANSFER (the last against the flattened set). [V-read] **Witnessed** since 2026-09-17: REPL/modules/DPOF.rep

id	summary	severity	verdict	evidence today
				l:2394, :2398, :2402 <<DPOF-G15>> hands a duplicated nonce list to all three capabilities. See §1.5.
C4 #4C	VST::C_Unreserve checks the issuer's ownership, not the reserver's — funds stuck	CRIT	**REFUTED**	Owner: Reserve/Unreserve is one-way escrow-for-purchase; the Token Manager is the intended sole collector. No code change. [REPORTED]
C5 #5C	DPOF|INFO_UpgradeBranding called OI|OI|UDC_DynamicKadenaCost — doubled prefix, unbound variable, every call crashed	CRIT	**FIXED**	**Fix site no longer exists.** 21_INFO-ONE+.pact was retired (commit 42fb75d); the successor Z_Reads/02_INFO-ONE+.pact:1198 INFO_DPOF|UpgradeBranding is a full rewrite delegating to DPOF::URCi_UpgradeBranding. Zero OI|OI| occurrences tree-wide. [V-cmd]
N1	DPOF::C_Transmit completely non-functional for every caller — C>TRANSMIT read "meta-data", schema field is "meta-data-array"	CRIT-equiv	**FIXED**	06_DPOF.pact:1027 reads "meta-data-array"; the schema/constructor at :1223 agrees. [V-read] C_Transmit is exercised by modules/DPOF.repl and Stage_02/[6.1.6]_DPOF.repl. [V-cmd]

HIGH

id	summary	severity	verdict	evidence today
H1 #6H	Four DALOS XE_* writers skip a SECURE/named-cap second gate	HIGH	**REFUTED**	UEV_IMC alone is equivalent to require-capability (SECURE); the sibling wrapper protects a shared writer, not an authorisation boundary.

id	summary	severity	verdict	evidence today
				[REPORTED]
H2 #7H	GLYPH|UEV_MsDc's charset fold seeded false/or — returned true if *any* character matched	HIGH	**FIXED**	1_Utilities/08_U_DALOS.pact:572 (and acc checkup), seed true. [V-read] **Witnessed** and gate-run: REPL/_scratch_udalos_h1_msdc.repl (2 asserts), listed in REPL/tools/_gate.py SCRATCH_PROOFS. [V-cmd]
H3 #8H	IGNIS::C_Collect had no per-leg zero filter — one legitimately-free leg aborted the whole bundle	HIGH	**FIXED**	02_IGNIS.pact:1896-1901 — (if (> amount 0.0) ... (with-capability (IGNIS|S>FREE) true)), comment names #8H. [V-read] **Witnessed** since 2026-09-17: REPL/modules/CUMULATOR.repl:139, :143, :147 <<CUM-G8>> — a free leg beside a billable one, with the IGNIS|S>FREE event asserted. See §1.5.
H4 #9H	DPTF::C_ToggleBurnRole/C_ToggleMintRole/C_ToggleFeeExemptionRole missing (UEV_IMC) — reachable bare, bypassing Talos, pause check and billing	HIGH	**FIXED**	05_DPTF.pact:3051, 3073, 3096 — (P|UEV_IMC) first statement on all three, matching the two siblings at :3029, :3119. [V-read] Partially witnessed by modules/CONFORMANCE.repl <<CONF-01>> for the sibling C_DeployAccount. [V-cmd]
H5 #10H	A zero-amount leg aborts the entire Multi(Bulk)Transfer batch	HIGH	**REFUTED**	Owner: dying in place on an invalid amount is intended; silently skipping would hide the caller's mistake. [REPORTED]
H6 #11H	TFT::C_MultiBulkTransfer refreshed the receiver's Elite tier but never the sender's — stale,	HIGH	**FIXED**	09_TFT.pact:1840 binds contains-eazs; :1920-1923 refreshes the sender. Mirrors

id	summary	severity	verdict	evidence today
	over-generous OURO overdraft bound			C_MultiTransfer at :1781/:1820. [V-read]
H7 #12H	DPOF|C>UPDATE-SPECIAL's immutability guard tested vzh-tag 2 twice instead of 2 then 3 — hibernation links silently re-pointable	HIGH	**FIXED**	06_DPOF.pact:1094-1096 — the three branches are now 1/2/3. [V-read]
H8 #13H	LIQUID::C_RegisterOuronetAccountForUrstoaHoldings had zero ownership check — account hijacking	HIGH	**FIXED (by removal)**	Function gone; only the tombstone comments remain at 12_LIQUID.pact:73 and 3_Talos/03_TS01-C2.pact:139. Zero live references. [V-cmd]
H9 #14H	U_VST::UEV_MilestoneWithTime had no lower bound — a negative duration mints a lock already unlockable	HIGH	**FIXED**	11_U_VST.pact:233-235 (and (>= offset 0) (>= duration 0)). [V-read] **Witnessed**: REPL/modules/UTILITIES.repl <<UTIL-11>>, both the negative-offset and negative-duration cases, by message. [V-read]
H10 #15H	ATS|INFO_Coil's third leg built its cumulator from the wrong token with sender/receiver reversed	HIGH	**FIXED**	**Fix site no longer exists.** The successor Z_Reads/02_INFO-ONE+.pact:2220 INFO_ATS|Coil no longer hand-assembles legs at all; it takes one ref-ATSU::URCi_Coil cumulator. The defect class is structurally gone. [V-read]
H11 #16H	ATS|INFO_ColdRecovery bound two locals both named ifp3 — the Transfer leg's cost was computed then shadowed away	HIGH	**DEFERRED** → now moot	Deferred in 2026-08 to the INFO-function project. That project landed: Z_Reads/02_INFO-ONE+.pact:2428 now derives cost from a single ref-ATSU::URCi_ColdRecover

id	summary	severity	verdict	evidence today
				y call. No duplicate binding survives. [V-read]
H12 #17H	PYTHIA::A_UpdateDeployPrice/ A_UpdateRenamePrice never wired into Talos — unreachable even for the admin	HIGH	**FIXED**	3_Talos/06_TS01-C4.pact:96-97 (interface), :324, :335 (impl). [V-read] **Witnessed**: REPL/modules/PYTHIA.repl:55 <<PYTHIA-PRICE>> — six assertions that the Talos wrapper *moves the value the readers return*, plus restore. [V-read]
H13 #18H	U_VST::UC_MakeVestingDateList silently dropped offset when milestones = 1 — a cliff vest releases early	HIGH	**FIXED**	11_U_VST.pact:138 [(add-time first-time duration)]. [V-read]
H14 #19H	The KDA/USD price oracle is a hardcoded stub, 1.0, with the real call commented out	HIGH	**FIXED (interim)**	Renamed and re-denominated: 01_U_CT.pact:364 UR_STOA-PID|Price returns 0.1 with the #19H comment and the dia-oracle call still commented at :365. **Still a stub.** See §1.5. [V-read]
H15 #20H	U_DEC::UC_AddHybridArray returns garbage on an empty column list and hard-crashes on an all-empty row list	HIGH	**FIXED**	07_U_DEC.pact:141-146 — (if (= maxl 0) [] ...) guard with a #20H fix comment; the original body is untouched inside. [V-read]
H16 #21H	~Half of TalosStageOne_AdminV1 's surface has zero REPL exercise; [6.4]_Admin.repl asserts nothing	HIGH	**DEFERRED →** largely closed	[6.4]_Admin.repl now has 3 assertions (was 0); [6.12]_DALOS-ADMIN.repl has 35 and is in the default Stage01_Tester.repl path; modules/DALOS-ADMIN.repl has 96 (expect openers. [V-cmd] (**Corrected 2026-09-18** — this said 94. The file has held 96 since commit 6f4175d

id	summary	severity	verdict	evidence today
				added the <<DALOS-G7>> block on 2026-09-17, the same day this chapter was written.) Not formally re-closed in the audit tracker.
H17 #22H	DemiourgosPactOrtoFungibleV2/ TalosStageOne_ClientOneV2 cascaded locally but not deployed	HIGH	**FINALIZED (status note)**	Superseded — DPOF now ships DemiourgosPactOrtoFungibleV2 and TS01-C1 TalosStageOne_ClientOneV2 as the *live* versions; the redeploy phase absorbed it. [V-cmd]
H18 #23H	OUROBOROS::C_SublimateV2 is live and actively called but absent from OuroborosV1	HIGH	**FIXED**	13_OUROBOROS.pact:72 declares it; :791 implements it; 3_Talos/03_TS01-C2.pact:147/1816 and 04_TS01-C3.pact:875 call it. [V-read]
H19 #24H	Four live C_* functions absent from CodexV1, which declared no C_ at all	HIGH	**FIXED**	2_Core/ 21_CODEX.pact:120-123 — all four declared. [V-read]

MEDIUM

id	summary	severity	verdict	evidence today
M1 #25M	DALOS::C_RotateKadena read the address *after* overwriting it — old reverse-index row orphaned forever	MED	**FIXED**	Now C_RotateStoa, 01_DALOS.pact:1865-1881 — old-stoa bound before XI_RotateStoa, comment names #25M. [V-read]
M2 #26M	Smart-account deploy validation lived inside the XI_* writer, not the client cap	MED	**FIXED**	01_DALOS.pact:847 — new DALOS|A>DEPLOY-SMART-OURONET-ACCOUNT composes GOV|DALOS_ADMIN + the shared validation cap; used at :1727. [V-read]
M3 #27M	GAS_PAYER's allowlist matches module names by string prefix, not identity	MED	**NOT A BUG**	Owner: intentional — every real Talos module is deliberately TS-

id	summary	severity	verdict	evidence today
				prefixed. [REPORTED]
M4 #28M	TFT::C_ClearDispo unconditionally force-unfroze the Elite-Auryn account even if pre-frozen for an unrelated reason	MED	**FIXED**	09_TFT.pact:1655-1662 — ico3 now carries ico1's own (if (not frozen-state) ...) guard. [V-read]
M5 #29M	dispo-data snapshotted once before the multi-transfer fold and reused per leg — **real OURO-overdraft inflation**	MED (under-ranked)	**FIXED**	09_TFT.pact:1797-1803 and :1867-1869 — recomputed inside each leg's let, both functions, comments name #29M. [V-read] **Witnessed** since 2026-09-17: REPL/modules/DPTF.repl :3006 <<DPTF-G14>> — an EA-draining leg in front of the overdraft leg, against a control at :2971 where the same overdraft alone succeeds. See §1.5.
M6 #30M	DPTF::UR_Hibernation, an unprotected "read", performed a live table update	MED	**FIXED**	05_DPTF.pact:1212-1229 — pure getter; the #30M fix comment records the live-chain check that made the backfill provably dead. [V-read] **Witnessed** since 2026-09-17: the companion gate entrypoint REPL/_scratch_dptf_m6_urhibernation_purefetch .repl now carries three <<M6>> assertions (:49, :53, :56) — the witness is **gas**, not the return value. See §1.5.
M7 #31M	DPOF::URC_Parent contained a direct enforce, violating the URC_ contract	MED	**FIXED, then hoisted clear**	06_DPOF.pact:1732 (now URCv_Parent) has no enforce; UEV_ParentOwnership has it at :2044, **above**

id	summary	severity	verdict	evidence today
				the parent binding. The in-source note at :2025-2043 records that the shadow this chapter described was only **partial**, and that the guard is now pinned. [V-read] **Witnessed**: REPL/modules/DPOF.repl:2441 <<DPOF-G16>>. See §1.4.
M8 #32M	DPOF::C_WipeHeavy reaches a table-scanning URDC_* from a public C_ without the HEAVY prefix	MED	**DEFERRED → closed**	The CC_/AA_ HEAVY convention was applied by the StoicSyntax sweep; e.g. 10_ATSU.pact:81 CC_RemoveSecondary, :72 AA_RemoveSecondary. [V-cmd]
M9 #33M	AHU/AUP_OrtoFungible* authorise via a hardcoded account literal, not GOV|DPOF_ADMIN	MED	**NOT A BUG**	Completed one-time DPMF→DPOF migration utility. Documented in place: 06_DPOF.pact:762-782. [V-read]
M10 #34M	Two dead capabilities in TS01-A, one the sole path to a governor slot registered on DALOS's own vault	MED	**DEFERRED to red-team**	**Still open.** 3_Talos/01_TS01-A.pact:147 P|GOVERNING-SUMMONER and :151 P|SECURE-SUMMONER still have zero callers; P|TRG at :139 is composed only by the former and registered as a guard at :242. [V-cmd]
M11 #35M	DPOF|C_TogglePause/C_ToggleFreezeAccount bind a dead ref-TS01-A and skip the mandated result string	MED	**FIXED**	3_Talos/02_TS01-C1.pact:1099-1113 — dead binding removed, format messages added, comment names #35M. [V-read]
M12 #36M	DPMF carries stale duplicate Elite-Auryn accounting,	MED	**NOT A BUG**	DPMF permanently out of scope. [REPORTED]

id	summary	severity	verdict	evidence today
	missing the 6th term ELITE adds			
M13 #37M	Malformed format in DPMF|C>UPDATE-SPECIAL	MED	**NOT A BUG**	Same. [REPORTED]
M14 #38M	Elite-tier update functions run on nearly every transfer but are never asserted; BRD's admin path has zero exercise	MED	**DEFERRED to REPL phase**	Partly closed by the Part II module testers; not re-closed in the tracker. [INFERRED]
M15 #39M	DPTF|INFO_ClearDispo carries a raw enforce, undeclared, zero callers	MED	**DEFERRED to INFO project**	INFO project landed; the whole family was rewritten. Not re-closed per-finding. [INFERRED]
M16 #40M	Six LIQUID|INFO_*/ORBR|INFO_* implemented but undeclared	MED	**DEFERRED to INFO project**	Z_Reads/02_INFO-ONE+.pact declares INFO_LIQUID|*/INFO_ORBR|* in its interface. [V-cmd]
M17 #41M	Near-total absence of asserted returns across 93 INFO functions — root enabler of C5/H10/H11	MED	**DEFERRED to REPL phase**	Closed: REPL/modules/INFO-ONE.repl exists, and per-module testers carry INFO_* "quote vs measured charge" blocks (e.g. modules/ATS.repl:2066 <<ATS-I1>>). [V-cmd]
M18 #42M	U_RS::UEV_EnforceReserved over-blocks legitimate non-principal names	MED	**NOT A BUG**	Verbatim port of Kadena's own coin.pact; zero callers here. Pinned anyway: REPL/_scratch_urs_m18_overblock.repl, a gate entrypoint. [V-cmd]
M19 #43M	Several U_LST/U_VST UC_* functions perform enforce	MED	**DEFERRED to StoicSyntax sweep**	Sweep landed; the v-specialisation (UCv_/URCv_) now formalises the legitimate cases and renames the rest. [V-cmd]
M20 #44M	UC_IzUnique can never return false — its own comment	MED	**FIXED**	05_U_LST.pact:222 UEV_IzUnique, doc rewritten to state

id	summary	severity	verdict	evidence today
	promises a two-valued predicate that never existed			the true contract. Zero UC_IzUnique code references remain. [V-cmd] **Witnessed**: REPL/_scratch_ulst_m20_uev_izunique.repl, a gate entrypoint. [V-cmd]
M21 #45M	UC_MaxInteger crashes *uncatchably* on an empty list; reachable unguarded through a real DPDC-S client entrypoint	MED	**FIXED**	06_U_INT.pact:193-194 — renamed UEV_MaxInteger with (enforce (> (length lst) 0) ...) first. [V-read] **Witnessed**: modules/UTILITIES.repl <<UTIL-11>> asserts both the value and the refusal message. [V-read]
M22 #46M	UEV_ContainsAll checks set membership, not multiset containment	MED	**NOT A BUG**	Sole live caller is DPMF, permanently out of scope. [REPORTED]
M23 #47M	Zero expect-failure assertions anywhere in CODEX/PYTHIA suites	MED	**DEFERRED to REPL phase**	Closed: modules/PYTHIA.repl, modules/CODEX.repl exist with negative assertions. [V-cmd]
M24 #48M	Both suites commented out of the default pipeline	MED	**DEFERRED to REPL phase**	Still commented in Stage01_Tester.repl:62 and :64, but both run under ZALL.repl:54-55 and their own module testers. [V-cmd]
M25 #49M	The PYTHIA flush-gas-probe REPL is broken — batch sizes exceed the module's own cap	MED	**DEFERRED to REPL phase**	[INFERRED — not re-checked]
M26 #50M	INTERFACE_VERSIONING.md doesn't document the additive/opt-in dual-implementation pattern	MED	**DEFERRED to StoicSyntax sweep**	[INFERRED]
M27 #51M	MODULE-INDEX.md's "latest: X" points at	MED	**DEFERRED to StoicSyntax sweep**	Superseded — the frozen-interface

id	summary	severity	verdict	evidence today
	frozen, never-deployed interfaces			convention was abandoned entirely; all three Stage-1 registry files now declare zero interfaces. [REPORTED — DEFECT-LEDGER.md §8.6]
N2	TS01-C1::DPTF|C_DeployAccount/DPOF|C_DeployAccount ungated — any signer could force any account to associate with any token	MED-equiv	**FIXED**	3_Talos/02_TS01-C1.pact:710 — (ref-DALOS::CAP_EnforceAccountOwnership account); new admin path DPTF|A_DeployAccount/DPOF|A_DeployAccount at 01_TS01-A.pact:574, :601 behind P|ADMINISTRATIVE-SUMMONER. [V-read]
N3	Three TS01-A treasury admin functions gated only by bare-true P|TS	CRIT?	**REFUTED**	Core layer independently composes GOV|DPTF_ADMIN. Pinned: REPL/_scratch_ts01a_n3_treasury_gate_check.repl, a gate entrypoint with one negative assertion. [V-cmd]

LOW

id	summary	verdict	evidence today
#52L	URD_AccountCounter — dead, mis-sectioned	DEFERRED → gone	Zero references tree-wide; the function no longer exists. [V-cmd]
#53L	A_UpdateUsagePrice has no bound check on new-price	**FIXED**	01_DALOS.pact:1783. [V-read]
#54L	Self-referential ref-DALOS:: call; hardcoded-account migration tool	NOT A BUG	Same verdict as M9. [REPORTED]
#55L	[6.1]_Cumulator.repl has zero assertions, never invokes C_Collect; [6.4]_Admin comments out C_RotateKadena	DEFERRED → closed	REPL/modules/CUMULATOR.repl exists; [6.1]_Cumulator.repl carries the 75 leg-level pricing assertions CLAUDE.md now cites. [V-cmd]
#56L	Two vestigial ELITE boilerplate items	**FIXED**	07_ELITE.pact:84 tombstone; zero references. (Note: a separate GOV|CollectiblesKey survives at

id	summary	verdict	evidence today
			06_DPOF.pact:438 — out of #56L's stated scope.) [V-cmd]
#57L	Stale "DPMF" naming in Talos docs/REPL labels	DEFERRED to sweep	[INFERRED]
#58L	Dead account-ea-supply binding in C_ClearDispo	**FIXED**	09_TFT.pact:1626 tombstone comment. [V-cmd]
#59L	DPTF/TFT REPL coverage gaps	DEFERRED → largely closed	modules/DPTF.repl carries <<DPTF-G7>> etc. [V-cmd]
#60L	DPOF's dedicated REPL has zero expect/expect-failure	DEFERRED → closed	modules/DPOF.repl has 30+ negative assertions. [V-cmd]
#61L	Dead total-ouro binding in OUROBOROS::C_Compress	**FIXED**	13_OUROBOROS.pact:694 tombstone. [V-cmd]
#62L	LIQUID::UEV_Amount defined, never called	DEFERRED to red-team	**Still open.** 12_LIQUID.pact:472 — exactly one occurrence in the file, the definition. [V-cmd]
#63L	Native KDA install-capability supplied by off-chain code, untestable in the Pact REPL	NOT A BUG	Documented architectural limitation. [REPORTED]
#64L	Broad coverage gaps across VST/LIQUID/OUROBOROS	DEFERRED → closed	modules/VST.repl, modules/LIQUID.repl, modules/OUROBOROS.repl all exist. [V-cmd]
#65L	UC_LpFuelToLpStrings (a UC_) contains a raw enforce	DEFERRED to sweep	[INFERRED]
#66L	UCX_*/UCXX_* pre-migration spelling	DEFERRED → closed	Renamed to UCx_ (e.g. 13_U_BFS.pact:382 UCx_GraphNodeLinks). [V-cmd]
#67L	VST|INFO-HibernatedNonce(s)Display hyphenated, undeclared, zero callers	DEFERRED to INFO project	Superseded by the rewrite. [INFERRED]
#68L	Dead constant PYTHIA|FLUSH-GAS-TARGET	**FIXED**	22_PYTHIA.pact:410 tombstone; zero references. [V-cmd]
#69L	UR_AWT|ListByCodex scans but is named UR_	DEFERRED to sweep	[INFERRED]
#70L	CODEX defcap body-order deviations	DEFERRED to sweep	[INFERRED]
#71L	PYTHIA price-setters acquire SECURE inline	DEFERRED to sweep	[INFERRED]
#72L	Stale header on [6.10b]_PYTHIA-ledger-v2.repl	**FIXED**	[REPORTED]
#73L	Tautological or in CT_DPTF-	**FIXED**	01_U_CT.pact:244-247 —

id	summary	verdict	evidence today
	FeeLock		single named condition. [V-read]
#74L	Typos in enforce messages	**FIXED (partial, by design)**	11_U_VST.pact:170 tombstone. “succesfully” deliberately left — it is the codebase’s consistent spelling: **133** occurrences in STAGE_01/**/*.pact, **156** across all production Pact. [V-cmd] (**Corrected 2026-09-18** — this said “119-occurrence” without naming a scope, and no scope reproduces 119.)
#75L	UEV_StringPresence’s [bar] sentinel doesn’t cover a real empty list	**FIXED**	05_U_LST.pact:262 — (and (!= item-lst [bar]) (UC_IsNotEmpty item-lst)). **Witnessed**: REPL/_scratch_ulst_75l_stringpresence_empty.repl, gate entrypoint. [V-read]
#76L	Self-referential module-ref style in UC_NonceSplitter	DEFERRED to sweep	[INFERRED]
#77L	UR|KDA-PID section-placement mismatch	DEFERRED to sweep	Now sits under {5.3} Read in 01_U_CT.pact:361-364. [V-cmd]
#78L	Off-by-one in UC_Search’s enumerate	DEFERRED to sweep	Standing instruction: never touch U|LST::UC_Search. [REPORTED]
#79L	[1]_Utilities.repl has zero assertions for any of 8 in-scope modules	DEFERRED → closed	REPL/modules/UTILITIES.repl carries <<UTIL-01>>…<<UTIL-11>>+. [V-cmd]
#80L	Talos wrapper naming drift vs core counterparts	DEFERRED to sweep	[INFERRED]
#81L	Misnamed GOV|MD_DPTF constant in 04_BRD.pact	DEFERRED to sweep	[INFERRED]
#82L	~12 client wrappers missing the mandated format result string	DEFERRED to sweep	[INFERRED]
#83L	Dead/orphaned frozen interfaces	NOT A BUG	Superseded — the frozen-copy convention was abandoned. [REPORTED — §8.6]
#84L	Inconsistent “Frozen —” @doc labelling	DEFERRED to sweep	Moot with the convention. [INFERRED]
#85L	[0.1]_Interfaces.repl is a smoke test by design	NOT A BUG	Still true, and now *says* it is. [REPORTED — §8.6]

Plus one non-finding: Fix #7 is an owner-requested IGNIS Compress/Prime gas optimisation, not a Round-I item. It is present: 02_IGNIS.pact:1510 UC_FindKeyIndex, used at :1206 and :1268. [V-read] The audit measured 35 gas

saved on an 8-leg batch and 85 on a 20-leg batch [REPORTED], and proved byte-identical output against retained _OLD copies before deleting them.

1.4 Six findings worth the telling

(a) C2 — the revoke that revoked the wrong account, and why one test could never catch it

DPOF::C_MoveCreateRole is supposed to be one atomic grant-and-revoke: the new holder gets the create/mint role, the old holder loses it. It called XI_UpdateVerum4 (write the new holder of record) and then XI_SwitchCreateRole (clear the old holder's flag, set the new one's). XI_SwitchCreateRole derives "who is the old holder" *itself*, internally, via (UR_Verum4 id).

By the time it ran, that read already returned the **new** holder. Both of its writes landed on the same row — set false, then immediately true, a no-op — and the real previous holder was never touched. Every past holder of a DPOF's create role kept mint access forever.

The interesting part is not the bug; it is why it survived. The master defcap DPOF|S>X_SWITCH-CREATE-ROLE already computed the correct previous holder from pre-write state. The correct value existed. It was simply never threaded through to the writer, which re-derived its own, wrong copy. A reviewer reading the defcap would conclude the logic was right.

And **a single move cannot detect it.** After one move the previous holder is the token's owner, who retains the role for an unrelated reason (UR_R-Create has an account == owner-konto OR-clause that masks the result). It takes *two* consecutive moves, so that the second one's previous holder is a third party with nothing else granting them the role. The audit's own harness did exactly that — emma (owner) → aoz, then aoz → patron — and that structure is preserved in the live pin today, REPL/modules/DPOF.repl:272, which explains the reasoning in its own comment before making the assertion:

> *"A single move cannot detect this — after one move the previous holder IS the owner, who keeps the role for other reasons. It takes TWO consecutive moves..."*

The fix was a reorder of two lines. 06_DPOF.pact:3093 and :3095, under the comment at :3088-3092 that names #2C. [V-read]

(b) N1 — the bug found while building the proof for a different bug

While constructing the REPL proof for C3 (nonce uniqueness), the audit needed a *working* C_Transmit call to attack with duplicated nonces. The very first ordinary, non-duplicated call crashed: Key "meta-data" not found.

DPOF|C>TRANSMIT's defcap read (at "meta-data" td). The TransmitData schema's real field is meta-data-array — as both the defschema and the constructor UDCX_TransmitData agree. Every call to C_Transmit, from the core or through the Talos wrapper, had crashed unconditionally, for any input, since it was written.

This is the finding class that no amount of reading produces, because the code reads correctly at every level *except* the one string. It was fixed as a one-string change (06_DPOF.pact:1027 [V-read]) and logged as its own numbered finding rather than folded into C3's write-up — the right call, since the two are causally unrelated.

The general lesson the audit drew, and it is worth keeping: **building the attack harness for one finding is itself an audit technique**, because it forces you to execute the happy path first.

(c) M5 — ranked MEDIUM, and it was an exploit

The finding as written was a code-organisation complaint: TFT::C_MultiTransfer and C_MultiBulkTransfer compute dispo-data once, before the fold over legs, and reuse the snapshot for every leg's debit check.

dispo-data captures the sender's current Elite-Auryn holdings. UC_OuroDispo — the maximum permitted OURO overdraft — is directly proportional to that. So a *single batch* containing both an Elite-Auryn-reducing leg and an OURO-overdraft leg gets its overdraft checked against the sender's **pre-batch** holdings, regardless of leg order (the snapshot precedes everything either way).

The audit re-verified it live per the owner's standing instruction and reported the numbers:

```
patron EA before: 114907.2904 / half transferred away: 57453.6452
max overdraft under STALE dispo:    44446.6688...
max overdraft under CORRECT dispo:  22223.3344...
chosen overdraft amount: 33335.0016
BATCH SUCCEEDED. patron's OURO balance after: -33335.0016
```

[REPORTED — ROUND-02-FIXES.md Fix #24] That is a ~50% inflation of a credit line, in one transaction, with no special privileges. The fix entry opens with an explicit severity correction — *"this finding turned out more serious than its MEDIUM rank suggests... Flagging this explicitly since the ranked tier undersells it"* — which is the correct behaviour and worth naming, because the alternative (quietly fixing it at its original rank) leaves the ranking looking calibrated when it was not.

The fix is present: 09_TFT.pact:1797-1803 and :1867-1869 recompute per leg. [V-read] **The staleness itself has no witness in the running suite** — see §1.5.

(d) M7 — the fix that moved the problem instead of removing it, and said so

DPOF::URC_Parent contained (enforce (!= fourth BAR) "Sleeping LP Tokens not allowed..."). A URC_ contractually never enforces, and this one made *every* caller pay for a rejection only one of them needed. The fix relocated the enforce to UEV_ParentOwnership, the one caller whose own @doc already claimed it.

That is where a normal audit entry would stop. This one did not. The enforce was moved *into* a function whose own let bound parent by calling URCv_Parent — the function it had just been moved *out of* — before the enforce could run, and the in-source note said exactly that, in those words, marking the guard **UNPINNED** and shielded.

> **CORRECTED 2026-09-18.** This section used to quote that note from 06_DPOF.pact:2030-2042 and conclude that reaching the enforce "requires a real issued sleeping-LP token, which no suite creates". The quote no longer resolves and the conclusion was too strong: the source was rewritten on 2026-09-17 after measurement showed the shadow was **partial**, not total.

The measurement is the part worth keeping. URCv_Parent reads the properties table only for ids prefixed V|, Z| or H|; every other shape falls through to its default with **no read at all**. So Z|X|NOSUCH-... did abort in that read exactly as the note claimed — while ABC|NOSUCH-... reached the enforce and produced its written message. Half the input space had always reached the guard, and nothing had ever tried it. An unreachability claim asserted from reading rather than from execution is the kind of claim that is wrong in one direction only: it stops anyone from writing the test.

fourth is a pure string slice of id and needs no table access, so the repair was to hoist the enforce above the binding. It is now the first form in the guard's let body, 06_DPOF.pact:2044, sitting above the let that binds parent, under a note at :2025-2043 that records the old claim and why it was half wrong. Both shapes are pinned by REPL/modules/DPOF.repl:2441 <<DPOF-G16>> — asserted *separately*, because they used to differ: revert the hoist and the Z| assertion goes red while the other stays green, which localises the regression to the line that caused it. A third assertion (:2451) keeps the pair non-vacuous, an id that gets past this guard and fails later, elsewhere. [V-read]

The general observation survives its own correction. This is the eager-let pathology the book's Rule 1 exists for, appearing in a *fix* rather than in original code; and the code documenting the incompleteness of its own repair, at the site, is what made the repair findable at all — the note is the only reason anyone went and measured it.

(e) H14 — the stub that was fixed, then quietly re-denominated

The finding: U_CT::UR|KDA-PID, the sole KDA/USD price feed used across asymmetric-LP IGNIS taxation and DemiPad pricing, is a hardcoded 1.0 with the real dia-oracle.get-value "KDA/USD" call commented out. Every dependent computation silently assumed permanent KDA/USD parity.

The owner's ruling was pragmatic: no oracle exists yet, set it to 0.1 (mainnet's approximate KDA price) and defer the real wiring. Fix #17 did that.

Today the function is UR_STOA-PID|Price at 01_U_CT.pact:364, it returns 0.1, and the commented oracle call above it reads "STOA/USD". [V-read] The rename came from commit 0b0ad31, *"Rename KDA/Kadena -> STOA across all production code (native chain is STOA)"* [V-cmd], and it is architecturally correct — the native chain token really is STOA, not KDA.

But **the number did not change when the denomination did.** 0.1 was chosen, on the record, as "mainnet's approximate KDA/USD price". It is now serving as the STOA/USD price. Whether those two quantities happen to coincide is not something this chapter can establish, and nothing in the tree asserts that they do. [INFERRED — the risk; V-read — the value and the label]

The finding's substance is in any case still open: it is a stub, and #58L in the sibling SWP audit recorded the same value fanning out to *"308 occurrences across 17 files"* at audit time, under the old KDA-PID name. After the rename the tree holds **41** references to UR_STOA-PID|Price in 11 .pact files, and **120** STOA-PID tokens across 15; KDA-PID survives only in the non-production 0_Sample/ files. [V-cmd] See §1.5.

> **Corrected 2026-09-18.** This wrote the id as #L58 and quoted the 308/17 figure in the present tense, as if it described the tree today. It describes the tree in 2026-08, before the rename.

(f) H4 — the escape hatches were checked before "different from its siblings" became "bug"

Three of the five Toggle-Verum client recipes in 05_DPTF.pact lacked (UEV_IMC). Two had it. The tempting inference — *inconsistent, therefore broken* — is exactly the inference that had already produced one refuted finding in this same audit (H1, where a missing SECURE wrapper turned out to be structural rather than authorising).

So the audit enumerated the two ways it could be intentional, and killed both before proposing a fix: (1) *maybe UEV_IMC belongs inside the capability instead* — checked: none of the five DPTF|C>TOGGLE-* defcaps reference it, consistently; (2) *maybe these three are meant to be called from within DPTF itself, where UEV_IMC would be wrong* — checked by repo-wide grep: zero internal callers, the only callers anywhere are cross-module ref-DPTF:: calls from the Talos wrapper, exactly like the two correct siblings. Only then: *"it seems I simply forgot to add the line."*

The consequence was real. Neither DPTF nor TFT independently checks the Global Administrative Pause, and IGNIS billing happens only in the Talos wrapper. A direct core call, signed by the token owner alone, bypassed all of it — no pause check, no gas, nothing but CAP_Owner. Two of the three were reproduced live; the third was confirmed by byte-identical code structure, and the report says so rather than implying three live reproductions. [REPORTED]

(P|UEV_IMC) is present on all three today: 05_DPTF.pact:3051, 3073, 3096. [V-read]

1.5 What remains open

Findings the audit left open on purpose

item	state today	why
M10 #34M — two dead capabilities in TS01-A, one the only path to a	**Open.** P|GOVERNING-SUMMONER (01_TS01-A.pact:147) and P|SECURE-	Owner: *"if it were needed it would already be used."* Deferred to the

item	state today	why
governor slot on DALOS's own vault	SUMMONER (:151) still have zero callers; P|TRG (:139) is composed only by the former and is registered as a capability guard at :242. [V-cmd]	red-team pass to confirm empirically that DALOS's governor resolves without this path. The red-team pass ran (REPL/RedTeam/), but this specific question does not appear to have been put to it. [INFERRED]
#62L — LIQUID::UEV_Amount defined, zero callers	**Open.** Exactly one occurrence in 12_LIQUID.pact, the definition at :472. [V-cmd]	Deferred to the same red-team pass; the question (does DPTF::C_Mint/TFT::C_Transfer already enforce the same precision check via each token's own setting?) is unanswered.
H14 **#19H** — no live price oracle	**Open by design.** 01_U_CT.pact:364 returns a hardcoded 0.1; the real oracle call is commented one line above. [V-read]	Blocked on an oracle existing. Now carries the additional, unrecorded question of whether 0.1 is still the right number after the KDA→STOA re-denomination.
C3(weighted)-class residues — n/a to this tree	—	—

Findings closed as DESIGN or NOT A BUG that a reader should still know about

- **C4** — VST::C_Unreserve can only be called by the token manager, never the reserver. This is correct *given* that Reserve/Unreserve is escrow-for-purchase rather than a symmetric lock, but it means a reserver has no unilateral exit. That is a trust assumption, not a bug, and it should be visible to anyone integrating against VST.
- **M3** — the gas station's allowlist matches by **string prefix**. Safe today because namespace deploy governance controls who can create a TS-prefixed module. It is safe by governance, not by construction.
- **H5** — a zero-amount leg aborts an entire multi-transfer batch, deliberately. Integrators who build batches programmatically need to filter their own zero legs.
- **M22 / M12 / M13 / C1** — four findings closed as "DPMF is permanently out of scope". DPMF is dead code by design, but it is *loadable* dead code sitting in the deploy order; the closure rests on the promise that it will never be upgraded.

Seven fixes that were present in source with no witness — all seven now closed

This is the book's Rule 2 applied to this chapter, and on 2026-09-17 it was where the DALOS tree was weakest. The audit's proofs were built as REPL/_scratch_*.repl harnesses. **Six of them were later promoted to gate entrypoints**, listed by name in REPL/tools/_gate.py's SCRATCH_PROOFS and in REPL/regressions/MANIFEST.md. [V-cmd] The rest were moved to REPL/archive/, a directory whose own README states: *"Files here contain no assertions and are referenced by nothing… evidence of past investigation, not tests."* [V-cmd]

Seven DALOS fixes were left with **no assertion anywhere in the running suite** that would fail if the fix were reverted — checked by searching REPL/Stage_01, REPL/Stage_02, REPL/modules, REPL/RedTeam and the gate's own entrypoint lists. Each now has one, named below.

> **CORRECTED 2026-09-18.** Every row of this table used to read *"None found"*, and the paragraph under it called these seven *"the remainder"* of the failure mode. All seven were closed on 2026-09-17, while this book was being assembled, together with four in Chapter 6; DEFECT-LEDGER.md §8.26 records the set of eleven. What each witness had to do — and why a first attempt passed while proving nothing in four of them — is set out in the Part I introduction.

fix	in source?	witness today
C3 #3C — DPOF nonce-uniqueness on C>DEBIT/C>TRANSFER/C>BULK-TRANSFER	Yes, 06_DPOF.pact:908, 1046, 1080 [V-read]	<<DPOF-G15>>, REPL/modules/DPOF.repl:2394, :2398, :2402 — a duplicated **nonce** list to each of the three capabilities separately. UEV_IzUnique's message was already asserted at RedTeam/[RT-H]_InputDomain.repl:138 and Stage_01/[6.3]_SWP.repl:4767, but both are *other* call sites (swap input ids, pool tokens): a message shared across call sites says nothing about the site you care about. [V-cmd]
H3 #8H — IGNIS C_Collect per-leg zero filter	Yes, 02_IGNIS.pact:1896-1901 [V-read]	<<CUM-G8>>, REPL/modules/CUMULATOR.repl:139, :143, :147 — a free leg beside a billable one; the bundle settles, IGNIS|S>FREE is asserted to have fired, and the two legs are asserted **not** compressed into one. That last assertion is what makes it a test: two legs on one interactor merge, and the fix's branch never runs. [V-cmd]
M5 #29M — per-leg dispo-data recomputation	Yes, 09_TFT.pact:1797, 1867 [V-read]	<<DPTF-G14>>, REPL/modules/DPTF.repl:3006, against a control at :2971 — the same overdraft, with and without an Elite-Auryn-draining leg in front of it. The halves sit in **separate transactions** because expect-failure does not roll back REPL writes. <<DPTF-G7>> (:1089) pins the OURO dispo *floor* on a wipe, a different guard. [V-cmd]
M6 #30M — UR_Hibernation as a pure getter	Yes, 05_DPTF.pact:1212 [V-read]	<<M6>>, _scratch_dptf_m6_urhibernation_purefetch.repl:49, :53, :56 — the return value cannot change, so the witness is **gas**: the reader is measured against UR_Sleeping, its pure-read sibling on the same row, and a reinstated table write cannot fit inside the margin. [V-cmd]
M7 #31M — relocated Sleeping-LP enforce	Yes, hoisted to 06_DPOF.pact:2044 [V-read]	<<DPOF-G16>>, REPL/modules/DPOF.repl:2441, :2445, :2451. The source's "UNPINNED and unreachable" note turned out to describe a **partial** shadow — see §1.4(d). [V-read]
N2 — C_DeployAccount ownership gate	Yes, 02_TS01-C1.pact:710 [V-read]	<<DPTF-G12>>, REPL/modules/DPTF.repl:2895, :2910 —

fix	in source?	witness today
		the **Talos wrapper**, driven by a payer who is not the named account, with the account holder's signature added back as the control. modules/CONFORMANCE.repl <<CONF-01>> pins P|UEV_IMC on the *core* function and would have stayed green if the ownership line were deleted. [V-cmd]
M1 **#25M** — C_RotateStoa ledger cleanup	Yes, 01_DALOS.pact:1874 [V-read]	<<DALOS-G7>>, REPL/modules/DALOS-ADMIN.repl:1238-1251 — it reads UR_StoaLedger, which no .repl in the tree had ever read. The assertion that carries the fix is *gone from the old ledger*; *present in the new* holds with or without it. [V-cmd]

The witnessed list from 2026-09-17 therefore grows by seven. Witnessed before that date, with the assertion named for each: C2 (<<DPOF-MCR>>), H2 (_scratch_udalos_h1_msdc.repl, gated), H9 and M21 and #75L (<<UTIL-11>>), H12 (<<PYTHIA-PRICE>>), M20 (_scratch_ulst_m20_uev_izunique.repl, gated), M18 and N3 (gated scratch proofs), H4 partially (<<CONF-01>> on a sibling).

REPL/modules/PYTHIA.repl states the general problem in its own header better than this chapter can:

> *"PYTHIA\|A_UpdateDeployPrice and PYTHIA\|A_UpdateRenamePrice existed ONLY in _scratch_pythia_h12_price_wiring.repl — a file the gate does not run.* ***The ledger counted them as exercised; nothing re-executed them.****"* [V-read]

That is the failure mode, stated by the repository about itself. The seven rows above were the remainder of it in this tree. Closing them is worth reading as a set, because **not one was closed by simply calling the function** — each needed a specific input, fixture or measurement that tells the fixed code apart from the reverted code.

Two fixes whose target no longer exists

Neither is a regression, but both should be stated plainly rather than ticked as "fixed":

- **C5** and **H10** were fixes inside 2_Core/21_INFO-ONE+.pact. That module was retired (commit 42fb75d) and its function family rewritten. The current equivalents (Z_Reads/02_INFO-ONE+.pact:1198, :2220) do not contain the fixed lines because they no longer contain hand-assembled cost legs at all — they delegate to URCi_* readers. [V-read] The defect *class* is gone, which is a stronger outcome than the fix. But a reader auditing the audit would find the cited lines absent, and should know why.
- **H11** was *deferred*, not fixed, and is now moot for the same reason: the duplicate ifp3 binding cannot exist in a function that makes one URCi_ColdRecovery call. [V-read]

Corrections to the audit's own record

- The DALOS README's status tracker is accurate on every row this chapter re-checked. No discrepancies found between the tracker, ISSUES-RANKED.md and ROUND-02-FIXES.md. [V-cmd]
- The README's claim that #56L removed "two vestigial boilerplate items copied from the module sample template" is true for 07_ELITE.pact. A GOV\|CollectiblesKey defun referencing the same unrelated "dpdc-keyset" string still exists in 06_DPOF.pact:438. [V-cmd] That is outside the finding's stated scope (which named ELITE), so it is not a broken fix — but it is the same template artefact, in a second module, unremoved.

6. ATS — autostake pools

Source tree: Audit/module-audits/ATS/ (3,296 lines across 7 files, plus a .docx rendering of the report). Audit ran 2026-08-16 → 2026-08-19. This chapter was written 2026-09-17 and re-checks every recorded fix against the tree as it stands today.

Evidence labels are the same as Chapter 5: **[V-cmd]** (established by running a read-only command), **[V-read]** (established by reading the cited source), **[INFERRED]**, **[REPORTED]** (quoted from the audit trail or the ledger, not re-established here). **No test was executed for this chapter.**

2.1 What the module does

ATS is Ouronet's staking engine. A user deposits *reward tokens* (RT) into a pool — an **ATS-pair** — and receives a *reward-bearing token* (RBT) that represents their claim on the pool. The RBT is not a receipt for a fixed amount; it is a share. As the pool owner **fuels** the pool with more reward tokens, the pool's *index* (reserves ÷ RBT supply) rises, and every existing RBT becomes redeemable for more RT than it was minted against. That is the yield.

Getting value back out has three doors, and they are the reason the module is large:

- **Cold recovery** — the ordinary unbonding path. You burn RBT, your claim is recorded into a numbered *position* (P0 through P7) on a per-account ledger row, a cull-time is stamped on it, and after the wait you **cull** it and receive the tokens. The wait length is a table indexed by your Elite tier, built so higher tiers wait less.
- **Hot recovery** — you receive an NFT-like DPOF nonce instead of a timer. It can be traded, branded, repurposed, or **redeemed** early for a decaying fee.
- **Direct recovery** — instant exit at an owner-configurable fee.

Around this sit the pool owner's levers: royalty rate, syphon floor, hibernation fees, cold-recovery duration curves, fee-threshold ladders, an elite-mode toggle, a parameter lock, and KickStart — the genesis operation that sets a virgin pool's initial index by declaring how much RT backs how much RBT.

What breaks if ATS breaks. Two things, and they are different in kind. The first is direct: positions are stored as a *positional array* of per-reward-token amounts, aligned by index against the pool's live reward-token list. If that alignment ever drifts, a staker's claim is paid in the wrong token — not lost, *misattributed*, which is worse because the ledger still balances. The second is indirect: KickStart sets the index a virgin pool starts at, and every later depositor's mint is floor(rt_amount / index, precision). An unbounded genesis index is the classic vault inflation-attack setup, where a later depositor's real tokens are credited in full to the pool while their own RBT rounds down to a negligible fraction of fair value.

Both of those are in this chapter. The first is C2. The second is M2.

2.2 How it was audited

Round I — four parallel lenses, read-only, plus a hand-run lead pass on the priority target. The scope was split by role rather than by file: admin/lifecycle, usage/token-custody, utility maths, and Talos wiring + interfaces. Each lens was briefed with the same contract as the sibling AQP audit — load StoicSyntax.md first, work read-only, **assume nothing is correct despite being live on mainnet**. [REPORTED — ATS/README.md § Method]

One thing was done differently here and it is the methodological high point of the tree. **C2, the reward-token remove/re-add mechanic, was the audit's declared priority target going in**, and the lead verified it end-to-end by hand across 08_ATS.pact + 10_ATSU.pact + 09_U_ATS.pact **before reading any lens result**. Two lenses then

independently converged on the same defect from different directions — ATSU's X_RemoveSecondary precondition gap, and U_ATS's UC_ReshapeUnstakeObject gating bug — giving three independent derivations of one root cause. [REPORTED]

VST was declared an explicit **boundary**, not an audited module: ATS|C_VestedCoil/VestedCurl call into it, but its internal correctness was out of scope. The tree says so up front, which is the right way to scope an audit.

Round I — the C1 correction, and what it taught. The Talos lens reported, as CRITICAL, that ATS|GOV — a (defcap () true) wired as the Autostake vault's governor guard — was forgeable by any caller, allowing a full drain. It claimed an *empirical reproduction* in an isolated two-module Pact 5.4 REPL.

The owner refuted it: Pact requires a foreign caller to already hold the target module's own admin before with-capability will grant that module's capabilities. The lens's repro had not isolated the claimed threat model — it ran without error, but it did not test the thing it said it tested. The audit re-verified the refutation independently with a fresh two-module repro using a foreign module holding zero admin, and recorded the conclusion in its own method section:

> *"This is a reminder that even an 'empirically reproduced' finding needs its repro checked for whether it actually isolates the claimed threat model, not just whether it runs without error."* [REPORTED — ATS/README.md § Method, "Correction, 2026-08-16"]

The correction also *narrowed* the rest of the audit usefully. ATS|GOV is the documented "Simple vault" pattern (StoicSyntax.md §14.5) and four sibling modules use it identically. The residual question — *does any of this module's own public functions compose its GOV cap without an ownership check first?* — is a per-module reading task, and it is exactly what C5 turned out to be.

Round II — 18 sequential fixes. Each owner-green-lit, each with a diff and a proof. The discipline is visible in the fix entries and is worth naming because it repeatedly changed the answer:

- **Verify under correct preconditions before touching code.** For C3, the owner suspected the dead REPL call might be a benign "nothing to redeem" state. Three escalating checks followed; the second one *failed for a different reason* (a fixture whose block-time was set two years before the nonce's mint time), which would have been reported as confirmation by a less careful pass. The third, under honestly forward-moving time, hit the real defect at the predicted line. [REPORTED]
- **Own the process failures in the record.** Fix #12's entry opens: *"a first pass at this fix was applied **without owner authorization** — the owner had only said 'let's do next'… Reverted immediately in full once caught."* [REPORTED]
- **Two fixes had no write-up until the audit audited itself.** Fixes #13, #14 and #17 were applied to code during the session but never logged; they were caught while re-auditing the tally in a fresh worktree and back-filled, each with a note saying so and a re-verification that the code fix was genuinely present before the entry was written. [REPORTED]

Round III was never created. The cycle table lists ROUND-03-REVERIFY.md as *"not yet created"*; it does not exist. [V-cmd] Unlike the SWP tree, no later systematic re-verify pass covered ATS either. This chapter is, as far as can be determined, the first.

Was the audit thin anywhere? The Round-I coverage of the Hot-RBT surface and ~12 configuration functions was zero — the audit says so as finding L4/#22L, and Fix #15 closed it by writing nine new transaction groups. That is the correct response. But note what that implies about Round I itself: for a dozen owner-facing functions, the Round-I verdict rested entirely on reading, because there was nothing to run. Two real bugs (N2 and N3) fell out the moment tests were actually written — see §2.4.

2.3 The findings

35 findings: #1C–#31L from Round I, plus #32N–#35N appended after the original list was published (deliberately appended rather than renumbered, so the original numbering never shifts). Severity is as recorded. The audit uses two ID schemes in parallel — a ranked #nnX and a per-severity C1/H2/M6/L9/N1 — and this chapter gives both, because the tree's own cross-references use whichever was to hand.

CRITICAL

id	summary	severity	verdict	evidence today
#1C / C1	ATS|GOV is (defcap () true) and is the vault's governor guard — claimed forgeable, full drain	CRIT	**REFUTED**	Pact requires foreign module-admin first. The pattern is intact and still used: 08_ATS.pact ATS|GOV, plus VST|GOV, LIQUID|GOV, ORBR|GOV, SWP|GOV. [REPORTED + V-cmd]
#2C / C2	Reward-token remove-then-re-add corrupts every pre-existing position's token attribution — **three** sub-mechanisms: (a) cull pays the wrong token, (b) royalty permanently stranded, (c) cold recovery permanently unusable for existing accounts	CRIT	**FIXED**	(a)+(c): 1_Utilities/09_U_ATS.pact:394-404 — UC_ReshapeUnstakeObject now calls UCv_SolidifyUnstakeObject unconditionally, comment names #1C / C2c. (b): 2_Core/10_ATSU.pact:1530-1596 — XI_RemoveSecondary derives the account list on-chain via URH_ExistingAutostakePairs and migrates RUR bucket 3 (royalty) alongside 1 and 2. [V-read] **No witness in the running suite** — see §2.5.
#3C / C3	ATSU::C_Redeem passes a :decimal where Pact's if requires :bool — **every** call reverts; the only exit from hot recovery is permanently dead	CRIT	**FIXED**	10_ATSU.pact:2117 — (have-fee-rts:bool (!= are-fee-rts 0.0)), with a comment naming #3C / C3; used at :2131. A second instance exists correctly at :1162. [V-read] **Witnessed**: REPL/Stage_01/[6.6]_ATS.repl:524 (early

id	summary	severity	verdict	evidence today
				redeem withheld a real fee — paid *strictly less* than full value) and :611 (matured redeem paid *exactly* full value). [V-read]
#4C / C4	syphon floor has no monotonicity, lock or timelock — the owner can re-lower it and extract ~95% + of pool backing in one call	CRIT	**NOT A BUG**	Owner: full at-will discretionary control (bounded >= 0.1) is intended; stakers trust the pool owner with this lever. Proposed ratchet explicitly rejected. [REPORTED]
#5C / C5	C_HOT-RBT|UpdatePendingBranding /UpgradeBranding have **no** owner or entity-linkage check at all	CRIT	**FIXED**	08_ATS.pact:736 — new unevented core ATS|C>HOT-RBT-BRD resolves the owning pair from the hot-rbt id, CAP_Owner, then composes ATS|GOV; :746 and :750 are the two @event leaves; used at :2993 and :3004. [V-read]

HIGH

id	summary	severity	verdict	evidence today
#6H / H1	Parameter-lock protects fee-schedule config but not royalty, syphon, hibernation fees, ownership rotation or the recovery switches	HIGH	**FIXED / CLOSED** (2 of 8 fields; 6 confirmed intentionally exempt)	08_ATS.pact:638 and :660 — (UEV_ParameterLockState atspair false) is now the first statement of ATS|S>SET-HIBERNATION-FEES and ATS|S>ROYALTY, each with a comment naming #6H / H1. [V-read] **No witness** — see §2.5.
#7H / H2	Royalty ceiling is 99.9%, applies instantly, no lock or delta cap	HIGH	**FIXED**	08_ATS.pact:662 — (enforce (<= royalty 500.0) "Royalty cannot exceed 500.0 promile (50%)"), layered over the shared UEV_Fee. [V-read]

id	summary	severity	verdict	evidence today
				Witnessed: [6.6]_ATS.repl:3965 (999.0 rejected), :3969 (500.0001 rejected), :3978 (500.0 accepted), :3987/:3993 (both off-sentinels still work). [V-read]
#8H / H3	URC_RBT's abs() masks the -1.0 "uninitialised index" sentinel — Coil/Curl bypass KickStart, genesis inflation / zero-mint donation	HIGH	**NOT A BUG** (both scenarios)	Scenario 1: bare-Coil bootstrap is the intended alternative to KickStart. Scenario 2: refuted on tracing — DPTF|C>CREDIT's UEV_Amount reverts a 0.0 mint atomically, so no silent-donation window exists. [REPORTED]
#9H / H4	UEV_ColdDurationParameters' soft branch calls enforce with 3 arguments — soft cold-recovery duration can never be set post-genesis	HIGH	**FIXED**	1_Utilities/09_U_ATS.pact:730-758 — one correctly-formed 2-arg enforce per branch; the @doc names both #9H / H4 and #16M / M7. [V-read] **Witnessed**: REPL/modules/UTILITIES.repl:158-163 <<UTIL-03>> — soft branch accepted with valid params, and refused with a message that *names the values*, asserted as a pair against the hard branch's anonymous message. [V-read]

MEDIUM

id	summary	severity	verdict	evidence today
#10M / M1	UEV_HibernationFees has a malformed (= () 0.0) term — C_SetHibernationFees **always** fails	MED	**FIXED**	09_U_ATS.pact:699-705 — the stray term is gone; the @doc records what it was (() is Pact's unit value; the

id	summary	severity	verdict	evidence today
				comparison is always false, dragging the whole (fold (and) ...) to false). [V-read] Indirectly witnessed: modules/ATS.repl:2361 <<ATS-I5>> executes C_SetHibernationFees through the real Talos path. [V-cmd]
#11M / M2	C_KickStart has no bound on the rt-amounts : rbt-request-amount ratio — vault inflation-attack setup	MED	**FIXED**	Layered per StoicSyntax §14.7: 10_ATSU.pact:370 ATSU|C>X_KICKSTART (shared core, >= 0.1 floor), :340 ATSU|C>KICKSTART (@event leaf, CAP_Owner + <= 100.0 ceiling), :359 ATSU|C>ADMINISTRATIVE-KICKSTART (@event leaf, GOV|ATSU_ADMIN, floor only). New helper 09_U_ATS.pact:303 UC_KickStartIndex returns -1.0 on a non-positive divisor rather than crashing in a let. [V-read] **Witnessed**: modules/ATS.repl:956 <<ATS-G9>> — *"an owner-path KickStart above index 100.0 is refused"*; :679-694 <<ATS-G5>> covers the core's four sequential guards. [V-read]
#12M / M3	XE_UpdateRUR has no floor-at-zero on any of its three buckets	MED	**NOT A BUG**	UDC_RT's constructor enforce fires during the let-binding phase, before any write — a complete atomic backstop, not partial defence. [REPORTED]
#13M / M4	C_Fuel doesn't gate on the lock-state	MED	**NOT A BUG**	Coil/Curl skip all four locks too; the

id	summary	severity	verdict	evidence today
	flags RemoveSecondary requires			locks exist to protect the RT-list reshape specifically. [REPORTED]
#14M / M5	Elite toggle switches the position-selection algorithm on already-populated ledger rows	MED	**NOT A BUG**	The toggle changes only the *search window* for new deposits; stored P1–P7 rows are always read literally. [REPORTED]
#15M / M6	UEV_CRF|FeeThresholds never validates threshold *values* despite its @doc promising [1,100]	MED	**FIXED (doc-only)**	09_U_ATS.pact:601-606 — @doc now says the bound is on the **count**, and says why values have no ceiling. [V-read] **Witnessed** for the count bound: modules/UTILITIES.repl:392 (a single-rung ladder is accepted — the minimum size) and :394 (an empty one is refused by the size guard), both <<UTIL-07>>. [V-read] (**Corrected 2026-09-18** — this cited :350-355, which is <<UTIL-06>>, the atspair name-length guard.)
#16M / M7	Hard-branch cold-duration params never enforce growth > 0 — a negative, evenly-dividing growth inverts the whole wait curve	MED	**FIXED**	09_U_ATS.pact:745 and :754 — (> growth 0) on both branches (soft had the identical gap). [V-read] **Witnessed**: modules/UTILITIES.repl:167-169 <<UTIL-03>> — *"a zero growth is refused before it can divide by zero"*. [V-read] (**Corrected 2026-09-18** — this cited :142-144, which is the block's namespace/banner lines, and §2.6

id	summary	severity	verdict	evidence today
				attributed the assertion to <<UTIL-02>>.)
#17M / M8	UC_SplitByIndexedRBT has no zero-guard on resident-sum — reachable div-by-zero	MED	**NOT A BUG**	Proven from URC_Index's own formula: resident-sum = 0.0 is the *only* way index reads exactly 0.0, so a strictly-positive index guarantees a nonzero divisor by construction. [REPORTED]
#18M / M9	UC_SplitByIndexedRBT trusts positional alignment of two arrays with no length-parity guard	MED	**NOT A BUG**	Both arrays are a 1:1 map over the same reward-tokens list; they cannot desync. [REPORTED]

LOW

id	summary	verdict	evidence today
#19L / L1	ATS|F>OWNER — dead capability, never composed	**FIXED**	Gone. Zero F>OWNER occurrences in 08_ATS.pact; the sibling DALOS|F>OWNER survives at 01_DALOS.pact:873 and is composed twice, confirming the pattern itself was fine. [V-cmd]
#20L / L2	UR_P-KEYS/UR_KEYS perform raw keys scans under a UR_ prefix	**NOT A BUG (deferred)**	Repo-wide convention; folded into the post-audit sweep. [REPORTED]
#21L / L3	can-upgrade is permanently true with no setter — a V1→V2 vestige gating C_Control	**FIXED**	New setter: 08_ATS.pact:255 (interface), :3202 (impl), plus cap and XI_; Talos wrapper 3_Talos/03_TS01-C2.pact:70/:614. [V-read] **Witnessed**: [6.6]_ATS.repl:2835 (*"can-upgrade is now false"*), :2836 (C_Control then refused), :2886 (restored). [V-read]
#22L / L4	Hot-RBT surface + ~12 config C_* functions have **zero** REPL coverage; the suite that exists isn't in the default pipeline	**FIXED**	[6.6]_ATS.repl now carries **42** expect/expect-failure forms across the twelve functions. [V-cmd] (**Corrected twice on**

id	summary	verdict	evidence today
			2026-09-18. It first said 45, which was wrong when written rather than drifted. The correction said 43, which came from grep -o '(expect' — a *substring* match that also catches (expected-release-sum:decimal …) at :3889, a let binding. Word-bounded, \(expect(-failure)?\b, the file holds 42. Two successive corrections of the same figure, each wrong in a different way, and the second was produced by the looser pattern of the two.) **But the pipeline half regressed** — see §2.5.
#23L / L5	Hibernation fee computed but never separately tracked, asymmetric vs royalty	**NOT A BUG**	Traced in C_Brumate: the full pre-fee amount is credited to resident while the coiler receives less RBT — the fee stays in the pool, raising the index for existing holders. [REPORTED]
#24L / L6	URC_RewardBearingTokenAmounts hardcodes dayz=1	**NOT A BUG**	dayz only matters when hibernation is on, and every caller of the plain variant is gated to hibernation-off by its own cap. [REPORTED]
#25L / L7	XI_Normalize's 16-branch position reshuffle not hand-verified	**VERIFIED CORRECT**	Full trace of all 9 top-level branches + the take/drop slicing; occupied slots are never dropped or duplicated. No defect. [REPORTED]
#26L / L8	C_KickStart's rt-amounts array is caller-order-trusted	**NOT A BUG**	Owner-only function; a wrong order misconfigures only the caller's own pool. [REPORTED]
#27L / L9	U_DPTF::UC_UnlockPrice's @doc says "ATS" (copy-paste)	**FIXED → since deleted**	**The fixed function no longer exists.** 02_IGNIS.pact:1501-1509 records that the escalating unlock ladder (U|DEC/U|ATS/U|DPTF

id	summary	verdict	evidence today
			UC_UnlockPrice) *“had been dead code since the flattening and was DELETED 2026-09-10”*; the flat replacement is UC_FeeUnlockPrice. [V-read]
#28L / L10	UC_IzStoicTagIndexChar/ UC_IzStoicTagIndex/ UEV_StoicTagIndex are dead and collide by name with the live CODEX StoicTag feature	**ONGOING (kept)**	**Still present, still dead.** 09_U_ATS.pact:57-58 and :63 declare all three. [V-cmd] Owner: *“leave them.”*
#29L / L11	defcap P|ATS in 05_TS01-P.pact — dead code shadowing the real P|ATS|CALLER machinery	**FIXED**	Gone; zero defcap P|ATS in that file. [V-cmd]
#30L / L12	Several ATSU master defcaps put a bare-ref validation call before local enforces	**ONGOING (kept)**	Deferred to the StoicSyntax sweep; confirmed harmless (affects only which message fires first). [REPORTED]
#31L / L13	Talos wrapper ATS|C_SetHotRecoveryFee (singular) vs core C_SetHotRecoveryFees (plural)	**NOT A BUG (left as-is)**	Renaming would cascade into a new interface version plus two citizen-module consumers plus a coordinated redeploy. [REPORTED]

Appended after the original list

id	summary	severity	verdict	evidence today
#32N / N1	URC_MultiCull returns a raw [decimal] on its “nothing cullable” branch but an object on the other — XI_MultiCull’s :object binding makes it a hard crash; reachable by **any** account with nothing currently ripe	—	**FIXED**	10_ATSU.pact:641-681 — the empty branch now returns the same 4-key object shape (after-cull unchanged, two empty lists, summed-culled-values: zr-output). Talos reports it distinctly: 3_Talos/03_TS01-C2.pact:1030 *“Nothing to Cull just yet for ATS-Pair {} - no positions have reached their cull-time”*. [V-read] **No witness** — see §2.5.
#33N / N2	C_WithdrawRoyalties	HIGH	**FIXED**	10_ATSU.pact:1651 —

id	summary	severity	verdict	evidence today
	hands its full per-RT royalty vector to TFT::C_MultiTransfer, which debits every leg unconditionally — any 0.0 leg crashes the whole withdrawal			nonzero-idx filters both lists before the multi-transfer; the RUR-reset loop still iterates every RT, so no accounting is skipped. The same filter is mirrored in the preview at :766-777. [V-read] **Witnessed**: [6.6]_ATS.repl:3821 (royalty accrued, nonzero) and :3827 (all buckets reset after withdrawal). [V-read]
#34N / N3	P|A_Define in 01_TS01-A.pact never registered ATS or ATSU as permitted Talos-admin callers — ATS|A_RemoveSecondary and ATS|A_KickStart were **unreachable, unconditionally, for any signer**	HIGH	**FIXED**	3_Talos/01_TS01-A.pact:249-250 — (ref-P|ATS::P|A_AddIMPmg) and (ref-P|ATSU::P|A_AddIMPmg), in the list alongside the eight that were already there; the missing ref-P|ATSU binding is at :234. The in-source comment at :219 records the history. [V-read]
#35N / N4	The audit added 5 new public functions across 5 interfaces with no version bump — 3 of them already live on mainnet under those exact names	—	**ONGOING (deployment prerequisite)**	**Resolved by the later redeploy phase.** All three required bumps are present: UtilityAtsV2 → UtilityAtsV3 (09_U_ATS.pact:2), AutostakeUsageV1 → AutostakeUsageV2 (10_ATSU.pact:6), AutostakeV2 → AutostakeV3 (08_ATS.pact:6). [V-cmd]

A note on this tree's own counting

The audit's two summary documents disagree with each other and with the list they summarise. This is worth stating precisely, because Rule 3 of this book exists for it.

- ISSUES-RANKED.md's Tally says **"FIXED: 19"** and then enumerates **18** ids.
- AUDIT-REPORT.md §1 says **"19 fixed… 13 closed as not-a-bug… 3 deferred"**, summing to 35.
- Counting the verdict tag on every entry gives **18 FIXED, 14 NOT A BUG, 3 ONGOING = 35**. [V-cmd]

So the report's total is right only because two errors cancel: fixed is overstated by one and not-a-bug understated by one. The correct figures are 18 / 14 / 3. None of this changes a verdict; it changes the headline number an outside reader would quote.

2.4 Six findings worth the telling

(a) C2 — one bug, three symptoms, and the sentinel that made it invisible

This was the audit's declared priority target, and it is the best-diagnosed finding in the tree.

A staker's cold-recovery claim lives in an Awo object: a [decimal] array of per-reward-token amounts, positionally aligned against the pool's live reward-tokens list, plus a cull-time. When the pool owner removes a secondary reward token, every stored Awo on every account has to be reshaped — fold the removed index's value into slot 0 (the primal RT) and shrink the array by one.

UC_ReshapeUnstakeObject did that *conditionally*, gated behind UC_IzUnstakeObjectValid — which is true only when the object already has a nonzero claim. The overwhelming majority of ledger rows are untouched, all-zero P0–P7 slots. Those were silently skipped. Every one kept its pre-removal array length forever.

Now the three symptoms:

- **(c) cold recovery becomes permanently unusable.** URCX_PosObjSt and XI_StoreUnstakeObject answer "is this slot open?" by **structural equality** against a freshly length-derived zero sentinel. Once the live reward-token list shrank, a stale longer array no longer equals the sentinel — so every empty slot on every pre-existing account reads as permanently *occupied*.
- **(a) a cull pays the wrong token.** A stale, longer-than-current array read positionally against the shorter live list attributes each amount to the wrong token.
- **(b) royalty is stranded.** Separately, X_RemoveSecondary migrated the removed token's *resident* (RUR 1) and *unbonding* (RUR 2) buckets into the primal token but not *royalty* (RUR 3). The tokens stayed custodied in ATS|SC_NAME with no reward-token entry left to reference them; C_WithdrawRoyalties could never reach them again.

The fix is two diffs and a deletion of trust. UC_ReshapeUnstakeObject now calls UCv_SolidifyUnstakeObject unconditionally — merging a 0.0 removee into slot 0 is a value no-op, it only ever needed to shrink the array. And X_RemoveSecondary (now XI_RemoveSecondary) **stopped accepting an account list from its caller at all** and derives the complete list on-chain via URH_ExistingAutostakePairs, closing a latent gap where the admin path could under-supply the list. 09_U_ATS.pact:394-404 and 10_ATSU.pact:1530-1596. [V-read]

Note what the audit did *not* claim. Its original plan was to drive the proof through A_RemoveSecondary with a deliberately incomplete caller-supplied list. It records honestly that this input *no longer exists to construct*, because the fix removed the parameter — so the proof demonstrates the equivalent property (an account never mentioned in the removal call is still found and correctly reshaped) and says so. [REPORTED]

The proof itself was strong: a brand-new account, funded, coiled, cold-recovered through the pool's own real split maths ([6.0627 PKOSON, 6.0175 EKOSON, 8.3403 AKOSON]), then EKOSON removed through the production Talos owner path, then culled — with the actual DPTF balance deltas asserted: PKOSON +12.080149529322803393554764, AKOSON +8.340265093244058451673906, bit-for-bit matching the pre-cull stored amounts. [REPORTED]

And that proof no longer runs. It was written into REPL/_audit_ats_baseline.repl, which now sits in REPL/archive/ and is excluded from the gate by name. On 2026-09-17 a replacement was written that does not depend on it: REPL/modules/UTILITIES.repl:1081-1093 <<UTIL-15>> drives UC_ReshapeUnstakeObject on the **all-zero** object the removed gate rejects — asserting the rejection first, so the case is known to be the one the fix bought — and shows the array shrinks anyway, 3 slots to 2. Reinstating the gate turns it red. [V-cmd] See §2.5.

> **CORRECTED 2026-09-18.** This paragraph ended *"See §2.5. This is the single most important caveat in this chapter"*, and §2.5 listed C2 as unwitnessed. That was true when the chapter was written on 2026-09-17 and was closed the same day; the caveat now applies to the archived proof, not to the finding.

(b) C3 — the function that had never once worked, and the fixture that nearly hid it

C_Redeem is the only exit from hot recovery. It fed are-fee-rts — a summed *decimal* fee amount — straight into (if are-fee-rts ...). Pact's if requires a :bool and does not coerce a decimal either way, including 0.0. Every call reverted. Anyone who had gone through C_HotRecovery was permanently locked out of their underlying tokens.

The reason this survived is visible in the REPL file it should have been caught by. The canonical "Redeem Test" section had the C_Redeem call **commented out**, with C_Reverse substituted in its place, under a fixture that set block-time to 2024-10-11 — two years *before* the nonce's own mint time of 2026-10-10. A dead test, with a wrong date, standing where the live test should be.

The owner's instinct was that the comment-out might be benign — "nothing to redeem". The audit ran three escalating checks rather than accept either story:

1. An isolated Pact 5.4 repro confirming (if <decimal>) fails for both a nonzero decimal and exactly 0.0.
2. Uncommenting the real call as-is. **It failed — for a different reason**: ico3's TFT::C_MultiTransfer rejected a negative amount, traced to the backwards fixture date. This is the step that matters. A pass that stopped here would have "confirmed" the finding with evidence that proves nothing about it.
3. The same call under honestly forward-moving time (mint 2026-10-10, redeem 2026-10-12, past the decay window — the cleanest possible input). It failed exactly where predicted: 10_ATSU.pact:1067:28: expected bool value, got 0.0.

The fix is one added binding (10_ATSU.pact:2117 [V-read]) and the dead test was **rewritten rather than patched around** — into two real assertion-backed branches. It is one of the few ATS proofs that is still live today: [6.6]_ATS.repl:524 asserts the early redeem paid *strictly less* than full value (proving a real fee was withheld, not just "didn't crash"), and :611 asserts the matured redeem paid exactly full value. [V-read]

(c) N3 — two admin functions that were dead on arrival, found only because someone wrote a test

P|A_Define is the function each Talos module runs once at init to register itself as a permitted caller into every core module its admin functions must reach — one (ref-P|<MODULE>::P|A_AddIMP mg) per module. Every core module's UEV_IMC checks that whitelist before any key-authorisation logic runs at all.

TS01-A's own P|A_Define registered into DALOS, IGNIS, BRD, DPTF, DPOF, LIQUID, OUROBOROS and SWP — but not ATS or ATSU. ATS was even **bound as a local variable (ref-P|ATS) and then never used in the body**: a half-finished registration, not a typo elsewhere. Every *other* Talos module's P|A_Define registers into both.

Consequence: ATS|A_RemoveSecondary and ATS|A_KickStart — real admin entrypoints, not scaffolding — were **completely unreachable via their real Talos admin path, unconditionally, regardless of who signed**, failing inside UEV_IMC with *"None of the guards passed"* before the admin's own key check could matter.

Two things about this are instructive. First, it is invisible to code review of ATS: nothing in 08_ATS.pact or 10_ATSU.pact is wrong. The defect is an *absence* in a third file. Second, it was found by **writing the missing tests from finding L4** — the coverage gap and the bug were the same event, and the audit chose to log it as its own numbered finding rather than fold it into L4's narrative, on the explicit precedent that C5 and H4/M1 were also "bugs testing revealed" and still got their own IDs. That is the right call: a coverage gap that hid a bug should leave two entries in the record, not one.

3_Talos/01_TS01-A.pact:249-250 [V-read].

(d) M2 — the finding whose premise was refuted, and which survived anyway

M2 (#11M) originally rested on the same "silent zero-mint donation" story as H3, which the audit had just **refuted**: an exact-0.0 mint reverts the whole transaction atomically via DPTF|C>CREDIT's UEV_Amount, so there is no silent donation.

The easy move here is to close M2 as a dependent of a refuted finding. The audit re-examined it instead and found the premise wrong but the attack intact, in a regime the original write-up had not described: a genesis ratio extreme enough that a later Coil's floor(rt_amount / index, p) lands on a tiny **nonzero** RBT amount. The transaction then succeeds. The depositor's real RT is credited in full to the pool's resident bucket; their RBT is a negligible fraction of fair value; the difference is a permanent transfer to whoever already held RBT. That is the classic first-depositor vault inflation attack, and it does not need a zero-mint to work.

The fix is the shape worth copying. Rather than pick a single bound, it **layered** the capability per StoicSyntax §14.7 — one unevented core holding the shared checks and the floor, two distinctly-@event-tagged leaves on top:

```
ATSU|C>X_KICKSTART          (core, unevented)  — shared checks + index >= 0.1
 ├─ ATSU|C>KICKSTART        (@event leaf)      — CAP_Owner + index <= 100.0
 └─ ATSU|C>ADMINISTRATIVE-KICKSTART (@event leaf) — GOV|ATSU_ADMIN, no ceiling
```

So the pool owner is bounded on both sides; module governance can exceed the ceiling for legitimate high ratios but still cannot go below the floor; and the two paths emit distinguishable on-chain events. 10_ATSU.pact:340, 359, 370 [V-read].

The audit also disclosed a limitation of its proof rather than glossing it: a full end-to-end run through a brand-new virgin pool was out of scope (pair creation needs a separate multi-step token registration flow), so the bound checks were driven via test-capability against the real deployed capabilities on an already-kickstarted pair, with real signatures — proving every bound exactly as it runs in production, with only the orthogonal virgin-pool gate inferred rather than re-demonstrated. [REPORTED] That coverage has since been replaced by real assertions: modules/ATS.repl:956 <<ATS-G9>> refuses an owner-path KickStart above index 100.0. [V-read]

(e) C5 — the owner worked out why the suspicious code was correct, and found the real gap underneath it

C_HOT-RBT|UpdatePendingBranding and C_HOT-RBT|UpgradeBranding wrapped themselves in with-capability (ATS|GOV) — the trivially-true "Simple vault" cap that C1 had just been refuted over. The obvious reading is "here it is again, and this time it's exploitable."

The owner reasoned it out the other way. A Hot-RBT's DPOF owner-konto is ATS|SC_NAME (rotated there by C_AddHotRBT), so DPOF's own branding gate resolves to *"prove you own ats-sc"* — something only ATS's own code can do, via ATS|GOV. ATS|GOV was never the bug; it was load-bearing and correct.

The real gap was one level up: **nothing checked *who* was allowed to trigger it.** A repo grep over the exact 21 lines of both functions returned zero CAP_Owner, zero UEV_IMC, zero enclosing defcap. Any account could rewrite or paid-upgrade branding on a Hot-RBT it did not own.

The fix is also where a documented StoicSyntax rule was born. The first pass used a **single @event capability shared by both functions**. The owner caught it: sharing one evented cap across two distinct client actions collapses their on-chain events into one indistinguishable signature — and, separately, the codebase already had an undocumented pattern for exactly this (one unevented "core" holding the shared validation, thin distinctly-@event leaves on top). The audit verified the pattern was already live in the same file (ATS|S>CONTROL-RECOVERY → two unevented middles → five @event leaves) before adopting it, then formalised it as StoicSyntax.md §14.7 with this case as the worked example. It is the same §14.7 that M2's KickStart fix then used.

08_ATS.pact:736 (core), :746, :750 (leaves) [V-read].

(f) N1 — the crash that was live on mainnet and had simply never been triggered

URC_MultiCull returns an object on its "here's what's cullable" branch and a bare [decimal] on its "nothing cullable yet" branch. XI_MultiCull binds the result as :object. Any account calling C_Cull with nothing currently past its cull-time gets a hard runtime type failure instead of "nothing to cull".

The audit did something here that most audits cannot: it checked the **live deployed module**. Using PYTHIA's public dirty-read console — reachable keyless via a Sec-Fetch-Site: same-origin header, which PYTHIA's own source documents as intentionally forgeable by non-browser clients, blast radius "public chain reads only" — it pulled describe-module "ouronet-ns.ATSU" and confirmed **the broken branch is byte-for-byte identical on mainnet**. It then enumerated all 11 real live ledger rows and called URC_MultiCull against each. All 11 succeeded — *purely because every existing live account happens to have something already past its cull-time right now.* [REPORTED]

That is a materially different statement from "we found a bug in the source". It is "this bug is deployed, and the only thing standing between it and every user is the current shape of eleven ledger rows."

The owner's ruling shaped the fix twice over: nothing lets funds move early, so this is a **soft-failure** problem, not a fund-safety one — and the correct repair is not a silent zero-value success but a *distinguishable* message. So URC_MultiCull's empty branch returns the same 4-key object shape (10_ATSU.pact:641-681) and ATS|C_Cull reports *"Nothing to Cull just yet for ATS-Pair {} - no positions have reached their cull-time"* (03_TS01-C2.pact:1030). [V-read]

Worth noting for its own sake: a first attempt at this fix was applied **without owner authorisation** and fully reverted once caught, and the fix entry says so in its own opening paragraph. An audit that records its own procedural breaches is more trustworthy than one that doesn't have any.

2.5 What remains open

Deliberately open

item	state today	why
#28L / L10 — three dead StoicTag functions in U_ATS that collide by name with the live, unrelated CODEX StoicTag feature	**Present.** 09_U_ATS.pact:57, 58, 63 declare UC_IzStoicTagIndexChar, UC_IzStoicTagIndex, UEV_StoicTagIndex. [V-cmd]	Owner, explicitly: *"leave them."* Recorded as a deliberate keep, **not** a not-a-bug verdict — the tree is careful about that distinction and so is this chapter.
#30L / L12 — ATSU master defcaps place a bare cross-module ref call before their own local enforces	Deferred to the StoicSyntax sweep.	Harmless by construction: every statement in a defcap body runs top-to-bottom to first failure, so the only effects are which message a caller sees when several conditions fail at once, and a little gas on a doomed call. No input gets through that shouldn't.
#20L / L2 — UR_P-KEYS/UR_KEYS are raw keys scans under a UR_ prefix	Repo-wide, off any client mutation path.	Owner: *"by design so far"*; belongs to the module rehaul, not a piecemeal ATS fix.
#31L / L13 — Talos/core naming asymmetry on SetHotRecoveryFee(s)	Left as-is.	Renaming cascades into a new interface version plus two citizen-module consumers plus a coordinated redeploy, for a cosmetic gain.

Closed as NOT A BUG, but a reader should know

- #4C / **C4 — the syphon lever is unbounded above 0.1 and has no timelock.** A pool owner can re-lower syphon at will and extract the great majority of a pool's commingled RT backing (principal *and* yield) in a single call. The audit confirmed the mechanics and the owner confirmed the design: stakers trust the pool owner with this parameter, and a monotonic ratchet was explicitly rejected because 0.6 → 0.5 must remain an ordinary adjustment. **This is the single largest trust assumption in the module** and it is not mitigated by anything technical. Anyone evaluating an ATS pair is evaluating its owner.
- #6H / **H1's six exempt fields.** Only royalty-promile and the two hibernation-fee fields got the parameter-lock gate. owner-konto (rotation), the three C_Control toggles (can-change-owner/syphoning/hibernate) and the three recovery on/off switches were *confirmed* exempt under the same trust model. The lock therefore protects less than its name suggests.
- #17M / **M8's invariant is protocol-maintained, not self-enforced.** The div-by-zero cannot fire *provided* resident-sum and rbt-supply stay in lockstep — which is exactly the class of invariant C2's bug was about. The audit noted this explicitly: no caller of UC_SplitByIndexedRBT's chain checks index > 0 before calling in. Safe by construction today; safe by a construction that has been wrong before.

Four fixes that were present in source with no witness — all four now closed

This is where the ATS tree was in materially worse shape than it looked, and the cause was mechanical rather than anyone's neglect.

Most ATS Round-II proofs were appended to **REPL/_audit_ats_baseline.repl**. That file now lives in REPL/archive/, and the gate excludes it *by name*, with a reason that is the finding:

> ("_audit_", "one-off audit baselines; archive/_audit_ats_baseline.repl **does not run to completion**, so its 32 assertions are NOT coverage") — REPL/tools/_gate.py EXCLUDED [V-read]

Thirty-two assertions that the audit trail treats as its proof of record do not execute and would not go red. Fix #15 (#22L) migrated *some* of them into the canonical [6.6]_ATS.repl, and the later Part II work added REPL/modules/ATS.repl (291 assertions) and REPL/modules/UTILITIES.repl, which between them re-cover several more. Four fixes were left with nothing in the running suite that would go red — and all four were closed on 2026-09-17, while this book was being assembled:

> **CORRECTED 2026-09-18.** Every row of the table below used to read *"None"* or *"Not found"*, and this section was headed by the claim that ATS's evidence had rotted. All four were closed on 2026-09-17, together with seven in Chapter 5; DEFECT-LEDGER.md §8.26 records the set of eleven. Each needed a purpose-built fixture, not a test — see the Part I introduction for what each one had to do.

finding	fix present in source?	witness in the running suite (all four added 2026-09-17)
#2C / **C2** — the audit's priority finding	Yes: 09_U_ATS.pact:394-404, 10_ATSU.pact:1530-1596 [V-read]	<<UTIL-15>>, REPL/modules/UTILITIES.repl:1081-1093. It drives UC_ReshapeUnstakeObject itself — not the inner UCv_SolidifyUnstakeObject that <<UTIL-05>> (:267-281) exercises — on the **all-zero** object the removed gate rejects, asserting the rejection (:1081) before showing the reshape happens anyway (:1086). It also gives UC_IzUnstakeObjectValid (09_U_ATS.pact:47, :291) its first caller in the suite. The royalty half (C2b) is still unasserted: [6.6]_ATS.repl's Secondary Remove 1|5–5|5 blocks

finding	fix present in source?	witness in the running suite (all four added 2026-09-17)
		(:1420–:1847) contain **zero expect forms**. [V-cmd]
#6H / H1 — parameter-lock gate on royalty + hibernation fees	Yes: 08_ATS.pact:638, :660 [V-read]	<<ATS-G28>>, REPL/modules/ATS.repl:4996 and :4999 — the **same owner**, the same arguments, refused on both setters once the pair is locked, against a working unlocked call at :4987. UEV_ParameterLockState's message (08_ATS.pact:2388) had appeared in no .repl at all; <<ATS-G16>> (modules/ATS.repl:1395) tests the lock's own two-sided rule, not that either setter sits behind it. [V-cmd]
#32N / N1 — URC_MultiCull soft failure	Yes: 10_ATSU.pact:641-681, 03_TS01-C2.pact:1030 [V-read]	<<ATS-G29>>, REPL/modules/ATS.repl:5052 — a cull from an account that has a ledger row in the pair but nothing ripe, which returns a sentence instead of crashing on the type. The account choice is the test: one with no row at all dies earlier, at a bare read. The string *"Nothing to Cull"* had appeared only in archive/_audit_ats_baseline.repl. [V-cmd]
#5C / C5 — Hot-RBT branding ownership	Yes: 08_ATS.pact:736 [V-read]	<<ATS-G30>>, REPL/modules/ATS.repl:5101, with the fixture asserted at :5097 and the owner's identical call succeeding at :5120. None of the usual owner's pairs has a Hot-RBT, so the fixture is a third account's — which makes the refusal *more* legible, since the message names a key that is neither the attacker's nor the suite's usual owner. [6.6]_ATS.repl exercises branding on the happy path only (:3539-3541, :3624-3625). [V-cmd]

The rest were already witnessed, and this chapter names the assertion for each: C3 ([6.6]_ATS.repl:523, :524, :611), #7H (:3965–:3993), #21L (:2835, :2836, :2886), #33N (:3821, :3827), #11M (modules/ATS.repl <<ATS-G9>>, <<ATS-G5>>), #9H and #16M (modules/UTILITIES.repl <<UTIL-03>>) and #15M (<<UTIL-07>>), and #34N indirectly (the A_KickStart/A_RemoveSecondary blocks in [6.6] can only pass because the registration exists).

> **Corrected 2026-09-18.** This list carried the pre-drift line numbers throughout, and attributed #16M to <<UTIL-02>> and #15M to a bare modules/UTILITIES.repl:350-355, which is <<UTIL-06>>, the atspair name-length guard. They are <<UTIL-03>> and <<UTIL-07>> respectively; every number above was re-resolved against the current files.

One regression in the audit's own fix

Fix #15's write-up states that it *"uncommented the [6.5]_DPOF.repl/[6.6]_ATS.repl load lines in Stage01_Tester.repl"*, and records that it found them disabled in that worktree — *"the same class of silent-revert data loss documented in the Claudstermind handoff, this time surviving the worktree transition undetected until this pass."*

They are commented out again today: REPL/Stage01_Tester.repl:57-58. [V-cmd]

The reason given is legitimate and is not a revert of the fix's intent: a #12a GUARD comment explains that the Stage-2 AQP path self-loads [6.5]_DPOF.repl, so loading it in Stage 1 too re-issues a token and aborts on duplicate-insert. The coverage itself did **not** disappear — [6.6]_ATS.repl is loaded by ZALL.repl:51 and by REPL/modules/ATS.repl:19, and ZALL.repl is what the exhaustive path runs. [V-cmd]

The guard's own justification had gone stale in a way worth flagging. It used to read:

> *"ATS tests run via their own driver _audit_ats_baseline.repl (Stage-1-only, no double-load)."*

_audit_ats_baseline.repl is in archive/, is excluded by the gate, and does not run to completion. The comment that told a reader why it was safe to skip [6.6] pointed at a file that had not executed for weeks. The coverage is real; the reason written next to the exclusion was not. That is the documentation form of the same pathology this programme keeps finding in code — a statement that outlived the thing it described, and is indistinguishable from a true one from the outside.

> **Corrected 2026-09-18.** This section quoted that justification from Stage01_Tester.repl:47 in the present tense. It no longer exists: Stage01_Tester.repl:49-56 now carries a "CORRECTED 2026-09-17" note retracting it in the same terms used here, and DEFECT-LEDGER.md §8.20 records the fix. [V-read]

Not re-verified for this chapter

#25L (the XI_Normalize 16-branch trace), #12M, #13M, #14M, #17M, #18M, #23L, #24L and #26L were closed by tracing arguments that this chapter did not re-derive. Their verdicts are **[REPORTED]**, not re-established. Of these, #25L carries the most weight — it is the only finding in the tree closed by exhaustive manual trace of a 9-branch, 7-slot reshuffle with no automated check behind it, and nothing in the running suite would catch a regression in XI_Normalize.

---

7. SWP — the automated market maker

Source tree: Audit/module-audits/SWP/ (8,453 lines across 7 files, plus a 5,720-line reference/ dump of Kaddex/KDX mainnet source used for comparison). Audit ran 2026-08-16 → 2026-08-29 on a dedicated swp branch, 103 commits. It is by a wide margin the largest of the three Stage-1 trees. This chapter was written 2026-09-17.

Evidence labels are the same as Chapters 5 and 6: **[V-cmd]**, **[V-read]**, **[INFERRED]**, **[REPORTED]**. **No test was executed for this chapter.**

3.1 What the module does

SWP is Ouronet's automated market maker — the thing that lets one token become another without a counterparty. It is eleven files, and unlike ATS or DALOS it is not one module with helpers; it is a layered machine where each layer does one job:

- **U|SWP** is the maths. Three pool types share it: **Stable** pools (Curve-style, a Newton solve over an invariant D with an amplification coefficient), **Weighted** pools (Balancer-style, $\prod x_i^{w_i}$), and **Standard** pools (plain constant product). Six formulas in total — each of the three types, in both the forward direction ("I give X, how much Y do I get?") and the inverse ("I want Y, how much X must I give?").
- **U|BFS + SWPT** are the router's graph engine. There is rarely a direct pool between the two tokens a user wants to trade, so the system maintains an adjacency graph of every token that shares a pool with every other, and runs a breadth-first search to find a route. SWPT owns the graph and a path cache.
- **SWP** owns the state: pool properties, reserves, weights, amplifier, fees, the LP-token registry, ownership, the principal-token list, and the admin levers.
- **SWPI** issues pools and computes swap amounts (the "hopper" — a hop-by-hop route evaluation).
- **SWPL/SWPLC** add and remove liquidity, including *asymmetric* deposits (putting in an unbalanced ratio, which dilutes existing LPs and is therefore taxed).
- **SWPU** executes swaps, enforces slippage, and runs the smart router.
- **MTX-SWP** is the multi-transaction (defpact) variant of issuance and liquidity-adds, for operations too large for one transaction's gas budget.
- **TS01-C3** and **TS01-P** are the Talos wrappers — the only supported client doors.

What breaks if SWP breaks. Three distinct things, and the audit found instances of all three.

The first is **money moving wrongly in a single trade**: a rounding bias that always favours the trader is a free, repeatable extraction from every liquidity provider; a solver that converges to the wrong root can output more than the pool holds.

The second is **routing**: if the graph's node set is narrower than its edge set, long paths are silently corrupted or lost — a user is told "no route" when one exists, or is routed through the worst of several parallel pools. This is not a safety failure but it is a continuous, invisible value leak.

The third is **pricing everything else**. SWP's URC_OuroPrimordialPrice publishes the canonical dollar price of OURO, the gas token, to an on-chain oracle. DEMIPAD launchpad payments convert with it. The Explorer displays it. A pricing error here is not contained to swaps at all — and §3.4(f) is exactly that case.

3.2 How it was audited

Round I — nine parallel deep-read passes over eleven files, read-only. Same contract as its siblings: load StoicSyntax.md first, work read-only, assume nothing is correct despite being live on mainnet. Round I produced **71 findings — 13 CRITICAL, 12 HIGH, 14 MEDIUM, 32 LOW**. [V-cmd]

Two things make this round methodologically different from ATS and DALOS.

There was no reference implementation to diff against. AQP had the UrStoa RPS vault as ground truth; SWP had nothing. So the AMM maths was verified against *first principles*: standard Curve-style StableSwap and Balancer-style weighted invariants re-derived algebraically and checked term-by-term against the source, then **numerically simulated** — replicating the exact Pact floor/fold sequence — and tested against conservation, monotonicity and round-trip neutrality. [REPORTED — SWP/README.md § Method]

Several findings were found twice, independently. URC_BestEdge was found by both the SWPT and the SWPI auditor; the rounding-direction bias by both the U|SWP and the SWPI auditor; the defpact Global-Admin-Pause gap by both the MTX-SWP and the Talos auditor. All thirteen CRITICALs were then lead-re-verified directly against source, and the two mathematical ones independently re-derived and simulated. Cross-corroboration, recorded as such rather than counted twice.

Round I — owner feedback, and the rule that exists because it was broken. Findings were presented one at a time in ISSUES-RANKED.md order. The README carries a HARD RULE in capitals:

> *"This is not optional bookkeeping — it's what 'closed' means… This rule exists because C4 (#11C), C5 (#12C), and C6 (#13C) were skipped over silently mid-session on 2026-08-17/18 — not refuted, not deferred on purpose, just missed — and the gap wasn't caught until the owner asked to audit the audit."* [V-read — SWP/README.md]

Three CRITICALs were lost mid-session and recovered only because someone audited the audit. That is in the tree's own front matter, in bold, and it is the most useful sentence in it.

Round II — 49 numbered fixes. The methodology is stated as non-negotiable in HANDOFF.md and is stricter than either sibling's:

> *"(1) verify every claim against live source before accepting it, don't trust the finding doc or your own memory; (2) apply the fix; (3) build a REPL reproduction; (4)* ***adversarially prove it*** *— revert just the fix (surgically, by hand, never via git), confirm the bug reproduces, restore the fix, reconfirm; (5) run the full regression suite; (6) log in all four places."* [V-read]

Step 4 is the one that matters, and the handoff is explicit that the owner *"explicitly and repeatedly demanded"* it: **do not trust a green test you have not watched fail first.** The "never via git" clause is not stylistic — it cites a confirmed data-loss incident where a git stash raced two concurrent sessions in a shared worktree and silently reverted files for both, twice.

Three sub-programmes ran alongside the findings list, and they are why this tree is twice the size of its siblings:

1. **#34M / M2, promoted to a MASTER ISSUE.** What began as "BFS keeps only one chain per node and routing does no cross-route comparison" became a 13-phase programme, because fixing it surfaced a worst-case *execution-gas* crisis: 6–7 million gas against a ~2,000,000 ceiling at realistic scale, 56.9% of it from a previously-unknown source (XE_UpdateStoaValue), not from the routing itself. The outcome was a redesign — dirty-read path injection, a bundle-based C_SmartSwap — and a measured **7,145,298 → 397,043 gas** on the identical worst-case swap. [REPORTED]
2. **#65bL, a 9-phase gas-optimisation arc.** Cold-cache worst case **5,094,054 → 1,296,898** (74.5%); warm cache 1,143,255 (77.6%). [REPORTED] Phase 3 of it is worth naming because it *failed*: a binary search over a sorted raw-graph list looked like a win on a synthetic benchmark and **regressed by +27,527 gas**

in the real integrated measurement. It was reverted and recorded as a genuine negative result rather than dropped. That is rare and it should be credited.

3. **A Kaddex/KDX comparison.** The audit pulled real Kadena-mainnet DEX source into reference/ and diffed design decisions. Almost nothing survived scrutiny — the audit's conclusion on whether SWP needs Kaddex's MUTEX reentrancy lock was *no* — but three findings trace back to it (#72C, #73C, #74). And the comparison produced its own correction: the first supporting claim ("Pact detects recursion") was accepted from Kaddex's own source comment without proof, then went back and was independently established — finding along the way that the first proof attempt was confounded by try forcing read-only execution on anything it wraps.

Round III was ruled out of scope on this branch (owner, 2026-08-27): *"this branch's job was the initial fix, scoped and now complete."* ROUND-03-REVERIFY.md was never created and does not exist. [V-cmd] **But unlike ATS and DALOS, SWP did eventually get one** — as part of the Part III programme, recorded in Audit/records/DEFECT-LEDGER.md §8.2: *"Every FIXED finding in the SWP audit tracker was re-verified against current source — 75 rows, 42 fixes, 100% covered, not sampled. Result: 41 VERIFIED-PRESENT, 1 NOT-FOUND."* [V-read] This chapter re-checks a sample of that re-verify and agrees with it, including on the one that was missing.

3.3 The findings

Counting, with the denominator stated. Round I produced **71** findings (#1C–#71L: 13C, 12H, 14M, 32L). Twelve more were added off-cycle afterwards — #32bM, #34bM, #65bL, #65cL, #65dL, #65eL, #65fL, #65gL, #65hL, #72C, #73C, #74 — for **83 distinct finding IDs**. The tree presents them in **75 tracker rows** in README.md (several findings share a row) and **79 entries** in ISSUES-RANKED.md (the four #65dL/#65eL/#65fL/#65hL never got their own ranked entry; they live in the tracker and in ROUND-02-FIXES.md). [V-cmd] All three numbers are correct for their own denominator; none of them is "the number of findings" without saying which.

The table below follows the ranked order. Severity is as recorded.

CRITICAL

id	summary	verdict	evidence today
#1C / C10	Asymmetric liquidity-add mints the naive LP amount, not the invariant-fair taxd-lp	**REFUTED**	The deficit *is* priced and charged, via an oracle-based IGNIS tax the caller cannot supply; the original walkthrough omitted that mandatory payment. Substance folded into H8. [REPORTED]
#2C / C13	SWPLC::C_Fuel's indirect branch can credit unbacked reserves; sole gate is a trivially-true IMC cap	**REFUTED**	Self-caught via the audit's own REPL PoC: Pact 5 requires module-admin of the target module before with-capability grants its caps from outside. [REPORTED]
#3C / C2	Stable-pool Newton solver has no domain guard — oversized swaps converge to the wrong root, output can exceed	**FIXED**	1_Utilities/12_U_SWP.pact:326-355 — UC_ComputeY reseeds from D (Curve-style reference), comment names the C2 fix at :355.

id	summary	verdict	evidence today
	pool balance		Scope was narrowed to stable pools only — W/P are closed-form with no seed-dependent root. [V-read] **Witnessed**: SWP|TX 015 in [6.3]_SWP.repl + [6.2+3]_DPTF-SWP_Issuance-Only.repl. [V-cmd]
#72C	UC_ComputeInverseY — C2's inverse-direction sibling, left explicitly open and never picked up: asking for ≥ the pool's own output reserve either crashed **uncatchably** (div by zero, decimal) at the ceiling or **silently returned a fabricated number** past it	**FIXED**	12_U_SWP.pact:420-422 (now UCv_ComputeInverseY) — (enforce (< output-amount xo) ...) placed *before* the invalid coefficients are computed, with 10 lines of @doc on why no seed choice can fix it. [V-read] **Witnessed**: SWP|TX 015b in [6.2+3]_DPTF-SWP_Issuance-Only.repl — but that file is **excluded from the gate by name**, so this witness does not execute in the run that decides green. See §3.5. [V-cmd]
#73C	URC_WorthWSTOA — **two** bugs: the OURO shortcut ignored the primordial pool's own weights; the general fallback simulated a swap of the *entire* reserve instead of pricing one unit and scaling	**FIXED**	16_SWPI.pact:1797 — URC_SingleOuroWorthWSTOA now performs a real 1-unit weighted swap via UC_ComputeWP; the general fallback prices 1 unit and scales across all three WorthWSTOA variants. [V-read] **Witnessed**: SWP|TX 032z8e, SWP|TX 032z6f. [V-cmd]
#74	UEV_Issue has no distinctness check on a caller's pool-tokens list	**VERIFIED SAFE, documented, no fix**	Rejected one layer down by TFT::C_MultiTransfer's own UEV_IzUnique, reached because both issuance paths share XE_IssueWrite. @doc added recording where the protection lives. **Witnessed**: [6.3]_SWP.repl:4703 SWP|TX 032o3, whose assertion at :4767 names the exact message "Unique Items Required, duplicate item found: OURO-98c486052a51". [V-read]
#4C / C12	C_ToggleSwapCapability has no ownership check	**REFUTED**	The original trace stopped at the first with-

id	summary	verdict	evidence today
	anywhere — free DoS on any pool		capability block and missed the second, which gates the actual write behind SWP|C>ADD-OR-SWAP → CAP_Owner, unconditionally, on both toggle directions. [REPORTED]
#5C / C11	Slippage bound is checked against the fee-exclusive gross quote, never the net amount delivered	**DESIGN**	Feeless-vs-feeless comparison correctly protects against reserve/price movement; the residual fee-rate-change gap is answered by the fee-lock primitive. See §3.5. [REPORTED]
#6C / C1	URC_BestEdge picks the pool with the **least** output — argmin instead of argmax	**FIXED**	16_SWPI.pact:1655 — comment: *"C1 fix: keep the index with the LARGER output (argmax), not smaller (argmin)"*. [V-read]
#7C / C3	All six swap formulas round toward the trader — repeatable, fee-free round-trip profit	**STABLE FIXED** / **WEIGHTED ACCEPTED**	Root cause was **not** floor/ceiling placement: Pact's native ^ silently uses float64 for decimal exponentiation. Fixed for the stable pool via exact-multiplication UC_IntPow (12_U_SWP.pact:508) and by flooring the *final* output rather than the intermediate (:378, :450, :635, :702). Weighted remains open by design — see §3.5. [V-read]
#8C / C7	C_ModifyWeights has no length-parity check and a dead, never-enforced precision check	**FIXED**	15_SWP.pact:503-532 — UEV_UniformList for parity, plus a real per-weight enforce (precision + >= 0.1 floor) replacing the discarded map, plus = 1.0 sum. [V-read]
#9C / C8	C_UpdateAmplifier has **zero** bound check, including on the module's own -1.0 "not a stable pool" sentinel	**FIXED**	15_SWP.pact:606-607 — (and (>= new-amplifier 1.0) (<= new-amplifier 2000.0)). The ceiling is evidence-backed: :598-602 records a REPL convergence-degradation sweep on a skewed pool, not a guess.

id	summary	verdict	evidence today
			[V-read]
#10C / C9	Pools issued via the MTX-SWP defpact are never registered in SWP|LP — every such pool's LP token hard-aborts AQP LP-stake admission	**FIXED**	15_SWP.pact:1744-1787 — the insert SWP|LP folded into XE_Issue, which both paths call; comment names the C9 fix. [V-read]
#11C / C4	UEV_Issue never checks individual weights > 0 — a 0.0-weight token makes a W pool permanently untradeable	**FIXED**	16_SWPI.pact:2564-2578 — the dead map now wraps a real enforce combining the precision check with a >= 0.1 floor, matching SWP|S>WEIGHTS's bound so issuance and modification agree. [V-read]
#12C / C5	UEV_Issue never checks individual genesis reserves > 0	**REFUTED**	Validated on every issuance path by the mandatory funding transfer itself (C_MultiTransfer → XB_DebitTrueFungible → DPTF|C>DEBIT → UEV_Amount), just not inside UEV_Issue. The original trace stopped one defcap too shallow. [REPORTED]
#13C / C6	The routing graph's node set is narrower than its live edge set — paths of 4+ hops corrupted or silently lost	**FIXED**	12_U_SWP.pact:993 UC_MakeGraphNodes now builds from the full caller-supplied swpairs universe. [V-read] Adversarial repro: reverting only this function broke *pool issuance itself* (a 3-hop connectivity check failed), which is a stronger demonstration of reach than the finding claimed. [REPORTED]

HIGH

id	summary	verdict	evidence today
#14H / H9	Reentrancy ordering window in XI_Swap via a smart-account guard callback	**REFUTED**	Callback is reachable, but an isolated Pact 5 repro proved any write attempted from guard evaluation is blocked

id	summary	verdict	evidence today
			(Operation disallowed in read-only or sys-only mode) and is not even try-catchable. [REPORTED]
#15H / H10	Global Admin Pause not honoured on defpact continuation steps	**DESIGN**	Step-0-only GAP gating is intentional and consistent across all 6 defpact flows in the codebase. Residual time-window exposure rides on #68L (no TTL). [REPORTED]
#16H / H12	SWP|S>UPDATE-SUPPLIES accepts non-positive reserve values unchecked	**FIXED**	15_SWP.pact:536-552 — an unconditional non-negativity enforce inside the map lambda. [V-read]
#17H / H11	can-add gates both deposits **and** withdrawals — one owner key can freeze every LP's exit	**FIXED**	18_SWPLC.pact:962-963 — UEV_RemoveLiquidity's @doc now states *"intentionally does NOT gate on <can-add>"* and the check is gone. Decided after industry research: Curve and Balancer both structurally exempt LP exit from their pause. [V-read]
#18H / H6	A_DefinePrimordialPool reads primality but never enforces it	**FIXED**	15_SWP.pact:800-804 — primality is now the sixth term of the fold; comment names the H6 fix. Live adversarial repro was found *structurally impossible* (UEV_CheckAgainstMass blocks any duplicate 3-token pool once the real one exists) and the audit said so rather than claiming a repro. [V-read]
#19H / H4	Tracer graph is append-only; disabled/frozen/sleeping pools never filtered; no fallback route	**FIXED**	16_SWPI.pact:1354-1360 — URC_BestEdgeFiltered restricts edge candidates to the call's own active universe; URC_HopperActive (:111) is the live-swap entry, URC_Hopper stays unfiltered for issuance-time pricing (:1504-1505). A **second layer** was

id	summary	verdict	evidence today
			needed after adversarial testing caught a disabled *parallel* pool still being selected. [V-read]
#20H / H2	URC_ComputeGraphPath crashes (at 0 []) instead of returning a clean “no path”	**FIXED**	14_SWPT.pact:604, :963, :1046 — (if (> (length fp) 0) (at 0 fp) [BAR]) in all three variants. [V-read] The first repro attempt hit a pre-existing short-circuit instead of the real crash; the audit records the correction. [REPORTED]
#21H / H3	Removing a principal token permanently orphans every Tracer entry filed under it, with no resync path	**FIXED (redesign)**	Not a patch: SwapTracerV1 → V2 (now V3), the principal-keyed SWPT|Tracer replaced by a plain token-adjacency SWPT|Graph (14_SWPT.pact:350). **Nothing in SWPT is keyed by principal identity any more**, so orphaning is structurally impossible. Migration utility SWPI::A_RebuildGraph (16_SWPI.pact:2711), proven idempotent. [V-read] Three follow-ups: principals capped at 7, removal floored at 2 remaining, A_RotatePrincipal added (15_SWP.pact:158), and majors made permanently fixed (URC_IsMajorPrincipal, :106). [V-cmd]
#22H / H7	Two unreconciled asymmetric-deficit pricing models stack in Standard mode	**DESIGN**	Both charges are meant to stack, and both were verified scoped exclusively to the asymmetric portion — two distinct levies on one base, not double-billing. [REPORTED]
#23H / H5	Weight-precision validation in UEV_Issue computed then discarded — dead code	**FIXED**	Closed as a byproduct of #11C: H5’s exact location *is* the map that C4’s fix wrapped in a real enforce. Logged rather than silently dropped. [V-

id	summary	verdict	evidence today
			read]
#24H / H1	UC_ComputeD/UC_ComputeY use a fixed iteration count with no convergence check	**FIXED**	12_U_SWP.pact:370-374, :445-446, :548 — all three bumped to 12 iterations, @doc at :521-530 records the measurement: at 1000× skew UC_ComputeD at 6 iterations was **0.0078 short** and fully converged (bit-identical to a 255-iteration reference) by iteration 10; 12 gives 2 iterations of margin. Pact is Turing-incomplete, so a dynamic convergence break is not expressible. Cost measured: **+64 gas flat** per stable swap. [V-read]
#25H / H8	Asymmetric-deficit compensation never returned to the diluted pool's own LPs	**DESIGN**	Protocol-wide value capture (treasury / special targets / primordial-pool boost) is the intended model, not per-pool LP protection. [REPORTED]

MEDIUM

id	summary	verdict	evidence today
#26M / M9	Slippage bound is symmetric (min *and* max), not a pure floor — a swap can revert because the price moved **in the user's favour**	**FIXED**	19_SWPU.pact:1389 and :1449 — upper bound commented out, not deleted, at both consumer sites, with a cross-reference between them. Researched against 5 protocols first: floor-only everywhere, zero counter-examples. [V-read]
#27M / M13	SmartSwapNoSlippage recomputes the pools to refresh via a fresh post-swap BFS instead of the swap's own traversed edges	**FIXED**	3_Talos/04_TS01-C3.pact:963, :1045 — (if (= (length out) 4) (at 3 out) []), the swap's own recorded distinct-edges. The (= (length out) 4) guard is a *later* repair on top of the fix: indexing (at 3 out) unconditionally turned every refused swap into an index fault. [V-read]

id	summary	verdict	evidence today
#28M / M10	kda-pid snapshotted at defpact Step 0, reused at Step 1, no re-validation or TTL	**DESIGN**	A fixed price across one logical multi-step event is intentional; re-pricing mid-flow would be internally inconsistent. Residual exposure rides on #68L. [REPORTED]
#29M / M8	Draining LP supply to exactly zero re-triggers genesis-ratio pricing regardless of dust in tracked reserves	**REFUTED**	*"If dust is left, it hasn't been drained to zero."* Reserves and LP supply hit zero together by construction; the finding's premise is a state the code cannot produce. [REPORTED]
#30M / M6	C_ChangeOwnership is one-phase — a fat-fingered destination permanently strips the true owner	**DESIGN**	Mechanics confirmed; the mitigation (addresses are pasted or Stoic-tagged, and the UI previews the destination pre-sign) is client-side and complete for the threat. Consistent with every other one-shot transfer here. [REPORTED]
#31M / M7	C_EnableFrozenLP/ C_EnableSleepingLP have **no** pool-owner authorisation at all	**FIXED**	15_SWP.pact:762-778 — CAP_Owner swpair added to both defcaps, each with a comment naming the #31M/M7 fix. [V-read]
#32M / M11	Permissioned pool issuance charges IGNIS+STOA **before** the admin gate that can reject it	**DESIGN → REOPENED → FIXED**	See §3.4(e). The design verdict rested on a premise retracted in 2026-08 and never revisited until 2026-09-17. Fixed today: 20_MTX-SWP.pact:1007-1027 hoists the whole (if p …) form into step 1, ahead of UEV_Issue and therefore ahead of all money. **Witnessed**: RedTeam/[RT-F]_Griefing.repl:244 <<RT-F-002>>, with an attribution control at :252. [V-read]
#33M / M12	Explicit rollback costs strictly more than silent abandonment; no TTL on open pacts	**DESIGN (split)**	Rollback-costs-more **verified and measured** in Part III: 53.00 IGNIS extra, nothing refunded.

id	summary	verdict	evidence today
			No-TTL correctly closed at #68L. [REPORTED — DEFECT-LEDGER.md §8.4]
#32bM	**Off-cycle correction:** M11/M12's "MTX-SWP has zero Talos wiring, unreachable" premise is factually wrong — TS01-CP has wired it since the repo's first commit	—	Verdicts were **left as-is** and deferred. 3_Talos/05_TS01-P.pact:221-330 wires **eight** starter wrappers over the four MTX-SWP defpacts (ref-MTX-SWP:: at :234, :245, :256, :270, :285, :300, :315, :330). [V-cmd] Reopened and resolved 2026-09-17.
#34M / M2	BFS keeps only one chain per node; routing does zero cross-route value comparison — **and the worst-case gas crisis found while fixing it**	**FIXED (13 phases)**	Phase 1 (best-of-3 by actual payout) stays live as the self-searching fallback; Phases 6–10 built the bundle/dirty-read redesign. 16_SWPI.pact:868-980, 14_SWPT.pact PathCache. [V-cmd]
#34bM	UEV_Issue's Stable-pool anchoring allows *transitive* connectivity to DLK instead of requiring direct adjacency to any current principal	**FIXED**	16_SWPI.pact — rewritten to test the first token's direct neighbours against the full principal list; comment tag #34bM present. [V-cmd] Also repaired a test fixture that had been *relying on the bug* to build a long BFS chain.
#35M / M1	Stable-swap maths silently drops all but the first input position	**REFUTED**	URC_Swap and UC_BareboneSwap both independently enforce length(input-amounts) == 1 for pool-type "S" before UC_ComputeY is reached. [REPORTED]
#36M / M5	C_Issue and MTX|C_Issue duplicate the entire write-side issuance sequence, including a duplicated hardcoded genesis-mint constant	**FIXED**	16_SWPI.pact:2653 XE_IssueWrite is the single chokepoint; :401 GENESIS_LP_SUPPLY replaces the duplicated 10000000.0 literal, used at :2694. [V-read] Wiring the new call surfaced a genuine pre-existing gap: MTX-SWP's own P|A_Define had never registered MTX-SWP as

id	summary	verdict	evidence today
			an approved IMC caller on SWPI.
#37M / M3	Several helpers crash on empty input (enumerate 0 -1 evaluates to [0 -1], not [])	**FIXED**	Guards at the 5 real root-cause sites. The owner initially pushed back, suspecting a botched rename; a repo grep confirmed zero callers on 4 of 7 named functions, and tracing the chain further than the finding did surfaced a **5th, previously unflagged** site (SWPT::URC_MakeGraph). Reachable from the live CC_SmartSwap whenever no pool has ever been issued. **Witnessed**: SWP|TX 003b. [V-cmd]
#38M / M4	BFS graph-node lookup linear-scans on every pop ($O(V^2)$), no size cap	**FIXED**	13_U_BFS.pact:382 UCx_GraphNodeLinks rewritten to a single filter pass. Measured on the real ~102-pool topology: byte-identical 7-hop route, **423,762 → 256,867 gas (~39%)**. [V-read/REPORTED]
#39M / M14	ClientThreeV2/ClientPactsV2 overwritten in place instead of archived	**FIXED → SINCE DELETED → CLOSED AS SUPERSEDED**	**The only fix in this tree that is not present today.** See §3.4(d). [V-cmd]

LOW

id	summary	verdict	evidence today
#40L	UC_LpID calls a cross-module UEV_* from inside a nominally pure UC_*	**FIXED**	Traced the only caller: both lists are structurally guaranteed equal length, so the enforce could never fire. Removed; 12_U_SWP.pact:773. [V-cmd]
#41L	The UC_* surface transitively inherits enforce from U|LST bounds-guards	**DESIGN — documented exception**	Unlike #40L, these are load-bearing guards inside live maths; deleting them strips real crash protection. Codified as StoicSyntax.md §6.1 (v1.8.0 → v1.9.0). [REPORTED]
#42L	UC_ComputeWP/InverseWP	**CLOSED — covered by**	Both issuance and

id	summary	verdict	evidence today
	divide by a weight with no zero-guard	**#11C + #8C**	modification floor every weight at >= 0.1. [V-read]
#43L	UC_ComputeD's docstring claims 5 iterations; the code runs 6	**CLOSED — byproduct of #24H**	Doc and code both say 12 now (12_U_SWP.pact:521, :548). [V-read]
#44L	UCX_*/UDCX_* is an uncodified naming tier	**TAGGED → since done**	StoicSyntax-Prefixes.md defines the tier; the rename landed (UCx_GraphNodeLinks, 13_U_BFS.pact:382). [V-cmd]
#45L	URC_AllGraphPaths misleadingly named — returns one shortest chain per node	**FIXED**	Renamed URC_ShortestChainPerNode (14_SWPT.pact:116, :610, plus FromRaw/FromGraph variants). [V-cmd]
#46L	Smart Swap REPL coverage is one transaction with no route assertions	**FIXED**	SWP|TX 016a now captures its return value and pins the measured output; the assertion was adversarially proven by corrupting the expected value and observing a real FAILURE. [V-cmd]
#47L	Unrecognised pool-type falls back to a silent -1.0 sentinel	**REFUTED — unreachable by construction**	Every swpair ID is built by one exhaustive function (UC_PoolID → UC_Prefix, only "S"/"W"/"P") called from one insertion point. Structurally closed, not merely unreached. [REPORTED]
#48L	Undocumented magic constants 5040000.0, 10000000.0	**DESIGN**	10000000.0 is the same fixed genesis LP mint as GENESIS_LP_SUPPLY; 5040000.0 is owner-verified share-computation. [REPORTED]
#49L	URC_Hopper's doc says "cheapest available edge"	**FIXED**	16_SWPI.pact:1326 — corrected to "highest-output edge", matching what #6C's fix made it do. [V-read]
#50L	UR_StoaValue performs an ungated table write as a side effect of a read	**FIXED**	15_SWP.pact:1020-1032 — the update is gone; @doc records that the first-instinct fix (with-default-read) was tested and found *wrong* before being proposed, because its default doesn't cover a

id	summary	verdict	evidence today
			field missing from an existing row. [V-read]
#51L	C_ModifyWeights composes bare (SECURE) instead of a named master cap	**REFUTED — correct shape**	It is the module's one C_* whose write half is XB_* (genuinely called cross-module from SWPL) rather than XI_*; XB_ModifyWeights carries its own UEV_IMC + full named cap. [REPORTED]
#52L	XE_Issue/XI_ToggleFeeLock return meaningful values with no @doc	**FIXED**	[REPORTED]
#53L	UEV_PoolFee's 320.0 upper bound has non-obvious units	**FIXED (doc)**	Fee is per-mille, so 320.0 = 32%; mirrored across LP/special/boost components so the combined worst case is 960‰, leaving 4% fees can never consume. 15_SWP.pact:1529. [V-cmd]
#54L	AHU/AUP_SwapPair(s) falls outside the module's prefix vocabulary	**DESIGN — AU_ formalised**	The identical pattern repeats across 6 modules; AU_ added to StoicSyntax.md v1.10.0, existing instances deferred to a cross-module sweep. [REPORTED]
#55L	XE_CanAddOrSwapToggle redundantly re-derives a check UEV_IMC already made	**FIXED**	15_SWP.pact:1766-1774 — removed, with @doc explaining that reaching the line already proves the guard list passed. [V-read]
#56L	URC_AreAmountsBalanced contains a raw enforce inside a URC_*	**FIXED + new convention**	8 of 11 callers pass raw unvalidated amounts, so it is genuinely reachable and not tautological, and no single non-URC_ chokepoint exists. Rather than choose between "leave incomplete" and "duplicate 8×", the **v (validating) specialisation** was formalised (StoicSyntax v1.10.0 → v1.11.0). Renamed URCv_AreAmountsBalanced (17_SWPL.pact:187, :1667) and a missing per-element

id	summary	verdict	evidence today
			negative check added. [V-read]
#57L	XI_AddLiqSendAndMint performs two distinct writes in one XI_*	**DESIGN**	Called from 3 sites — real reuse, matching the owner's own criteria for a shared wrapper. [REPORTED]
#58L	|KDA-PID-qualified defun names deviate from prefix-only naming	**TAGGED → since done**	308 occurrences across 17 files at the time. The rename landed: 01_U_CT.pact:364 UR_STOA-PID|Price. [V-cmd]
#59L	Reserve bump precedes the token transfer in XE|KDA-PID_AddLiqudity	**FIXED (doc)**	Traced the real worry — does MTX-SWP's defpact split bump and transfer across steps? It does not; the whole function runs inside Step 1's step-with-rollback. @doc records the invariant for future callers. [REPORTED]
#60L	LP-branding fee attribution resolves via DPTF/DPOF Konto, not SWP::UR_OwnerKonto	**DESIGN**	entity-owner never drives a debit, only a 25% smart-account interactor credit; the pool owner's normal account wouldn't qualify either, so "fixing" it would lose the incentive rather than redirect it. [REPORTED]
#61L	C_UpgradeBrandingLPs commented out of the REPL suite — untested	**FIXED**	New SWP|TX 002 proving propose→upgrade moves pending branding to live and flips Gray→Blue; the original call was stale (wrong Talos module). [V-cmd]
#62L	C_Fuel has zero REPL coverage despite moving real funds	**FIXED**	New SWP|TX 038c asserting the real DIRECT path bumps reserves by exactly the fuelled amounts, no LP minted. [V-cmd]
#63L	SWPLC defines no XI_* at all	**DESIGN**	SWPLC owns no domain tables; every C_* is pure orchestration and all real writes happen in the modules it calls. The absence is the natural shape of a 100%-

id	summary	verdict	evidence today
			orchestration module. [REPORTED]
#64L	URC_Swap/URC_InverseSwap's validation:bool conditionally enforces from inside a URC_*	**TAGGED → since done**	The v specialisation was applied: 19_SWPU.pact:1284 calls ref-SWPI::URCv_Swap. [V-cmd]
#65L	URC_HopperActive computed **twice** per self-searching Smart Swap — once in the defcap, once in execution	**FIXED**	h-obj computed once and threaded through the cap chain. Measured: **5,094,054 → 4,593,400 gas (~9.8%)** on the ~102-pool checkpoint. [REPORTED]
#65bL	Off-cycle: the graph-search engine rebuilds the whole graph on every routing attempt, and the true dominant cost (per-pool STOA repricing, 56.9%) shares it	**FIXED (7 phases, 6 shipped)**	SWPT|PathCache wired (14_SWPT.pact, 25 references), plus a topology-versioning repair (the cache was insert-only, never refreshable). Phase 1 alone: **477,825 → 11,491 gas (~41.6×)** warm. Phase 3 was built, measured, found to **regress**, and reverted. [V-cmd / REPORTED]
#65cL	Naming: URC_WorthDWK* inconsistent with the "Stoa" vocabulary	**SUPERSEDED — actioned as #65gL**	[REPORTED]
#65dL	Re-verification of #21H after the owner independently re-raised it	**No code gap; one stale @doc corrected**	Re-grepped every UR_Principals consumer; exactly one real caller outside the setters. [REPORTED]
#65eL	Major vs. minor principal distinction	**FIXED**	15_SWP.pact:106 URC_IsMajorPrincipal — a "major" is a live member of the primordial pool; majors are permanently fixed, never removable or rotatable. [V-read]
#65fL	Zero-search OURO/DLK shortcuts, boost-path caching	**FIXED**	Phase 8a measured **276,994 → 26,783 gas (90.3%)** on a warm hit. [REPORTED]
#65gL	DWK/DLK → WSTOA/SSTOA rename	**FIXED**	Caught a real near-miss: a word-boundary sed first pass renamed a hardcoded REPL pool-lookup string, because

id	summary	verdict	evidence today
			the genesis-configured tickers are *literally* "DWK"/"DLK". Caught by full regression (Load failed), reverted, redone surgically. [REPORTED]
#65hL	Targeted/early-exit BFS	**FIXED**	13_U_BFS.pact:274 UC_BFSTargeted. Verified first that BFS as an algorithm *class* is optimal here (unweighted shortest path, no edge weights or admissible heuristic for Dijkstra/A* to exploit) — the gap was in the implementation. [V-read]
#66L	Failure-branch OutputCumulator objects hand-built rather than via a UDC_*	**FIXED**	Confirmed byte-identical reproduction via a standalone REPL check before the change, not just a code trace. [REPORTED]
#67L	MTX|C_AddSleepingLiquidity burns a Step-0-cached nonce amount	**CONFIRMED SAFE, no change**	DPOF::C_Burn enforces live supply at execution time; a stale cache can only cause a revert, never an over-burn. This investigation is what surfaced #32bM. [REPORTED]
#68L	No TTL/expiry on any of the 6 defpact flows	**DESIGN — structural limitation**	Pact has no scheduled execution; nothing can run against an abandoned pact without someone submitting a continuation. Not expressible as ordinary Pact logic. **Still open.** [REPORTED]
#69L	MTX-SWP|S>ADD-LQ's doc implies a bounded kda-pid lock window	**CLOSED — premise doesn't hold**	The current @doc makes no such claim. [REPORTED]
#70L	SWP|C_Fuel/SWP|C_Firestarter public on TS01-C3 but undeclared on its interface	**FIXED**	3_Talos/04_TS01-C3.pact:88-89 declares both; implementations at :625, :827. [V-read]
#71L	SWPU/SWPLC call SWP::C_ToggleAddOrSwap directly rather than via	**DESIGN — documented**	Rerouting to the existing XE_CanAddOrSwapToggle would silently strip real

id	summary	verdict	evidence today
	an XE_*		IGNIS billing, LP-role bootstrap, and **the only ownership check in the chain**. 15_SWP.pact:2165-2167 records why. [V-read]

3.4 Six findings worth the telling

(a) #7C / C3 — the rounding bias that was not a rounding bug

Two independent auditors reported the same thing: all six swap formulas round toward the trader rather than the pool, giving a repeatable fee-free round-trip profit. Numeric simulation confirmed the bias empirically. The obvious diagnosis is floor/ceiling placement — the classic AMM mistake — and the obvious fix is to move the roundings.

The diagnosis was wrong. The audit traced it to **Pact's native ^ silently dropping decimal exponentiation into IEEE-754 double precision**, confirmed empirically: on one call the result differed from exact repeated multiplication by ~0.013 *absolute*. Pact's +, -, * and / on decimals genuinely are arbitrary-precision; ^ is not, and nothing in the language says so.

That reframes the problem completely. The stable pool's exponents are always whole numbers (token count, and count+1), so exact multiplication *is* available: UC_IntPow (12_U_SWP.pact:508) replaces ^ there, and both directions were then proven exact — zero bias — live. [REPORTED] A second, genuinely separate correction went in alongside: **floor the final output, not the intermediate solved balance** (:378, :450, :635, :702 [V-read]), keeping intermediates at internal precision 24.

The weighted pool cannot be fixed the same way, and the audit says so at the site. x^weight is a genuine fractional power; no exact-multiplication trick exists in pure Pact. 12_U_SWP.pact:590-603 carries the reasoning verbatim [V-read]: writing a from-scratch high-precision power routine (Newton's method or a power series) was *"assessed and explicitly declined as disproportionate to the residual risk"* — the bias scales with float64's ~1e-16 *relative* precision, is orders of magnitude below anything resembling insolvency, stays internally consistent (the same computed value backs both the transfer and the reserve update), and for realistic non-24-decimal token precisions is routinely swallowed entirely by final settlement rounding. It closes: *"Accepted as a bounded, documented limitation of the underlying language, not tracked as an open bug."*

That is the right way to leave something open. The alternative — quietly fixing the stable half and marking C3 "FIXED" — would have made the tracker look cleaner and the system less understood.

(b) #21H / H3 — the finding that was answered by deleting the concept, not the bug

SWPT maintained a routing graph keyed by *principal token*. Remove a principal and every Tracer entry filed under it was orphaned, with no resync path. The natural fix is a resync function.

The audit built one — and then did not stop there. It replaced the principal-keyed SWPT|Tracer with a plain **token-adjacency** SWPT|Graph (14_SWPT.pact:350 [V-read]). After the redesign, **nothing in SWPT is keyed by principal identity at all**, so orphaning is not fixed, it is *unrepresentable* — regardless of what any future remove or replace does. SwapTracerV1 became V2 (and is now V3), and a migration utility SWPI::A_RebuildGraph (16_SWPI.pact:2711 [V-read]) was added and adversarially proven idempotent. Every prior #13C/#19H/#20H/#11C route assertion was re-run and confirmed byte-identical post-redesign.

Then three follow-ups, each of which is its own small lesson:

- **Fix #13** capped principals at 7 and added A_RotatePrincipal as the retirement path.

- **Fix #14** re-allowed removal (safe now) but floored it at 2 remaining, and split "rotate into self" into its own distinct enforce so the error message tells you which rule you broke.
- **Fix #44 (#65eL)** — the owner, months later and unprompted by any code reference, re-raised the original concern as a possible "major vs. minor principal" gap. The audit **re-verified against current code rather than trusting its memory of the earlier fix**: grepped every consumer of UR_Principals, found exactly one real caller outside the setters, confirmed no regression — and then implemented the owner's actual new idea anyway. URC_IsMajorPrincipal (15_SWP.pact:106 [V-read]) defines a "major" as a live member of the primordial pool and makes majors permanently fixed. Not because rotation was unsafe — #65dL had just proven it safe — but because there is no legitimate reason to retire OURO/WSTOA/SSTOA from the principals list, so the operation should not exist.

That last move is worth naming: *"we proved it can't break, and we removed the ability anyway"* is a defensible position, and the tree records both halves rather than presenting the second as though it repaired the first.

(c) #26M / M9 — the guard that punished the user for good luck

SWPU's slippage protection bound the output on **both** sides: a minimum *and* a maximum. A maximum means a swap reverts because the price moved **in the user's favour** between quote and execution. The user pays gas for a refusal that cost them nothing and would have profited them.

The audit did not argue from first principles; it checked what everyone else does. Five protocols surveyed, **zero counter-examples** — floor-only, universally. Only then was the upper bound disabled, and the choice of *how* is deliberate: **commented out, not deleted**, at both UC_SlippageMinMax consumer sites, with a cross-reference between them (19_SWPU.pact:1389, :1449 [V-read]). The intent is recoverable if someone ever wants it back.

The detour is the instructive part. Testing the change required a *full swap execution*, not just a module load — and that turned up **a real Pact 5 runtime bug**: a single-argument and parses fine and fails at execution. A load-only check would have gone green with a broken swap path.

#26M also leaves a trace that Part III later picked up. The only tests naming *"out of Slippage bounds"* are this finding's pair, and **both assert the message must NOT appear** — positive slippage is deliberately allowed. So 19_SWPU.pact's (>= feeless-final min), which the source itself calls *"the real protection this whole check exists for"*, was asserted nowhere. Part III's RT-A-004 closed that. [REPORTED — DEFECT-LEDGER.md §8.3]

(d) #39M / M14 — the fix a later sweep deleted, and why the deletion was right

M14 was small: the historical interfaces TalosStageOne_ClientThreeV2 and ClientPactsV2 had been *overwritten in place* rather than archived, unlike the sibling ClientFour block in the same file. Fix #25 reconstructed both from git history (commit df2d72e) and archived them.

Even the fix had a lesson in it. The first placement attempt — into the central interface registry, which is what the finding literally described — **genuinely failed to load**. The finding's premise was already out of date: live V3 had moved to deploy-with-module files, and V2's Smart Swap functions need a module-owned type not resolvable that early. Relocated alongside their live V3 siblings instead.

On 2026-09-02, commit 6833a21 deleted them both. [V-cmd] Its message reads:

> *"deleted dead old versions (ClientFourV1-V6BlockTime, ClientPactsV2, ClientThreeV2 — all 0/0)"*

Zero implementers, zero consumers — which is **the defining property of a deliberately-frozen archive**. The tool's own heuristic guaranteed it would delete exactly the artefact the fix created, and a usage counter cannot distinguish "unused because obsolete" from "unused on purpose". The sweep also deleted ClientFourV1-V6BlockTime, the very precedent Fix #25 had cited as justification.

Part III's re-verify caught it — the single NOT-FOUND out of 42 re-checked fixes — and recorded it as a silent deletion. **And then corrected itself, which is the part worth reading.** DEFECT-LEDGER.md §8.6 [V-read]:

> *"The mechanism is right.* ***The intent was wrong, and the correction matters more than the finding.*** *6833a21 was not a rogue tool. It was step 5b-2.4 of a sequenced phase executing a dated canon amendment — StoicSyntax-Prefixes.md §7.10, 'Retire the pool: delete the 0_Interfaces/ files; git history preserves old versions', written the same day. The frozen-copy convention was* ***deliberately abandoned****."*

And then the argument that settles it, which is about this repository's own cascade rule: a restored archive **cannot be both loadable and historical**. The deleted V2 text carries 12 type references to interface versions that no longer exist (SwapperV3, SwapperUsageV2, BrandingV1), so a verbatim restore *fails to load* — proven at the time, since Fix #25's own first attempt died on Module SwapperUsageV2 has no such member: Slippage. Rewriting those refs to today's versions fabricates a "V2" typed against types V2 never saw. Meanwhile the archive that **did** live in-tree was rewritten by three separate canon sweeps in ten days (+92% in size) until its @doc described a surface that never existed.

> *"An in-tree archive is a strictly worse provenance store than git: mutable, swept, deployed, and silently falsifiable."*

M14 is closed as SUPERSEDED. Nothing was restored, and nothing should be. [V-cmd — the interfaces are absent tree-wide] This chapter records it as the tree's one deleted fix not because the deletion was wrong, but because a reader checking this audit's claims will find a FIXED verdict whose artefact is gone, and deserves to know which of the two possible reasons applies.

(e) #32M / M11 — the verdict whose premise was retracted, and sat for three weeks

This is the case the book's own front matter is describing when it says a reachability claim needs re-checking.

2026-08-19. M11 reported that permissioned pool issuance through the MTX-SWP defpact charges IGNIS and STOA *before* the admin gate that can reject it. Verdict: **DESIGN — accepted, confirmed non-live.** Fee-before-gate ordering is a deliberate anti-abandonment incentive for defpact flows; and, independently, *"MTX|C_Issue and every MTX-SWP defpact have zero Talos wiring anywhere in the codebase — unreachable through the only supported client/gas-station path."*

2026-08-27. While investigating an unrelated LOW (#67L), the audit discovered the premise was **factually wrong**. TS01-CP (05_TS01-P.pact) had wired MTX-SWP::C_IssueStablePool/ WeightedPool/StandardPool **since the repository's first commit**. This was logged honestly as #32bM, marked as *"the one open item"* in MERGE-HANDOFF.md — and, per owner direction, the verdict was **left as-is** and deferred to a main-branch red-team pass rather than reopened.

2026-09-17. The red-team pass reached it. 05_TS01-P.pact:221-330 wires **eight** starter wrappers over the four MTX-SWP defpacts (:234, :245, :256, :270, :285, :300, :315, :330); the wrapper takes p:bool straight from the client and gates nothing but P|TS. [V-cmd]

> **Corrected 2026-09-18.** This said *"all six defpact starters"*, here and in #32bM's row above. Six is the number of defpacts in the whole tree — four in 20_MTX-SWP.pact, two in 07_MTX-AQP.pact — which is the figure #15H's row uses. The number of SWP starter wrappers on TS01-P is eight, because three issue variants and five liquidity variants fan out over four defpacts. And M11 was proven **by execution**, each step in its own committed transaction [REPORTED — DEFECT-LEDGER.md §8.4]:

step	what happens
0	SWPI::UEV_Issue … p — **no authorisation check of any kind.** p=true *skips* the spawn-limit branch, so the permissioned path is the **laxer** one.
1	**commits 5,506.0 raw IGNIS + 600.0 raw STOA** (2,919.77 / 459.0 net)
2	GOV|MTX-SWP_ADMIN refuses: "MTX-SWP Ownership not verified"

No refund, no cancel. Rollback costs a **further 53.00 IGNIS**; abandoning leaves the pact open forever (#68L). Three controls were run, because an unattributable refusal proves nothing: *attribution* (the same drive at p=false fails with a **different** message, so the p=true refusal is the admin gate and nothing else), *non-vacuity* (it completes when the master key signs), and the contrast that makes it a defect — **the single-transaction twin refuses the identical operation and charges zero**, because SWPI|C>ISSUE (16_SWPI.pact:424) hoists its conditional admin gate above UEV_Issue, exactly as the 2026-09-14 authorise-first ruling requires.

> Same logical operation, two live Talos doors, and only one charges you for a refusal — selected by p, an undocumented raw bool on the Talos signature whose @doc never mentions it.

And it falsified a claim in the ledger written by the same programme. DEFECT-LEDGER.md §7.3 had recorded MTX|C_Issue as *"the counter-example, and it is the specification — validation before money"*. That is true for p=false and false for p=true: *shape* validation precedes the money; *authorisation* follows it. The op cited as the specification was carrying the unrepaired case, and the owner had ruled with that framing in front of them. §8.4 says so in the first person.

Fixed 2026-09-17. 20_MTX-SWP.pact:1007-1027 [V-read] hoists **the whole (if p …) form** into step 1, ahead of UEV_Issue and therefore ahead of all money. Not the bare acquire — the in-source comment is explicit about why:

> *"THE WHOLE CONDITIONAL FORM IS HOISTED, not the bare acquire: only a PERMISSIONED issuance needs the admin key, so unwrapping the branch would convert a conditional gate into an unconditional one and lock out every ordinary pool issuance. That is the exact mistake CLAUDE.md records a scripted reorder making during the sweep."*

Pinned by RedTeam/[RT-F]_Griefing.repl:244 <<RT-F-002>> [V-read], with an attribution control the block needs more than most: PK_AncientHodler is *both* an ordinary account key and a DemiurgoiSithMasters member in this fixture, so an ANHD-signed drive would pass the gate and prove nothing. The same call with the same signer and only p flipped must fail for a **different** reason — and does (:252).

M12 splits the same way. "Rollback costs more than abandonment" is now **verified and measured** (53.00 IGNIS extra, nothing refunded); "no TTL" remains correctly closed at #68L.

One inference is named rather than hidden: the fixture conflates identities, so what was *measured* is that step 1 charges and that step 2's first and only obstacle at p=true is the master keyset. That a caller who cannot satisfy it therefore pays and is refused is **one inference step**, not a measurement. Closing it needs a non-Demiurgoi account that owns a DPTF.

(f) #73C's twin — the follow-up that was filed, not chased, and was worth 38%

MERGE-HANDOFF.md closed the SWP branch with a section headed *"The one item genuinely worth a look, not just deferred by design"*:

> **URC_OuroPrimordialPrice's dollar-denominated math** — flagged in #73C's own writeup as *likely* carrying the identical weight-omission bug just fixed on the WSTOA-denominated side. Both build off the same shared core… This is **not confirmed** — nobody has traced or reproduced it — but given the WSTOA sibling's bug was real and the two functions share the exact same math shape, it's the most likely next real finding in this file. **Recommend a 15-minute trace.**

The lead was also written into the source, verbatim, in URCx_PrimordialValueAndOuroSupply's own @doc. It sat unopened for nineteen days, because it was filed as a *lead* rather than a *defect*.

The 15 minutes were eventually spent, and it was real. URC_OuroPrimordialPrice computed ((R_wstoa + R_sstoa·k) × pid) / R_ouro — a flat reserve ratio that **reads no weight at all**. It was proven not by reading but by controlled experiment: reserves held constant, weights varied through the live SWP|C_ModifyWeights path [REPORTED — DEFECT-LEDGER.md §8.1]:

pool weights [SSTOA, OURO, WSTOA]	URC_OuroPrimordialPrice	weight-aware sibling	error
[0.4, 0.4, 0.2]	0.09200067782692156…	0.11998182062980…	−23.3%
[0.2, 0.6, 0.2]	0.09200067782692156… *(bit-identical)*	0.17996364248400…	−48.9%
[0.3, 0.5, 0.2] (genesis)	0.09200067782692156… *(bit-identical)*	0.14997348886490…	**−38.65%**

The output does not move by a single digit across three weightings of the pool it prices. And it is not latent: SWP|C>DEFINE-PRIMORDIAL-POOL (15_SWP.pact:781) enforces a *weighted* pool of exactly three tokens — three equal decimal weights cannot sum to 1.0 — so the pool it serves **cannot** be equal-weighted. Genesis ships [0.3 0.5 0.2].

Blast radius: the oracle write publishes it as the canonical OURO price on every swap touching the primordial pool; DEMIPAD launchpad payments convert with it; the Explorer derives display values from it. OUROBOROS's compress/sublimate clamp below $1.00 and so are masked *today* — a masking that ends the moment OURO's true price crosses $1 while the understated feed is below it.

Fixed 2026-09-17 by **delegation, not re-derivation**: 16_SWPI.pact:1768-1796 [V-read] now calls URC_TokenDollarPrice → URC_SingleWorthWSTOA → URC_WorthWSTOA's OURO short-circuit → URC_SingleOuroWorthWSTOA, a real 1-unit weighted swap through UC_ComputeWP — the only maths in the family that consumes the weights. That path is #73C's own repair, already live and proven, so the fix introduces no new arithmetic. *The defect was that a second implementation existed at all.*

And the commit says the thing that makes this the most useful story in the chapter:

> *"THE FIX CHANGED NO ASSERTION, WHICH IS THE POINT. The gate came back at 22,941 both before and after, because nothing in the suite pinned an OURO price. A 38% repricing of the token behind the oracle write, DEMIPAD launchpad payments and the Explorer moved not one test.* ***That is how the bug shipped and how its repair could have regressed unnoticed.****"* [V-cmd — git log 15dd7b7]

The witness added, <<SWP-G27>> (REPL/modules/SWP.repl:5532 [V-read]), pins the **invariant** rather than a figure — OURO's price must equal what the generic weight-aware pricer returns for OURO — because a literal would pin today's reserves and go red on any ordinary trade, while the invariant only breaks if someone re-derives the price by hand again. And it carries its own counter-example: the *removed* formula is recomputed inline from the same shared core and asserted to **disagree** (:5544), so every run re-proves both that the guard discriminates rather than comparing something to itself, and that the original defect was real on live reserves.

One adjacent defect found in the same trace is still latent. URCx_PrimordialValueAndOuroSupply hard-codes reserve positions 0=SSTOA, 1=OURO, 2=WSTOA, while C>DEFINE-PRIMORDIAL-POOL checks only membership and length — no order check, no sort on the issuance path. A primordial pool issued with the same three tokens in a different order silently transposes the reserves. Unreachable today only because genesis and the fixture happen to use the matching order. [REPORTED — §8.1]

3.5 What remains open

Open by design, and material

item	state	why
#7C / C3, weighted half — round-trip bias in weighted pools	**Accepted known limitation.** Documented in place at 12_U_SWP.pact:590-603. [V-read]	x^weight is a genuine fractional power; Pact's native ^ drops to float64 and there is no exact-multiplication trick. A from-scratch high-precision power routine was assessed and declined

item	state	why
		as disproportionate: the bias is ~1e-16 *relative*, internally consistent, and usually swallowed by settlement rounding. **It is a real, permanent, bounded arbitrage against weighted-pool LPs.**
#5C / **C11** — slippage compares fee-exclusive quotes	**Design.**	The feeless-vs-feeless comparison correctly catches reserve/price movement. The residual — a pool owner changing the *fee rate* between quote and execution — is answered by the fee-lock primitive, which is real, enforced and publicly queryable. But **it is unlocked by default**, so the protection is opt-in by pool owner. An integrator quoting against an unlocked pool has no fee-rate guarantee.
#25H / **H8** — asymmetric-deficit compensation is not returned to the diluted pool's LPs	**Design.**	It is captured protocol-wide (treasury, special targets, primordial-pool boost). LPs in a pool that receives a large asymmetric deposit are diluted and compensated *indirectly*, not directly. This is a legitimate design choice and it should be visible to anyone providing liquidity.
#68L — no TTL on any of the 6 defpact flows	**Structural.**	Pact has no scheduled execution; nothing can force-expire an abandoned pact. Open pacts persist forever. #15H and #28M's residual time-window exposure both ride on this, explicitly.
#33M / **M12** — explicit rollback costs more than silent abandonment	**Design, now measured.**	53.00 IGNIS extra, nothing refunded. For the AddLiquidity family this is a ruled anti-spam design; the MTX|C_Issue instance has no such ruling but is small next to what M11 was charging. [REPORTED]
#4C / **C4-equivalent trust levers**	—	#30M (one-phase ownership transfer), #60L (fee attribution), #48L (magic constants) are all closed as design with client-side or owner-side mitigations. None is a defect; all are trust assumptions a reader should be able to see.

Latent, found but not chased

- **URCx_PrimordialValueAndOuroSupply hard-codes reserve order.** 0=SSTOA, 1=OURO, 2=WSTOA, while the defining capability checks only membership and length. A primordial pool issued with the same tokens in a different order transposes the reserves silently. Unreachable today only because genesis and the fixture use the matching order. [REPORTED — DEFECT-LEDGER.md §8.1] **No finding ID, no fix, no pin.**
- **A measured aside from the M11 work, not chased:** MTX|C_Issue step 0 is the only Talos-reachable SWP client op found that collects **zero IGNIS while doing real work** — 4,124 gas at p=true, **76,934 at p=false** — while its three sibling defpacts all take LQ|INITIATION-FEE in step 0 as anti-spam. GAS_PAYER Case 1 whitelists the ouronet-ns.TS... prefix, so the station pays it. [REPORTED — §8.4]

Tagged-for-sweep items that have since been done

Four LOW findings were deferred to a protocol-wide StoicSyntax pass rather than fixed piecemeal. That pass ran, and all four are closed, though the SWP tracker was never updated to say so:

finding	then	now
#44L	UCX_*/UDCX_* uncodified tier	UCx_GraphNodeLinks, 13_U_BFS.pact:382 [V-cmd]
#58L	KDA-PID → STOA-PID across 308 sites in 17 files	01_U_CT.pact:364 UR_STOA-PID|Price [V-cmd]
#64L	URC_Swap → URCv_Swap	19_SWPU.pact:1284 calls ref-SWPI::URCv_Swap [V-cmd]
#54L	AU_ category formalised, instances deferred	StoicSyntax.md v1.10.0 [REPORTED]

For completeness, the interface cascade also completed: every SWP-family interface is now one to two versions ahead of where the audit left it — UtilitySwpV2, BreadthFirstSearchV2, SwapTracerV3, SwapperV4, SwapperIssueV4, SwapperLiquidityV2, SwapperLiquidityClientV2, SwapperUsageV3, SwapperMtxV4, TalosStageOne_ClientThreeV4, TalosStageOne_ClientPactsV4. [V-cmd]

Fix presence, re-checked

Part III re-verified all 42 SWP fixes against current source and found **41 present, 1 not found** (M14). [REPORTED — DEFECT-LEDGER.md §8.2] This chapter independently spot-checked **32** of them by locating the fixed construct in current source, and agrees on every one, including M14:

> Present and read: #3C, #72C, #73C, #74, #6C, #7C, #8C, #9C, #10C, #11C, #13C, #16H, #17H, #18H, #19H, #20H, #21H, #24H, #26M, #27M, #31M, #36M, #38M, #45L, #49L, #50L, #55L, #56L, #65eL, #65hL, #70L, #71L. [V-read / V-cmd]
> Absent: #39M / M14, by deliberate later decision. [V-cmd]
>
> **Corrected 2026-09-18.** The sentence above said **22**, and the list under it has always enumerated 32. The list is the evidence — every id in it carries a source citation in §3.3 — so the number was the error, not the list.

Proof tags survive — eight of nine in a gated file. All nine SWP|TX proof identifiers named in the fix write-ups still exist in the suite files — 015, 015b, 002, 003b, 016a, 038c, 032o3, 032z6f, 032z8e. [V-cmd] Eight of them live in [6.3]_SWP.repl, which ZALL.repl:43 and REPL/modules/SWP.repl:20 both load.

015b does not. It exists only in [6.2+3]_DPTF-SWP_Issuance-Only.repl, and that file is in _gate.py's EXCLUDED list by name — *"ALTERNATIVE to [6.2]+[6.3], both of which ZALL runs"* (_gate.py:160-161). Stage01_Tester.repl:42 loads it as the committed fast-path default, but the gate does not, so #72C's only witness is not executed by the thing that decides whether the tree is green. [V-cmd]

> **CORRECTED 2026-09-18.** This paragraph said both files *"run under ZALL.repl and REPL/modules/SWP.repl"*, and concluded *"unlike ATS, SWP's proofs did not rot… that single decision is why this tree's evidence is*

> *still alive and ATS's largely is not."* Neither half holds. The issuance-only file is gate-excluded, so one of the nine tags is in exactly the position the chapter was contrasting itself against; and ATS's four unwitnessed fixes were all closed on 2026-09-17 (<<UTIL-15>>, <<ATS-G28>>, <<ATS-G29>>, <<ATS-G30>>, DEFECT-LEDGER.md §8.26), so the comparison it drew is no longer true either.

The point the paragraph was reaching for still stands, narrowed: writing proofs into the canonical suite rather than into scratch harnesses is why eight of the nine are alive. 015b is the counter-example that shows the filing decision has to be re-checked, not assumed — a file can be a canonical suite file and still be outside the gate.

Three ordering observations from the re-verify, still open

Part III's SWP re-verify independently found CAP_Owner sitting **after** the business check in SWP|C>ENABLE-FROZEN, C>ENABLE-SLEEPING, C>ADD-OR-SWAP, S>WEIGHTS, S>RT_CAN-CHANGE and SPW|S>UPDATE_SPECIAL-FEE-TARGETS. [REPORTED — §8.2] The 2026-09-14 authorise-first sweep matched compose-capability (GOV|...) forms and never saw the CAP_* ownership ones.

Per CLAUDE.md's standing engineering position these should **not** be resolved by reordering — ordering is only a proxy for testability, and swapping it hides the other guard instead. The remedy is a fixture that satisfies the first guard so both are exercised. As of this chapter, that work is recorded but not done for the SWP sites.

And one shape flagged as worth watching, not yet actioned: C_ToggleAddOrSwap's ico1 binding performs real DPTF::C_Toggle*Role **writes inside an eager let, above the with-capability** — the same eager-binding pathology this book's Rule 1 exists for, in a function whose only ownership check lives in the capability it has not acquired yet. [REPORTED — §8.2]

---

8. DPDC — collectables

Source tree: Audit/module-audits/DPDC/ (7 files, 5,093 lines) **Audited:** 2026-08-18 → 2026-08-27, on a dedicated dpdc branch (48 commits) **Scope:** all 11 modules, 01_DPDC-UDC.pact through 11_EQUITY+.pact **Tracked items:** 58 · **Fixed and proven:** 35 · **Refuted:** 13 · **Deferred:** 4 **Verification pass for this chapter:** 2026-09-17. Every fix recorded below was looked for in current source. Results are in §7.

---

1. What DPDC is, and what breaks if it breaks

DPDC — *Digital Property / Deed Collectable* — is Ouronet's NFT and SFT stack. Where DPTF handles plain fungible tokens and DPOF handles metadata-carrying "ortofungibles", DPDC handles the two asset families where **individual units are distinguishable**: DPSF (semi-fungibles — many identical copies of a numbered *nonce*, like a concert-ticket tier) and DPNF (non-fungibles — one holder per nonce, the ordinary NFT). It is not one module. It is eleven, sharing one set of tables:

module	what it owns
01_DPDC-UDC	pure constructors — no tables
02_DPDC	**the tables**: Properties, Nonces, VerumRoles, Account, AccountSupplies, for both DPSF and DPNF
03_DPDC-C	the credit/debit ledger primitive — every balance change in the family passes through it
04_DPDC-I	collection issuance and account association
05_DPDC-R	the twelve role toggles (freeze, burn, mint, royalties, transfer, …)
06_DPDC-MNG	admin supply operations: add-quantity, burn, respawn, and the Wipe* family
07_DPDC-T	transfer, bulk transfer, repurpose, and IGNIS royalty collection
08_DPDC-S	**sets** — combine N nonces into one "set" nonce and break it back apart
09_DPDC-F	**fragments** — split one nonce into 1,000 tradeable pieces held in protocol escrow
10_DPDC-N	per-nonce metadata mutation (name, description, score, URIs, royalties)
11_EQUITY+	shareholder collections — a cap-table instrument built on DPSF

Three things about that shape matter for an audit. First, **one shared table set, ten thin writer modules** is the widest fan-out of any module family in Ouronet: a missing check in 03_DPDC-C is a missing check in all ten callers at once. Second, DPDC holds *escrow*: when a nonce is fragmented, the real asset moves into the protocol's own dpdc system account and 1,000 fragment claims are issued against it. Third, 11_EQUITY+ is a **financial instrument** — package shares with a dilution-sensitive cap table — and at the start of this audit it had zero test coverage anywhere in the repository.

If DPDC breaks, the failure modes are: supply minted from nothing (the ledger primitive), user value permanently stranded (a set that can be made but never broken), escrow collateral destroyed under outstanding fragment claims, or a collection owner locked out of their own collection with no recovery path. All four of those were found.

Some of this code is **live on StoaChain**. That is not a hypothetical audit.

---

2. How it was audited

The tree records five documents and one honest gap. The real sequence was:

Round I — parallel deep reads (2026-08-18). Eleven auditors, one per module file, working read-only, each told to load StoicSyntax.md first and to assume nothing is correct despite the code being live-adjacent. Cross-module composition was traced explicitly rather than assumed, because the shared-table decomposition means no single file contains a complete operation. Output: ROUND-01-FINDINGS.md, 1,589 lines, **frozen** — never edited after the fact.

Ranking. ISSUES-RANKED.md flattened the eleven per-module reports into one severity-ordered list, #1C … #55L, each carrying a back-reference to its module-local id (DPDC-C·C1 and so on). This is the working document: findings were taken **one at a time, in this order**.

Round I feedback — the owner loop. ROUND-01-OWNER-FEEDBACK.md (938 lines), append-only, one entry per finding. Thirteen findings died here: the owner corrected a technical premise and the finding did not survive it. Those are recorded with the correction, not deleted.

Round II — sequential fixes. ROUND-02-FIXES.md (1,638 lines), 35 numbered Fix #N entries, each with root cause, diff, and REPL proof. Code changed **only** here, one fix at a time, each green-lit individually.

The hard rule. Written into README.md and worth quoting, because it exists for a reason:

> A finding that was only discussed in chat and never landed in these files **is not closed**, no matter how thoroughly it was reasoned through.

That rule is inherited verbatim from the SWP audit, where findings silently skipped mid-session went uncaught until an audit-of-the-audit. Four things had to happen in the same turn a verdict was reached: feedback entry, tracker row, ISSUES-RANKED annotation, and — if code changed — a numbered fix entry with its proof.

The method's one real discipline

Findings were proved by **reverting the fix and re-running** (git stash), so the pre-fix bug shape was observed rather than assumed. FINAL-AUDIT-REPORT.md names eleven: C1, C7, H1, H2, H5, H8, H11, #24M, #31M, #32M, #35M. This caught two methodology errors before they were presented as proof — in both cases an expect-failure that "passed" because an unrelated UEV_IMC guard rejected the call, not the check under test. The audit recorded those errors rather than quietly correcting them.

It also produced the round's sharpest result, on **#7C**: the C_UpdateSetMultiplier type bug was confirmed **byte-identical on mainnet** through a keyless Pythia dirty-read against the deployed ouronet-ns.DPDC-S (hash Qslr8IXA10HEYsiHPnjvvCy4hYNIh3bfPQvD7w5QEoU). The feature had never worked in production, and the audit could say so with a hash rather than a guess.

Round III never happened

README.md names a Round III re-verify and says *"(not yet created)"*. It was not created. The FINAL-AUDIT-REPORT.md closes the audit on Round II plus Z.repl green. **The DPDC audit is a two-round audit that planned a third.** Everything below is verified against current source by this chapter, which is the closest thing the tree has to the Round III that was skipped.

New test coverage built

Three canonical integration suites, all wired into the live Stage02_Tester.repl pipeline rather than written and left disconnected [VERIFIED by command — Stage02_Tester.repl lines 38, 41, 45]:

suite	covers	born from
REPL/Stage_02/[6.1.1]_EQUITY.repl	Issue baseline, Make/Convert/Break exact conservation, 5 negative paths	#22H
REPL/Stage_02/[6.1.2]_DPDC-FRAGMENTS.repl	Make→Merge conservation, repurpose-without-consent	#27M
REPL/Stage_02/[6.1.3]_DPDC-S.repl	Primordial/Composite/Hybrid Make→Break round trips, admin mutations	#39L

All three modules (EQUITY, DPDC-F, DPDC-S) had **zero reachable coverage** before. The pre-existing [6.1]_DPDC.repl contained EQUITY test code, but the file was disabled, crashed on an earlier unrelated bug when run directly, never created the collection it tested, and its assertions were print-only. That is worth stating plainly: the prior coverage was not thin, it was **fictional**.

---

3. The findings

Severity is as **recorded by the audit**. Verdict is as closed by the audit. The evidence column is **this chapter's** verification — what, in current source as of 2026-09-17, would have to be deleted for the finding to come back.

Line numbers drift. Every citation below was resolved fresh on 2026-09-17.

CRITICAL (8)

id	summary	verdict	evidence today
#1C	Unsigned amount on every Credit/Debit path — a negative amount inverts direction and mints supply from nothing	**FIXED**	03_DPDC-C.pact:594 UEV_Amount, called at :1086 from the sole write path. [VERIFIED by reading]
#2C	C_IgnisRoyaltyCollector debits an arbitrary patron with no ownership check	**REFUTED, hardened anyway**	07_DPDC-T.pact:396 CAP_EnforceAccountOwnership sender inside IGNIS|C>DEBIT. [VERIFIED by reading]
#3C	Same unsigned-amount hole reached through C_MakeFragments/C_MergeFragments	**ALREADY CLOSED by #1C**	Same chokepoint. Rejection traced live to 03_DPDC-C.pact UEV_Amount. [VERIFIED by reading]
#4C	C_RepurposeCollectableFragments moves a holder's balance with no consent/freeze/wipe gate	**REFUTED — design**	Deliberate admin account-recovery tool, gated on CAP_Owner via wipe-mode=true. No code change.
#5C	Burn/wipe orphans fragment collateral held in the dpdc escrow account	**FIXED (+narrowed)**	06_DPDC-MNG.pact:439-470 C>REMOVE-CLASS-ZERO-NONCES, escrow block narrowed to *currently fragmented* nonces via UR_SplitNonceData. [VERIFIED by reading]
#6C	Composite set with	**FIXED**	08_DPDC-S.pact:974-977 — (fold (and) true (map (lambda

id	summary	verdict	evidence today
	allowed-sclass = 0 — Make succeeds, Break can never succeed, value stranded		(sc) (> sc 0)) ...)). [VERIFIED by reading]
#7C	C_UpdateSetMultiplier crashes on every call — a let type annotation bug. Confirmed on mainnet	**FIXED, then removed entirely**	Function deleted by #15H. 08_DPDC-S.pact:99 and :1143 carry removal notes; Talos wrappers removed at 01_TS02-C1.pact:148/1190, 02_TS02-C2.pact:145/1041. [VERIFIED by command — 0 live definitions tree-wide]
#8C	how-many-sets unbounded on Make/Break	**FIXED**	08_DPDC-S.pact:323 and :338 — (enforce (> how-many-sets 0) ...) in both C>MAKE and C>BREAK. [VERIFIED by reading]

HIGH (14 + 2 sub-findings)

id	summary	verdict	evidence today
#9H	Talos DPSF|C_UpdatePendingBranding passes 7 args to a 6-parameter function — feature 100 % broken	**FIXED**	01_TS02-C1.pact:418-427 — 6 params, 6 args forwarded. [VERIFIED by reading]
#10H	NFT issuance billed at the SFT price — 20 % silent revenue shortfall	**FIXED**	04_DPDC-I.pact:263-264 and :275 — son selects "issue-sft" vs "issue-nft". **The implementation was later rewritten by the Part II re-pricing work and the distinction survived it.** [VERIFIED by reading]
#11H	NFT genesis owner==creator denied its own royalty/exemption/creator roles	**FIXED**	04_DPDC-I.pact:505-511 — the NFT owner==creator branch writes true for all three. [VERIFIED by reading]
#12H	Shared XE_* write surface performs zero value-level validation	**REFUTED — verified live**	Every real call site either self-derives (current+1) or is pre-validated by the calling defcap.
#12Hb	*(found verifying #12H)* nonce name/description/meta-data/asset-type/uri-* had no content validation, ever	**FIXED**	02_DPDC.pact:1150/1157/1181/1190/1205 — five validators, wired into 03_DPDC-C.pact:553-556 and each DPDC-N update cap. [VERIFIED by reading]

id	summary	verdict	evidence today
#12Hc	*(found verifying #12Hb)* meta-data.composition — the escrow record for an NFT set — overwritable to arbitrary values	**FIXED**	10_DPDC-N.pact:456 UEV_NotSetInstance, wired at :241 and :352 into the shared C>DATA/C>SET-DATA chokepoints. [VERIFIED by reading]
#13H	can-freeze gates unfreeze as well as freeze → permanently bricked account	**FIXED**	05_DPDC-R.pact:239-242 — (if frozen (UEV_CanFreezeON …) true). [VERIFIED by reading]
#14H	Pause gates no mutating entrypoint in DPDC-MNG	**REFUTED — design**	Pause halts transfers only (MultiversX token semantics); DPDC-T enforces it, DPDC-MNG correctly does not.
#15H	Set-class score-multiplier unbounded and retroactively re-priceable	**FIXED ×3**	08_DPDC-S.pact:1031-1047 UEV_ScoreMultiplier — precision 3 and [1.0, 100.0]; wired into all three Define caps (:351, :362, :375); Update path deleted. [VERIFIED by reading]
#16H	UEV_TransferRoles reads sender twice — receiver-side role check is dead code	**FIXED**	07_DPDC-T.pact:688 — (r:bool (… UR_CA|R-Transfer id son receiver)), with the DPOF sibling cited in the comment. [VERIFIED by reading]
#17H	C_RepurposeCollectable skips frozen/transfer-role gates	**REFUTED — design**	Same admin escape hatch as #4C; owner explicitly rejected freezing the source afterward.
#18H	Native NFT Credit never checks the nonce isn't already held	**REFUTED — verified live**	All three real callers (Transfer, CreateNewNonce, RespawnNFT) gate at their own layer.
#19H	UR_N|Score fails to clamp the -1.0 unscored sentinel in 3 of 4 branches	**FIXED, then renamed**	08_DPDC-S.pact:554-570 URC_N|Score — sentinel checked **once, at entry, on the raw value**. [VERIFIED by reading]
#20H	C_UpdateNonceIgnisRoyalty has no upper bound at all	**REFUTED — design**	Owner: no principled ceiling exists. Precision check only.
#21H	score-multiplier unvalidated at Define, checked at Update	**ALREADY CLOSED by #15H**	Subsumed by the fix chain above.

id	summary	verdict	evidence today
#22H	EQUITY — the entire financial-instrument module had zero test coverage	**FIXED**	REPL/Stage_02/[6.1.1]_EQUITY.repl, wired at Stage02_Tester.repl:38. [VERIFIED by command]

MEDIUM (16)

id	summary	verdict	evidence today
#23M	16-branch capability dispatch cond fails **open** on an unmatched shape	**FIXED**	03_DPDC-C.pact:859 — (enforce false (format "Unreachable nonce/amount shape ...")). [VERIFIED by reading]
#24M	NFT amount=1 enforced on native Credit, absent on 3 fragment/hybrid variants	**FIXED**	03_DPDC-C.pact:600 UEV_FragmentCreditAmount — positive multiple of 1000 — wired at :325, :345, :373. [VERIFIED by reading]
#25M	Royalties read live at transfer time, no snapshot or max-price guard	**REFUTED — design**	Three options weighed; owner: a timelock buys nothing against a role holder who controls the lock.
#26M	C_UpdateNonceRoyalty mutates a field with zero on-chain consumers	**FIXED (doc)**	Confirmed intentional — a hook for the unbuilt Escrow/marketplace. Documented at schema, reader and entrypoint.
#27M	Fragment make+merge round trip and repurpose-without-consent executed but never asserted	**FIXED**	REPL/Stage_02/[6.1.2]_DPDC-FRAGMENTS.repl, wired at Stage02_Tester.repl:41. [VERIFIED by command]
#28M	EQUITY “shareholder collection” identity is a self-checked "E|" string prefix	**REFUTED**	Two stacked walls: iz-special=false is hardcoded on the only public issuance path, so | cannot appear in a publicly-issued ticker; and DPDC-I|C>ISSUE enforces ownership of dpdc itself.
#29M	EQUITY’s 50 % packaging cap is an undocumented magic constant	**FIXED**	11_EQUITY+.pact:239 PACKAGING_CAP_DIVISOR, used at :388. [VERIFIED by reading]
#30M	C_EnableSetClassFragmentation skips the active-state gate its four siblings enforce	**FIXED**	08_DPDC-S.pact:405 — (UEV_SetActiveState id son set-class true) with the audit reference in the comment. [VERIFIED by

id	summary	verdict	evidence today
			reading]
#31M	Primordial set bounds check only the running maximum, not each value	**FIXED**	08_DPDC-S.pact:917-929 — per-element 0 < abs(n) <= nu. [VERIFIED by reading]
#32M	Hybrid set Make-time and Break-time constituent ordering are opposite conventions	**FIXED**	08_DPDC-S.pact:1080 — normalised to primordial-first, with cross-referencing comments at both sites. [VERIFIED by reading]
#33M	Collection id keyed only on prev-block-hash — same-block same-ticker collides	**REFUTED — accepted**	Live-verified twice; the second check corrected the first assumption (NFT+SFT same ticker **also** collides, in the shared BRD|BrandingTable, before DPDC's own tables are reached). Documented on UDC_Makeid (08_U_DALOS.pact:231).
#34M	URD_AccountNoncesWithSupplies returns [{}] instead of [] when empty	**FIXED**	02_DPDC.pact:892-913, now URH_AccountNoncesWithSupplies, empty case [] with the audit note in place. [VERIFIED by reading]
#35M	XB_DeployAccount{SFT,NFT} never verify the caller controls the target account	**FIXED by removal**	04_DPDC-I.pact:55 and :396 removal notes; 01_TS02-C1.pact:65/484 the Talos halves; DemiPad redirected, see 05_TS02-DPAD.pact:207/278. [VERIFIED by command — 0 live definitions]
#36M	AUP_Account/AUP_Property slice composite keys by hardcoded offsets	**REFUTED**	08_U_DALOS.pact:490 — every valid DALOS account is hard-enforced to exactly 162 characters. The offsets rest on a real system-wide invariant. [VERIFIED by reading]
#37M	UDC_ZeroNonceData called cross-module through a typed ref but absent from that interface	**FIXED, +3 more**	01_DPDC-UDC.pact:273; sweep found CAP_OwnerOrCreator (02_DPDC.pact:221), UEV_CanWipeON (:193), C_DefineHybridSet (08_DPDC-S.pact:127). [VERIFIED by reading]
#38M	UDC_NoPrimordialSet/	**ALREADY CLOSED**	Primordial [0] blocked by

id	summary	verdict	evidence today
	NoCompositeSet sentinels indistinguishable from real data		#31M's per-element bound; Composite -1 by the sc > 0 check. Both present. [VERIFIED by reading]

LOW (17 + 1 unnumbered)

id	summary	verdict	evidence today
#39L	Zero REPL coverage for DPDC-S Make/Break and admin mutations	**FIXED**	REPL/Stage_02/[6.1.3]_DPDC-S.repl, wired at Stage02_Tester.repl:45. [VERIFIED by command]
#40L	The Wipe* family has zero REPL coverage	⏸ **DEFERRED**	Open. See §6.
#41L	Branding functions untested; stark SFT-vs-NFT test asymmetry	⏸ **DEFERRED**	Open. See §6.
#42L	No negative-path coverage of the ownership gate on any of 12 role functions	⏸ **DEFERRED**	Partially closed later, not by this audit. See §5.
#43L	XI_RegisterCollectionElement returns a display string instead of ending on a write	⏸ **DEFERRED**	Open. See §6.
#44L	Dual implements DpdcTransferV1 + V2 deviates from latest-version-only	**CLOSED → since moot**	The dual implement is gone: 07_DPDC-T.pact:102 carries only (implements DpdcTransferV2), and DpdcTransferV1 has zero hits tree-wide. [VERIFIED by command] (**Corrected 2026-09-18** — this said the arrangement was *"already self-documented as intentionally additive in DpdcTransferV2's own @doc"*. No such @doc exists, and the thing it described no longer does either.)
#45L	UDC_ScoreMetaData is dead code	**FIXED by removal**	01_DPDC-UDC.pact:276 and :654 removal notes. Removed rather than rewired — wiring it in would have reset a real set-instance's composition. [VERIFIED by command]
#46L	Constructors take 5-8	**CLOSED — no live bug**	Every call site audited

id	summary	verdict	evidence today
	same-typed positional params in a row		safe in Round I.
#47L	C_RepurposeCollectableFragments Multi Mode has no length > 0 guard	**FIXED**	09_DPDC-F.pact:236 — (enforce (and (= l1 l2) (> l1 0)) …). [VERIFIED by reading]
#48L	DPDC-F|C>MERGE omits id/son; two caps missing @event	**FIXED**	09_DPDC-F.pact:268-272 — params and @event present, with the audit reference. [VERIFIED by reading]
#49L	EQUITY Make/Break reimplements DPDC-S's pattern divergently	**FIXED (doc)**	Cross-referencing @doc notes flag the intentional divergence for future review.
#50L	URC_SingleSharePerMillions has no declared return type	**FIXED**	11_EQUITY+.pact:60 and :374 — :integer on both interface and module. [VERIFIED by reading]
#51L	Empty set-definitions crash with Array index out of bounds	**FIXED**	08_DPDC-S.pact:891-897 and :951-956 — MAX_SET_DEFINITION_SIZE length bound on both validators. [VERIFIED by reading]
#52L	URC_NoncesSummedScore discards a nested set's own multiplier	**REFUTED — design**	Raw-sum by design; a multiplier applies once at its own set's level. Matches observed mainnet Bloodshed scores.
#53L	creator-account bound with no ownership/consent check	**REFUTED — design**	Same "owner has complete dominion" trust model as #4C/#17H/#20H.
#54L	C_DeployAccount{SFT,NFT} carry no UEV_IMC/cap/@doc	**MOOT**	The functions no longer exist — removed by #35M.
#55L	UR_AS-KEYS is a full table scan named with the point-read prefix	**FIXED**	02_DPDC.pact:541, now URH_AS-Keys. [VERIFIED by reading]
(unnumbered)	DPNF|AccountRoles interface @doc states the reverse composite-key field order	**FIXED (doc)**	Doc-only.

A note on the count. FINAL-AUDIT-REPORT.md opens with *"Findings: 56 (8 CRITICAL, 14 HIGH, 16 MEDIUM, 17 LOW)"*, and 8+14+16+17 is 55. The tracker's own wording is the correct one: **55 numbered findings** + **1 unnumbered doc note** = **56;** + **2 sub-findings (#12Hb, #12Hc)** = **58 tracked items.** The headline compresses two different denominators into one number. This book's third rule says a count is reported with its exclusions or not at all, so it is stated here in full rather than repeated as published.

4. Seven findings worth the retelling

These are chosen for what they teach, not for severity. Four of them start with a hypothesis that was **wrong**.

4.1 #1C — the fix was one function, because the bug was one function

The finding read like eleven bugs. A negative `amount` passed to `DPDC-T::C_Transfer` inverts the sign of a credit into a debit; the same hole is reachable through fragments (#3C), through hybrid credits (#24M), through set composition (#8C). Each auditor found it in their own module and reported it as their own module's bug.

They were all the same bug. Every SFT and fragment balance change in all eleven modules funnels through one function in `03_DPDC-C.pact`. A single `UEV_Amount` there closed #1C, closed #3C outright with no additional code, and closed the negative half of #8C. The list-form callers came along for free, because they `map` the same single-value function.

The exploit was reproduced before the fix, as a real signed transfer: sender balance **+50** from a `-50` "debit", receiver driven to **−50**. After the fix, submitted as a real uncaught transaction, it aborts before `commit-tx` and nothing persists.

The detail worth keeping is what the enforce actually says. It is **not** `> 0`:

```
(enforce (>= amount 0) "Amount cannot be negative")
```

Zero is legal, and the `@doc` explains why — EQUITY issues tier nonces with zero initial supply — with a separate guard one line below rejecting a zero-amount *debit* against a zero balance. A reviewer grepping for `> 0` would conclude the fix was weakened. It was made correct.

Verified 2026-09-17: `03_DPDC-C.pact:594`, called at `:1086`. [VERIFIED by reading]

4.2 #2C — refuted twice, hardened anyway, and the hardening is the point

Reported as a CRITICAL drain: `C_IgnisRoyaltyCollector` debits a caller-named `patron`'s IGNIS balance with no authorisation check anywhere in the chain. Name someone else's smart account, collect their balance to a collectable's creator.

The first refutation was mechanical — `IGNIS|C>DEBIT` already rejects smart-account patrons through `UEV_EnforceAccountType`, so the "any smart account" shape was never reachable. Narrowing to *standard* accounts, the second refutation was subtler and more interesting: the royalty-bypass in `C_IgnisRoyaltyCollector` and the fee-ownership check in `C_Collect` **key off the same toggle**. "Royalty is nonzero" and "the ownership check gets skipped" cannot both be true. No reachable drain.

The owner asked for `CAP_EnforceAccountOwnership` to be added to `IGNIS|C>DEBIT` anyway — *so the debit does not depend on that external invariant holding forever.* That is the entire argument for the fix, and it is a better one than the finding. A correct refutation tells you the system is safe **today**, by a coincidence of two toggles in two modules. It does not tell you the system will be safe after someone edits one of them.

Live-proven both directions: a real owner still collects a genuine 500.0 IGNIS royalty; an attacker naming someone else's account is rejected at the new line with a keyset failure.

Verified 2026-09-17: `07_DPDC-T.pact:396`, inside `IGNIS|C>DEBIT`, alongside the `UEV_EnforceAccountType sender false` at `:395` that the refutation rested on. **Both are present** — the refutation's premise and the belt-and-braces fix. [VERIFIED by reading]

4.3 #5C → Fix #19 — the fix that broke EQUITY, and the test that found it

#5C: nothing stopped a collection owner from freezing and wiping the protocol's own dpdc escrow account, destroying the collateral behind every outstanding fragment claim. Fix #3 added one check to the shared C>REMOVE-CLASS-ZERO-NONCES chokepoint: account != dpdc. Both burn and wipe rejected at the identical line, ordinary burn still working, Z.repl green.

Three days later, building real EQUITY test coverage for **#22H**, that blanket check turned out to block EQUITY's legitimate Convert/Break flows. EQUITY uses dpdc as a *same-transaction escrow* — transfer in, burn the old tier, credit the new tier, transfer out — which has nothing to do with fragments.

The narrowed check asks the question #5C actually cares about: is this nonce **currently backing an outstanding fragment claim**? Since the capability only ever handles Class-0 nonces, that reduces to "does it have non-zero split-data", answerable without a forward reference to DPDC-F (which deploys *after* DPDC-MNG and cannot be referenced by interface type this early — confirmed live, not assumed: referencing module{DpdcFragmentsV2} there throws *"Cannot find module"* at DPDC-MNG's own deploy step).

Two lessons stacked. The first: **a fix that passes the full suite can still be wrong, if the thing it breaks has no test.** #5C's fix broke EQUITY and Z.repl stayed green for three days, because EQUITY had no reachable coverage at all. The second: writing the missing test is what found it. The audit did not discover this by re-reading its own fix.

Verified 2026-09-17: 06_DPDC-MNG.pact:439-470. The narrowed form is present, the reasoning is in the source comment, and the deploy-order constraint is recorded there too. [VERIFIED by reading]

4.4 #15H — a bound, then immutability, then a *tighter* bound, and the reason for each

The finding was ordinary: a set-class score-multiplier with no bound, retroactively re-pricing every outstanding member of a set instantly (the same shape as the SWP audit's C_ModifyWeights).

Fix #13 added UEV_ScoreMultiplier: (0, 100] with 3-decimal precision, wired into all three Define paths (which previously had **zero** validation) and the Update path (which previously had precision only).

Fix #14, after the owner asked whether the multiplier is stable afterwards: not bounded — **immutable**. C_UpdateSetMultiplier and its capability, XI_* and Talos wrappers were deleted outright. Write-once, at Define, forever. This is also what closed #7C permanently: the type bug that made the function crash on every call stopped mattering when the function stopped existing.

Fix #16, surfaced while discussing #19H: a multiplier is meant to **boost** a score, never quietly shrink it. The floor moved from 0 to 1.0. 0.5 — previously legal — is now rejected; 1.0 is the neutral no-op.

Three edits to one validator over three days, each one triggered by a question the previous one raised. That is what a sequential, one-finding-at-a-time process buys, and it is the argument against batch fixing.

There is a coda. While fixing #15H the auditor checked whether the multiplier was consumed anywhere live, and found UR_N|Score — the only function that applies it — had **zero callers**. The bound was being placed on a value nothing read. A brief was handed off to a separate reviewer asking whether AQP staking was meant to apply it, and FINAL-AUDIT-REPORT.md records that the answer *"has not yet come back"*.

It has since come back, and the answer was yes. 02_SCORE.pact:2134 now calls ref-DPDC-S::URC_N|Score for NFT score model 0, with a comment naming DPDC #15H. The bound is load-bearing today in a way it was not when it was written.

Verified 2026-09-17: 08_DPDC-S.pact:1031-1047 (the bound), :99/:1143 and the four Talos sites (the removal), 02_SCORE.pact:2131-2134 (the consumer). [VERIFIED by reading]

4.5 #35M — removing the function was the fix, and the removal broke the launchpad

XBv_DeployAccountSFT/NFT associate an account with a collection, and never check the caller controls the target account. Any signer could force any account to associate with any collection. State-bloat griefing, not theft — the write is idempotent-safe.

The obvious fix is an ownership check at the XB_ layer. **That would have been wrong**, and the audit established why before doing anything: the same XB primitive is used for legitimate auto-associate-on-transfer, where the caller is by definition not the target. The fix that preserves the feature is to remove the *standalone entrypoints* — the Talos DPSF|C_DeployAccount / DPNF|C_DeployAccount wrappers and the orphaned DPDC-I::C_DeployAccountSFT/NFT — while leaving the shared primitive alone. That is the #15H removal precedent applied a second time.

Then it broke. Every real caller was traced first, and one — TS02-DPAD::A_RegisterAssetToLaunchpad, DemiPad's launchpad registration — genuinely depended on the removed path. It was redirected to call XBv_DeployAccountSFT/NFT directly, which required TS02-DPAD's own guard to be registered as a trusted DPDC IMC peer. **The redirect compiled cleanly and did not work.** It was caught by running the real end-to-end launchpad scenario, which is not in the default test profile.

FINAL-AUDIT-REPORT.md singles this out, correctly, as the round's best argument for live verification over a clean compile. The "Talos module needs its own IMC registration" shape had already been fixed once each in the AQP and ATS audits — three modules, three times, found three separate ways.

A handoff was drafted for DPTF/DPOF, where the identical shape was confirmed to exist.

Verified 2026-09-17: 04_DPDC-I.pact:55/396, 01_TS02-C1.pact:65/484, and the DemiPad redirect documented at 05_TS02-DPAD.pact:207 and :278, with the live call at :285-286. No live definition of either removed function remains. [VERIFIED by command]

> **Corrected 2026-09-18.** This section wrote the surviving primitive as XB_DeployAccountSFT/ NFT throughout. The v-marker sweep this chapter's own §7 describes renamed it: XBv_DeployAccountSFT / XBv_DeployAccountNFT (02_DPDC.pact:229, :1342). The unprefixed names have zero hits.

4.6 #19H — one check at the entry, not four checks in four branches

UR_N|Score is the public "cooked" score reader. DPDC stores -1.0 as the *unscored* sentinel, and the reader is supposed to clamp it to 0.0. It did — in one of its four arithmetic branches. The other three either omitted the check or compared against -1000.0, a copy-paste leftover from the fragment-scaling arm. Any composite-class nonce, and any fragment nonce created without explicit metadata, read a real negative score.

The finding notes something that ought to make a reader uncomfortable: **a sibling audit (AQP) had already recorded a false "already fixed" assumption about this exact function** in its own audit trail.

The fix could have been four patches. It was one: hoist the sentinel check to function entry, on the raw untouched value, before any multiply or divide. Three broken branches close at once and there is no fifth branch to get wrong later. Eight-check live proof across NFT class-0 and SFT set-member, scored and unscored, native and fragment; git stash reproduced the exact pre-fix values (-0.001, -2.5, -0.0025).

Fix #18 then renamed it UR_N|Score → URC_N|Score, because **the prefix is the contract**: it reads three tables and derives a computed value, which is URC_, not UR_. Zero callers existed at the time, so nothing needed updating.

Verified 2026-09-17: 08_DPDC-S.pact:554-570. The sentinel test is the first form inside the let body, on raw-nonce-score, before either branch. [VERIFIED by reading]

4.7 #33M — the refutation whose own first assumption was wrong

UDC_Makeid derives a collection id from prev-block-hash, a block-level constant identical for every transaction in a block. Two same-ticker issuances in one block collide, and the second aborts with an opaque low-level error.

Confirmed live. Then, while documenting it, a second assumption was checked — *"at least NFT and SFT with the same ticker work, they're in different tables"* — and it was **also false**. They collide first in the globally-shared BRD|BrandingTable, used by seven modules (DPDC-I, DPTF, ATS, MTX-SWP, DPOF, DPMF, SWPI), long before DPDC's own type-split tables are reached.

So the finding got *worse* during its own refutation, and was still accepted. The reasoning is worth reading because it is a genuine engineering judgement rather than a dismissal: no fix is possible inside UDC_Makeid, a Stage-1 utility that deploys before the BRD core module and cannot read it without violating deploy order. A real fix needs collision-probe-and-retry logic added to all seven consumer modules — disproportionate to a failure mode that is atomic, self-healing (the next block has a different hash), and not exploitable beyond forcing a retry.

The outcome was a @doc on UDC_Makeid and nothing else. That is a defensible close, and it is recorded as accepted-by-design rather than as fixed.

Verified 2026-09-17: 08_U_DALOS.pact:231. [VERIFIED by reading]

---

5. What later rounds found in DPDC that this audit did not

The DPDC audit closed in August 2026. Two later programmes — the guard-reachability round and the red-team round — went back over the same modules. What they found is not a criticism of this audit so much as a demonstration that **reading a call chain to decide what refuses first is unreliable**, which is this book's first rule.

The guards this audit added, that could not speak

#8C's fix put (enforce (> how-many-sets 0) ...) inside DPDC-S|C>MAKE. Correct check, correct capability. It sits in the let body, and the let binds (iz-active (UR_IzSetActive ...)), which funnels to UR_Set's bare read. **Pact evaluates binding groups before the body**, so a set-class that does not exist died on the raw table key and *neither* the "is not active" message nor #8C's own message could ever speak for it. Set classes are 1-based, which puts 0 — the natural off-by-one — squarely in the mute half.

That is G-44 in Audit/records/DEFECT-LEDGER.md §1.2.1, fixed 2026-09-17 with a hoisted URC_SetExists guard above the let. The entry records that **the defcap fix alone was not enough**: the first raiser was in the *Talos wrapper*, which bound (nonce (UR_NonceOfSet ...)) eagerly, purely to print it in the success message.

G-45 is the sibling — DPDC-S|C>RENAME had no domain guard at all, and unlike C>MAKE never called UEV_SetClass. *The module already contained a live guard for that exact input and RENAME was simply not wired to it.*

Two more in the same family: G-11 (02_DPDC.pact UEV_Nonce, all three predicates shadowed by UR_NonceValue), G-12 (08_DPDC-S.pact UEV_SetClass, both enforces shadowed by UR_Set), and G-10 (06_DPDC-MNG.pact:299 C>ADD-QUANTITY, the guard at :307 — *partially* shadowed: nonce 0 aborted in the row read, but a *negative* nonce reached the guard, because the reader keys on (abs nonce) and found a real row. Mute for one input, live for another).

G-08 is the most expensive of the set and belongs to EQUITY: C_IssueShareholderCollection's *"24 IPFS links must be provided"* check sat below a let binding that **issues the collection**. So every rejected call **paid for a full collection issuance before its arguments were inspected**, and whenever the issuance failed first for its own reasons — a duplicate name, the ordinary case — that error was reported instead. Pinned by <<EQ-G1>>.

None of these are new *checks*. They are checks this audit and its predecessors wrote, which no input could reach. The audit's own careful expect-failure proofs did not catch them, because a test that drives a guard from a state where it *can* fire proves the guard works and says nothing about the states where it cannot.

#42L was deferred, and the red team partly closed it

#42L — *no REPL coverage exercises the ownership gate's negative path for any of the 12 role functions* — was deferred to a planned suite refactor. It was the finding the audit itself called *"the single highest-value missing assertion class for this module"*.

The red-team round arrived at the same place from the other direction. DEFECT-LEDGER.md §7.2h measured **which owner gates have ever been observed refusing anybody**, and found that the entire transfer family — DPDC-T|C>TRANSFER, DPDC-T|C>BULK-TRANSFER, DPDC|C>TG_TRANSFER-R and their DPOF siblings — gate on CAP_EnforceAccountOwnership sender and **none had ever been shown refusing a non-owner**. Their negative tests drove list-shape and nonce guards with the owner as sender.

RT-D-004 wrote the first one for the collectable door, deliberately pinning **the victim's key by name** rather than a bare Keyset failure — because on that family an unqualified match would not separate "the owner gate refused" from "the attacker's own signature was rejected", which would pass while proving the opposite.

Two DPDC-S|C> set witnesses and two DPDC nonce-level witnesses were added in the same sweep; [6.1.3]_DPDC-S.repl has since grown from the audit's 40 assertions to a 101-assertion, ~70 KiB suite carrying them (TX-SET-013 is the collection-owner gate's first witness). [VERIFIED by command, 2026-09-18 — this said "67 KB"; the file is 71,491 bytes]

#42L is therefore partly closed, by a different programme, and its deferral stands for the rest.

The exec diagnosis bug behind a preview

RT-K-004 went after INFO_DPNF|Burn and found the preview died on a raw properties read. The *exec* turned out to be worse: DPDC-MNG|C>BURN-NFT checked the burn **role** before establishing the collection exists, so for a non-existent id it answered *"NFT Burn Role for NOSUCHCOL-… must be set to true"* — true, and useless, because there is no collection to hold a role on. UEV_id now runs first, and the preview calls the same function.

---

6. What remains open

Four findings were deferred, deliberately and with reasons. None is a correctness gap; all four are coverage or discipline items folded into larger planned work.

id	what	why it is still open
#40L	The Wipe* family (Heavy/Pure/Clean/Dirty) has zero REPL coverage — the checked-in "Wipe Tests" transaction is commented out	The family itself needs renaming as part of a StoicSyntax architecture pass. Building coverage for names about to change is premature.
#41L	Branding functions untested; DPNF|C_* gets one exercised call in the entire suite against dozens for DPSF|C_*	Folded into a planned top-to-bottom Stage 1→2 REPL refactor.
#42L	No negative-path ownership coverage across 12 DPDC-R role functions	Same refactor. **Partly closed since, by the red-team round** — see §5.
#43L	XI_RegisterCollectionElement returns a	Touches the INFO-function

id	what	why it is still open
	formatted string instead of ending on a write	architecture (UI cost/description read-points), rearchitected separately.

Two cross-references were left open at close and have since moved:

- **The AQP multiplier-wiring question** (raised during #15H) — *closed*. 02_SCORE.pact:2134 now consumes URC_N|Score. [VERIFIED by reading]
- **The DPTF/DPOF handoff** (from #35M) — the identical standalone-DeployAccount shape was confirmed to exist there. Whether that audit acted on it is **not verified by this chapter.**

Two design positions are open by decision rather than by omission, and a reader should know they are positions and not oversights:

- **The owner-dominion trust model.** #4C, #17H, #20H, #25M and #53L were all refused on one consistent principle: *a collection owner has complete, trusted dominion over their own token's economics and admin structure.* An unconsenting third party can be named "creator". A role-modify-royalties holder can raise a fee mid-flight against a pending buyer. There is no ceiling on an IGNIS royalty. These are deliberate, applied consistently across five findings rather than re-litigated each time, and the mitigation is off-chain: royalty changes are @event-logged in real time, a marketplace UI can re-read before signing, and Ouronet admin can red-flag an abusive collection. A reader who does not accept that model should treat those five as open.
- **The admin escape hatches.** C_RepurposeCollectable and C_RepurposeCollectableFragments deliberately bypass freeze and transfer-role gates, gated only on CAP_Owner. They exist for stolen and deceased-account recovery. The owner explicitly rejected freezing the source account afterward, because that would route through the real freeze path and become conditional on can-freeze=true (#13H) — defeating the point of an always-available escape hatch. HEIR-SYSTEM-PONDERING.md in the audit tree sketches a future design that removes admin discretion from the succession case.

---

7. Verification result for this chapter

Every finding recorded as FIXED was searched for in current source on 2026-09-17.

Nothing was found missing. All 35 fixes are present. This is a stronger result than it sounds, because the codebase has been through at least four substantial rewrites since the audit closed:

- the **interface cascade** to V2 — the audit's DpdcV1, DpdcUdcV1, DpdcCreateV1 are now V2, and every fix survived retyping;
- the **Part II pricing rehaul**, which replaced #10H's literal "dpnf" price key with the URCi_IssueCollectionPrice/URCi_IssueCollectionStoa reader pair — the *implementation* of that fix was rewritten and the *distinction* survived;
- a **StoicSyntax prefix sweep**, which renamed #34M's URD_AccountNoncesWithSupplies → URH_AccountNoncesWithSupplies and #55L's URD_AS-Keys → URH_AS-Keys. Both were briefly invisible to a grep on the audit's own wording, and both are correct. A fix verified by name is not verified;
- XI_CreditOrDebitDPDC → XIv_CreditOrDebitDPDC (03_DPDC-C.pact:1068), the v marker for an intrinsic guard. (**Corrected 2026-09-18** — this named XI_CreditOrDebitCollectables as the renamed function. That one is still called exactly that, at 03_DPDC-C.pact:798; the rename happened one layer down, to the per-nonce writer it dispatches into.)

The reason the fixes are findable at all is a convention the audit adopted and stuck to: **35 of them left a source comment naming the finding.** Counted 2026-09-17 [VERIFIED by command]:

file	DPDC Audit # markers
08_DPDC-S.pact	17
10_DPDC-N.pact	8
02_DPDC.pact	7
03_DPDC-C.pact	5
01_DPDC-UDC.pact, 04_DPDC-I.pact, 11_EQUITY+.pact	3 each
09_DPDC-F.pact	2
05_DPDC-R.pact, 06_DPDC-MNG.pact, 07_DPDC-T.pact	1 each

That is the mechanism that made this chapter's verification possible in an afternoon, and it is the practice worth copying. A fix with a finding id attached can be found after four refactors. A fix without one is indistinguishable from ordinary code the moment its line number moves.

Two caveats on scope, stated because the book's rules require it.

1. This chapter verified that each fix is **present**. It did not re-run the audit's proofs. Where the table says a fix is present, that is what was checked; where it says *live-proven*, that is the audit's claim, quoted.
2. The audit's Z.repl-green gate is **weaker than it reads**, though less so than this chapter said. The three suites this audit built are in the fast path, and so, now, is the pre-existing DPDC scenario suite: Stage02_Tester.repl:75 is an **uncommented** (load "Stage_02/[6.1]_DPDC.repl"). The gate that actually covers this family is still python3 REPL/tools/_gate.py, because Z.repl skips other things — [5.3]_Launchpad.repl (:73) and [6.3]_STOAICO.repl (:77) are both still commented out.

   Corrected 2026-09-18. This read *"Z.repl skips [6.1]_DPDC.repl entirely"*. It does not. Stage02_Tester.repl:8's header still lists *"[6.1] DPDC suite"* among the skips, which is where the claim came from and which is itself now stale — the same failure this chapter documents in code, in the comment that describes the code.

Next: Chapter 9 — DEMIPAD, the sovereign launchpad, and the smallest audit tree in the book.

---

9. DEMIPAD — the launchpad

Source tree: Audit/module-audits/DEMIPAD/ (4 files, 792 lines) **Audited:** 2026-08-29 → 2026-08-30, on main, no worktree **Scope at the time:** 6 modules, 3,664 lines — the smallest core family in Ouronet **Findings:** 17 · **Fixed:** 15 · **WONTFIX:** 2 **Verification pass for this chapter:** 2026-09-17.

---

1. What DEMIPAD is, and what breaks if it breaks

DEMIPAD is Ouronet's launchpad: the venue where Demiourgos.Holdings sells assets to the public. It is **permissioned** — not anyone can list — which matters for several of the verdicts below, because the seller is a trusted party by construction.

At the time of the audit it was six modules:

module	lines	role
00_Demipad.pact	1,298	the launchpad **rules**: register an asset, toggle open-for-business, define a price, deposit, withdraw, transmit the four asset kinds, track dollars raised, retrieval
05_STOAICO.pact	739	the STOA ICO — a staking vault with RPS reward distribution (inject / stake / unstake / collect)
01_Spark.pact	543	the Spark token sale, bonding-curve priced, with redemption
04_STOICPAY.pact	417	the StoicPay (KPAY) sale, per-period buy cap
03_Custodians.pact	351	the Ouronet Custodians collection sale
02_Snakes.pact	316	the Demiourgos ShareHolder collection sale

It handles **KDA payments** (native and wrapped), **token custody** (inventory sits in a shared launchpad account until sold or retrieved), **bonding-curve pricing**, and **RPS reward distribution**. For 3,664 lines that is a lot of money-touching surface.

The failure modes are concrete. A launchpad that lets a seller pull deposited inventory mid-sale is not a launchpad, it is a rug-pull with extra steps. A custody ledger that can be credited without tokens arriving lets one seller drain another's proceeds from the shared account. And a reward vault whose "last claimant takes the dust" branch tests the wrong thing pays the entire remaining vault to whoever asks first.

All three of those were found here. So was a sale module that had been copied from its twin and half-wired.

Where these modules live now

The audit's single most consequential outcome was **not a code fix**. Finding #15L asked why several C_ entrypoints lacked UEV_IMC, the sovereign inter-module gate. The answer was that they should not have it: the per-asset sales are **citizen** modules that wire into the sovereign launchpad's rules, and citizen client functions correctly carry no sovereign gate. The defect was **physical filing** — they sat under 1_SOVEREIGN/…/2_Core/02_DEMIPAD/, which made them look sovereign.

They were moved. Today [VERIFIED by command]:

```
1_SOVEREIGN/STAGE_02/2_Core/02_DEMIPAD/00_Demipad.pact     <- sovereign: the rules
1_SOVEREIGN/STAGE_02/3_Talos/05_TS02-DPAD.pact             <- sovereign launchpad Talos
2_CITIZEN/7_Launchpad/{1_Spark,2_Snakes,3_Custodians,
                4_StoicPay,5_StoicIco}/            <- the five citizen sales
2_CITIZEN/7_Launchpad/99_TS02-CPAD.pact                    <- citizen launchpad Talos, deployed last
```

Five of the six modules this chapter covers are no longer in the directory whose audit tree this is. Anyone reading the audit documents without that fact will not find the code. The round-docs were deliberately left as accurate-when-written rather than retro-edited, which is the right call for an append-only record and a trap for a reader.

---

2. How it was audited, and what the tree does not contain

The DEMIPAD audit is **792 lines across four files**. The DPDC tree is 5,093 and the AQP tree is 4,233 across 14 files (it was 4,112 across 13 when this chapter was written — RPS-SPLIT-SCOPING.md landed 2026-09-17). This chapter is shorter than its neighbours because its source is, and padding it would misrepresent the coverage.

What was run

README.md describes a five-step method, and steps 1–2 are the part that distinguishes this audit from its siblings:

1. **Round-01 lens fan-out.** Six independent read-only auditors, each hunting **one dimension** across all relevant modules — rather than DPDC's and AQP's one-auditor-per-file. The roster: access-control/capability; arithmetic/precision/payment-math; economic/MEV/sale-accounting; a STOAICO reward-distribution deep dive; custody/state-lifecycle/asset-integrity; and StoicSyntax conformance plus cross-module boundary.
2. **Adversarial verification.** Every candidate was re-checked against the code by an independent validator and classified CONFIRMED / REFUTED / STYLISTIC **before** reaching the owner.
3. Owner verdicts → 4. Round-02 fixes with diffs and REPL proof → 5. ranked index and final report.

The lens structure is why ROUND-01-FINDINGS.md carries a **corroboration count** — "4 lenses", "2 lenses", "1 lens" — that the other audits do not. It is a genuinely useful signal: the critical finding was found by four independent lenses; most of the mediums by one.

The findings file also carries something the other trees lack: a **"Cleared by the lenses"** section listing what was checked and found correct — Spark's bonding curves, the Snakes/Custodians cost chains (floor-at-precision, favouring the protocol), Demipad royalty math, STOAICO's settle-before-score ordering and last-rps snapshot-after (so a late staker cannot claim pre-stake injects), the over-unstake block, and the open-for-business enforcement path. A negative result recorded is worth more than a negative result assumed.

What the tree does not contain

Being specific about this is the point of the section:

- **No ROUND-01-OWNER-FEEDBACK.md.** Steps 3 of the method produced no file. Owner verdicts are embedded in the fix entries instead — readable, but not separable, and not append-only.
- **No FINAL-AUDIT-REPORT.md.** The audit closes on ROUND-02-FIXES.md's last line: *"Closes the DEMIPAD audit."* There is no consolidated sign-off document.
- **No re-verify round.** Same as DPDC.
- **README.md's status checklist is still four unticked boxes** — *"Round-01 lens fan-out (in progress)"* — while ISSUES-RANKED.md shows all 17 closed. The tracker was never updated. That is a one-line clerical gap, and it is exactly the kind of gap that makes a reader distrust the rest, so it is named here rather than smoothed over.

What the tree **does** have, and has well, is proof discipline. **Eight fix entries record a deliberate pre-fix reproduction** — the fix reverted or neutered and the original failure captured — covering nine findings: #2H, #3H+#4H, #5M, #6M, #8M, #9M, #10M and #16L. [VERIFIED by reading — ROUND-02-FIXES.md] #2H neutered the new capability body to (enforce true ...) and watched the non-admin block assertion fail. #4H reverted one identifier and captured the exact runtime error (Unbound free variable ... DEMIPAD.GOV|LAUNCHPAD|SC_NAME). #5M forced the flush branch on an empty vault and captured Arithmetic exception: div by zero at the exact line. #6M removed the phase gate and watched 1,568 urSTOA be minted twice.

That is a higher bidirectional-proof rate than either neighbouring audit, in a quarter of the documentation. **The tree is thin; the work in it is not.**

One live-vs-workspace check that changed a finding into a non-finding

Finding **#11M** deserves separate mention as method. The audit measured StoicPay's team-split accounting against the **workspace** copy of 04_STOICPAY.pact and derived a real inconsistency: the module moves 2× amount KPAY per buy while UR_KpayLeft derives sold = 100M − 0.4·resident, so the per-period cap drifts from real inventory.

Pulling the **deployed** module through the Pythia dirty-read gateway (describe-module → code) showed the on-chain contract is correct: a 5-recipient **1.5**× team split, not the workspace's 3-recipient **1.0**×. With the real split, resident drops 2.5× per buy and the formula tracks buyer amount exactly. The "20 % under-count" existed only in a test simplification that had been left in the workspace.

The fix was to re-sync the workspace to live. The general lesson recorded in the fix entry is the transferable part: **for "does the workspace match live" questions, pull the deployed code and diff — the workspace may carry deliberate test simplifications**, and an audit measuring the wrong artefact produces a confident, wrong finding.

---

3. The findings

All 17, in the audit's own ranking. Severity as recorded; evidence as verified 2026-09-17.

#	sev	module	summary	verdict	evidence today
#1C	CRITICAL	STOAICO	C_Collect is drainable — non-idempotent, and unclaimed-count == 1 pays the whole vault	**FIXED**	05_STOAICO.pact:153 last-collected-round, :169 distribution-round, cap at :243, flush family at :468 (URH_UncollectedAccounts), :1046 (Ap_FlushUncollectedSlice) and :1069 (AA_FlushUncollected). [VERIFIED by reading]
#2H	HIGH	Demipad	the retrieval toggle is dead state — the anti-rug lock is never enforced	**FIXED**	00_Demipad.pact:412 DEMIPAD|C>RETRIEVAL-GATE, composed by all four RETRIEVE-* caps at :477, :482, :487, :492. [VERIFIED by reading]

#	sev	module	summary	verdict	evidence today
#3H	HIGH	Custodians	C_Acquire never opens CUSTODIANS|ACQUIRE — supply cap and policy caps dropped	**FIXED**	03_Custodians.pact:496-499 — with-capability (CUSTODIANS|ACQUIRE …) with the #3H note. [VERIFIED by reading]
#4H	HIGH	Custodians	calls a non-existent GOV|LAUNCHPAD|SC_NAME → runtime unbound variable	**FIXED**	03_Custodians.pact:302-303 — GOV|DEMIPAD|SC_NAME, with the #4H note. [VERIFIED by reading]
#5M	MEDIUM	STOAICO	A_Inject divides by vault-score with no zero guard	**FIXED**	05_STOAICO.pact:170 zombie-rewards + :393 reader — escrow-on-empty, so the division only runs when the divisor is positive. [VERIFIED by reading]
#6M	MEDIUM	STOAICO	urSTOA double-credited across stake rounds — re-mints already-claimed urSTOA	**FIXED**	05_STOAICO.pact:996-997 and :1034-1035 — (if (= (UR_Global11) 0) (XI_UpdateUrstoaEarned …) true). [VERIFIED by reading]
#7M	MEDIUM	Demipad	NF transmit guarded by the SF capability — NF assets cannot move, SF-as-NF type mismatch	**FIXED**	00_Demipad.pact:1427-1439 — the 2×2 son branch wires FUEL/RETRIEVE-NON-FUNGIBLE (defined at :470, :490). [VERIFIED by reading]
#8M	MEDIUM	Demipad	direct-injection credits withdrawable funds with no tokens in — phantom funds	**FIXED**	00_Demipad.pact:1208 UEV_DirectInjection — unconditional hard block — composed at :580. [VERIFIED by reading]
#9M	MEDIUM	Custodians	UC_NonceQuintesse	**FIXED**	03_Custodians.pact:

#	sev	module	summary	verdict	evidence today
			nce is declared pure and enforces		271-274 — pure mapping, with the enforce relocated and documented. [VERIFIED by reading]
#10M	MEDIUM	Custodians	UR_NonceSaleAvailability enforces; the twin Snakes does not	**FIXED**	Enforce relocated to 03_Custodians.pact:245, the first line of CUSTODIANS|ACQUIRE; reader at :295-303 is now a pure read. [VERIFIED by reading]
#11M	MEDIUM	STOICPAY	workspace diverged from the deployed module (3 addr/1.0× vs 5 addr/1.5×)	**FIXED**	Workspace re-synced to live. See §2.
#12M	MEDIUM	all sales	no on-chain slippage or max-cost bound on any buy	**FIXED**	00_Demipad.pact:1218 UEV_SlippageCost (composed at :561), :776 UC_SlippageFactor (used at :1036); URC_Acquire gained slippage in all five modules. [VERIFIED by reading]
#13L	LOW	STOAICO	unclaimed-count/nzs-count have no lower bound	**WONTFIX**	Subsumed by #1C; declined to keep parity with the canonical AQP counter. See §5.
#14L	LOW	STOICPAY	fractional team-split for buys not divisible by 4	**WONTFIX**	Precondition (decimals < 2) not met — KPAY has 24 decimals. See §5.
#15L	LOW	Snakes/Cust/STOAICO/Spark	missing UEV_IMC on several C_ entrypoints	**FIXED by reclassification**	The sales are citizen modules and correctly carry no sovereign gate;

#	sev	module	summary	verdict	evidence today
					they were relocated to 2_CITIZEN/7_Launchpad/. [VERIFIED by command]
#16L	LOW	Demipad	open-for-business reject message has a {} placeholder and no argument	**FIXED**	00_Demipad.pact:582 — (format "{} is not open for business, to allow deposits" [asset-id]). [VERIFIED by reading]
#17L	LOW	Demipad	interface omits URC_Acquire/URCI_Acquire; URCI_ is a mis-prefix	**FIXED, then superseded**	URC_Acquire is now declared in all five interfaces. URCcap_ lived one day. See §4.5. [VERIFIED by command]

4. Five findings worth the retelling

4.1 #1C — the vault that paid the whole balance to whoever called last

Four of the six lenses found it independently, which is the strongest corroboration signal in the tree.

STOAICO|REDEEM-CONTRIBUTION enforced two things: that the caller owns the account, and that a user row exists. **No already-claimed flag. No pending > 0 gate.** The @doc said *"Can only be done once."* Nothing enforced it.

Meanwhile URC_ClaimableRewards implemented the dust sweep as: when unclaimed-count == 1, return the **entire remaining wstoa-supply**, ignoring the account's own entitlement. And C_Collect decremented unclaimed-count **unconditionally** after paying.

Compose those three and the attack is not clever, it is arithmetic. Any account holding a user row — any past contributor, even one with zero score — calls C_Collect on itself repeatedly. The first call pays its real share and sets last-rps = current. Every subsequent call pays approximately zero **and still decrements the counter**. Drive the counter to 1, call once more, and the whole remaining vault transfers out, stranding every other contributor and driving wstoa-supply negative.

It also broke in honest operation: an accidental double-collect desynchronised the counter and stole the last-claimant dust branch from whoever should have had it.

The fix is not a flag. The owner-agreed design is a **distribution-round (generation) model**, which preserves the vault's ability to run multiple inject rounds:

- UserContributionSchema gains last-collected-round; GeneralContributionSchema gains distribution-round, initialised to 0.
- The redeem capability enforces (< last-collected-round distribution-round). A second collect in the same round is refused, so the counter cannot be walked.
- A_Inject enforces unclaimed-count == 0 before opening a new round. A round cannot open until the previous one is fully collected.
- A new staker is initialised with last-collected-round := current distribution-round — **born already-collected for the round they joined**, so a mis-ordered post-inject stake is not eligible for the already-injected round.

- Because “fully collected” is now a precondition for the next inject, an **admin flush family** was added to push-collect stragglers: URH_UncollectedAccounts (a Hydra preflight, the one heavy scan) feeding Ap_FlushUncollectedSlice (parallel, retryable, idempotent per account) and AA_FlushUncollected (solo). The flush delivers each straggler **its own** wSTOA and urSTOA — to the rightful user, not to the admin and not to a burn address.

The proof is a conservation assertion, not a pass/fail: emma self-collects her exact share, the double-collect is refused, A_Inject is blocked while lumy is outstanding, URH_UncollectedAccounts lists exactly lumy, AA_FlushUncollected delivers to lumy sweeping the dust as the last, the counter reaches 0, the vault reaches 0, and **emma** + **lumy wSTOA** == **10,000,000 injected**.

Verified 2026-09-17: the schema fields, the reader, the capability enforce, and the whole flush family are present in 2_CITIZEN/7_Launchpad/5_StoicIco/05_STOAICO.pact. [VERIFIED by reading]

And it was not the end of the story. See §5.

4.2 #2H — a promise in a @doc that the code never made

00_Demipad.pact’s documentation promises buyers: *“if retrieval = false the only way to retrieve Assets is a buy.”* That is the launchpad’s anti-rug guarantee. Deposited inventory is locked in; it leaves through sales or not at all.

UR_Retrieval — the reader for that flag — was called in exactly **one** place in the whole module: inside TOGGLE-RETRIEVAL, to detect a no-op flip. The four RETRIEVE-* capabilities that actually gate withdrawal were byte-for-byte the shape of the FUEL-* capabilities: they enforced owner-or-admin ownership and nothing else.

So the flag existed, had an admin toggle, emitted events, appeared in the UI — and gated nothing. The asset owner could pull deposited inventory at any time, including mid-sale.

Three lenses found it. The fix is six lines: a shared DEMIPAD|C>RETRIEVAL-GATE (asset-id), composed by all four RETRIEVE-* capabilities and by none of the FUEL-* ones (deposits are always allowed). The owner chose **Option A, admin override**: the gate is an enforce-one over [admin guard, (enforce (UR_Retrieval asset-id))], so admin may always retrieve, a non-admin owner may retrieve only when retrieval = true, and everyone else is still stopped by ownership.

The bug direction was proved by neutering the new gate body to (enforce true ...) and watching the non-admin BLOCK assertion fail with *“expected failure, got result: ()”*.

One footnote a careful reader should have. Per DEFECT-LEDGER.md §1.2.3 G-29, **every enforce nested inside an enforce-one is mute** — Pact discards the failed branch’s message and raises the outer one. So the inner "retrieval disabled" string is documentation, not a diagnostic. Here that is harmless, because the outer message is the informative one and names the asset: *“Asset {} retrieval is LOCKED (retrieval=false) — only the Launchpad admin may retrieve until a sale or an admin re-enable.”* It is worth knowing that the inner string will never be seen.

Verified 2026-09-17: 00_Demipad.pact:412-419, composed at :477, :482, :487, :492. [VERIFIED by reading]

4.3 #3H + #4H — the half-wired twin, and why fixing one alone would have crashed

Custodians is a copy of Snakes. It is not a finished copy, and the audit’s cross-cutting note says so plainly: *“Custodians is a half-wired copy of Snakes — the whole module needs a line-by-line reconcile against its working twin.”*

#3H: C_Acquire dropped straight into its let. Snakes wraps its body in (with-capability (SNAKES|ACQUIRE nonce amount) ...). CUSTODIANS|ACQUIRE was defined and had **zero** with-capability sites. Everything in it was skipped: the per-nonce supply cap (amount <= available-supply), both policy capability composes that downstream DPDC-T::C_Transfer relies on for caller authorisation, and the @event.

#4H: the capability's own reader called (ref-DEMIPAD::GOV|LAUNCHPAD|SC_NAME). That member does not exist. The module defines GOV|DEMIPAD|SC_NAME; Snakes reads the correct one. Because modref :: dispatch is dynamic, this is a **runtime** Unbound free variable the moment the function executes — latent only because no test had ever exercised the Custodian availability read.

The two had to be fixed together, and this is the instructive bit: **adding the capability without fixing the identifier would have made a previously-silent path crash on every Custodian purchase.** The unguarded code path worked precisely because it never entered the broken capability. Wiring it correctly wires in the runtime error.

Both directions proved. Post-fix: UR_NonceSaleAvailability(-1) resolves to -1 (the pad holds none) with no crash, and an over-supply acquire of 999999999 is rejected with *"Insufficient Assets for Acquisiton"* — the capability's enforce firing for the first time. Pre-fix reproduced by reverting the identifier and capturing Unbound free variable … DEMIPAD.GOV|LAUNCHPAD|SC_NAME at Custodians:193.

Verified 2026-09-17: 03_Custodians.pact:496-499 and :302-303. [VERIFIED by reading]

4.4 #8M — the stub had to be unreachable, not silent

direct-injection is an **unbuilt feature**. In C_Deposit, with the flag true, the inbound token transfer became a no-op — no tokens entered the shared launchpad account — while the seller-ledger credit XI_DepositForAsset still ran, raising that asset's withdrawable funds-* balance. A later C_Withdraw would then pull those phantom funds out of **other sellers' proceeds** in the same custody account.

Doubly latent: all four live callers hardcode direct-injection = false, and the old capability only let true through when an admin flag was on. But the flag could be flipped. It was a landmine, not dead code.

The design conversation is what makes this finding worth reading. Building the real path needs AQP vaults live, an injection profile, a finalize-and-redeploy cycle — not something testable now. So the decision was to leave a stub. And then the interesting part: **a silent no-op body was explicitly rejected as the stub.** If a flipped flag made the deposit silently take no payment while the acquire flow still delivered the asset, that is a free-asset hole — a worse bug than the one being stubbed out. The stub had to make direct-injection **unreachable**, not quiet.

The result is UEV_DirectInjection, an **unconditional** hard block that ignores the admin flag entirely, composed into DEMIPAD|C>DEPOSIT in place of the old flag-gated branch — plus defence-in-depth: the seller credit is additionally wrapped in (if (not direct-injection) … true), so even if the capability is loosened during the future AQP build, the credit cannot fire before the transfer is wired. The UR_DirectInjection state is kept, reserved, to gate the real path when it exists.

Proved with the admin flag turned **ON**: UEV_DirectInjection true still fails. Bug direction proved by restoring the flag-gated form, at which point with the flag on it returns true — *"expected failure, got result: true"*.

Verified 2026-09-17: 00_Demipad.pact:1208-1216, composed at :580. The @doc records the whole reasoning, including what the reserved state is for. [VERIFIED by reading]

4.5 #17L — the fix that was correct and lasted one day

URCI_Acquire installs capabilities so a buyer can pay without pre-signing transfer caps. The finding had two halves. The first — *the interface omits it, so cross-module calls "don't resolve"* — was **REFUTED**: :: dispatch is dynamic and the suite loads these calls green. What survived was a convention gap (declare it for drift-catching) and a real contract violation: **URCI_ was an improvised prefix**, and worse, the whole UR*/URC* family is defined side-effect-free while this function performs install-capability under a read-looking name.

The chosen fix was not an ad-hoc rename. StoicSyntax-Prefixes.md gained a formally registered cap lowercase specialisation marker, a URCcap_ prefix meaning *"a URC_ that installs the caps it derives"*, a distinct CAP-INSTALL

colour family, an interface-membership corollary, and a flat longest-first prefix index. URCI_Acquire was renamed URCcap_Acquire across five modules, and URC_Acquire + URCcap_Acquire were declared in every relevant interface.

One day later, URCcap_ was removed. StoicSyntax-Prefixes.md:107 records it:

> **Removed 2026-08-31.** A cap-installing helper uses the **CAP_** prefix (Validate/ENFORCE) — installing a defcap is what CAP_ denotes.

The function is CAP_Acquire today, in all five modules and all five interfaces [VERIFIED by command]. The half of the finding that survives — *declare it in the interface, and do not hide a state effect behind a read-shaped name* — is fully honoured. The specific remedy did not survive its own week.

This is a small thing, and it is here because of what it does to the tree: **URCcap_ appears in the DEMIPAD audit documents and nowhere in the code**, and a reader checking that fix by grepping the name the audit records will find nothing and reasonably conclude the fix was lost. It was not. It was superseded. That distinction is only visible if you look for what replaced it, and it is the single most likely place for a future reader of this tree to draw a wrong conclusion.

---

5. What later rounds found, including one thing this audit built and got wrong

The #1C fix was correct, and the reader underneath it was not

The red-team round came back to STOAICO on 2026-09-14 and attacked the dust sweep from the other side. DEFECT-LEDGER.md §1.1d records it as **GS-06**.

URC_ClaimableRewards still branched on one condition:

```
(if (= (UR_Global7) 1) (UR_Global4) (URC_AvailableRewards account))
```

UR_Global7 is unclaimed-count, a property of the **vault**. UR_Global4 is the whole remaining wstoa-supply. The branch asks *"is exactly one claimant left?"* and never *"is **this** account that claimant?"* — so whenever the count happened to be 1, **every** caller was told the entire vault was theirs.

Measured at RedTeam/[RT-E]_Sequencing.repl <<RT-E-001>>, reached by a legitimate admin action rather than an attack:

```
post-stake: unclaimed=1  nzs=3  newcomer-owed=0.000000  newcomer-OFFERED=690.525983
```

No theft was possible, and that is precisely why it survived this audit. The thing preventing it is #1C's own repair — A_Stake stamps a newcomer's last-collected-round to the current round, and the collect capability enforces (< last-collected-round distribution-round), so a newcomer is born already-collected. The money never moved.

But the same reader feeds URCi_Collect and INFO_Collect, so the **preview told such an account it would receive the whole vault**. And the thing preventing the theft was a stamp written in a *different function*, with nothing connecting the two. The owner's ruling, recorded in the ledger: *a wrong reader is an error whether or not it is exploitable.*

Fixed by URC_IzDustSweepClaimant, which adds the two missing O(1) conditions — is this a real staker, and has it already collected this round [VERIFIED by reading — 05_STOAICO.pact:406-421, all three conjuncts present, consumed at :449].

What this means for #1C's verdict. The fix was real, correct and load-bearing: it is what made the defect un-exploitable. But **#1C's own description of the root cause named the reader** — *"stop deriving 'final claimer gets everything' from a mutable global"* — and the reader was not changed. The audit fixed the capability and the counter, which closed the attack, and left the mis-derived number in place. Sixteen days later it was still lying to every preview.

And the sibling it was ported from had the same shape, worse

The same session found **GS-08** in AQP: AQP-RPS::URC_CollectClaimableRewards had the identical counter-only sweep branch — and there, no last-collected-round stamp exists. A fully-exited account with zero weight collected the whole vault while the rightful sole claimant received 0.0. Measured, on the deployed stack. See Chapter 10, §5.

Both are ports of the Stoa coin UrStoa vault, whose comments the code still carries (*"coin step 1"*, *"coin step 2"*). coin is correct: its guard is (and (= unclaimed-count 1) (> available 0.0)), and the repair is preserved in genesis with the pre-fix version commented out directly above it. **The caller conjunct was dropped in both ports.**

The verdict on #13L rests on a premise that a sibling counter later failed

#13L — *unclaimed-count/nzs-count have no lower bound* — was declined, and the reasoning was: *use the canonical AQP implementation, verified bug-free — no clamp.* AQP's WU_Score|NzsCount updates by a computed delta with no floor and no enforce, trusting the *guarded delta*: a decrement is only ever reached on a genuine non-zero→zero transition, after a matching increment. Adding a clamp to STOAICO would diverge from a settled pattern.

That premise **still holds as stated** [VERIFIED by reading — 02_SCORE.pact:3121-3132, WU_Score|NzsCount is still an unclamped (+ old-nzs nz-delta)].

But a different AQP counter, in a different file, failed exactly the way #13L worried about. **GS-09**: XI_1|BookCollectUnclaimed in 04_RPS.pact decremented on deb == 0 alone, with no check that the caller was ever counted — so a zero-weight account decremented the counter *for somebody else*, once per call, for the price of gas. Walk it to 1 and GS-08 re-arms against honest stakers.

So the appeal-to-canonical-sibling argument was **correct about the counter it cited and not transferable to the family**. "This pattern is proven elsewhere" is an argument about one instance of a pattern. The general defence — *trust the guarded delta* — is only as good as the guard, and the AQP-RPS guard was deb == 0 with no membership test.

#13L's disposition is not overturned by this, because STOAICO's counter genuinely is protected — by the round stamp #1C added, plus the flush's uncollected-only walk. But the **form** of the argument should be read with care.

Every citizen launchpad preview quoted a purchase it would refuse

RT-K-006 found that Spark, Snakes, Custodians and StoicPay all price through DEMIPAD::URCi_Deposit, and the amount check lived **inline in DEMIPAD|C>DEPOSIT**, where no reader could share it. Buying **zero** was quoted as valid; buying **negative** was refused in a helper's words rather than the operation's.

The repair is the one worth copying: the check was extracted to UEV_DepositDollarAmount — *one definition, four sales* — and both the capability and the preview call it [VERIFIED by reading — 00_Demipad.pact:518, called at :558 by the capability and :1072 by URCi_Deposit, with a source comment stating *"The op's own gate, not a copy of it"*]. The sale's own amount <= remaining-supply gate was deliberately left alone, because remaining supply changes as other buyers act and a preview should not validate what the caller cannot control.

This is the direct descendant of #12M. The audit added a slippage bound to the execution path in August; the red team found in September that the *quote* the buyer reads before signing did not share the execution path's arithmetic guards. A guard in one of two paths is half a guard.

Two guard-reachability defects in the launchpad core

DEFECT-LEDGER.md §1.2.1 records G-05, and calls it *"the clearest instance"* of the eager-let class in the codebase. The message *"Asset … is not registered to the Demiourgos Lauchpad"* (00_Demipad.pact:550, inside C>DEPOSIT at :527) was unreachable: the caller got a raw Ledger table key instead. The fix was already in the module three lines up — UR_CheckRegistration deliberately wraps its read in (try false …), while its three **sibling** reads of the same row were bare. All three switched to with-default-read. Pinned by <<TX-DEP-02>> and <<DEMIPAD-G2>>.

G-32 is DEMIPAD|C>WITHDRAW's type enforce (00_Demipad.pact:588), and it is **provably dead**: C_Withdraw binds (URv_Funds asset-id type) before entering the capability, and that reader opens with the identical predicate. It was **ruled KEPT** rather than fixed, because <retrieval-amount> is a parameter of the @event capability and moving the read inside would change an event signature indexers consume. That is a legitimate close, and the ledger records it as accepted-and-annotated rather than repaired.

---

6. What remains open

Two WONTFIX verdicts, both argued rather than waved away:

- **#13L** — no clamp on the two counters. Declined to match the canonical AQP pattern. The root cause (#1C's ungated collect) is gone; the residual is defensive. See §5 for the one caveat on the form of the argument.
- **#14L** — fractional team splits for buys not divisible by 4. The precondition is KPAY decimals < 2; KPAY has **24**, confirmed in the test fixture and on-chain via the Pythia dirty-read (DPTF.UR_Decimals "STOICPAY-64EvuR4kgZHd" → 24). Decimals are fixed at issuance, so the condition cannot arise. Guarding against it was declined as defending an impossible state. **This chapter did not re-run the live read** — the 24-decimal figure is quoted from the audit, not independently re-verified. *(not verified)*

Two features deliberately stubbed, with the real work scheduled:

- **Direct injection** (#8M). Hard-blocked. The real path routes the cod royalty portion into an AQP injection profile, or collects it locally for a daily drip automaton. Needs AQP vaults live and a redeploy cycle. Logged in POST-AUDIT-MAIN-ROADMAP.md under STAGE 3 / post-AQP.
- **NF transmit end-to-end** (#7M). The capability wiring is fixed and unit-proved via UEV_AssetFungibility in both directions, but a full non-fungible transmit needs a real DPNF collection and the UEV_IMC/"after-Upgrade" transmit path, which is separate Talos work. The SF direction is proved; **the NF direction is proved at the guard, not end-to-end.**

One product decision, not a code question: the StoicPay sale is **suspended**, and whether to resume it — and on what tokenomics — is open. #11M only re-synced the workspace to what is deployed.

One open question the audit raised and the owner answered by policy, not by code: #12M's original finding was that the launchpad is permissioned, so the admin is trusted, which tempers the front-running risk. The slippage bound was built anyway. The 50 % tolerance ceiling, however, **is a UI policy and not an on-chain bound** — UEV_SlippageCost enforces only the absolute max-cost the buyer signs. The on-chain code holds no poll-time baseline to recover a percentage from, which the @doc states explicitly. A buyer whose wallet signs a bad max-cost is not protected by the chain.

---

7. Verification result for this chapter

Every fix recorded as FIXED was searched for in current source on 2026-09-17.

Nothing was found missing. All 15 fixes are present, in the five files' new citizen locations where applicable. One — #17L's URCcap_ prefix — was **superseded within a day** by URCcap_ → CAP_, which a reader grepping the audit's own wording will misread as a deletion. It is not. See §4.5.

Three caveats a reader should hold against this chapter:

1. **The tree is the thinnest in Part I and the chapter reflects that.** There is no owner-feedback file and no final report; the README's status boxes were never ticked. **Nine** of seventeen findings carry a bidirectional REPL proof — eight fix entries, one covering two findings — which is still a better rate than either neighbouring audit, but the *consolidation* step the sibling audits performed did not happen here.

Corrected 2026-09-18. This said *"Fifteen of seventeen"*, contradicting §2 of this same chapter, which counts the eight Bug reproduced / Bug direction entries in ROUND-02-FIXES.md and names the nine findings they cover. Fifteen is the **FIXED** count in ISSUES-RANKED.md, not the proof count.

2. **Most of this audit's proofs are outside the default pipeline.** [5.3]_Launchpad.repl and [6.3]_STOAICO.repl are **both commented out in Stage02_Tester.repl** (lines 73 and 77), which is what Z.repl loads [VERIFIED by command]. They run under ZALL.repl (lines 92 and 94), which REPL/tools/_gate.py executes, so they are gated — but a developer running the fast path sees **none** of the #1C, #2H, #3H/#4H, #5M, #6M, #7M, #8M, #9M or #10M proofs go green. This is the exact hazard CLAUDE.md names about Z.repl: *the fast path, not the gate.* The fix entry for #1C flagged the wiring gap as a follow-up at the time; it is still the case, by design, and the suites have since been joined by modules/DEMIPAD.repl (37 assertions), modules/LAUNCHPAD.repl (39) and modules/STOAICO.repl (12), all globbed into the gate.
3. **The audit's scope no longer matches its directory.** Five of six audited modules are citizen modules under 2_CITIZEN/7_Launchpad/. The audit tree that describes them sits under 1_SOVEREIGN/. That is a consequence of the audit's own #15L reclassification and is correct; it is also the first thing that will confuse a future reader, so it is stated twice in this chapter on purpose.

Next: Chapter 10 — AQP, the acquisition-pool family, and the largest single body of design work in Part I.

---

10. AQP — acquisition pools

Source tree: Audit/module-audits/AQP/ (14 files, 4,233 lines — it was 13 and 4,112 when this chapter was written; the corrections described in §6 grew it) **Audited:** 2026-08-11 → 2026-08-24, with follow-on hardening to 2026-08-30 **Scope at the time:** 5 modules, ~15,300 lines — the largest core family in Ouronet **Findings:** 33 tracked rows · **Design documents produced:** 10 **Verification pass for this chapter:** 2026-09-17.

---

1. What AQP is, and what breaks if it breaks

AQP is Ouronet's **staking and reward engine**. A user stakes an asset — a fungible token, an LP token, an ortofungible, a semi-fungible or an NFT — into a *pool*; the stake is converted into a *score*; the score becomes a *weight*; rewards injected into a *vault* are divided among weights. That sentence hides five modules and two nested layers of RPS accounting, which is why this audit produced more design documentation than findings.

module	what it owns
01_ANK (Anchors)	*boosts*. An **anchor** says "holding this asset raises your score by N promile". Anchors group into **boost classes**; a user's holdings across a class fold into one aggregate-promile.
02_SCORE	the **score ledger**. Per-user and per-score-total base, boost and deb (Elite-DEB) columns, one write path, and the definitions that convert an asset into a weight.
03_AQP (POOL)	the **pools**: custody, the stake tracker, the rollup, and which of ≤7 scores a pool employs.
04_RPS + 05_FVT	the **reward vaults** — Farms, Vaults and Treasuries. Two-tier RPS: Tier 1 splits an injection across *members*, Tier 2 splits a member's share across its *users*.
06_VCT (Vacate)	the **exit**: unwind every position in a pool, return every asset, zero every aggregate.

Two structural facts drive nearly every finding.

It is an accounting system, and accounting systems fail conservatively or catastrophically. The invariant that has to hold is that the sum of what everyone can claim equals what was injected. A divisor read one instruction too early, a weight stored as a fluctuating value rather than a stable quantity, a "last claimant takes the remainder" branch that tests the wrong thing — each of those breaks conservation, and the symptom is either *rewards stranded forever* or *the vault drained by whoever asks first*. Both happened.

Gas is a correctness property here, not a performance one. A reward settle runs on every stake, every unstake, every collect and every inject. If its cost scales with the number of stakers in the pool, the pool has a size beyond which nobody can transact — a denial of service that arrives by success. Two findings (H5, M2) are scans on that hot path, and the owner's verdict on both was the same three words: *absolute no-go.*

The reward math has a ground truth, and the audit used it: the **UrStoa RPS vault** in 00_StoaSandbox/coin.pact lines 1520–1940, a proven implementation of the same model. Several findings are literally *"the port dropped a guard the model has"*. Several more, found a month later by a different round, are the same thing again.

---

2. How it was audited

The declared cycle, and the one round that did not happen

README.md lays out an append-only cycle: **Round I findings** (frozen) → **Round I owner feedback** (frozen) → **Round II fixes** (sequential, one at a time, each green-lit) → **Round III re-verify** (re-read fixed code cold, enumerate every path, prove correctness; new findings restart the cycle) → repeat until a re-verify round is clean.

Rounds I and II happened. **ROUND-03-REVERIFY.md does not exist.** Like DPDC and DEMIPAD, AQP is a two-round audit that specified a third.

This matters more here than in the sibling chapters, because Round II's fix entries are littered with a field labelled **"Not yet covered (Round III regression)"** — a list of the regression tests each fix *should* have and does not. Fix #7 (H1) names two. Fix #8 (H5) names two. Fix #9 (H4) names the full negative test for its own lock. Those are not oversights; they are a deliberately maintained backlog for a round that was never run.

Round I

One deep-read auditor per module, five modules, ~15,300 lines. The two CRITICALs and M6 were **lead-verified against code** before being written up, which is why ROUND-01-FINDINGS.md tags each finding [CONFIRMED] (the lead re-read it) or [PLAUSIBLE] (auditor-reported, to be confirmed in a REPL round). Three of the five HIGHs went in as PLAUSIBLE. All three were real.

The findings document also contains something the DPDC tree does not: a **"What is VERIFIED CORRECT"** section, enumerating the invariants that were checked and hold — ANK's insert-once issuance and idempotent aggregate recompute; SCORE's single weight-mutation path and exact Σuser == vault-total; POOL's custody↔tracker↔rollup sign consistency; FVT's RPS ordering (bank at the **old** rps before the score changes, checkpoint **after**); VCT's fully-bound TF vacate and on-chain gas ceilings. That section is the sign-off backbone, and it is where the audit stated, of VCT, that *"discipline [is] exemplary"*. Section 5 of this chapter records what a later round found in VCT.

Round I feedback — where the owner changed the work

ROUND-01-OWNER-FEEDBACK.md is 97 lines and unusually consequential. Five findings were converted from bugs into **StoicSyntax rules** (R1–R5) rather than fixed:

- **R1** X-cm_ naming for X functions that return an IGNIS cumulator — L5 was not a bug, it was an unnamed convention.
- **R2** an X function may write more than one table — L2 dismissed.
- **R3** a C_/A_ (or any dependency) that unavoidably uses a scan is **HEAVY** and must be renamed CC_/AA_ *for instant observability*. L3 was confirmed as a principle and softened in practice: *avoid at all costs, never on a daily-hot path, allowed where genuinely unavoidable.*
- **R4** X functions returning a specific value document it in @doc.
- **R5** consolidate all Ouronet-specific rules into one chapter.

That is an audit changing a language convention rather than patching five call sites, and CC_/AA_ is now a load-bearing part of how the whole codebase is read.

The feedback file closes with an owner note that is the most honest thing in the tree:

> Pact is auditable by reading: a careful reader can predict failure events … from the written logic alone — REPL testing confirms but only catches these *if all possibilities are actually exercised*. **The AQP modules grew too complex to hold entirely in one human's head, which is why this audit was delegated.**

Round II — 25 numbered fixes, plus the design documents

ROUND-02-FIXES.md is 1,057 lines and carries **23** ## Fix # headings (26 ## headings in all). What distinguishes it from the DPDC tree is that **five findings were too large to fix and produced a specification instead**. **Ten** design documents live alongside the round files — the eight tabled below plus DSA-DELEGATED-STAKING-DESIGN.md, which this chapter's own §6 uses, and RPS-SPLIT-SCOPING.md, added 2026-09-17:

> **Corrected 2026-09-18.** This said *"1,057 lines and 25 entries"* and *"Eight design documents"*. 1,057 lines is exact; **no counting rule in the file reproduces 25**, so the two mechanical counts are given instead. Eight was the count of the table, not of the directory.

document	what it specifies	landed?
LP-SCORING-REDESIGN.md	two-level RPS: Level-1 = stable LP **amount**, Level-2 = wrapped-STOA **value** at inject only	**partly** — the Level-1 fix shipped (Fix #7); Level-2 refinements deferred
M3-DEB-DESIGN.md	the corrected boost/deb score model + the deb-staleness subsystem	**yes**
SWEEP-VACATE-DESIGN.md	the on-chain re-score sweep that lets an employed anchor be retired	**yes**
ANCHOR-STALENESS-INVENTORY.md	the map of every place the "stale-until-restake" property bites	(an inventory, not a build)
STREAMED-INJECT-DESIGN.md	linear-vesting reward release over a duration	**yes**
VACATE-V2-DESIGN.md	"fast vacate" — separate the cheap transfer work from the expensive accounting work	**partly**
FVT-SPLIT-DESIGN.md + FVT-SPLIT-MANIFEST.md	split the 7,501-line FVT past StoaChain's deploy cliff	**yes**

That is the real shape of this audit: **a third of it is architecture, not repair.** Section 6 reports which claims in those documents are still true.

The proof standard

The tree records green-gates at three levels: a **golden** triplet-collect suite, the fast Z.repl, and — unique to AQP — bash REPL/run-aqp-audit.sh, a comprehensive multi-suite run. At the time it was six suites (comprehensive, core-vct, deb-staleness-proof, sweep-cc, inject-cc, triplet-collect-golden); it declares **15** today, the DSA and stream suites having been added since. The **1,384**-assertion figure is the audit's, and the tree's only source for it (STREAMED-INJECT-DESIGN.md:4) attributes it to **seven** suites, not six. [VERIFIED by command, 2026-09-18]

Running it surfaced three things worth keeping, all recorded in README.md:

- an **MVST double-load** regression from an ATS merge;
- a **coincidental test** (A04) that asserted an arbitrary-nonce stake landed a specific bunny and passed by luck — made deterministic by filtering on URC_ConformNonces;
- a measured gas fact that contradicts intuition: **NF-stake cost is O(anchors defined on the collectable), flat per stake** — 5-anchor stake 92,552 gas vs single-anchor 92,486, Δ 66, 0.07 %. *"The gas lever is how many anchors a collectable defines, not how many a given NFT hits."*

And one Pact gotcha that cost time and is worth passing on: **`select` is disallowed inside an `enforce` predicate** (read-only/sys-only mode). Compute the scan in a `let`, enforce on the value.

3. The findings

33 tracked rows. Severity as recorded; evidence as verified 2026-09-17. Note that 04_FVT.pact was split into 04_RPS.pact + 05_FVT.pact and 05_VCT.pact renumbered to 06_VCT.pact after this audit, so the audit's own file references no longer resolve.

CRITICAL (2)

id	summary	verdict	evidence today
C1	OF/DPSF/DPNF vacate legs are not bound to staked rows — five validators defined and never wired; a pool owner could name any victim's nonce	**FIXED**	06_VCT.pact — all five now appear 2–4× (definition **plus** call sites): URC_VacateOrtoLegBeneficiaryOk, …OrtoNoncesSufficient, … CollectableLegBeneficiaryOk, … CollectableNoncesSufficient, … CollectableRollupSufficient. [VERIFIED by command]
C2	Farm inject divisor captured **before** the in-transaction ghost-TVL sync → over- or under-distribution, vault insolvency	**FIXED**	04_RPS.pact:4477-4483 — the denominator is bound *inside* XI_DistributeInjectAmount, after the sync, via URC_FarmInjectDenominatorFresh for farms. [VERIFIED by reading]

HIGH (6, including one discovered mid-fix)

id	summary	verdict	evidence today
H1	LP base weight marked to market by delta with no floor → **negative base** on a full exit, poisoning the reward divisor	**FIXED** (after one reverted attempt)	02_SCORE.pact — LP Level-1 raw weight is lp-amount × mx, not a reserve-derived value. No clamp needed. See §4.2.
H2	Stake re-open ignores vacate-in-progress — a stake landing mid-vacate strands funds	**FIXED**	03_AQP.pact:1694-1707 — URC_PoolStakeAdmissionOk's @doc names the vacate guard and the audit finding; a separate unstake-direction predicate at :1707. [VERIFIED by reading]
H3	finalize=true re-enables pool-wide stake with no remaining-inventory check; LP pools have two independent streams	**FIXED**	06_VCT.pact:536 — (enforce (URC_PoolFullyVacated pool-id) "Finalize: pool not fully drained (nns != 0)"). [VERIFIED by reading]
H4	Anchor revoke leaves every holder's aggregate promile stale — a dead anchor keeps paying	**TEMP-PATCHED, then completed**	Lock: 01_ANK.pact:682-685 — revoke refused while UR_BC|ScoreLinkCount > 0. Unwind: 05_FVT.pact:3092 CC_SweepRevokeAnchor +

id	summary	verdict	evidence today
			:3142 paginated twin → 01_ANK.pact:2158 XE_SweepRevokeAnchor; decrement at :131 XE_UnbumpBoostClassScoreLinks. **Both halves live.** [VERIFIED by reading]
H5	Triplet Tier-2 divisor runs an unbounded select on the hot path — O(stakers) per stake/collect/inject	**FIXED**	04_RPS.pact:431 total-lane-weight (the maintained divisor) and :386 contrib-weight (the matching per-user snapshot); URD_UserScoreStakerAccounts deleted from SCORE. [VERIFIED by reading]
S4	*(surfaced by H5)* triplet reward math branches on **FVT class** instead of the true-triplet flag → numerator/divisor basis mismatch	**FIXED**	Branches on UR_SCR|TripletTrueTriplet (any class): true → lanes, non-true → Σ-deb.

The 2026-09-19 round — found by MAKING THE MONEY MOVE (6)

These came from a different method than everything above, and the difference is the point. The earlier rounds were found by READING. These were found by building the Custodians delegation vault end to end and then *running the economics* — open a second agency through the public path, oracle it, inject, collect. Every structural assertion in the suite was green throughout: class, unit-score, template active, split rows registered. **Checking that rows exist is not checking that value moves.**

id	summary	verdict	evidence today
N1	ANK|BoostClass has **no owner field at all** — attaching an anchor to an EXISTING class validates only *active* and *a slot is free*. Anyone holding any anchorable asset could attach it to another vault's class and grant their own holders a boost inside that vault's scoring. Six free slots per class	**FIXED**	00_AQP-SCHEMAS.pact ANK|BoostClass.class-owner; enforced in 01_ANK.pact UEV_IssueAnchor via CAP_EnforceAccountOwnership, so no defcap signature moved. Pinned by <<TX-BOOT-OG7>>, which refuses with the class owner's keyset **by name**
N2	The FVT admission rule was **INVERTED** — vault admitted SF/NF, treasury admitted OF. It contradicted URC_TripletCategoryMatchesFvtClass, which was right. Nothing caught it	**FIXED**	05_FVT.pact URC_ScoreClassMatchesFvtClass → vault=TF/OF (1,2), treasury=SF/NF (3,4); Step 8 issues class 2. Four fixtures encoded the inversion and were corrected

id	summary	verdict	evidence today
	because C_Step8 issued four Treasury-NAMED entities at class 1, which worked *only because* the rule was inverted		
N3	The pure Bloodshed score is employed by its pool but admitted to **no FVT**, so every stake on DHBloodshed aborts at 05_FVT.pact:1031. Three of Step 4's four core scores had a treasury; this one had none	**FIXED**	Fifth treasury BloodshedTreasury added across Steps 8/9/12. <<TX-BOOT-12-REWARDS>> reads the reward count back out of RPS: 2 for Bloodshed, [1 1 1 1] for the rest
N4	A delegation vault could not guarantee its own scoring: a score's boost class was settable only AFTER issue, by the score's owner — **the agency operator**. An agency could decline the vault's anchor or point at another class	**FIXED**	The class now rides on SCR|ScoreEntityModel.boost-class-id and is applied by XI_IssueOneFromModel at issue. <<TX-BOOT-OG3>> proves it on an agency opened through the PUBLIC path that never linked anything
N5	Quality-split **LANES read the SILVER score's base only**, so a staker in any other tier produces zero lane weight and a heterogeneous split routes everyone on one row	**OPEN**	04_RPS.pact URC_ComputeTripletLanes. Measured: lane-b/s/g = 0 for both agencies while both were paid. Two resolutions, both with real economic consequence — recorded, not chosen
N6	**Royalty disposal is not isolated per vault** — an inject into one vault funded another vault's disposal	**OPEN**	Found when the inject added to [6.2.9] flipped dsa-grand-tour GT-09 from expect-failure "dispo capabilities exceeded!" to success. The experiment now rolls back; the property is real and unfixed

Three of these were nearly missed by tests that passed for the wrong reason. <<TX-BOOT-OG7>> took three attempts: it first died in coin.TRANSFER before reaching any guard (an expect-failure that "passed" proving nothing), then the hijack **succeeded** because the admin key happened to be in env-sigs for the enclosing transaction. It only became evidence once it failed with the class owner's keyset named in the message. <<AQP-G25>> was **shadowed** by N1's new enforce and would have kept passing while never reaching the revocation check it exists to test.

MEDIUM (7)

id	summary	verdict	evidence today
M1	unclaimed-count == 1 dust sweep never implemented — residual locked in the vault	**DONE**	Two-tier dual sweep built (FVT|T|MemberVault + global). **Later found exploitable — see §5.1.**
M2	Vault/treasury inject denominator runs a keys scan **inside a defcap** — cross-tenant gas coupling	**DONE**	URD_FvtScoreEntityLinkKeysForFvt gone; the divisor point-reads a maintained total-deb-score mirror (02_SCORE.pact:411). [VERIFIED by command — 0 hits for the deleted reader]
M3	A boost-class link with aggregate promile < 1000 **zeroes** rewards — adding a boost reduces effective weight to 0	**DONE**	02_SCORE.pact:2455/2464/2471 — boost is now the **additive part** (base × promile/1000), not a replacement, with the comment *"M3: boost is ADDITIVE"*. [VERIFIED by reading]
M4	Boost-class / boost / deb-boost links settable **after** positions exist → previously-staked base erased	**DONE**	02_SCORE.pact:777, :933, :968 — all three caps enforce nzs-count = 0, message *"vacate to reconfigure"*. [VERIFIED by reading]
M5	OF/collectable unstake recovers the beneficiary from a **self key** while stake writes a caller-supplied one → non-self stakes unrecoverable	**DONE**	03_AQP.pact:1773/1808/1847 — *"beneficiary-id is caller-supplied (self OR foreign), not a self-key derivation"*; both self-key readers deleted. [VERIFIED by command]
M6	TF anchor promile is pro-rated; the README documents a whole-step model with a 1000 cap	**DOC-FIX + new guard**	Code was right. 01_ANK.pact:1617-1619 UEV_Promile — precision exactly 3, promile in [1, 10000]. [VERIFIED by reading]
M7	*(ATSU)* C_Coil/C_Curl revert when a tiny input's pool-index conversion rounds below token precision	**REFUTED — by design**	A precision artefact; amounts within ~1 ulp of maximum precision are unusable for Coil/Curl by design.

LOW (10)

id	summary	verdict	evidence today
L1	An enforce inside a URC_	**DONE — and the whole check was tautological**	03_AQP.pact:721 records the deletion: DPOF::C_Transfer

id	summary	verdict	evidence today
			moves *whole* nonces and ignores amounts, and callers sourced nonce-amounts from the very reader the check compared against. [VERIFIED by reading]
L2	One X function writes two tables	**CONVENTION R2** — allowed	
L3	select/URD on a sync path	**CONVENTION R3** — CC_/AA_ rename	
L4	ANK|C>REVOKE has no liveness gate; UEV_LiveAnchor exists and is unused	**DONE**	01_ANK.pact:679 — (UEV_LiveAnchor anchor-id) is the first form in the capability. **The validator itself was later found mute — see §5.2.** [VERIFIED by reading]
L5	XE_ returns an OutputCumulator	**CONVENTION R1**	
L6	SF/NF incremental promile has no floor → negative promile	**DONE**	01_ANK.pact:1815-1824 — floored at the WW_Anchors write chokepoint, the sole write path for TF/SF/NF, incremental and absolute. [VERIFIED by reading]
L7	Negative score weight	**DONE** — after a **wrong** first close. See §4.5	02_SCORE.pact:2116-2133 (clamp at source), :841 (definition validators), 10_DPDC-N.pact UEV_Score tightened. [VERIFIED by reading]
L8	Trailing non-write X returns	**CONVENTION R4** — allowed, document in @doc	
L9	Dead VACATE-MAX-LEGS = 16 + an unused parity helper	**DEFERRED, then removed**	Both gone. [VERIFIED by command — 0 hits tree-wide]
L10	CC_Collect settles the member Tier-2 twice	**DONE**	Already resolved by M3's redesign; three orphaned functions deleted. [VERIFIED by command — XI_CollectRpsPreScore, XI_1|CollectSettleAndBank, URDC_BuildCollectScorePlan all absent]

New findings raised during Round II (2)

id	summary	verdict
N1	The comprehensive suite's "negative payout" and "over-accumulate" failures	**RESOLVED — no core bug, three *test-layer* defects**: a driver double-loading a suite that self-loads, two assertions testing an absolute value where a delta was meant, and one coincidental pass
N2	The enforced-fresh inject was excluded from farms on an incomplete rationale	**DONE** — the class ≠0 guard's reasoning (*"farms use a fresh denominator"*) covered only the farm Tier-1 S; the Tier-2 L_i divisor is stale-able for singular and non-true-triplet members, which a **mosaic farm** hits

The vacate rehaul's own findings (6)

Raised by an owner-requested adversarial review *after* the single-transaction agnostic vacate shipped. They sit inside H4 phase 4 and change no verdict above.

id	sev	summary	verdict	evidence today
V1	CRIT	CC_FullVacate orphans the class-0 Z| sleeping-LP satellite (satellites wrongly gated to class == 1)	**FIXED**	06_VCT.pact:1569-1574 URC_VacatePoolOfIds — class 0 → Z|, class 1 → Z| **and** H|. [VERIFIED by reading]
V2	HIGH	CC_FullVacate orphans the F| frozen-TF lane (the scan was native-only)	**FIXED**	06_VCT.pact:1551-1567 URC_VacatePoolTfIds — native asset-id **plus** its F| counterpart when one exists. [VERIFIED by reading]
V3	CRIT	An empty scan aborts the whole vacate (enumerate 0 -1 → out of bounds; a 0.0 TF debit → UEV_Amount) → satellite-only or empty pools unvacatable	**FIXED**	Empty-guards no-op each lane; pinned [6.2.5] TX-VCT-CC02.
V4	HIGH	The multi-transaction C_Vacate*Legs path is **not parallel-safe** — per-leg unwind writes shared Tier-2 aggregates, so two disjoint	**OPEN at the time — since SHIPPED**	06_VCT.pact:71-76 CCp_BatchVacate{TrueFungible,OrtoFungible,Collectables} — the begin → parallel drain → finalize shape. [VERIFIED by reading]

id	sev	summary	verdict	evidence today
		slices write-conflict		
V5	LOW	Collectable amount uses round where floor was proven	**FIXED**	
V6	—	VCT|C>VACATE drops the R| reserved-asset gate	**REFUTED**	The stake path already rejects R| legs, so no R| stake row can exist to vacate. UEV_TrueFungibleStakeNotReserved remains at 06_VCT.pact:2230. [VERIFIED by reading]

4. Seven findings worth the retelling

4.1 C1 — five validators, written, correct, and connected to nothing

The TF vacate path binds every leg to a real staked row: URC_VacateTfLegsOk → URC_VacateTfLegBalancesOk enforces amount > 0, amount == staked-bal(owner, beneficiary), and rollup-bal >= amount. The OF and collectable paths validate only that the asset matches the pool, that the arrays are the same length, that the gas and nonce totals are under the ceiling, and that the caller owns the pool.

Nothing tied the supplied owner-ids / beneficiary-ids / nonces to any actual staked row.

The five validators that would have closed it existed. The auditor found them by counting occurrences: each appeared **exactly once** in the file — the definition — against the TF validator's two. A defined-and-never-called function is invisible to every kind of review except a call-graph count, and it looks *more* reassuring than an absent one, because grep finds it.

The exploit is fund theft by the pool owner, and the shape is worth spelling out because it is not the obvious one. The OF vault transfer is **whole-nonce and receiver-directed**: the pool owner signs a vacate with owner-ids = [attacker] and nonces = [[victim's staked nonce]]. The amount resolver reads the *attacker's* empty tracker key and returns [[0.0]]. Every check passes, because every check is about shape. The bulk transfer then moves the **victim's** whole nonce from the vault to the attacker, and the unwind — at amount 0 — leaves the victim's tracker, score and RPS position completely intact. Stolen inventory plus pool insolvency at the next honest unstake.

DPSF and DPNF are worse: the amount is caller-controlled and the balance write has no >= 0 floor and no sufficiency check on the vacate path.

The fix is exactly what the finding said: wire the five, as TF already does.

Verified 2026-09-17 by the same method that found it — an occurrence count. All five now appear 2–4 times in 06_VCT.pact, and URC_VacateTfLegsOk still appears twice. [VERIFIED by command]

4.2 H1 — the fix that broke the exit, and the redesign that replaced it

The finding: an LP position's **value** — its lp-denominator equivalent at *current* SWP reserves — is stored as the user's persistent base score, and reversed on unstake at the *then*-current reserves. Stake 1,000 LP when it is worth 100; reserves drift so it is worth 105; unstake and the base goes to **−5** after a full exit that should be 0. FVT then reads that as a Tier-2 weight — a negative in the reward divisor.

Attempt 1 was the obvious one and it was wrong. Clamp the base at 0. It broke three DPNF vacate assertions immediately, and instrumenting the base explained why: the DPNF vacate legitimately drives base **transiently negative and then corrects it** in a later step. A per-step clamp destroyed the correction, and the pool never registered as fully vacated. Reverted.

The lesson the fix entry records is the transferable part:

> A blanket floor cannot distinguish a *final* bad negative (LP) from a *transient* legitimate one (DPNF).

Three readers then went at the architecture rather than the symptom, producing LP-SCORING-REDESIGN.md. The diagnosis is a **category error**: an LP token has no fixed value — it is a claim on two fluctuating reserves — and the code stored a fluctuating quantity as if it were fixed.

The intended design is a **two-level RPS**:

- **Level 1, within one LP family.** All LP tokens of a family are fungible, so the user's stored score is **LP amount × mx** (the frozen/sleeping variants simply count as more or less amount). Stable. A full unstake nets to exactly 0 **by construction** — no clamp required.
- **Level 2, across families.** Each FVT entity's weight is its current wrapped-STOA value, computed **only at inject time** to split *that* injection, and **never persisted as anyone's score**.

The striking result of the study is that FVT's Level-2 side **already did this correctly** (ghost-tvl-weight = SWP::UR_StoaValue, consumed transiently at inject). The bug was one decision in one module: SCORE storing value at Level 1.

The corrected fix is three lines and no schema change: two raw-weight expressions switch from equiv(reserves) × mx to amount × mx, and the now-callerless URC_LpAmountToLpDenominatorEquivalent is retired. The shared delta path is untouched, so DPNF's legitimate transient negatives survive.

Verified 2026-09-17: the LP Level-1 weight is amount-based. [VERIFIED by reading]

Still open, and recorded as such in the design document: whether Level 2 should split by *whole-pool TVL* (what it does) or by *staked value*; and whether FVT's Level-2 persist-and-sync should become fully transient. Both are gas/purity questions, not correctness ones — the C2 fix already makes the current shape conserve.

4.3 H5 — deleting a scan without breaking conservation

URC_FarmTripletTier1Denominator ran a full select over SCR|T|UserScore, mapped over **every** staker in the pool, and re-derived each staker's lane weight **live** — on every stake, every collect and every inject touching a triplet farm member. O(stakers) on the hot path, and the owner's verdict was *absolute no-go*.

Replacing a scan with a cached number is where reward systems get quietly broken, because the numerator and the divisor must always come from the **same basis**. Cache the divisor and keep a live numerator and you have created a slow leak.

The design is careful about exactly that. The farm triplet was the **only** score-entity that recomputed live; singular scores and vault triplets already point-read maintained totals. So:

- each user's lane weight is **stored** as contrib-weight in a new FVT|T|MemberUserWeight table;
- the per-member sum is **maintained** as total-lane-weight on the ScoreEntityLink row;
- the **numerator** switches from live derivation to the stored contrib-weight;
- the **divisor** switches from the scan to a point read of total-lane-weight;
- a new XI_SyncTripletLaneWeights at **phase 4.6** advances both by delta — placed after the existing phase-4.5 mirror sync, mirroring its read-old-at-2.3 / write-after-SCORE ordering.

The conservation argument is structural rather than empirical: at stake, phase 2.3 banks the user's pending at the **old** stored contribution and settles against the **old** stored total — both before 4.6 — and 4.6 then advances

both together. Numerator and divisor always share one snapshot, so Σ contrib-weight ≡ total-lane-weight **by construction**.

URD_UserScoreStakerAccounts was then deleted from SCORE entirely, which is the part that makes this a real fix rather than a faster path: the scan cannot come back by accident.

The proof asserts the invariant directly rather than a total: at [6.4]_AQP-TRIPLET-COLLECT <<TX-AQP-CL04>> (:365-370), both stakers' w-user are asserted positive and the maintained divisor is asserted to equal their sum, within 1e-6.

> **Corrected 2026-09-18.** This said *"ANHD has w-user = 10, EMMA has w-user = 4, and the maintained divisor reads 14"*. No such literals appear in the file. The pins are **relational**, which is the stronger form — they hold at any reserve state, where literals would go red on the next ordinary stake.

The cost is honestly stated: the farm triplet now accepts the same **eventual consistency** as every other deb-based score — a user's weight is a snapshot from their last stake, stale until they restake. That property was already system-wide, and cataloguing it is what produced ANCHOR-STALENESS-INVENTORY.md.

Verified 2026-09-17: 04_RPS.pact:431 and :386; the deleted reader has zero hits. [VERIFIED by command]

4.4 H4 — the fix that admits it is temporary, in the source

Revoking an anchor deactivates it and strips it from the boost-class and asset bookkeeping. It never decrements any holder's stored aggregate-promile, and it never zeroes the per-user anchor promile. SCORE reads the stored aggregate directly. **So a revoked anchor keeps paying its boost** until each user happens to trigger an update on some *other* live anchor in the same class.

There is no O(1) correction. You cannot enumerate 10,000 holders on revoke. The owner offered two directions: **(A)** a lazy method that is still mathematically correct, or **(B)** disallow revoking an anchor used by ≥1 score — to truly revoke, first flush all stakers through Vacate.

B was chosen, and only half of B was built. Fix #9 is the lock: a new ANK|T|BoostClassLinkCount table (forced onto ANK by deploy order — ANK deploys before SCORE and cannot read it at revoke time), incremented by SCORE at the single link-creation site through a forward entrypoint, and enforced in ANK|C>REVOKE. The other half — freeze, run an on-chain re-score sweep, unlink, then revoke — did not exist.

The consequence is stated bluntly in the inventory and in the fix entry: **once a boost-class is linked to a score, its anchors are locked forever.** Intentionally conservative. Safe, and stricter than the final design.

This is the right way to ship half a fix: the lock is correct on its own, the gap is named in two documents and in the source, and the constraint it imposes is described in terms of user-visible behaviour rather than internal state.

The other half has since landed, which neither document records. CC_SweepRevokeAnchor (05_FVT.pact:3092) is the single-transaction re-score sweep, with a paginated defun+gate twin at :3142 for sets that exceed one transaction; both terminate in ANK::XE_SweepRevokeAnchor (01_ANK.pact:2158), whose capability deliberately enforces liveness and ownership but **not** the #9 link lock — *"the sweep has already refreshed every affected holder, so no staleness remains"*. The decrement exists as XE_UnbumpBoostClassScoreLinks. [VERIFIED by reading]

ANCHOR-STALENESS-INVENTORY.md said **"NOT BUILT YET."** It now carries a correction box at :58-77 naming both Talos doors and the live decrement at 01_ANK.pact:2184, written while this book was assembled. See §6.

4.5 L7 — the audit closed it as a misdiagnosis, and the close was the misdiagnosis

This one is included because it is the only finding in Part I where the audit **wrote a formal "no fix, the finding's premise is wrong" entry and then had to supersede it**, and both entries are still in the file.

L7 reported that negative score weight was reachable. The obvious fix — floor the base-delta chokepoint at 0, the analogue of L6's fix — broke three DPNF vacate assertions. Instrumenting the base showed the probe stakes an NFT whose trait-score definition is **−1**: on stake the user base goes to −1, on unstake back to 0, a correct round trip. The auditor concluded that **SCORE base is a sum of trait-score values, trait scores can be negative by design** (a trait that *reduces* weight), and therefore a negative user base is legitimate. Fix #19 was written as NO FIX — misdiagnosis, with a careful L6-versus-L7 distinction:

> #18's floor was correct because ANK promile is count × ank-promile with ank-promile ∈ [1,10000] and count ≥ 0 ⇒ promile is always ≥ 0 … SCORE base has no such lower bound, so the same floor corrupts valid data. **Before flooring any accumulator, confirm the negative is actually invalid.**

That is good reasoning from a false premise. **The owner flagged that negative scores are not a designed feature**, which forced a trace of where the −1 actually came from.

It came from DPDC. The probe mints a **metadata-less** nonce, and UDC_NoMetaData → UDC_MetaData {} → UDC_NonceMetaData -1.0 defaults an unscored nonce's native score to the **-1.0 unscored sentinel**. SCORE's model-0 weight path read the **raw** score and counted the sentinel as a real negative. DPDC's *cooked* reader already maps -1.0 → 0 — and the weight path bypassed it. Compounding it, DPDC's UEV_Score enforced only >= -1.0, so genuine negatives in [-1.0, 0) were settable too.

The corrected fix is at the source, in three places, with no aggregate floor anywhere: clamp each per-nonce native score at the model-0 weight reader; make the SF and NF definition validators enforce score >= 0 so negatives cannot be authored; and tighten DPDC's UEV_Score to (or (= score -1.0) (>= score 0.0)) — the exact sentinel, or a non-negative value, and nothing between.

Three things make this the most instructive entry in the tree. The wrong close was *carefully argued*, and its careful argument is what made it convincing. It was caught by the owner contradicting a premise, not by any test — the suite was green under both closes. And the audit **kept the wrong entry in the file**, marked superseded, rather than editing it away. A correction with the error deleted teaches nobody anything.

There is a fourth thing. This is the **same -1.0 DPDC sentinel** that DPDC's own audit found leaking through three of four branches of UR_N|Score (#19H, Chapter 8 §4.6), found independently, in a different module, by a different team, weeks apart. And DPDC's finding notes that *a sibling audit (AQP) had already committed a false "already fixed" assumption about this exact function to its own audit trail.* One sentinel, two audits, three wrong conclusions between them.

Verified 2026-09-17: 02_SCORE.pact:2116-2133 carries the clamp and the L7 #19 reference; the definition validators are at :841. [VERIFIED by reading]

4.6 M3 — the footgun where adding a boost removed your rewards

nominal-boosted = floor(base × promile/1000, p), where promile is the user's aggregate across the boost class and **defaults to 0.0** with no 1000 baseline.

Link a boost class and hold none of its anchors, and boosted = deb = 0. A real stake earns **nothing**. Adding a boost class *reduced* effective weight to zero. The code matched the README formula literally, which is what made it survive.

The owner's model, locked in M3-DEB-DESIGN.md, is that boost is a **bonus, not a replacement**:

- **boost-part** = base × promile/1000
- **pre-deb total** = base + boost-part
- **deb** (Elite-DEB) is an **end multiplier on the sum**, final = (base + boost) × deb, and is always ≥ 1.0

The last of those was not obvious and the document records why it is necessary: a **base-0, boost-N** score — a pure-boost position, which the triplet case produces — would vanish entirely under a deb applied to base only.

The consequence is five stored score-level totals rather than two, because the two post-deb totals are **not derivable** from the pre-deb ones (deb is per-user): total-base, total-boost, total-base-deb, total-boost-deb, and total-deb-score — the reward denominator. At user level, store two and derive three.

The design then grew a whole subsystem, because a per-user deb that can change means a stored deb-score can go **stale**, and a stale numerator against a fresh divisor breaks conservation. Part 2 is the deb-staleness architecture: a staleness detector, a collect-time backstop, an enforced-fresh inject, a chunked fallback for spikes, and an IGNIS surcharge so a user who forces someone else's inject to fix their staleness pays for it.

Verified 2026-09-17: the additive model is live at 02_SCORE.pact:2455-2471, the five totals at :409-413. [VERIFIED by reading — :411-413 holds three of the five]

And 2b was found defective by the audit itself, in writing. M3-DEB-DESIGN.md records the collect-time backstop as *"WRITTEN, DEFECTIVE, REBUILDING"*: the refresh mutated SCORE's deb-score **without resyncing FVT's total-deb-score mirror**, desyncing the divisor and breaking conservation on a real deb change. The sentence explaining why the suites stayed green is the best one-line summary of this entire audit's method:

> Passed the suites only because deb is static there (refresh = permanent no-op → the branch was never executed).

4.7 L1 — the misplaced enforce that was not a misplaced enforce

Reported as discipline: URC_OrtoStakeWholeNonceAmounts contains an enforce, and a URC_ must not. Move it to a UEV_ or return a bool. A ten-minute fix.

Investigating it found that the **whole check was tautological**. DPOF::C_Transfer moves *whole* nonces and ignores the amounts array entirely, and every caller sourced nonce-amounts from UR_NoncesSupplies — so the check compared a value against its own source. It could not fail.

The helper was deleted rather than relocated.

This is the pattern the codebase later named and hunted systematically: DEFECT-LEDGER.md §1.2.3 catalogues **tautologies and key-echo checks** — G-30 in this same 02_SCORE.pact, where one word in a reader name (UR_AccountNoncesSupplies versus UR_NoncesSupplies) is the entire difference between a live guard and a dead one, *and the two enforces look identical at the site*; and G-31, two VCT beneficiary checks where the beneficiary is a **component of the lookup key** and the reader is a with-default-read whose default is built from that key, so the function cannot return false in either direction.

L1 is the first instance in the programme of a class that turned out to be common. A prefix violation is often a **symptom**: the check is in the wrong place because nobody could work out what it was for — including, sometimes, its author.

---

5. What later rounds found in AQP

5.1 M1 — the audit built the dust sweep, and wrote the test that hid the bug

M1 reported that the unclaimed-count == 1 full-supply branch from the canonical vault was never implemented in AQP, so residual dust accumulated locked in the vault. Fix #10 built it, as a **dual** sweep: per-member Tier 1 plus global Tier 2, the mathematically complete port extended to both tiers. It shipped drain-proof green.

The fix entry states its own verification target:

> a wind-down test — last user in a member sweeps member.available; last user globally sweeps the Tier-2 remainder; **both vaults drain to exactly 0**.

And it states the decrement condition it implemented:

Collect decrement added to XI_1|BookCollectUnclaimed at the same deb == 0 condition.

On 2026-09-14 the red-team round found both of those sentences to be the defect. DEFECT-LEDGER.md §1.1d records it as **GS-08** and **GS-09**, and the owner asked the question that found it: *"the audit of the STOA ICO allegedly fixed the dust sweep following the canonical model of the coin module — can we verify this? because if this is incorrect, the implementation in the AQP module also might be off."*

GS-08. URC_CollectClaimableRewards branched on **counters alone**:

```
(if (= gc 1) (UR_FVT-RG|AvailableRewards ...)          ;; the WHOLE global vault
  (if (and (= (UR_FVT|FvtClass fvt-id) 0) (= mc 1))
    (UR_FVT-MV|AvailableRewards ...)                   ;; the WHOLE member vault
    (URC_UserTier1AvailableRewards ... deb-user)))
```

gc is a property of the **vault**; mc of the **member**. deb-user — the one value identifying the caller — is bound one line above and used **only in the else-branch**. Measured on the deployed stack, driving [6.4]_AQP-TRIPLET-COLLECT to its wind-down with ANHD the sole real claimant (deb = 10.0) and EMMA fully exited (deb = 0.0):

```
gc=1   emma-claimable == anhd-claimable      <- the reader could not tell them apart
EMMA (exited) collected the sweep, vault -> 0
ANHD (rightful sole claimant) gained 0.0
```

UEV_CollectContext checks pool, FVT, link and ownership, and never that the caller is a staker, holds a claim, or has already collected. And gc == 1 is a **normal end-of-life state**, not an attack precondition.

GS-09, exposed by fixing GS-08. With the reader corrected, ANHD received 0 and the dust was *stranded* — because XI_1|BookCollectUnclaimed decremented on deb == 0 alone, with no check that the caller had ever been counted. A zero-weight account decremented the counter **for somebody else**, once per call, for the price of gas. Enough calls and gc reaches 1 while honest stakers are still staked, re-arming GS-08 against them. *Fixing GS-08 alone converts theft into a fund-lock; the two must be fixed together.*

Why the suite was green through both. The audit's own assertions checked that the vault **drained** (ar-final → 0). The ledger's sentence:

> **Conservation is satisfied whether the money reaches the rightful claimant or someone who just left.**
>
> A test that asserts an outcome does not assert who caused it.

Both are ports of the Stoa coin UrStoa vault — the AQP comments still name the steps (*"coin step 1"*, *"coin step 2"*). coin is correct: its guard is (and (= unclaimed-count 1) (> available 0.0)), and its own repair is preserved in genesis/stoa-genesis-4.pact with the pre-fix version commented out immediately above. **The caller conjunct was dropped in the copy — in AQP and, separately, in STOAICO (Chapter 9 §5).**

The second dropped guard is the subtler one and it caused GS-09. In coin, a zero-amount payout **aborts** at step 1 (C_Transmit → UEV_Amount). AQP turned that abort into a **skip** ((if (<= payout 0.0) (UC_EmptyOc) ...)). That reads like making a no-op graceful; it exposed a write three steps downstream.

> **A guard can be load-bearing for code it does not mention.** Removing an abort relaxes everything sequenced after it.

Both fixed and verified 2026-09-17. 04_RPS.pact:1891 and :1894 now read (and (= gc 1) (> deb-user 0.0)) and (fold (and) true [(= class 0) (= mc 1) (> deb-user 0.0)]) respectively, with a 25-line source comment carrying the measurement. XI_1|BookCollectUnclaimed additionally requires unsettled pending, and PHASE 3 was resequenced before PHASE 2 so that value is still readable. Pinned by [6.4]_AQP-TRIPLET-COLLECT <<TX-AQP-CL04>>'s sweep-drain block (:436-525, eight assertions) — including the two a careless repair would break: the rightful claimant must still sweep, **and** the vault must still drain to 0. Returning 0.0 to everyone satisfies "the non-claimant gets nothing" while destroying the sweep. [VERIFIED by reading]

A codebase-wide scan for the shape — *a branch on (= <counter> 1) selecting a whole-balance reader* — returned **10 sites, and every live one is now guarded.** 06_VCT.pact has no sweep branch at all; 05_FVT.pact is a facade delegating to RPS, so the RPS repair covers it.

5.2 L4 — the audit wired in a validator that could never speak

Fix #17 closed L4 by adding (UEV_LiveAnchor anchor-id) as the first check in ANK|C>REVOKE. The validator already existed and was unused; wiring it in means a double-revoke aborts cleanly at the top instead of deep inside a list utility. Correct fix, correct placement, proven at [6.2.1] TX002·06b.

RT-K-007 found that UEV_LiveAnchor had never been able to deliver its own message.

```
(defun UEV_LiveAnchor (anchor-id:string)
  (let ((iz-anchor-active:bool (UR_ANK|State anchor-id)))
    (enforce iz-anchor-active (format "Anchor {} must be alive for operation" [anchor-id]))))
```

UR_ANK|State was a bare read. Pact evaluates let bindings before the body, so for the one input that most needs the message — an anchor that does not exist — the raw table error No value found in table … ANK|T|Anchor for key: <id> fired **one line earlier, every time.**

The ledger records this as the **third instance of one shape and the first where the obstruction was a READER rather than an eager let in the caller**. C_Recover and C_HotRecovery had their capability below the bindings; C_DeployAccount had its check below the bindings; here the check sits in exactly the right place and *the value it reads raises before it can be tested.*

> **A guard being present is not a guard being reachable, and grep cannot tell the two apart.**

Fixed by defaulting UR_ANK|State — an anchor that does not exist is not active, which is what both callers mean by the question. The source comment at 01_ANK.pact:1055-1066 records the whole thing, including the DPTF precedent it follows. [VERIFIED by reading]

And it exposed a test that was vacuous about the gate it named. [6.2.10]_AQP-NEGATIVES.repl <<TX-AQP-NEG-OWNER2>> was labelled *"anchor-owner gate"* while actually pinning the raw missing-row error, because its anchor-id was derived from a block hash and no such anchor had ever existed. **Delete the owner gate and the line would still have passed.** RT-K-007 surfaced it by changing the message it was matching. It now pins the **liveness** gate, which is what that input actually tests.

5.3 Six more mute guards in AQP, and the count is the finding

DEFECT-LEDGER.md §1.2 catalogues the guard-reachability class across the codebase. AQP holds a disproportionate share, and several sit directly on top of this audit's work:

- **G-09** 05_FVT.pact UEV_AddScoreEntityTripletContext — *"Triplet must be issued in AQP-SCORE"* was dead until 2026-09-10: the let hard-read the triplet table **five times**, even though URC_TripletExists is a with-default-read written specifically to answer for a missing row.
- **G-13** 06_VCT.pact TRUE-FUNGIBLE-VACATE-BATCH — **a guard that knew and had no voice.** (gas-ok (URC_TfOwnerArraysGasOk …)) correctly computes false for an empty batch, but the sibling binding (legs (UC_TfLegsFromParallelArrays …)) faults first. The asymmetry proves intent: the Orto and Collectable leg builders already guard (if (> l 0) …); the TF one did not.
- **G-15 / G-16** 04_RPS.pact — sub-shape **C**, the one an audit for the other two walks straight past: a later conjunct of a fold (and) hard-reads the value an earlier conjunct is validating. (fold (and) true) **does not short-circuit** (verified directly), so passing BAR — the exact case the first conjunct exists to reject — aborts on key |. *The cheap check IS first; the ordering IS right; the code looks correct.*
- **G-20 / G-21 / G-37 … G-41** — seven sites in 02_SCORE.pact, 03_AQP.pact and 05_FVT.pact whose messages claim *"score must exist"* while the bindings above them bare-read the score row.

The last group carries the most useful methodological point in the ledger. **G-20 and G-21 were found by hand and recorded as two sites. The class is seven**, and five were invisible because the instrument reading them

truncated its window at 300 characters and the messages sat past it. One of the five is the direct twin of an already-ledgered site in the same file — *which is the strongest available evidence that hand-finding had been sampling this class, not enumerating it.*

5.4 The owner gates that had never refused anybody

DEFECT-LEDGER.md §7.2h asked a measurable question: **which owner-ownership capabilities has no test ever shown refusing anybody?** Four AQP gates sit behind a **latched** flag — once false it is irreversible, so the gate behind it is permanently unreachable for that entity. Exactly one had ever been reached by a non-owner:

defcap	shadowing enforce	non-owner test before 2026-09-16
FVT|C>CONTROL-FVT	can-upgrade	**yes**
SCR|C>CONTROL-SCORE	can-upgrade	none
FVT|C>ROTATE-OWNERSHIP-FVT	can-change-owner ∧ distinct	none
SCR|C>ROTATE-OWNERSHIP-SCORE	can-change-owner ∧ distinct	none

Every expect-failure on the latter three signed **as the owner**, so their CAP_EnforceAccountOwnership had never once been shown refusing anybody.

The remedy is worth reading because the obvious one is wrong. The 2026-09-14 owner ruling says authorisation precedes validation inside a defcap, and a naive reading says hoist all of them. **Two AQP suites deliberately depend on the current order** to reach an argument guard *without a signature* — [6.2.10] TX-AQP-NEG-SCRCTL and [6.4] <<TX-AQP-FA01>>. Hoisting the ownership gate there would make the *distinctness* clause unreachable instead: one unobservable guard traded for another.

> **Ordering can expose only ONE of two state-dependent guards at a time. A fixture that satisfies the first guard exposes BOTH.**

Closed **additively** in <<TX-AQP-NEG-OWNER2>>, using fixtures whose latched flag is still true so ownership is the only thing left that can refuse — and, for rotation, passing a foreign account as the *new* owner, so distinctness is satisfied by the same fact that makes her a non-owner. Each refusal is preceded by an expect pinning the flag it depends on, so a future suite that latches a flag early turns the **precondition** red rather than letting the refusal go quietly vacuous.

5.5 A bare-called defcap that silently no-opped the entire vacate owner gate

Not in either ledger — recorded only in the source, at 06_VCT.pact:2214:

> CAP_VctVacatePoolOwner … MUST be a defun (not a defcap): every VCT|C>* vacate/abort cap calls it BARE — (CAP_VctVacatePoolOwner pool-id) — as an inline enforce. **A defcap bare-called that way does NOT run its body**, which silently no-opped the owner gate on the WHOLE vacate surface (CC_FullVacate / XB_Vacate* / Cp_BatchVacate* / C_AbortVacate) — any non-owner could vacate or abort.

It is a defun today and the enforce runs [VERIFIED by reading]. Round I's finding C1 was about *which legs* a pool owner may vacate; this was about **whether the caller had to be the pool owner at all**, across every entry point in the module — and it is in the module Round I signed off as *"discipline exemplary"*.

This chapter did **not** establish when the defcap-versus-defun shape was introduced, or whether it predated the audit. git log -S on the current definition returns only the file-renumbering commit from the FVT split, which tells us nothing about the earlier form. *(not verified)*

---

6. What remains open

The design documents: what landed

Each of these was checked against current source on 2026-09-17.

design	claim	status today
FVT split (FVT-SPLIT-DESIGN.md, -MANIFEST.md)	04_FVT.pact at 7,501 lines is **past StoaChain's deploy cliff** (~6,635 lines, because the size charge grows as the **7th power** of transaction size against a 2,000,000-gas block limit). Split into RPS + FVT along the capability seam.	**SHIPPED.** 04_RPS.pact and 05_FVT.pact both exist; RPS owns the 7 reward tables and its own SECURE; FVT is a facade (URC_InjectDenominator at 05_FVT.pact:3769 delegates to RPS). The manifest's key result — *"0 functions directly touch both domains"* — is what made the seam clean. [VERIFIED by reading] The design document's own footer still reads *"awaiting owner GO to start P1"*; the manifest's does not.
Streamed inject (STREAMED-INJECT-DESIGN.md)	rewards may vest linearly over a duration; late stakers earn the portion released after they join; independent overlapping streams, no merge; slots capped by the owner's Elite tier	**SHIPPED.** FVT|RPS|Stream schema at 04_RPS.pact:349, table at :548, URC_ReleasableToNow at :1902. The instant and streamed paths share one distribution core (XI_DistributeInjectAmount), so *"a streamed release is IDENTICAL to an instant inject of the same amount"*. [VERIFIED by reading]
DSA — delegated staking (DSA-DELEGATED-STAKING-DESIGN.md)	a MultiversX-style delegation layer: 1 agency = 1 FVT member; operators run nodes to capture reward units and take a fee	**BUILT, PARTIAL.** 08_DSA.pact exists and its header states the phasing: data model + vault define + agency open (Phase 2), capture recompute + delegated oracle (Phase 3), **royalty disposal + collect (later)**. The document's own §11 defers elite-tier fee reduction and the Vesta conversion. **Not a shipped feature end to end.** [VERIFIED by reading]
Sweep/vacate (SWEEP-VACATE-DESIGN.md)	the re-score sweep that lets an employed anchor be retired; freeze stake **and** collect on affected pools while it runs	**SHIPPED.** See §4.4.
Vacate v2 (VACATE-V2-DESIGN.md)	separate the cheap transfer work from the expensive accounting work; *"~95 % of each batch's per-position gas is reward overhead, not the transfer"*; 13k NFTs at 30/tx ≈ **433 transactions**, versus ~17 at the transfer floor	**PARTLY.** The parallel-safe CCp_BatchVacate* family is live (this is V4's fix) and the vacate-generation field the document specifies is implemented. Whether the full drain/finalize separation is complete was **not established by this chapter.** *(not verified)*
LP scoring (LP-SCORING-	two-level RPS	**Level 1 shipped** (Fix #7). The

design	claim	status today
REDESIGN.md)		document's own §6 leaves G1 (make Level 2 fully transient) and G2 (split by whole-pool TVL vs staked value) open, and notes Vault/Treasury STOA-normalisation is deferred — *"LP-only for now"*.
M3-DEB (M3-DEB-DESIGN.md)	the score model plus the deb-staleness subsystem	**SHIPPED, and the document says otherwise.** See below.
Anchor staleness (ANCHOR-STALENESS-INVENTORY.md)	a map, not a build	Its S2/H4 row was stale; a correction box now sits above it. See below.

Two design documents contradicted the code — one of them, and only one, really did

This is the only place where this chapter's verification disagreed with the audit tree. Both documents now carry correction boxes; one of those boxes corrects **this chapter**.

1. **ANCHOR-STALENESS-INVENTORY.md §Status** said H4's half 2 was *"NOT BUILT YET"* and warned that a linked boost-class locks its anchors **forever**. That had not been true since the sweep shipped. The README tracker records *"half-2 sweep BUILT + PROVEN ✅(phase 3 done)"* and the code carries the full path. Corrected in place on 2026-09-17: the box at :58-77 names both Talos doors and the live XE_UnbumpBoostClassScoreLinks at 01_ANK.pact:2184, and §Status:87 now defers to it. The original text is preserved below the box. [VERIFIED by reading]
2. **M3-DEB-DESIGN.md §"Build order & STATUS"** marks 2c / 2d / 2e as **"⏸ NOT BUILT"**, and this chapter reported that the code disagreed. **It does not.** M3-DEB-DESIGN.md:276-283 now carries a *"PARTIALLY CORRECTED 2026-09-17"* box that settles it: C_InjectChecked, CC_InjectChecked and InjectSweep all have **zero hits tree-wide**, so 2c and 2d are correctly marked NOT BUILT. What shipped of 2e is the **counter** — FVT|T|ForcedFixCount, declared in 04_RPS.pact:393/551/5614, with its key builder UCk_ForcedFixCount (05_FVT.pact:1444) and the zeroing at collect — not the forced-fix-count × RATE surcharge. [VERIFIED by command]

   CORRECTED 2026-09-18. This chapter cited CC_Inject (05_FVT.pact:2926) as evidence that 2c had shipped. CC_Inject is a different function from CC_InjectChecked — the *exact* near-miss the correction box names, and the same mistake the box records the first pass making against ANCHOR-STALENESS-INVENTORY.md. It also placed the FVT|T|ForcedFixCount table in 05_FVT.pact; the table is declared in 04_RPS.pact. **An identifier must be checked by exact name in the exact file, or a plausible neighbour will answer for it.**

So one document was behind the code and one was not, and the chapter's own check was the thing that needed correcting on the second. The surviving finding is still the failure mode this book's third rule is about — **a true record whose coverage stopped** — and in the anchor case a reader who trusted the document would have planned around a permanent operational lock that does not exist.

Genuinely open

what	why
Round III	Never run. Each Round II fix entry names its own missing regressions under *"Not yet covered (Round III regression)"*: the LP reserve-drift round trip asserting base returns to **exactly 0**; the ≥3-staker triplet with a mid-life unstake; the anchor-promile-change-between-stakes eventual-consistency test; and the full

what	why
	negative test for H4's own revoke lock.
LP Level-2 basis (G2)	Whole-pool TVL or staked value. A design question, not a correctness one.
The AQP shared-reader question (G-37…G-41)	**DECIDED 2026-09-17, and the decided remedy is not the one proposed.** Defaulting the shared UR_SCR|Score* readers changes nothing a caller sees — the caller never reaches those guards — while turning three *deliberate* pins in [6.5]_AQP-INFO.repl red. The caller-visible defect is real and sits at the **first raiser**, which was guessed wrong **three times in a row** from reading the call chain. *On an eager-let path, the first raiser is found by executing, not by reading.* Actionable, with the wrong remedy ruled out.
INFO_AQP-ANK|RevokeAnchor still quotes a revoke of an anchor that never existed	Guarding it is two lines and it **breaks [6.5]_AQP-INFO.repl**, a *deliberately fixture-free* cost-shape suite that passes arbitrary ids to all **83** AQP readers on the sound principle that AQP prices are argument-independent. Making AQP previews validate ids needs anchor, score and boost-class fixtures for every one of the 83 — real work with a real design question inside it.
04_RPS.pact:3983-3995, inside XI_1|SyncFarmGhostTvlForEmployedScores	The eager-fold-operand shape **inside an if**, so a settlement plan containing a BAR fvt-id would abort the **whole batch settle** instead of skipping that entry — the opposite of what the guard was written to do. **Latent only because the plan builder does not emit BAR today.** The source comment says so at the site and names its pin, REPL/modules/AQP.repl <<AQP-G31>> (:2122). (**Corrected 2026-09-18** — this cited 04_RPS.pact:3876 XE_XI_SettleScoreRps. That name has **zero hits in any .pact**; it exists only in prose, and :3876 is a phase comment.)
04_RPS.pact:3983-3995 and :3053	G-27 and G-24: the remaining eager-fold read (the row above) and a *partial* shadow in UEV_QualitySplitContext that survived the G-16 repair. (**Corrected 2026-09-18** — this cited :3919 and :3027, both inherited from DEFECT-LEDGER.md:1141/1144 and both stale: :3919 is inside XI_1|EnsureScoreRewardRows and :3027 is a comment inside the neighbouring UEV_AddRewardLinkContext.)
The M7 refutation's scope	ATSU precision artefacts are by design, but the fix entry notes the interaction: M1's dust sweep cannot recover **sub-precision** triplet dust, because the ATS ladder cannot move it. Economically negligible, structurally permanent.

One accepted property that a reader should understand as a property, not a bug

Stale-until-restake. A user's stored score is a snapshot taken at their last stake or unstake, and nothing recomputes it when the *anchor* side changes. That is deliberate — you cannot re-price N stakers cheaply — and ANCHOR-STALENESS-INVENTORY.md exists precisely so every place relying on it is known rather than assumed. H5's

fix explicitly **adopts** this property for the farm triplet in exchange for deleting an O(stakers) scan, and the trade was made with the owner's eyes open. The mitigations are the collect-time backstop, the self-service CC_UnstaleMyScores, and the enforced-fresh inject.

---

7. Verification result for this chapter

Every finding recorded as fixed was searched for in current source on 2026-09-17.

Nothing was found missing. All fixes are present, including the ones whose surrounding code was substantially rewritten afterwards — the two CRITICALs both survived the FVT→RPS split, which moved C2's entire inject path into a different module and a different file.

Five things a reader should weigh against that result:

1. **One of the audit's own design documents was stale in the direction of understating what shipped** (§6), and this chapter originally claimed two. Both now carry correction boxes; the second box corrects this chapter, not the document. The most concrete finding here is still documentary rather than technical — including the part of it that was wrong.
2. **The audit's proof standard was high and its coverage question was left open by design.** Round III was specified, its regressions were enumerated fix by fix, and it was not run. Everything in the *"Not yet covered"* fields of ROUND-02-FIXES.md is still not covered by anything this chapter found.
3. **The single most serious defect in AQP was introduced by this audit's own fix** and found a month later by a different method (§5.1). M1 was a real finding, its fix was a faithful port, and the two guards it dropped were dropped from a model the audit had explicitly adopted as ground truth. The audit's verification target — *both vaults drain to exactly 0* — was met, and was satisfied by the defect.
4. **The Z.repl numbers quoted throughout the tree are not the gate.** They range from 225 to 242 assertions; REPL/tools/_gate.py — which globs modules/*.repl, RedTeam/* and ZALL.repl — runs **25,035** (20,042 positive, 4,993 negative), as of the 2026-09-18 green gate. Where this chapter quotes a suite figure it is the audit's, from the time.
5. **04_RPS.pact is 304 KB and 05_FVT.pact is 225 KB.** On 2026-09-15 five source-rewriting tools fired on a bare import and **silently deleted 111 lines of schemas from 05_FVT.pact**; the tools now refuse without --apply (DEFECT-LEDGER.md §7.2i, CLAUDE.md). The file is intact today — 05_FVT.pact holds 14 defschema and 04_RPS.pact 19 [VERIFIED by command] — but *"the tree was committed"* is not a safety property, and this family is the one that proved it.

The practice that made this chapter possible is the same one Chapter 8 recommends, applied less consistently here: **fixes that left a finding id in a source comment could be found after the module was split in two and renumbered.** Counted 2026-09-17: 01_ANK.pact 9, 02_SCORE.pact 9, 05_FVT.pact 7, 04_RPS.pact 3, 07_MTX-AQP.pact 1, 03_AQP.pact 1 — and **06_VCT.pact 0**, in the module that carried three of the six vacate findings. Every VCT fix was verified here by reading its behaviour rather than by finding its marker, which took an order of magnitude longer and is the only reason this chapter can say *"not verified"* about the one VCT question it could not settle (§5.5).

End of Part I. Part II documents the main-work round — the systematic construction of the preview and pricing surface across every module in the tree.

---

11. Part II — The main-work round

Roadmap Phases 1.1–1.5. Source records: POST-AUDIT-MAIN-ROADMAP.md, IGNIS-PRICING/, ARCHITECTURE/REPL_TEST_ARCHITECTURE.md, Audit/records/REPL-ROUND-REPORT.md, Audit/records/DEFECT-LEDGER.md, and 370 commits between 2026-08-30 and 2026-09-14.

What this round was

Part I audited modules that already existed. Part III attacked a system that was already finished. This Part covers the work in between: building, across the whole codebase, **one cost reader per priced operation and one free preview per client operation**, re-pricing everything those readers return, splitting the one module that had grown past the point where it could be deployed, and rebuilding the test suite around the result.

The roadmap describes its own value accurately, and this Part does not inflate it:

> *"mostly the systematic construction of the preview/pricing surface, and its value is in completeness rather than in individual findings."*

So the chapters demonstrate completeness with counts, and they state the exclusions those counts carry. Where something did go wrong — and several things did, including a repricing script that duplicated 1.78 million lines of contract source and a pricing headline that is **still** eleven operations short — the chapter says so with the file and the number.

The round, in numbers

[VERIFIED by command] — git diff --shortstat 0ab7639 b8284a5, the first commit of the phased roadmap board (2026-08-30) to the round's own closing checkpoint (2026-09-14).

commits	**370**
files changed	**500**
lines added / removed	**+154,877 / −42,312**

Per area, from git diff --numstat over the same range:

area	files	+	−
REPL/	290	74,454	6,177
1_SOVEREIGN/	90	45,809	29,163
OuronetInformational/	69	25,180	758
2_CITIZEN/	30	7,506	5,890

And what those deltas did to the two things a reader cares about — **[VERIFIED by command]**, git archive <rev> | wc -l at each endpoint:

	2026-08-30	2026-09-14	today (HEAD)
Pact source files	91	93	**109**
Pact source lines	96,310	114,156	**132,756**
.repl files (excl. archive/)	181	209	**222**
.repl lines	68,435	120,603	**148,965**
distinct assertions	**1,604**	**5,262**	**6,619**

2026-08-30	2026-09-14	today (HEAD)

written

> **These are a SNAPSHOT, and the book says so rather than implying permanence.** The distinct count moved four times on 2026-09-17 alone as witnesses were added; it was **5,555** when the figure-sync tool's own source of truth was found stale, **5,830** when that loop was closed, and **6,619**, measured when this book was built. The canonical value is whatever ARCHITECTURE/REPL_SUITE_STATS.md holds, which _figuresync.py --check now verifies **against the tree** rather than against itself — see DEFECT-LEDGER §8.22.

The test suite grew faster than the contracts it tests, by a factor of about four. That is the shape of the round.

What each phase produced, measured today

Every figure below is re-derived from the working tree, not quoted from a status document. The command is named in the chapter that owns it.

phase	what it built	measured in the tree
1.1 URCi_ cost architecture	one pure cost reader per priced operation	**322** implementations across **40** modules; **291** declared on an interface, **31** in-module-only and **none of those 31** called cross-module
1.2 INFO_ preview rehaul	one free preview per client operation	**426** implementations across **10** modules: **346** wrap a URCi_, **59** delegate to a sibling preview, **20** declare the op free, **1** is a data view
1.3 IGNIS re-pricing	the whole cost model moved into four constant maps	IG|DETER **54** keys · IG|COMPONENTS **396** · IG|WEIGHTS **14** · IG|LEGS **22**; the generated price sheet carries **442** priced rows + **5** declared unpriced
1.4 module splits	04_FVT.pact cut below the deploy ceiling	04_RPS.pact **5,621** lines + 05_FVT.pact **3,977**; **0 of 93** modules over the ~6,635-line cliff, **1 in the project's own "Danger" band**
1.5 REPL finalisation	a one-command gate over the whole suite	**94** gate entrypoints, **14** fatal static checks, **6,619** distinct assertions, orphaned asserting files **0**

The Part II findings register, and where each one ended

The owner's directive for this book (Audit/BOOK-DIRECTIVE.md, 2026-08-27) specifies that Part II carry a findings register: twelve numbered defects, six from the coverage-completion pass and six from the architectural-conformance pass. **Four of the twelve were absent from this book entirely** until 2026-09-18, when it was checked against the directive rather than against itself. All four turned out to be closed — so the gap was in the record, not the code, which is the easier kind to miss precisely because nothing is broken.

The full register, with each finding's status re-derived from the tree today:

#	where	defect	status now
1	05_STOAICO.pact	unguarded zero-amount urSTOA mint **deadlocks the distribution vault from round 2**	**FIXED** — the mint moved inside the (!= urSTOA-supply 0.0) guard
2	11_VST.pact	C_Merge/C_Slumber did not validate the dpof against its vzh-tag; the wrong-kind merge reported success and made the position **permanently unrecoverable**	**FIXED** — (enforce (= (take 2 dpof) "Z|") "Merge requires a Sleeping DPOF")
3	5 sites	(format "literal") with no argument list — arity error; INFO_ATS|Cull broken on every call	**FIXED** — zero such sites remain
4	12_LIQUID.pact, 01_DALOS.pact	migration guard message said “offline” while enforce gap requires the pause **ON**	**FIXED** 2026-09-14 (wording only; the logic was always right)
5	16_SWPI.pact	slippage bound measured in **fee-less** tokens; a breach returned a string instead of reverting	**FIXED** — and re-attacked in the red team as RT-H-002 (Chapter 21)
6	DEMIPAD|C>WITHDRAW	provably dead enforce — URv_Funds enforces the identical predicate first	**ACCEPTED, deliberately.** Kept as defence-in-depth and documented in source: removing it would change an @event signature indexers consume, and the capability should stay correct on its own terms for a future caller that does not read first
7	[2.1]_Dalos.repl	the **master keyset registered as a DALOS inter-module policy**, so P|UEV_IMC passed for any holder of it — an undocumented, unbilled exception to “Talos is the only supported client path”	**FIXED** — the registration is commented out; the mechanism is untouched
8	29 sites	state-dependent enforce in unprotected UR_/URC_/XI_ — validation living outside the defcap	**FIXED** — _conformance.py reports **0** across UR-no-enforce, UC-no-read and UR-no-write
9	02_DPDC.pact	XE_DeployAccountWNE had	**FIXED** — the defun now

#	where	defect	status now
		no P|UEV_IMC, unlike its immediate neighbour	opens on (P|UEV_IMC)
10	99_TS02-CPAD.pact ×4	calls TS01-A::XB_DynamicFuelSTOA, a protected X* on a sovereign module	**OWNER RULING** — CPAD is admin-authored and deliberately between the two roles. Accepted **and bounded**: four such calls are expected, a fifth is not
11	14 ops in 4 families	single C_/A_ prefix on ops reaching a heavy scan — the prefix promises bounded gas, the tree did not deliver it	**FIXED** — renamed to CC_/AA_, 126 replacements across 30 files, since the names are also IGNIS price-table keys
12	05_DPTF.pact	UEV_ReservationState's two messages **inverted** — each reported the opposite of the condition that produced it	**FIXED** — swapped, with the reasoning left in source

Three observations the register supports better than any single finding does.

Ten of the twelve were reachable without a red team. The directive says so itself, and it is the sharpest methodological result in this book: *"whatever the adversarial phase costs, it should not be spent re-finding what a first invocation would have."* Findings 1–6 came from nothing more adversarial than invoking every client entrypoint once and asserting on the observable outcome. Findings 7–12 came from comparing the code against the project's own **stated** design rules — which is why several of them are guards that do not exist at all, rather than guards that are wrong. An enforce that was never written cannot be found by reading enforce statements.

Three of the twelve are message defects, not logic defects (#4, #12, and #6's near-miss). In each case the guard was correct and the text stated the opposite of the condition that fired it. A system that pins refusals **by message** — which this one does, 1,296 times — cannot treat that as cosmetic: a wrong message makes a correct guard indistinguishable from a different guard, which is the whole problem Chapter 18 is about.

Two were closed by a ruling rather than a fix, and both rulings are *bounded*. #10 accepts four specific calls and states that a fifth is not accepted; #6 keeps a dead enforce and says exactly what would have to change for it to matter. An unbounded acceptance is indistinguishable from an unexamined one a year later.

Three published figures that the tree did not support

Found while writing this Part. Each was a live discrepancy when written, each was verifiable with one command, and each was a different failure mode.

> **ALL THREE ARE NOW CLOSED.** *(Verified 2026-09-18.)* They are kept in full, as they were written, because the point of this section is the three failure modes and not the three numbers — and because a finding deleted once it is fixed leaves a book that cannot show its own working. Each carries a dated closure note below.
>
> That they were all fixed within days is worth one caution, though. These three were found by re-deriving published figures from the tree. A staleness sweep of this book's own chapters on 2026-09-18 found **124** stale figures against ~516 that still held — in a book whose third rule is about counts. Finding three in someone else's documents is easy; the discipline is turning the same instrument on

your own, which is why the volatile figures in this book are now measured at build time rather than typed. Chapter 27 says how.

1. The roadmap's own correction notice is itself wrong. POST-AUDIT-MAIN-ROADMAP.md carries a 2026-09-17 banner warning that its dashboard is stale, and supplies replacement figures: *"267 distinct URCi_ readers"* and *"335 distinct INFO_ previews"*. The first is a count of distinct **names** (268 today) and undercounts implementations by 54, because names legitimately repeat across modules. The second matches nothing: the tree holds **426** implementations and **425** distinct names. *A correction notice is read with more trust than the thing it corrects.* Chapter 12. > > **CLOSED 2026-09-18.** The banner now reads *"256 distinct URCi_ readers (307 implementations)"* > and *"420 distinct INFO_ previews"*, scoped to 1_SOVEREIGN/ as it always claimed to be. The > maxim in italics above went on to apply to this book: Chapter 12 §2 published a correction > that was itself wrong and reverted a correct figure, and it was believed precisely because > corrections are.

2. The designated authoritative pricing reference quotes a superseded census. IGNIS-PRICING/IGNIS-PRICING.md §5 states *"345 of 365 INFO implementations are thin wrappers over a URCi_ reader; 14 declare their op free, 4 are data views."* Those figures were exactly right on 2026-09-06, when commit 6b7a85b measured them and said so. The tree today is **346 / 426 / 20 / 1**, and the document's own three sub-counts sum to 363, not 365. Chapter 12. > > **CLOSED 2026-09-17.** §5 was re-measured and now publishes 450 declared / > 441 client-facing / 428 measured, quoting the 345/365/14/4 > census explicitly as superseded rather than replacing it silently.

3. The generated price sheet publishes a total that omits eleven of its own rows. The footer reads *"431 Talos client functions"*. The sheet contains **442** priced rows. The generator computes the total as nsimple + ncomplex + nexempt and omits nstoaonly — so every operation priced in STOA and not in IGNIS (branding upgrades, the PYTHIA tolls, the CODEX StoicTag family) is counted in its own column and then dropped from the headline. This was recorded as an internal inconsistency in DEFECT-LEDGER.md §5 item 14 on 2026-09-15, at the then-current values of 420 and 430. It was not fixed; the numbers had since grown to 431 and 442. **And a control added the same week required the prose document to quote the wrong total, and failed the gate if it did not.** Chapter 13. > > **CLOSED 2026-09-17.** _ignis_price_sheet.py now sums nsimple + ncomplex + nstoaonly + nexempt; > the sheet's footer reads **442**, IGNIS-PRICING.md quotes 442, and the literal 431 no longer > appears anywhere. The fix was one identifier. What made it a finding worth a section was never the > eleven rows — it was that a **generated** artefact had been wrong for weeks while a gate check > enforced agreement with it, so the control was actively holding the error in place. A check that > enforces consistency between two things says nothing about whether either is right.

None of the three was a defect in the chain. All three were defects in what the project publishes about itself, which is what an audit book is made of.

Chapters

01-URCI-INFO.md	The cost-reader and preview surfaces: what they are, how complete they are, and what the completeness claim excludes
02-PRICING.md	The cost model, the decisions, the generator, and the gate mechanism that makes the generated artefacts re-derivable
03-SPLITS.md	The deploy ceiling, the FVT split, what the split cost, and what it did not finish
04-REPL.md	The test architecture: six gates, 92 entrypoints, and where the verification has holes

How to re-derive anything in this Part

The book's reproduction appendix (APPENDIX/01-REPRODUCTION.md) covers the gate and the two money defects. The counts specific to this Part come from the following, all read-only and all runnable from the repository root:

```
# URCi_ / INFO_ census, separating interface declarations from implementations
python3 - <<'PY'
import re, glob, os
PFX = re.compile(r'^(?:[A-Za-z0-9|_-]+\|)?([A-Z]+[a-z]?)_')
DEF = re.compile(r'\(defun\s+([A-Za-z0-9|_>-]+)')
impl = {}; decl = {}
for f in sorted(glob.glob("1_SOVEREIGN/**/*.pact", recursive=True)
              + glob.glob("2_CITIZEN/**/*.pact", recursive=True)):
   if "/Audit/" in f: continue
   src = open(f).read()
   mi = [m.start() for m in re.finditer(r'^\(module\s', src, re.M)]
   start = mi[0] if mi else None
   for m in DEF.finditer(src):
      p = PFX.match(m.group(1))
      k = p.group(1) if p else None
      if k not in ('URCi', 'INFO'): continue
      (impl if start is not None and m.start() >= start else decl
       ).setdefault(k, set()).add((f, m.group(1)))
for k in ('URCi', 'INFO'):
   print(k, "implementations:", len(impl[k]), " interface declarations:", len(decl[k]))
PY

# price-sheet categories, counted from the artefact's own row markers
f=OuronetInformational/IGNIS-PRICING/IGNIS-PRICE-SHEET.md
grep -c 'admin/exempt' $f; grep -c 'empty cumulator' $f; grep -c 'collects none of them' $f
grep -c 'collects no IGNIS and no STOA' $f; grep -c '| STOA only |' $f
grep -c '| COMPLEX |' $f; grep -cE '\| \$[0-9]' $f

# module size bands, against MODULE-SIZING.md section 1
find 1_SOVEREIGN 2_CITIZEN -name '*.pact' -not -path '*/Audit/*' -print0 | xargs -0 wc -l \
  | sort -rn | head -10

# gate entrypoint arithmetic, without running the gate
cd REPL && echo $(( 7 + $(ls deb-staleness-*.repl | wc -l) + $(ls modules/*.repl | wc -l) \
  + $(ls RedTeam/*.repl | grep -vc '/_') + 27 + 6 ))
```

Do not run REPL/tools/_suite_stats.py or REPL/tools/_toolindex.py to inspect them. Both rewrite a tracked artefact on a bare invocation, with no --apply flag. They are generators and regenerating is the intended action, but the repository's own rule about tools that write on import applies to the habit, not only to the five tools that caused the incident behind it.

---

12. The cost-reader and preview surfaces

Ouronet charges for almost everything a client does, in a virtual gas called IGNIS and sometimes in native STOA. Before this round, an operation's price was computed **twice**: once inside the execution path, and once again inside whatever free preview the UI called to quote it. Two derivations of one number, edited by different hands at different times.

This chapter is about replacing that with one derivation, and about what the replacement exposed.

---

1. The two prefixes

URCi_ — a pure, read-only cost reader. No enforce, no writes. It returns the cost of one operation, and it is called from **two** places: the execution path, to bill; and the preview, to quote. Registered in the project's prefix canon on 2026-08-30 (commit 02c9b79) with its own colour family.

INFO_ — the free client-facing preview. It returns a ClientInfo object carrying ignis-need, stoa-need and human-readable text, and in the intended shape its body is a thin wrapper over the operation's URCi_.

The design decision is dated and recorded. **Option A, 2026-08-27**: leaves expose URCi_, the executor composes as before, and the preview concatenates the same leaf readers. Option B — billing *through* the composer — was rejected as a top-to-bottom rehaul. One rule came with it and is worth quoting because Chapter 14 turns on it:

> A URCi_ lives in the **same module as the function it prices**, never in a shared cost module. If a module gets too big, split the module — do not exile its cost functions.

There is one architectural exception, and the project named it itself: **DALOS deploys below IGNIS**, so DALOS's nine client-op cost readers live *in* IGNIS under the scoped name DALOS|URCi_*. **[VERIFIED by command]** — those nine are the only scoped URCi_ names in the tree, and they are why a naive grep '(defun URCi_' undercounts the surface by nine.

---

2. The census

[VERIFIED by command] — the parser in PART-II/README.md, which separates interface declarations (before the (module ...) form, in this codebase's co-located interface files) from implementations (after it). This is the same rule REPL/tools/_scale_report.py uses, so the numbers below agree with the project's own generated statistics.

URCi_ — 322 implementations

implementations, across **40** modules	**322**
distinct names	269
declared on an interface	**291**

> **THE CORRECTION THAT WAS ITSELF THE ERROR.** *(Retracted 2026-09-18.)*
>
> This chapter published the figures above, then on 2026-09-17 published a **correction** replacing them with 313 / 260 / 282 across 39 modules. **The correction was wrong and the original figures were right.** They are restored above, and the episode is kept rather than quietly reversed, because of what made it convincing.
>
> The correction rested on one sentence: *"**595** URCi_ defuns exist in total, partitioning exactly into 282 declarations and 313 implementations."* **Exact partition is a strong signal** — it is the reason the correction was believed, and it is why the correction notice was trusted more than the thing it corrected. 282 + 313 = 595, and a total that reconciles looks checked.

> But 595 is exactly what grep -c '(defun URCi_' returns, and that pattern **misses the 18 scoped ENTITY|URCi_* defuns** — nine implementations and nine declarations. The tree holds **613**, which partitions just as exactly into **291** + **322**. Both partitions reconcile; only one of them is over the whole population.
>
> The sharpest detail is that this chapter had already warned about the scoped names **eleven lines above the correction** — DALOS's nine cost readers live in IGNIS as DALOS|URCi_* precisely *"because a naive grep '(defun URCi_' undercounts the surface by nine"*. The correction then used the naive grep. Knowing about an exception in one paragraph does not stop you applying a rule that ignores it in the next, and nothing in the process connected the two.
>
> Re-derive it:

```
grep -rhc '(defun URCi_' --include=*.pact 1_SOVEREIGN 2_CITIZEN | paste -sd+ | bc     # 595
grep -rhoE '\(defun [A-Za-z0-9|_-]*URCi_' --include=*.pact 1_SOVEREIGN 2_CITIZEN | wc -l  # 613
grep -rhoE '\(defun [A-Za-z0-9|_-]+\|URCi_' --include=*.pact 1_SOVEREIGN 2_CITIZEN | wc -l # 18
```

> Independently corroborated by _scale_report.py, which reports URCi_ 151 / 322.
>
> Two things follow, and the second is the uncomfortable one. **A reconciling total is not evidence the population was right** — it only proves the two halves were cut from the same cloth, however wrong that cloth is. And **a correction notice is read with more trust than the text it corrects**, so it needs *more* scrutiny than the original, not less. This book makes that claim elsewhere; here is the instance where the claim is about itself.

Attribution method, so the figures can be re-derived: comments stripped; for each URCi_-prefixed defun — scoped or not — compare the nearest preceding ^(interface against the nearest preceding ^(module and attribute it to whichever is closer. One file in the tree declares more than one module, which a simpler "everything after the first (module" rule mis-attributes.

The gap between 322 and 269 is real and correct: URCi_UpgradeBranding is defined independently in DPTF, DPOF, ATS, SWP, BRD, DPDC and others, because each module prices its own branding upgrade. Counting names rather than implementations is the mistake the roadmap's own correction notice makes.

The 291/31 split is the answer to an open roadmap decision. Item **1.1.1.1** — *"interface-richness policy: rich (declare cross-module functions) vs marker-lean"* — is still an unticked ⚠️ open decision on the board. The tree settled it: **[VERIFIED by command]** every one of the 31 in-module-only readers has **zero** cross-module :: call sites, so the policy applied is "declare what crosses a module boundary, keep the rest local", exactly as the walk's own instruction (1.1.2.1) specified. The unticked box is a bookkeeping gap, not an unmade decision.

The 31 are concentrated where you would expect a composer to be local: VCT's eight batch vacate/drain readers, RPS's six reward-flow readers, SWPU's four swap-core readers, VST's four special-link legs, DSA's three royalty composers, and MTX-SWP's two liquidity legs.

Distribution across the tree, largest first — **[VERIFIED by command]**:

module	URCi_	module	URCi_
11_VST.pact	26	02_SCORE.pact	13
05_DPTF.pact	25	08_DSA.pact	12
08_ATS.pact	24	05_DPDC-R.pact	11
06_DPOF.pact	20	08_DPDC-S.pact	10
04_RPS.pact	20	02_IGNIS.pact	9 *(the DALOS nine)*
09_TFT.pact	17	06_DPDC-MNG.pact	9
18_SWPLC.pact	15	03_AQP.pact	9
10_ATSU.pact	14	…23 further modules	1–8 each

module	URCi_	module	URCi_
15_SWP.pact	13	**citizen launchpad (5 modules)**	1–2 each

INFO_ — 426 implementations, in ten files

[VERIFIED by command]. The consolidation target of roadmap 1.2.2.1 — *“one INFO module per stage, relocated to Z_Reads/, deployed last”* — is **substantially but not entirely** met:

file	INFO_	
1_SOVEREIGN/STAGE_01/Z_Reads/ 02_INFO-ONE+.pact	180	the Stage-1 preview module
1_SOVEREIGN/STAGE_02/Z_Reads/ 01_INFO-TWO.pact	149	the Stage-2 preview module
1_SOVEREIGN/STAGE_02/2_Core/ 03_AQP/09_AQP-INFO.pact	**83**	**still in 2_Core/, not Z_Reads/**
21_CODEX.pact, 22_PYTHIA.pact	4 + 4	previews kept beside their own module
5 citizen launchpad sale modules	6 total	citizen previews, one per sale

01_INFO-ZERO.pact survives in STAGE_01/Z_Reads/ as an explicitly documented **tombstone**: an empty module retained as a deploy-slot placeholder, with a header recording where each thing that used to live in it went. **[VERIFIED by reading]** It defines no functions and implements no interfaces. That is a defensible choice for a module that may already hold a namespace name on chain, and the reason it is worth mentioning at all is that the alternative — deleting it — would have been invisible in every count in this book.

What is inside an INFO_ body

A complete, exhaustive partition of all 426 — **[VERIFIED by command]**, a script that splits each module body at top-level (defun boundaries and classifies the remainder of each block:

shape	n	
references a URCi_ / URCix_ reader	**346**	the intended shape
delegates to a sibling INFO_	**59**	the INFO_DPNF|* / INFO_DPSF|* wrappers over a shared son-discriminated INFO_DPDC-* body
declares the operation free	**20**	OI|UDC_NoIgnisCosts / NoStoaCosts / UC_EmptyOc — the gas-station-subsidised hydra slices, ORBR, the DSA oracle toggles
a data view, not a cost preview	**1**	returns a HibernatedNoncesView, no cost fields
total	**426**	

The 59 delegators are not a gap: INFO_DPNF|Issue is literally (INFO_DPDC-I|Issue patron owner-account collection-name false). They reach a URCi_ one hop further down. Counting them as unwrapped is a mistake the project made once and corrected — _info_measured.py carries the note, and the reason it matters is stated well there: *“a coverage report that cries wolf nine times is a coverage report nobody reads to the end.”*

The published figures this contradicts

source	says	tree says
POST-AUDIT-MAIN-ROADMAP.md dashboard banner (*as of 2026-09-17*)	267 URCi_, 335 INFO_	**322** and **426** implementations; 269 and 425 distinct names. *(The banner was wrong in **both directions at once** — low on INFO_ by 85, high on URCi_ — which is how it survived: a reader spot-checking one of the two would find it plausible. It has since been corrected to the sovereign-only scope it states, and now reads 256 / 420. This chapter's own second pass was wrong too; see the retraction in §2.)*
IGNIS-PRICING.md §5 (*as of 2026-09-06*)	*"345 of 365 INFO implementations … 14 free … 4 data views"*	**346 / 426 / 20 / 1.** *(§5 was re-measured on 2026-09-17 and now publishes 450 declared / 441 client-facing / 428 measured, quoting the old census as superseded. The row is kept because the discrepancy is what prompted the re-measure.)*

The second is worth being fair about. **[VERIFIED by command]** — git show 6b7a85b (2026-09-06) is the commit that produced 345/365/14/4, and it produced them by **auditing every INFO function against the URCi-wrapper rule** and fixing the two that failed it. The figures were correct and hard-won. They then sat in the document while the DPNF/DPSF preview family was built out, and nothing re-derived them. *This is the same failure the pricing artefacts were given a gate check for in Chapter 13 — and §5 is outside that check's scope.*

There is a third figure in the same paragraph worth flagging as **[INFERRED]**: §5 also says *"395 of 401 Talos client ops have an INFO preview."* But 401 is exactly the number _info_measured.py reports for **client-facing cost previews**, and the Talos client surface is **448 distinct entrypoints** (§5 of Chapter 15, independently reproduced). A preview count appears to be wearing an operation count's label. Not asserted as an error — but a reader should not take "401 Talos client ops" as the size of the client surface, because it is not.

---

3. How the surface was built

[VERIFIED by command] — git log --oneline --since=2026-08-30 --until=2026-09-03. The walk went in deploy order, one module per commit, and each commit's message states what was proven:

```
2026-08-30  DALOS-via-IGNIS (9) · DPTF (23) · DPOF (19) · ATS (22)
        TFT (14, a cross-module rename of an existing cumulator family)
        ATSU, SWP, then the Stage-2 DPDC family
2026-08-31  the composition tier: ATSU (11 composers), VST (29), LIQUID, OUROBOROS,
        SWPI, SWPL, SWPLC, SWPU, MTX-SWP, DEMIPAD, EQUITY
        then AQP vacate/drain/FullVacate — the "final 17"
2026-09-01  the INFO rehaul, entity by entity: DPTF, DPOF, VST, ATS (42/42),
        SWP (37/37), LIQUID, ORBR, DEMIPAD, EQUITY, the DPDC family, AQP
```

Two facts about the trajectory are worth recording because they explain a discontinuity anyone re-deriving these counts from git will hit. **[VERIFIED by command]**, counting (defun …URCi_ and (defun …INFO_ at each commit:

commit	date	URCi_ sites	INFO_ sites
0ab7639	2026-08-30	0	143

commit	date	URCi_ sites	INFO_ sites
a1eb868	2026-08-30	198	143
7ba53ac	2026-08-31	418	143
aea2e19	2026-09-01	455	**40**
1f714ec	2026-09-06	580	**623**
HEAD	2026-09-17	613	623

The INFO_ count collapses to 40 and then jumps to 623 because the rehaul first renamed previews into a URC_<scope>| interim form and the StoicSyntax canon sweep then renamed them back under a unified INFO_<scope>| scheme (commits 1d31d5a, 1d02de7, 2026-09-01). **No previews were deleted.** These raw counts include interface declarations, which is why they exceed the implementation census in §2; they are given only to show the shape of the work.

---

4. What the surface was for, and how it is checked

The point of one reader with two callers is that the preview and the charge cannot disagree. The project stated the specification as an owner rule:

> *"The INFO_ function must output the exact same cost as the real execution function."*

Three candidate tests for that rule were considered and two were rejected — this is the most transferable thing in the round, and it is recorded in DEFECT-LEDGER.md §1.1a:

candidate	verdict
INFO_X.ignis-need == <hardcoded expected>	**rejected — already failed silently.** Four AQP anchor costs were pinned at 1000.0 while the real price moved to a computed 574.0, and nothing went red
INFO_X.ignis-need == URCi_X	**rejected — tautological.** The preview is literally built from that reader
capture the patron's balance, run the real op through Talos, difference it, compare to the quote	**adopted** — the only one with an independent oracle

Everything in §5 was found by the third.

---

5. What it found

DEFECT-LEDGER.md §1.1a lists **17 preview-versus-charge entries covering the project's running count of 19 defects**; §1.1b lists **13 further mispriced or unbilled operations**. The full register is there and this book does not reproduce it. What belongs here is the **shape**, because the shapes recur and the counts do not.

Three failure modes, and they are not the same problem

(a) A literal zero for a currency the operation really charges. The preview returned OI|UDC_NoStoaCosts — rendered to a client as *"Operation is free of native Stoa"* — for operations charging 918, 500, 432.5 and 76.5 STOA respectively (INFO_EQUITY|IssueCompany, INFO_ATS|ToggleParameterLock, INFO_SWP|ToggleFeeLock, and all five INFO_VST|CreateSpecial*Link). Nine previews, one shape. The remedy was not nine fixes but a **detector**: REPL/tools/_infostoa.py reports any preview claiming STOA-free whose execution tree reaches STOA|C_Collect. It reports 0 today, and it was mutation-tested against the known defect before that 0 was trusted.

(b) A missing leg in a multi-part cost. The sharpest instance is P-05: four AQP inject previews quoted URCi_Inject — the gas leg — while phase 1 of XI_FvtInjectCore is a custody transfer carrying its own cumulator. Measured

276.13 quoted against 276.66 charged. A 0.53 discrepancy is not a rounding artefact; it is a whole leg, and the size of the gap is a property of the fixture, not of the defect. The same is true of P-07, where two citizen sale previews missed a royalty payment made *out of the patron* before the sale's own collect: measured 89.002 against 89.004, *"0.1% here only because this fixture's royalty is small."*

(c) The wrong source constant. P-06: two issuance paths price a liquidity pool differently — the single-transaction path at 500 STOA, the defpact path at 600 — and all six previews quoted the second, so the three single-transaction previews over-quoted by 100 STOA each.

The one that was wrong in both directions

P-10 is the entry to read if you read only one. Seven VST special-link previews added a vst-link deterrence of 279 raw IGNIS that the execution path **never charged**, while simultaneously modelling a transfer-role toggle as a hand-made 4.0 leg against the execution's real 59.0. Quoted 719.74, charged 601.02.

The preview was wrong. The execution was **also** wrong — it had been under-collecting a designed-in fee on *every special link ever created*. That is a revenue defect, not a quoting defect, and it is why the entry was escalated to an owner ruling rather than repaired on sight: there was no way to tell from the code which of the two numbers was the intended one.

The invariant, and its documented escape hatch

IGNIS-PRICING.md §5 states the guarantee as *"it is called inside the execution path for billing AND served to the UI for preview, so the two cannot drift."* The project's own defect ledger (§5 item 18) records that this is **not architecturally true**, and names the exception:

> **SWPLC is *variant-B*** — the execution path builds its cumulator **inline**, and the URCi_ reader is a **parallel reconstruction**. Both sides must always be edited together.

That escape hatch, plus an INFO_ layer free to hard-code NoStoaCosts, **produced six of the seventeen §1.1a defects.** The ledger's proposed wording is the honest one and this book adopts it: *the two cannot drift **where the execution path calls the reader***.

And a shape that survived a green pipeline

B-01, from the pricing walk rather than from measurement, because it is the clearest statement of what a test suite does not do. DPDC-MNG::URCi_Control and URCi_WipeNonce were already branchy — (if son (BigCumulator) (BiggestCumulator)) — and the migration script replaced only the **first** tier call it found. So son=true got the new price and son=false silently kept the old one.

> *"ZALL stayed green throughout: the code is syntactically valid and merely charges the wrong number, which no existing test asserted."*

The remedy was a detector with one rule — *flag any URCi_* whose body contains **both** UC_IgnisPrice and a UDC_*Cumulator*, meaning it is half-rewritten — and a dedicated suite, REPL/Stage_02/[6.1.9]_PRICE-SWEEP.repl, which exists specifically for this defect and asserts every son-taking reader **twice**, once per fungibility side. **[VERIFIED by command]** it carries 64 assertions.

---

6. Coverage of the measurement — and what its denominator excluded

> **CLOSED 2026-09-17.** Both halves of this finding are repaired, and the section is kept as written because the *shape* of the error is the reusable part.

The instrument no longer enumerates its subject from a hardcoded three-file list: it discovers it, globbing 1_SOVEREIGN/** and 2_CITIZEN/** for every .pact that defines a ClientInfo-returning preview. The published figure moved from **401 of 401 with 0 gaps** to **428 of 441 with 1 gaps** (450 declared) — and note that the *old* figure also read "0 gaps". A tool that defines its population to exclude the gaps will always report full coverage, and will report it in exactly the same words as a tool that has genuinely closed them.

The one preview the old denominator hid, STOAICO::INFO_Collect, is now measured. It became red-team attack **RT-K-008** — found not by suspicion about that function but by counting the population and subtracting, which is the only method that can find the case nobody suspected. Chapter 21 has it.

This is the section that matters, because the project's headline claim for this phase is a coverage claim.

REPL/tools/_info_measured.py answers *"how many previews have been measured against a real charge?"* Its proxy is stated in its own docstring and is deliberately generous in one direction: a preview counts as measured when its name appears, uncommented, inside a begin-tx block that also extracts "ignis-need" or "stoa-need". **[VERIFIED by command]**, run today:

```
cost previews declared : 410   (ClientInfo-returning only)
  INFO-internal helpers:   9   (called by another preview; exercised transitively)
  CLIENT-FACING       : 401   <-- the denominator the owner's rule is about
MEASURED (cost proof)  : 401
named but NOT measured :   0
client-facing, NEVER named: 0
```

401 of 401. That is a real result and the instrument is unusually well-documented: it states its proxy, states which direction the proxy is generous in, and tells the reader to treat the number as an upper bound.

But 401 is not the number of cost previews in the tree, and the tool cannot say so.

[VERIFIED by reading] _info_measured.py enumerates its subject as a hardcoded three-file list:

```
PREVIEW_SOURCES = [
   "../1_SOVEREIGN/STAGE_01/Z_Reads/02_INFO-ONE+.pact",
   "../1_SOVEREIGN/STAGE_02/Z_Reads/01_INFO-TWO.pact",
   "../1_SOVEREIGN/STAGE_02/2_Core/03_AQP/09_AQP-INFO.pact",
]
```

Those three hold 412 of the tree's 426 INFO_ implementations. The other **14** live in CODEX (4), PYTHIA (4) and the five citizen launchpad sale modules (6) — and **[VERIFIED by command]** every one of the 14 returns object{OuronetInfoV2.ClientInfo}. They are cost previews by the tool's own definition, and they are outside its denominator by construction.

So I applied the tool's own rule to the 14 by hand. **[VERIFIED by command]** — the same begin-tx / ignis-need proxy, over the same non-archived .repl corpus:

preview	named in a gated .repl	measured
the four INFO_CODEX|*	yes	**yes**
the four INFO_PYTHIA|*	yes	**yes**
INFO_BuySparks, INFO_RedeemSparks (SPARK)	yes	**yes**
INFO_BuyStoicPay (STOICPAY)	yes	**yes**
INFO_Acquire (SNAKES **and** CUSTODIANS, both measured in launchpad-groundtruth.repl)	yes	**yes**
INFO_Collect — 2_CITIZEN/7_Launchpad/5_StoicIco/05_STOAICO.pact:510	**no**	**no**

Thirteen of the fourteen are measured anyway. The result is therefore not that the project overstated its coverage by much — it is that:

> **The honest figure is 414 of 415, not 401 of 401, and the single hole is in the one place the instrument is structurally unable to look.**

STOAICO::INFO_Collect appears in no .repl file in the suite, gated or otherwise. That is one unmeasured cost preview on a live citizen sale, and it will stay unmeasured for as long as the instrument's subject is a list rather than a search — because *a hardcoded list cannot report its own incompleteness.* Chapter 22 records the same failure mode in _toolpaths.py and the remedy that was applied there: make the checker **discover** its subject and report any directory its list does not cover. The same remedy fits here, and has not been applied.

The instrument is also not in the gate

[VERIFIED by reading] REPL/tools/_gate.py runs twelve static checks as fatal pre-checks. Every tool in REPL/tools/ that has a --check mode is one of them. **_info_measured.py has no --check and no --selftest** — the gate byte-compiles it and moves on.

That is not an oversight in the same class as a missing check; there is nothing to check against, because the tool prints a census rather than asserting a threshold. But it does mean the project's own stated rule, written in _gate.py's own comments, is not satisfied for its Phase-1.2 headline:

> *"A number quoted in an audit document as evidence of a repair must be one the gate RE-DERIVES on every run. Otherwise it is a claim about the past, and the repair it certifies can be undone without anything going red."*

401/401 is a claim about 2026-09-15. Deleting every ignis-need assertion from launchpad-groundtruth.repl tomorrow would turn nothing red in the gate. Chapter 15 returns to this, because the same is true of five other coverage instruments.

---

13. Re-pricing IGNIS

> Source of record: OuronetInformational/IGNIS-PRICING/IGNIS-PRICING.md — the project's own single authoritative reference, which folded in and deleted five earlier pricing documents. IGNIS-PRICE-SHEET.md and IGNIS-DETER-WORKSHEET.md in the same folder are **generated** and **gate-enforced**; §5 of this chapter explains that mechanism, which is one of the stronger controls in the project.

Chapter 12 built one cost reader per operation. This chapter is about what those readers were made to return.

---

1. The model

```
IGNIS charged = deter(op) + components(op)        ;; UC_IgnisPrice
STOA  charged = dollars(deter(op)) / stoa_price   ;; UC_StoaPrice — issuance ops only
```

Four rules make it legible, and each is an owner ruling with a date:

- **1 IGNIS = 1 US cent.** A deterrence of 5000 is $50.
- **Deterrence is a multiplier and it is ADDITIVE, not the total.** An operation with "no deterrence" still pays its full computation; `usage = 1` is the neutral element.
- **Every price is denominated in dollars** and converted at the oracle. When a real oracle replaces the $0.10 peg, the STOA *amount* moves and the *value* the user pays does not.
- **All published prices are full prices.** Elite discounts apply on top — up to 49% on IGNIS, up to 24.5% on STOA, *exactly half*. Three things are explicitly non-discountable: PYTHIA tolls, StoicTag registration, and some legs of the asymmetric-liquidity path.

The deliberate exception, recorded in the function's own `@doc` so nobody "corrects" it: CODEX::UC_StoicTagStoaFee is **1 STOA per glyph**, fixed in STOA units. A glyph costs one STOA whatever the oracle says.

2. Where a price lives

Four constant maps, all in 1_SOVEREIGN/STAGE_01/2_Core/02_IGNIS.pact. **[VERIFIED by command]** — key counts parsed from the defconst bodies today:

map	holds	keys
IG|DETER	the per-op deterrence — the business decision	**54**
IG|COMPONENTS	per-op computed work cost, keyed ENTITY|FN, calibrated to measured gas	**396**
IG|WEIGHTS	the primitives components are computed from	**14**
IG|LEGS	per-write and per-item unit costs charged inside XI_/XB_ writers	**22**

IGNIS-PRICING.md §2 says IG|COMPONENTS holds *"~394"* and §8 says 396; DEFECT-LEDGER.md §5 item 16 already flagged that one document carries three different counts. The tree says **396**.

The calibration, and what it overturned

IG|WEIGHTS was not modelled and left alone — it was measured against real gas in the rehaul's sixth substage, and **three of its four parameters were wrong**:

parameter	modelled	measured
read multipliers (s/m/l/xl)	1 / 1 / 2 / 3	**1 / 2 / 5 / 9**
update cost	ceil(fields ÷ 2)	**ceil(fields ÷ 4)**
hydra wipe ceiling	120 nonces	**1000 (DPOF) / 500 (DPDC)**
write multipliers	1 / 2 / 3 / 5	unchanged

The wipe ceiling is the one with a user-visible consequence. Measured through the real Cp_WipeSlice path, a DPOF nonce costs **405.6 gas** and a collectable nonce **952.4** — 2.3× heavier — which puts roughly 4,900 and 2,100 nonces in a 2M-gas transaction against a configured ceiling of 120. *A ceiling set 40× too low charges users for transactions they did not need.* Both were raised to about a quarter of the measured ceiling, which is a deliberate margin rather than an estimate.

The distinction that prevented a compounding error

> **A legacy tier read that is MULTIPLIED BY A COUNT is a *unit*, not a price.**

Migrating such a site from a flat tier to deter + components replaces a per-item unit with a whole-operation price and then multiplies it by the item count. **Four readers were caught at the edge of exactly that** — DPOF::URCi_MoveCumulator, both Repurpose families, and DPDC-N::URCi_UpdateNonces (DEFECT-LEDGER.md B-13). They were lifted into IG|LEGS at parity instead, and no price moved. The ledger calls it *"the canonical structurally-wrong price shape in this corpus"*, and the check is one sentence long: **before migrating any tier read, check whether it is multiplied.**

---

3. What the round actually changed

The decision log in IGNIS-PRICING.md §9 is the authority. The entries worth reading as an auditor are the ones where a live price was wrong by an order of magnitude or more:

site	before	after	why it was wrong
fee-unlock ladder (U_DEC::UC_UnlockPrice)	base × (unlocks+1), **unbounded**	flat **$50 IGNIS + $50 STOA**	no ceiling existed; DPTF's *first* unlock already cost $100 and each subsequent one grew without bound. Its STOA leg was computed as ignis/100 — dollars-at-$1, **10× short** of the peg
AQP pool / FVT / score issuance	**0.02 STOA**	100 STOA	issuance had borrowed the account-creation price keys "as a convenient small number" — a **5000× gap** that also bypassed the account-creation STOA switch
blue-flag branding	**0.025 STOA/month**	250 STOA/month ($25)	
ATSU::C_ColdRecovery / C_Cull / C_HotRecovery	10 / 10 / 15 IGNIS	130 / 132 / 117	a flat placeholder ~13× below the measured computation
DPTF::C_Burn / C_Mint	2 / 7 IGNIS	72 / 87	approved only after

site	before	after	why it was wrong
			confirming C_Burn's 71 components are **11 cross-module calls plus 7 reads**, not a modelling artefact
seven AQP-DSA client ops	**free**	priced	they wore an A_ prefix while gating on CAP_EnforceAccountOwnership — owner-gated, therefore client ops — and a Talos wrapper prefixed A_ is classified admin and exempt. All seven renamed A_→C_

That last row is the one to keep. **A naming convention was load-bearing on money**: seven live client operations collected nothing because their prefix told the biller they were administrative.

The constants-only conversion, and how parity was proven

Owner, 2026-09-07: *"we run no more table values, but constants for determining prices now."* All **65** live UR_UsagePrice "ignis|*" reads became UC_IgnisLeg "tier-*" against six new IG|LEGS constants lifted verbatim from the old table values.

The proof of no-change is the part worth copying: **the regenerated price sheet was byte-identical to the pre-conversion one.** Not "the suite stayed green" — byte-identical output from an independent generator. No price moved; only where the number lives changed.

And the first attempt at that edit destroyed the source. From commit 981c708:

> *"the first attempt used a paren-depth scanner for defun boundaries, overran on a body it mis-counted, and duplicated content exponentially — **1.78M insertions across 19 files**."*

Recovered by git checkout of 1_SOVEREIGN only, preserving in-flight sandbox work. The rewrite used **line-based boundaries plus an assertion that the line count cannot change**, which for an in-line substitution is exactly true and turns a silent catastrophe into a clean abort. The same class of failure had already destroyed source once before in this project, deleting 624 lines across four utility files. IGNIS-PRICING.md §8 now carries the rule in imperative form: **never use paren-depth scanning to define an edit region in Pact.**

One claim in the authoritative document that the tree does not support

[VERIFIED by command] — grep -rn 'UR_UsagePrice' --include=*.pact 1_SOVEREIGN 2_CITIZEN.

IGNIS-PRICING.md §2 states: *"As of 2026-09-07 **no live pricing path reads a database table**."* The narrow version of that claim is true — the only surviving ignis|* **tier** reads are in the dead 00_DPMF module. The sentence as written is not. Seven live reads of the DALOS usage-price table remain on pricing paths outside DPMF:

site	key	what it prices
04_BRD.pact:408 (URCi_UpgradeBranding)	"blue"	blue-flag branding, per month
11_VST.pact:922, 1589	"dptf"	the STOA leg of the frozen / reservation links
11_VST.pact:934, 1662	"dpmf"	the STOA leg of the vesting / sleeping / hibernating links
20_MTX-SWP.pact:1048–1049	"dptf" + "swp"	the defpact pool-issuance STOA leg

plus their INFO-ONE+ preview twins, and the oracle read inside UC_StoaPrice itself — which is by design, since the whole dollar rule is *"divide by the oracle price"*, and the oracle lives in a table.

[VERIFIED by reading] REPL/Stage_01/[4.0]_Sovereign-Executor.repl explains how the two statements were reconciled in practice. The executor seeds those keys twice: first with legacy literals (line 236: DALOS|A_UpdateUsagePrice "blue" 0.025), then **overwrites them from the constants** (line 272: DALOS|A_UpdateUsagePrice "blue" (UC_StoaPrice "branding-blue")). So IG|DETER really is the source of truth — *through a deploy-time seeding step, not through the read path.*

[INFERRED] — the operational consequence, which the document does not state. §2's rule is *"a price is retuned there and nowhere else."* For these four keys it is not: retuning IG|DETER "branding-blue" changes nothing on a deployed chain until an administrator re-runs DALOS|A_UpdateUsagePrice "blue". That is a two-step retune wearing the description of a one-step one, and the second step is an admin entrypoint that the test ledger records as invoked 32 times with **zero adversarial assertions**.

---

4. The generated price sheet

IGNIS-PRICE-SHEET.md is the per-function price list and the declared input to the Chapter-2 user documentation. It is produced by REPL/tools/_ignis_price_sheet.py, which walks the Talos client surface and extracts each operation's real cumulator legs.

The generator's own defect history is the most useful part of it

Every defect found in it *made the published sheet disagree with a chain that was already correct* — so when a price looks wrong, the sheet is the first suspect, not the contract. The full-coverage pass of 2026-09-09 took it from 82 unresolved rows to **0**, resolving 80 and repricing 14, with **zero rows demoted**. Each fix was a distinct blind spot:

blind spot	what it hid
@doc prose not stripped (\\. cannot cross a newline, so no doc string ever closed — and the unmatched quote then stripped **real code**)	doc-mentioned functions charged to the wrong operation
a module alias read as a module name (ref-B|DPOF is module DPOF)	every branding operation
same-module delegation not followed	thin alias operations
defpact bodies invisible (defun_body matched only (defun)	all 5 SWP liquidity adds + 3 pool issuances
cumulator constructors not followed cross-module	the 100-IGNIS branding charge, and the **flat $50 fee-unlock, which published as 2 IGNIS**
price helpers not followed (UC_DeployPrice → UR_Config → UC_StoaPrice)	the PYTHIA tolls
a literal 0.0 cumulator counted as a price	**every SWP swap published as FREE**
no notion of a STOA-only or free-by-design operation	58 rows reading ? where the honest answer was "charges nothing"

Two rules the walk must keep, both of which were violated during the round, caught by an invariant, and reverted:

1. **Depth stays 3**, escalating to 5 then 7 *only for a row that resolved to nothing*. Raising it globally crosses into **sibling** operations — AQP-FVT|CC_Collect absorbed a neighbour's 500 deterrence and DPTF|C_Mint, moving its floor from 557 to 1150.
2. **Same-module C_* is followed for ONE hop.** Transitively it reaches neighbouring client operations — DPDC-F|C_MergeFragments absorbed C_MakeFragments, floor 18 → 152, though its code calls only C_Transfer. *One hop captures a delegation alias; more hops capture the neighbourhood.*

The invariant that caught both is worth stating on its own: **for a re-pricing refactor, an already-priced row must not change value** — so diffing the regenerated sheet at every step is what rejected two generator "fixes" before they shipped.

The one that is worst to get wrong

SWP|C_Firestarter is the **bootstrap** operation for a brand-new account, and it is *gated on the caller holding under 100 IGNIS*. The sheet published it at **≥ 93 IGNIS**. The wrapper builds three cumulators, reads one value out of one of them, collects none of them, and its own success message says *"with no IGNIS Costs!"*.

> The sheet told someone whose defining characteristic is having no IGNIS that they needed 93 of it first.

The fix is narrow on purpose. The obvious rule — *"no C_Collect in the wrapper means free"* — was tried and is **wrong**: billing shapes D, E and F bill in the core, in a defpact step, or in a nested Talos wrapper, and the loose rule flipped 14 rows to free including C_AddStandardLiquidity, whose entire point is a 1000-IGNIS churn deterrent. The correct rule requires a **bound-and-unused** cumulator.

---

5. The gate mechanism — how the artefacts stay true

This is the control a reader should understand, because it is stronger than what most projects have and because its boundary is exactly where this chapter's finding sits.

REPL/tools/_pricesync.py --check is **fatal inside REPL/tools/_gate.py.** It does three distinct things, and the third is the unusual one:

(a) Artefact equals generator. It re-runs both generators *in memory*, from the repository root, and diffs the output against the committed files. Not a timestamp check, not a hash recorded at generation time — a live regeneration compared byte-for-byte. A contract edit that changes a price and is not accompanied by a regenerated sheet fails the gate.

(b) The document must not contradict itself. legend_vs_table() extracts the sheet's own **EXEMPT legend paragraph** — the prose that says which operations are deliberately free — and checks that every operation it names is published as 0 in the sheet's own table, 500 lines further down. This exists because the legend named Firestarter as free while the table published it at ≥ 93, *and nothing compared the two halves of one document*. It is derived from the legend text rather than a retyped list, so a newly-exempt operation is covered the day it is added.

(c) The narrative must quote the artefact. narrative_tally() parses the generated sheet's footer tally and requires IGNIS-PRICING.md to contain that exact line, and the exact total, as literal text. This is the rule the project states elsewhere and enforces here:

> *A generated artefact guarded by the gate, quoted by a narrative nobody checks, just moves the stale number one file along.*

The reason it exists is specific: on 2026-09-15 the prose document still read *"Nothing on pricing. Every one of the 420 Talos client functions now carries a price"* — figures no longer produced by anything, because **both generators had been dead since the tools directory moved** and nothing had re-derived them. They died at *import*, on hard-coded sibling paths, so no output existed to diff. Meanwhile the artefacts had been **hand-edited**: their Regenerate: provenance lines were updated to name the new path, i.e. *the provenance line of a document that could not be regenerated was corrected to name a command that crashed.*

_figuresync.py is the sibling control for ARCHITECTURE/*.md, and _toolpaths.py --check is what makes a repeat of the dead-generator incident visible — it statically resolves every hard-coded path literal in every tool. Both are also fatal in the gate.

6. The finding: a gate-enforced headline that was eleven operations short

> **CLOSED 2026-09-17.** Everything in this section describes a defect that is now repaired. _ignis_price_sheet.py sums nsimple + ncomplex + nstoaonly + nexempt (line 743, having moved from 730); the sheet's footer reads **442**; IGNIS-PRICING.md quotes 442 in both places; the literal 431 appears nowhere. The section is kept unedited below, with the tense left as written, because its subject is not the eleven rows.
>
> Its subject is this: a **generated** artefact published a wrong total for weeks while a gate check enforced that the prose document agree with it. The control was not merely blind to the error — it was **holding it in place**, because correcting the prose by hand would have failed the build. A consistency check between two artefacts is evidence that they match. It is not evidence that either is right, and where one is generated it can convert a generator bug into a mandatory falsehood. That lesson survives the fix, which is why the section does.

[VERIFIED by command], counting the price sheet's own row markers:

```
admin/exempt ..... 41
core op returns an empty cumulator ... 5
builds cumulators and collects none .. 1
collects no IGNIS and no STOA ..... 3     → 50 exempt
| STOA only | ..... 11
| COMPLEX | ..... 199
| $n.nn |  ..... 182
                      ----
                total    442 priced rows
```

442 rows, plus a further **5** listed in the sheet's own UNPRICED section. **[VERIFIED by reading]** — the physical row count reconciles: 437 rows begin with a backticked name, and 5 more (the three HOT-RBT| rows and the two MTX-AQP|2| rows) carry an escaped pipe instead.

The sheet's footer reads:

```
182 simple (exact price) · 199 complex (floor price) · 11 STOA-only · 50 exempt
 · 0 unresolved · 5 unpriced · 431 Talos client functions
```

[VERIFIED by reading] _ignis_price_sheet.py:730 computes that last figure as:

```
f" · {nsimple+ncomplex+nexempt} Talos client functions"
```

nstoaonly is omitted. So **every operation priced in STOA and not in IGNIS is counted in its own column and then dropped from the headline** — the branding upgrades, the PYTHIA deploy and rename tolls, and the CODEX StoicTag family. 182 + 199 + 50 = 431; the sheet holds 442.

IGNIS-PRICING.md §6 repeats it twice — *"431 Talos client functions priced, 0 unresolved"* and *"**431** Talos client functions carry a price"*.

Three things make this worth a section rather than a footnote:

It is a known defect that was recorded and not fixed. DEFECT-LEDGER.md §5 item 14, dated 2026-09-15, states it precisely and names the line: *"the footer computes the total as nsimple+ncomplex+nexempt and omits nstoaonly. So the sentence 'every one of the 420 … carries a price' silently excludes 10 ops that do."* At that date the figures were 420 and 430. Today they are **431 and 442**, and the excluded set has grown from 10 to 11.

The control added the same week now enforces it. narrative_tally() requires IGNIS-PRICING.md to contain the literal string **431**. Correcting the prose document by hand — writing the true total — would **fail the gate**. The undercount is no longer merely published; it is *mandatory* until the generator is fixed.

It is the book's third rule, from the inside. *A count is reported with its exclusions, or not at all.* The sheet does report its exclusions — the 11 are in the breakdown, one field to the left of the total that drops them. A reader who adds the columns finds it in ten seconds. A reader who quotes the headline, as the authoritative document does twice, does not.

> The fix is one identifier. The instructive part is that a genuinely good control — regenerate, diff, and force the prose to quote the artefact — **propagates a wrong number as faithfully as a right one.** Consistency is not correctness, and a gate that enforces the first can make the second harder to reach.

---

7. What the pricing is actually tested by

[VERIFIED by command], counting (expect ...) forms in the three dedicated suites:

suite	assertions	what it pins
REPL/Stage_01/[6.1]_Cumulator.repl	**75**	the **leg-level** assertions
REPL/Stage_02/[6.1.9]_PRICE-SWEEP.repl	**64**	every son-taking reader, twice, once per fungibility side
REPL/Stage_02/[6.2.16]_AQP-PRICE-SWEEP.repl	**42**	the AQP family
	181	

Plus the measured preview-versus-charge proofs described in Chapter 12, which are balance deltas rather than table comparisons.

Which of them run where, stated exactly

CLAUDE.md warns that the fast-path runner skips pricing suites, and that warning has itself been corrected once for overstating the case. The precise position today — **[VERIFIED by command]**:

- Stage01_Tester.repl:32 carries ;(load "Stage_01/[6.1]_Cumulator.repl") — **commented out**. So Z.repl, the fast path, runs **106** pricing assertions (64 + 42) and skips **75**.
- **The gate runs all 181.** [6.1]_Cumulator.repl is loaded by ZALL.repl:27 **and** by REPL/modules/CUMULATOR.repl:6, and both are gate entrypoints.

The 75 matter more than their share suggests, and the reason is on the record: on 2026-09-14 [6.1]'s <<TX-IGC-008>> was **the only assertion in the entire suite** to catch a VST preview leg-split, because the total was unchanged and every total-level assertion agreed. *A cost split across a different number of legs prices differently, because the cumulator discounts per leg.*

So the honest formulation, which this book adopts: **a quarter of the pricing assertions, including every leg-level one, are outside the fast path — and none of them are outside the gate.**

---

8. What this chapter does not establish

- **components is a model, not a measurement, per operation.** The sheet says so in its own caveat: it counts the core operation's own module-internal work, and *"cross-module callee internals are not re-summed, so delegating ops read a little low."* The IG|WEIGHTS primitives were calibrated against measured gas; the per-operation totals built from them were not individually measured against a live charge. The preview-versus-charge work in Chapter 12 is what measures live charges, and it measures **previews**, which is the same arithmetic but not the same claim.

- **199 of 442 rows are floors, not prices.** A COMPLEX row publishes *"costs at least N"*, because the composition varies with a list length or a scan. That is honest, and it means the sheet cannot answer "what will this cost me" for 45% of the surface.
- **Two pricing judgement calls remain open** and are recorded as such in §6 of the reference: MTX-SWP::C_AddSleepingLiquidity still carries tier-token-issue 500, the last legacy number under a new name, inside a defpact that already carries a 5000 deterrence; and the double-piped MTX-AQP|2|C_* Talos names are either a deliberate defpact step marker or a naming slip, and nobody has said which.
- **Five entrypoints carry no row.** Three are admin operations that are free by rule; two (DALOS|C_UpdateEliteAccount and its Squared twin) are billing shape B — the Talos wrapper builds the cumulator and collects it itself, so there is no core operation for the sheet's row model to key on. The sheet names the authoritative reader for each rather than inventing a component cost, and **before 2026-09-15 it dropped them silently while the footer reported 0 unresolved.**

---

14. Module splits under the deploy ceiling

> Source records: OuronetInformational/MODULE-SIZING.md (the rule), Audit/module-audits/AQP/FVT-SPLIT-DESIGN.md and FVT-SPLIT-MANIFEST.md (the design and the verified move-list), OuronetInformational/DEPLOY-READY-GATE.md (the measured outcome), and commits 43c0ea6 → b37c08e (2026-09-03).

This is the shortest phase in the round and the only one with a hard blocker behind it: one sovereign module had grown past the point where any transaction containing it could be mined.

---

1. Why there is a ceiling at all

Chainweb charges a transaction for its size, and **the size charge grows as the seventh power** of that size. MODULE-SIZING.md tabulates the consequence against StoaChain's 2,000,000-gas block limit:

lines	size charge	exec	total	% of block	size as % of its own cost
2,000	1,569	101,463	103,032	5.2%	1.5%
3,500	20,944	177,560	198,503	9.9%	10.6%
4,500	112,188	228,291	340,479	17.0%	33.0%
5,500	449,422	279,022	728,444	36.4%	61.7%
6,635	1,669,856	336,805	2,006,661	100.3%	**CANNOT DEPLOY**

Read the last column. At 6,000 lines, 73% of what you pay is *the mere fact of being large*.

Two properties turn that curve into a design rule rather than a curiosity:

- **Every Pact upgrade redeploys the whole module.** A one-character fix in a 6,477-line module costs 1,733,784 gas — not once, every time, forever. A module's line count is a **permanent tax on every future change**.
- **You cannot split your way out afterwards**, because **Pact tables are module-scoped**. (create-table FVT|T) binds that table to that module and no other module can adopt it. Splitting a *deployed* module with live data means new tables plus a migration, and the old data stays where it is.

The failure mode to design against is: **unable to upgrade, and unable to split your way out of being unable to upgrade.**

The bands, and they are the project's own:

band	lines	action
Target	under 3,500	fine
Acceptable	3,500 – 4,000	only **with a plan**
Warning	4,000 – 4,500	**start designing the split now**
Danger	above 4,500	**split before deploying**
Impossible	~6,635	cannot be deployed at all, ever

And the rule for *where* to cut, which the round followed and which is the reason this chapter is not about line counts: **split by capability, never by line count.** Capabilities are module-scoped in Pact — that scoping is a real security boundary, and it was the subject of a critical vulnerability fixed at block 516,500, where compose-

capability could reach across module boundaries without the module guard. Anything sharing a defcap belongs in the same module; tables go with the module whose capabilities guard writes to them; and that choice is **permanent**.

---

2. The blocker

[VERIFIED by command] — git show <rev>:.../04_FVT.pact | wc -l:

when	lines
2026-08-27, when the rule was first applied	6,694 — already **past** the ~6,635 cliff
2026-08-30, start of this round	6,781
2026-09-03, after the canon let-staircase pass added ~800 whitespace lines	7,249 → **7,527**
immediately before the split (5a453d8^)	**7,527**

FVT loads fine in the REPL, which does not enforce the block cap. It could not be deployed or upgraded on chain. That is a pre-mainnet blocker of the worst kind, because it gets structurally harder the moment the module holds live data.

---

3. The seam, and how it was chosen

The design (FVT-SPLIT-DESIGN.md) reads as an **accountant** and an **estate registrar**:

- **RPS** — the reward-per-share ledger and all the arithmetic that moves it. A pure leaf: inputs in, ledger writes out. Deploys **first**, carries **its own SECURE**.
- **FVT** — the farms/vaults/treasuries themselves, their ownership, configuration and topology, vacate/sweep, and **every client entrypoint**. Deploys second, keeps the name.

Three things about the method are worth extracting, because they generalise past this module:

The seam was measured, not argued. REPL/tools/_fvtsplit.py classified all 407 definitions by which tables they transitively touch: **79 RPS-only, 164 FVT-only, 77 SEAM (touch both), 87 FREE (no table)**. The manifest's key result is a single sentence: *"0 functions directly touch both domains"* — so no atomic cross-domain write transaction had to be untangled, and all coupling was at the call-graph level.

The first cut was wrong and the measurement said so. A ledger-only extraction left FVT at 6,435 lines and ~84% of the gas budget — deployable, with no headroom — because the reward-settlement orchestration reads ScoreEntityLink throughout (36 functions). The revision moved the reward *distribution topology* with the ledger. *The seam that looks right on a diagram is not necessarily the one the call graph has.*

A cycle was found before it was built, and resolved by moving data rather than code. The orchestration writes five aggregate fields that live on FVT|T — if it moved to RPS, those writes would be RPS→FVT, a dependency cycle. The fix was a new RPS-owned table FVT|T|RewardAggregate holding seven denormalised fields, so FVT|T keeps only identity and configuration and RPS stays a clean leaf. That is a **permanent data-model decision taken because tables cannot move later** — the right time to take it, and the design says so explicitly.

The resulting dependency graph is acyclic by construction: RPS → {IGNIS, DALOS, DPTF, SCORE-reads}, FVT → RPS::XE_. RPS never reads an FVT table; the structure fields the arithmetic needs are computed by FVT and **passed as arguments**. And the hot-path risk — cross-module calls cost gas per call — is neutralised structurally: **loops live inside RPS**, so a settle/inject/collect pass crosses the boundary once per *phase*, never once per *user*.

---

4. What shipped, against what was designed

[VERIFIED by command]. This is the part the design documents do not reconcile, and it is the chapter's main finding.

	designed	shipped (b37c08e, 2026-09-03)	today (HEAD)
RPS	~3,288 lines	**5,617**	**5,621**
FVT	~2,765 lines	**3,878**	**3,977**
combined	~6,053	**9,495**	9,598
the monolith it replaced	7,527	7,527	—

Two facts follow, and both are verifiable in one command:

The split added 1,968 lines — 26% — to the codebase. That is not waste; it is the price of a module boundary, and the commit message itemises it: **33 XE_ SECURE-granting writer wrappers** so FVT can mutate the ledger through a guarded entrypoint, **53 facade re-exports** so every existing reader call site and the entire test corpus stay byte-identical, and the policy plumbing (P|T, P|MT, the IMC machinery) duplicated because it is per-module infrastructure rather than domain data. **[VERIFIED by command]** — 53 FVT functions call RPS. directly in the tree today, matching the facade count.

RPS landed 71% larger than designed, in the project's own Danger band. MODULE-SIZING.md §1 says of anything above 4,500 lines: *"split before deploying."* The design anticipated a risk here — Risk 3, *"if FVT lands over ~3,900 lines, shift the UC_ compute helpers"* — but it was written against the wrong half, and the half that overran was RPS.

The project recorded this honestly rather than rounding it away. DEPLOY-READY-GATE.md, written the next day, publishes the distribution with RPS labelled **danger** in its own table and reports (5617/6635)^7 × 2M ≈ 624K gas against the 2M cap. The verdict it draws is *"deploy-ready"*, and against the **cliff** that is correct: nothing is undeployable. Against the **band rule** in the same project's sizing document, it is not — and against §5's guidance that *"if size is more than ~25% of your total, the module is too big regardless of whether it fits"*, RPS at 5,621 lines sits at roughly 62% size share by the document's own table. **[INFERRED]**, by interpolation in that table; not measured on chain.

> **SUPERSEDED BY MEASUREMENT, 2026-09-18.** RPS's deploy gas has since been measured rather than interpolated: **353,662 of 2,000,000 — 18% of a block**, not 62%. The worst module in the tree is 02_INFO-ONE+ at 22%. The inference was wrong by more than a factor of three, and wrong in the alarming direction, which is the direction that gets acted on. Lines are a proxy for deploy gas and a poor one: a module of dense short UR_ readers and a module of long arithmetic bodies do not cost the same per line. The original text is kept, with its [INFERRED] label, because the label did its job — it is what made the claim checkable, and it is why the measurement was taken.

The band census today

[VERIFIED by command] — find 1_SOVEREIGN 2_CITIZEN -name '*.pact' -not -path '*/Audit/*' | xargs wc -l, classified against MODULE-SIZING.md §1:

band	n	modules
Impossible (>6,635)	**0**	—
Danger (>4,500)	**1**	04_RPS.pact **5,621**
Warning (4,000–4,500)	**2**	02_SCORE.pact 4,283 · 02_INFO-ONE+.pact 4,199
Acceptable (3,500–4,000)	**3**	05_FVT.pact 3,977 · 06_VCT.pact

band	n	modules
		3,508 · 03_AQP.pact 3,507
Target (<3,500)	**87**	
	93	

Against the same census on 2026-09-04: SCORE 4,122 → 4,283, FVT 3,878 → 3,977, AQP 3,421 → 3,507 and VCT 3,377 → 3,508 — the last two crossing out of Target. INFO-ONE+ shrank, 4,450 → 4,199. **Six modules are outside the Target band and the number is rising**, which is the expected consequence of adding a cost reader and a preview to every operation.

---

5. What Phase 1.4 did not finish

The roadmap lists three steps. **[VERIFIED by reading]** the tree, one is done and two are not:

step	status
1.4.1.1 split 04_FVT.pact along a capability seam	**done** — 04_RPS.pact exists, the seam is a capability seam, the DAG is acyclic, the suite was green across the flip
1.4.1.2 re-audit POOL/SCORE/ANK/VCT once their URCi_s are in; **split any in Warning/Danger**	**closed 2026-09-18 by owner ruling** — *"I don't think we need any splitting of modules, we already did that before we started this, to get to the final pact code shape."* MODULE-SIZING.md now carries the ruling and demotes the line bands from a threshold to a review prompt. The measurement above is why the ruling is sound rather than merely authoritative: the worst module in the tree uses 22% of a block
1.4.1.3 update MODULE-SIZING.md's applicability table with post-URCi measurements	**done** — the table now carries measured deploy gas, not interpolated line shares

The third is the one with a trap in it. MODULE-SIZING.md's applicability note still reads:

module	lines	band
.../03_AQP/04_FVT.pact	**6,694**	**IMPOSSIBLE**
.../03_AQP/05_VCT.pact	3,023	Target
.../03_AQP/08_AQP-INFO.pact	1,178	Target

All three filenames are pre-renumber. 04_FVT.pact no longer exists; 05_VCT.pact is now 06_VCT.pact at 3,508 lines; 08_AQP-INFO.pact is now 09_AQP-INFO.pact. So the document that states the sizing rule still describes a blocker that was cleared two weeks ago, under paths that resolve to nothing. A reader arriving at the rule is told the worst module is undeployable and pointed at a file that is not there.

---

6. The second split, and a stale line it left in the governing document

The round performed a **second** split along a different axis, and it is worth recording because it is a split for *authorisation* reasons rather than size ones.

The launchpad Talos orchestrator was one module doing two jobs: composing the per-asset citizen sales (a citizen-role concern) and orchestrating the sovereign DEMIPAD launchpad rules. Commit ae1f9b6 (2026-08-31) cut it in two, and git records the move as a rename plus an addition:

```
R057  2_CITIZEN/6_Launchpad/99_TS02-DPAD.pact  ->  1_SOVEREIGN/STAGE_02/3_Talos/05_TS02-DPAD.pact
A     2_CITIZEN/7_Launchpad/99_TS02-CPAD.pact
```

[VERIFIED by command] — both files exist at those paths today, and no 2_CITIZEN/7_Launchpad/99_TS02-DPAD.pact does.

CLAUDE.md described the outcome **twice, and the two descriptions disagreed**:

> **CLOSED 2026-09-17.** CLAUDE.md now carries an explicit retraction of the second description, naming the one-letter C/D distinction that caused it. The cited line numbers are also stale — the two rows are now at 127 and 140. The finding is kept because of the mechanism: the two sentences differed by **a single letter** in a filename (TS02-CPAD against TS02-DPAD), and the wrong one sat in the sentence that assigns the sovereign/citizen role. A one-character divergence between two passages of the same document is invisible to a reader and to every diff that is not looking for it.

- Line 109 (the repository layout table) is **correct**: 99_TS02-CPAD.pact is the *citizen* launchpad Talos; *"the **sovereign** launchpad Talos moved to 1_SOVEREIGN/STAGE_02/3_Talos/05_TS02-DPAD.pact"*.
- Line 122 (the Sovereign-vs-citizen architecture section) still says *"2_CITIZEN/7_Launchpad/99_TS02-DPAD.pact is the **sovereign** Talos orchestrator … co-located with the launchpad for readability, but sovereign-role and deployed last."*

The second names a file that does not exist, and describes an arrangement — sovereign code living in the citizen tree — that the split exists to have ended. A reader who takes line 122 at face value concludes that the sovereign/citizen boundary is a labelling convention rather than a directory.

---

7. What the split cost in coupling, measured

[VERIFIED by command] — cross-module calls written as MODULE.fn dot-notation rather than as a module reference (ref-M::fn …), counted over 1_SOVEREIGN + 2_CITIZEN with comments and strings stripped:

	2026-08-30	today
cross-module dot-notation call sites	**38**	**275**

Where the 237 new ones are: 06_VCT.pact 0→85, 09_AQP-INFO.pact 26→78, 05_FVT.pact 0→53 (the new shell), 08_DSA.pact 0→25, 04_RPS.pact 0→13, 07_MTX-AQP.pact 0→8, 99_TS02-CPAD.pact 0→1. **262 of 275 are in the AQP family**, and the growth is the direct consequence of two things this round did: splitting FVT (so VCT, DSA, MTX-AQP and the new FVT shell must now reach reward state in another module) and building the preview layer (so AQP-INFO must reach every cost reader).

CLAUDE.md's deployment section states the convention plainly:

> *Cross-module calls use **module references** with :: (e.g. (ref-M::some-fun …)), not module.function, so only the used interface members matter for coupling.*

[INFERRED], from the same document's own upgrade note rather than from an experiment this book ran: :: modref dispatch is live, so an in-place sovereign upgrade reaches every :: caller with no bless, while a . reference is a hash pin. On that basis the round added 237 hash-pinned sovereign-internal dependencies. The consequence is bounded — these are all our own modules, and re-deploying a caller is a normal operation — but it is a consequence, it is concentrated in the largest and most-upgraded family in the codebase, and it is not recorded in any of the three split documents.

It also had a measured cost inside the round itself. _ignis_price_sheet.py could not see dot-notation calls at all, which is one of the four blind spots that left 18 live client entrypoints with no published price while the generator's footer reported 0 unresolved (Chapter 13, §5). *A convention that a tool does not implement is a convention the tool is blind to.*

8. What this chapter does not establish

- **The deploy-gas figures are extrapolations, not chain measurements.** RPS load-exec 362K / FVT 250K came from the REPL's `table` gas model, which measures runtime load execution, not Chainweb's size formula; ~624K and ~47K are (lines/6635)^7 × 2M. Both are stated as such in the source documents. Nothing has been deployed to a chain to check them.
- **No claim is made that the seam is optimal.** It is acyclic, it is capability-aligned, it is measured, and it works. Whether a different cut would have produced two Target-band modules instead of one Danger and one Acceptable is not something this book can answer, and the design's own measurement pass rejected the two obvious alternatives (ledger-only, and accessor-split) as unviable rather than as inferior.
- **The `FVT|T|RewardAggregate` denormalisation is permanent and untested against migration.** It was taken pre-mainnet precisely so that no migration is needed. If the module is ever deployed and the seven fields turn out to be on the wrong side, the remedy is a migration, not an edit.

15. The test architecture

> Source records: ARCHITECTURE/REPL_TEST_ARCHITECTURE.md (the model), Audit/records/REPL-ROUND-REPORT.md (the round's own report), ARCHITECTURE/REPL_SUITE_STATS.md (generated), Audit/records/REPL-TEST-LEDGER.md (generated), REPL/modules/README.md, and REPL/tools/_gate.py — which is the gate.

The audits in Part I kept finding the same thing: functions with **zero** REPL coverage, and real bugs living in them precisely because nothing called them. ATS finding #22L alone named twelve untested configuration functions and two live bugs inside them. Phase 1.5 is the phase that closed that, and rebuilt the runner architecture around the result.

---

1. What was built, in size

[VERIFIED by command] — git archive <rev> REPL | wc -l, and the same (expect…) regex REPL/tools/_suite_stats.py uses for its own "distinct assertions written" row:

	2026-08-30	2026-09-04	2026-09-09	2026-09-14	2026-09-16	HEAD
.repl files (excl. archive/)	181	—	—	209	—	**222**
.repl lines (excl. archive/)	68,435	—	—	120,603	—	**148,965**
distinct assertions	**1,604**	1,663	2,431	**5,262**	5,555	**6,619**

CORRECTED 2026-09-18 — the HEAD column was written as literals and had rotted. It published **128,567** .repl lines, which was true around 2026-09-17 and is roughly nine thousand short today. All three HEAD cells are now build-time macros. The two size macros carry _scale_report.py's counting rule, reaching this chapter through REPL_SUITE_STATS.md; that rule differs from the historical columns' git archive … | wc -l by about twenty lines. Much smaller than the drift it replaces, but not zero, and worth knowing before reading the row as one measurement.

The suite roughly **3.3×'d** across the round, and most of that arrived between 2026-09-09 and 2026-09-14 — the guard-pinning phase, which is the bulk of the work by volume.

One fifth of the suite is comment, and the round's report argues that is a design decision rather than slack. The argument is testable and this book accepts it, because the mechanism is stated: several defects in the round were found **while writing the explanation**, because stating precisely what an assertion proved exposed that it proved nothing.

---

2. The model: three suites, six gates

The architecture document's opening distinction is the one that organises everything else:

suite	asks	derived from	can it be complete?
NORMAL	does the specified behaviour work?	the spec / the op list	**yes** — every entrypoint, exercised
ADVERSARIAL	is every door we built actually locked?	**the code itself** — every enforce, every defcap	**yes** — it is enumerable

suite	asks	derived from	can it be complete?
RED TEAM	is there a door we forgot to build?	attack hypotheses	**no** — open-ended

The middle one is the insight. An adversarial suite is *derivable*: list every enforce, write an expect-failure proving it rejects, and when the count matches, that suite is provably finished. Red teaming cannot be derived that way, because the bug it looks for is an **absence** — and you cannot derive what you forgot from the code that forgot it. The document says so in one sentence that belongs in this book:

> *Reaching 100% adversarial coverage does NOT mean the code is secure — it means every lock we installed has been tested. It says nothing about the doors we never built.*

That is also why **RULE 3** is stated without an exemption category: *an enforce you cannot write a failing test for is dead code — delete it.* There is no "unreachable, skip it" bucket. Either a caller can violate the condition, or the guard is noise and its removal makes the guarantee clearer.

The six gates, each a number recomputable from a script. **[VERIFIED by reading]** the generated statistics of 2026-09-16 and the architecture document's own order-of-work block:

gate	asks	at 2026-09-09	now
G1 Surface	every Talos entrypoint invoked **inside its own module tester**	~65%	**448 / 448 = 100%.** It read 447 until 2026-09-18, and the missing one was the ledger's blind spot, not a gap in the suite (§5)
G2 Adversarial	every enforce / defcap rejection has an expect-failure	293 tests / 1,015 sites = 29%	**live unpinned = 0**; 709 pinned unambiguously, 766 including shared wording
G3 Determinism	every test passes standalone **and** inside the full run	—	every module tester boots its own environment; the gate runs them as separate processes
G4 Conformance	every rule the architecture *states* is proven to hold	44 deviations / 4,919 defuns	**violations 0**, observations 114
G5 Doc claims	every @doc stating a RULE has a test proving it	100 unverified	**0 left** — scoped at ~100, the real worklist was 19
G6 Function reach	every client-reachable function is exercised	3,230 / 4,498 = 72%	**3,826 / 4,525 = 84%**, 699 never reached

CORRECTED 2026-09-18, re-measured by running the instruments rather than by reading the 2026-09-16 generated statistics this table originally quoted. The "now" column used to read **G1 448 / 448 = 100%**, **G2 live unpinned = 1** (707 unambiguous, 763 including shared wording), and **G6 3,816 / 4,524 = 84%, 708 never reached**. G2 and G6 simply moved: the last live unpinned guard was pinned, and the tree gained one client-reachable function while nine more were reached.

G1 never moved at all, in either direction. It read 100%, then 447/448, then 100% again — and the suite drove every entrypoint throughout. The dip was a defect in the measuring tool, found on 2026-09-18 and repaired the same day; §5 has it. It gets a section because a coverage number wrong in the **pessimistic** direction is the harder kind to notice: nobody investigates a number that is asking for more work, and a figure that has quietly *understated* coverage for months will be believed indefinitely. None of these four figures has a live-figure macro behind it, so all four are snapshots and will rot again. That is §7.1's subject.

G2's denominator, stated rather than assumed. The "live unpinned = 0" figure is conditioned on three exclusions that the generated statistics list beside it, so the subtraction is visible rather than taken on trust: **24** guards in the dead 00_DPMF module, **40** unreachable inside an enforce-one, and **31** proven unreachable by hand and annotated. All three counts re-derive unchanged on 2026-09-18; only the headline moved, from 1 to 0. A zero needs its exclusions stated more urgently than a one did, because a zero is the number a reader stops interrogating. An excluded guard is not reported as a gap — and the project's own rule for the third bucket is the thing that keeps it honest: *every ;;UNREACHABLE annotation is backed by a test that drives the route to whatever wall actually stops it and reads the message that comes back.* That is what turns an annotation from a claim into a regression trip-wire.

G5 is the one most projects do not have, and it can fail in a way G2 cannot. G2 asks whether the guard we wrote works; G5 asks whether the guard we **promised** exists at all. Closing it found two *vacuous existing proofs* — shared-graph equalities that short-circuited on OURO and never read the graph argument they were written to test.

G6 is the one that is not closed, and the round says so. The target was ~93%; the measured position is 84%, with **699 client-reachable functions never reached**, concentrated in per-field readers and internal constructors. The round report files it under *"Remaining, and deliberately not closed here"*, which is the correct disposition and the correct place for it.

---

3. The gate

python3 REPL/tools/_gate.py. It resolves its own paths and chdirs to its own directory, so it runs from anywhere. **[VERIFIED by reading]**, it does four things in order.

(a) It refuses to start on a dirty workspace

Any modules/_*.repl, RedTeam/_*.repl, or non-_verify_finding_ Kursan/_*.repl fails the gate immediately. modules/ is globbed into the entrypoint list, so a forgotten iteration probe becomes a real entrypoint — which happened, and failed the gate 200 seconds in with a message about wrong expected strings rather than about a leftover file.

(b) Fifteen static checks, every one fatal

[VERIFIED by reading _gate.py] — fifteen tools are invoked as --check subprocesses, and **every one of them sys.exits the gate on a non-zero return**:

check	what a failure means
_stagez_variant --check	the generated Stage-Z testing variant is stale against canonical — *"the STAGE-Z assertions are testing a stale copy and would still go green"*
_ladder --check	a citizen minter's batch ladder does not tile its collection — gap, overlap, or a rung over the per-transaction gas budget
_colproj --check	a projecting read names a column its own table's schema lacks
_redteam --check	a RedTeam/ block has a malformed or duplicate attack header — *"a shrinking register looks exactly like a clean one"*
_figuresync --check	an audit document quotes a figure the generated statistics do not support
_pricesync --check	a generated pricing artefact does not match its generator (Chapter 13 §5)

check	what a failure means
_toolpaths --check	a tool hard-codes a path that no longer resolves
_prefixsync --check	a function prefix live in the tree is unknown to the tooling's vocabulary
_eagerlet --check	an enforce is shadowed by an eager hard read in the same let, and nobody has annotated it
_booktables --check	a headline table in this book does not sum to its own total, or disagrees with the attack register
_auditbook --check	the assembled single-file book is stale against its chapter sources, or a source writes a chapter number as a literal
_modref --check	a (ref-X::member ...) call where member is defined **nowhere** in the implementing module
_vacuous --check	a positive assertion that **cannot fail**
_conformance --check	an architectural conformance **violation** (observations stay advisory)
_heavy --check	a single-prefixed C_/A_ whose tree reaches a heavy scan

CORRECTED 2026-09-18. This section was headed *"Twelve static checks"* and said that exactly twelve tools implement --check and that all twelve are gated. Three were missing — _eagerlet (_gate.py:368), _booktables (:380) and _auditbook (:392), all three fatal. The last two arrived with this book's own generator, so the chapter describing the gate was out of date about the check that exists to keep the chapter honest. The wording was wrong in a second way that caused the first: *"of the tools in REPL/tools/, exactly twelve implement a --check mode, and all twelve are run by the gate"* fuses two different counts into one number, and a fused count cannot show which half has drifted. What is stated now is the second question alone, read off _gate.py — which is also why a sixteenth tool implementing --check tomorrow would not make this sentence wrong, only incomplete.

Two of these deserve a sentence each. _modref exists because **Pact 5 resolves modref members dynamically**, so a call to a member that does not exist loads and runs and only raises if that branch is taken — invisible to every other check. And _conformance / _heavy were **added as gate checks on 2026-09-14 after a verification pass found their "0" was a hand-measured figure**: the gate byte-compiled them and ran one selftest, but never ran either tool, so re-introducing a defect that had been *verified exploitable* before it was fixed would have left the gate green.

(c) Tool integrity

Every _*.py in REPL/tools/ is byte-compiled, and --selftest is run on the nine that have one. CORRECTED 2026-09-18: this said **ten**. Nine _*.py in REPL/tools/ contain --selftest once _gate.py itself is excluded, and the gate excludes itself explicitly. Ten is what you get by grepping the directory without that exclusion — counting the gate as one of the tools it checks. This exists for a specific reason: an edit to _conformance.py's rule text left an unescaped quote inside a string, **and the gate went green twice while the conformance checker could not start.**

> *A tool that cannot run is worse than a missing tool, because its silence reads as clean.*

(d) The orphan check, which is the load-bearing one

The gate computes the transitive (load ...) closure of all 92 entrypoints and **fails if any .repl containing an assertion is outside it** and not on an 18-entry exclusion list, each entry of which must carry a reason. *"It is slow" is not a reason; "it is an alternative path to something already gated" is.*

This exists because it was missing. Four deb-staleness-* drivers were archived on an "assertion-free" heuristic — true of the drivers, false of what they *load* — and they were the only path to an entire AQP suite family. **~125**

assertions silently left the suite and the ledger never noticed, because the ledger counts files, not execution.

What the gate actually runs

[VERIFIED by command], reconstructing the entrypoint list from _gate.py without running it:

```
 7  named runners  (ZALL, AQP-FULL, AQP-core-vct, triplet-collect-golden,
                    launchpad-groundtruth, Stage00b_Run, Stage00b_RunGas)
 4  deb-staleness-*.repl
34  modules/*.repl          (the per-module standalone testers)
14  RedTeam/*.repl          (globbed, so a new attack is gated the day it exists)
27  Kursan/                 (scale + finding-verification harnesses)
 6  named _scratch_ audit proofs
—
92  entrypoints
```

The report's measured parallel economics: **6,386 seconds of work serial — 1 hour 46 minutes — completing in ~427 seconds wall on 16 workers, a 15.0× speedup.** The speedup is deliberately sub-linear and the bound is stated: **wall time can never fall below the slowest single entrypoint**, and ZALL.repl alone is ~354 s of the ~375 s total. Adding workers past 16 buys almost nothing.

That ratio is not a performance note. It is the difference between running the gate once a day and bisecting a regression across twenty edits, and running it after every coherent change so the regression names itself. Most of what this round found was found in the second mode.

---

4. "One command tests the entire system" — what that actually means

Roadmap step **1.5.1.3** asks for a *"single comprehensive run — the whole codebase, one boot… one hermetic run that boots once and exercises ALL Pact code."*

[VERIFIED by command] — computing the transitive (load …) closure of ZALL.repl, the one-boot runner, and intersecting it with the set of .repl files that contain assertions:

asserting .repl files in the suite (excl. archive/)	**149**
…inside ZALL.repl's closure	**34**
distinct assertions in the suite	**5,804**
…inside ZALL.repl's closure	**1,437 (25%)**

The 5,804 here excludes archive/; §1's 6,619 is the project's own _suite_stats.py rule, which walks all of REPL/ including the 68 archived files. Same corpus, two denominators — quoted separately rather than reconciled, because each is the right one for its own question. CORRECTED 2026-09-18: these four rows read **148 / 34 / 5,747 / 1,434 (25%)** when the chapter was written on 2026-09-16, and drifted with the suite. What did **not** move is the pair the section's argument rests on — the closure is still 34 files and still 25% of the assertions — so the one-boot runner has taken on none of the suite's growth.

So the single-boot runner exists, is gated, deploys the entire stack in dependency order, and carries **about a quarter of the suite's distinct assertions.** The other three quarters live in the 34 module testers, the Kursan harnesses, the 14 red-team files and the named runners — each of which boots its own environment.

The goal was met by a different mechanism than the one specified, and the substitution is a good one. One command does test the entire system: it is _gate.py, and it is a 92-way parallel run rather than one boot. The reason it is better is the reason the parallelism rule exists — a single hermetic boot is bounded by its own serial length, and at 354 s for a quarter of the assertions, a one-boot run of the whole suite would be the 1-hour-46

figure. But the roadmap step is not ticked and nothing in the tree records that it was consciously superseded, so a reader checking whether “one boot exercises all Pact code” would conclude, correctly, that it does not.

The two figures should not be conflated the way they have been elsewhere: DEPLOY-READY-GATE.md cites *“ZALL.repl … runs 787 assertions green”* as the proof of deploy readiness. That is a true statement about deployment order — ZALL does deploy the whole stack fresh — and it is 787 of what was then a much larger suite. **Deploy-order proof and coverage proof are different claims and ZALL only carries the first one well.**

---

5. The entrypoint surface, independently reproduced

REPL-TEST-LEDGER.md is generated by REPL/tools/_test_ledger.py. Running that generator on 2026-09-18 reports:

metric	value
client entrypoints (the auditable contract)	**448**
exercised at least once	**448 (100%)**
exercised by a file the gate runs	**448 (100%)**
exercised only in an ungated file	**0**
never exercised	**1** — P|A_AddIMP
exercised but with **no adversarial assertion in any of its blocks**	**170**
total invocations across the suite	3,649

[VERIFIED by command] — parsing every ENTITY|C_/CC_/A_/AA_/Cp_/CCp_ defun from the eleven Talos modules gives **482 definitions and 448 distinct names**, with 7 names defined in more than one Talos file. The ledger’s denominator reproduces exactly.

CORRECTED 2026-09-18. This table published **448 (100%)** exercised, **448 (100%)** gated and **289** without an adversarial assertion — and the sentence that used to close the section said REPL-TEST-LEDGER.md *“was last regenerated on 2026-09-14 and is not re-derived by the gate”*, then quoted that artefact’s figures as the current position anyway. Naming a staleness hazard and then relying on the stale number is worse than not naming it, because the caveat reads to a reviewer as though it had been applied. The figures above come from running the generator, not from reading the committed .md, and the committed .md has not been regenerated since. The 289 had fallen to 167 as the adversarial phase landed. The 100% is the subject of the rest of this section, because it did not fall for the reason the ledger gives.

G1 read 447 of 448, and the one that was missing was the instrument

> **REPAIRED 2026-09-18.** The one-line fix described at the end of this section has since been made. _test_ledger.py now accepts (rollback-tx) as a block terminator, and G1 reads **448 of 448 (100%)**, never-exercised **0**. The section is kept in full, in its original tense, because the diagnosis is the valuable part and because it is a worked example of the rule this book keeps returning to: **a coverage number is an instrument reading before it is a fact about the suite.** What changed was the instrument.
>
> One figure moved in the uncomfortable direction, and it is the reason the repair mattered beyond the headline. Entrypoints with **no adversarial assertion** in any of their blocks went from 167 to **170** — the newly-visible blocks brought three more operations into view whose neighbourhoods are asserted only positively. A blind spot does not only hide coverage; it hides gaps, and it hides them in the column that matters most for a security suite.

The single entrypoint the ledger reported as **never exercised** was P|A_AddIMP. It is worth the space, because the honest answer is not “an admin entrypoint shipped untested” — it is “the tool that owns G1 cannot see the test that covers it”.

What P|A_AddIMP is. It appends a guard to the m-policies list that P|UEV_IMC checks, which is the mechanism by which one Ouronet module is permitted to call into another — the inter-module boundary itself. Every Talos module defines its own copy. P|A_AddIMP is one of three names, with P|A_Add and P|A_Define, defined in **all eleven**, which is most of the gap between the 482 definitions and the 448 distinct names above. The ledger keys its rows by name, so eleven definitions collapse into one row, and that row is covered the moment any one copy is driven.

It is driven, twice, in a file the gate runs. REPL/modules/LAUNCHPAD.repl carries the blocks LPAD-N7 (:631-672) and LPAD-N7b (:676-702). The first calls TS02-CPAD.P|A_AddIMP as a stranger and asserts that the refusal names the **admin keyset** rather than the attacker's key — which is what separates "the admin gate refused" from "that signature was rejected". The second calls it as the admin and asserts the call completes, so the refusal above is the gate and not a function that is broken outright. That is a bidirectional pin on an admin door, and modules/LAUNCHPAD.repl is one of the 34 module testers in the gate's entrypoint list, so both run on every gate.

Why the ledger cannot see them. _test_ledger.py attributes invocations by transaction block, and it finds blocks with the regular expression \(begin-tx.*?\(commit-tx. Both LPAD-N7 blocks end on (rollback-tx), and LAUNCHPAD.repl's **last** (commit-tx) is at :441. Everything after that line — five blocks, :447-702, including both P|A_AddIMP call sites — matches no block at all, so it is never scanned. Nothing about the test is wrong; the parser stops reading before it.

[VERIFIED by command] — applying the same rule across the suite, the blind spot is not confined to one file:

non-archive/ .repl files with begin-tx blocks after their final (commit-tx)	**46**
blocks in them the ledger never scans	**350**
(expect…) forms inside those blocks	**898**

The error also runs the other way. A rollback-terminated block in the *middle* of a file is not skipped — the lazy match runs past it to the next (commit-tx), swallowing the following block, so the two pool their assertion counts and every operation in either is credited with the union. So the assertion columns are not merely the neighbourhood proxy the ledger honestly declares them to be: they are a neighbourhood proxy computed over blocks whose boundaries are sometimes wrong in both directions. And the invocation counts, which the ledger calls *exact*, are exact only over the part of the corpus it reads.

The irony is load-bearing. LPAD-N7's own header says it was written *because* an earlier enumeration found this exact function at zero coverage — *"FOUND BY COUNTING THE POPULATION, not by suspecting this function"* — and records why it had never been driven: both of its production call sites are commented out and labelled "Needed on Mainnet". The test was written, it runs, it is adversarial, it substantiates a claim RedTeam/[RT-G]_HostileCitizen.repl:91 had only asserted in prose. And the counter that motivated it still reads zero. **A coverage instrument that cannot see the fix it caused will keep asking for the fix.**

This was a defect in _test_ledger.py, not in the suite. The repair is one line — accept (rollback-tx) as a block terminator — and it was made on 2026-09-18, along with the tool's own prose, which described its attribution rule as (begin-tx … commit-tx) and so documented the bug as though it were the design.

Why the blind spot fell where it did is the part worth keeping. RULE 9 of this suite requires that *an expect-failure on a state-mutating operation gets its own (rollback-tx)*, because expect-failure does not roll back REPL writes. So the suite's own discipline guarantees that adversarial tests on mutating operations are exactly the blocks that end on rollback-tx — which were exactly the blocks the ledger could not see. The instrument was blind in precise anti-correlation with where the evidence is densest, and it had been for as long as both rules existed. Neither rule is wrong; nobody had put them side by side.

Reading the 167

Assertions are attributed by transaction block, credited to every operation invoked in the same (begin-tx … commit-tx). So the columns measure how well an operation's *neighbourhood* is asserted, not that an assertion

targets it. The pattern the ledger exists to surface sits at the top of its own gap table: **P|A_Define, invoked 58 times with zero positive and zero adversarial assertions in any block that calls it** — the deploy-time registration every module tester runs and none of them checks. Raw invocation counts hide exactly that.

CORRECTED 2026-09-18 — this paragraph named **ATS|C_ColdRecovery, invoked 273 times, with zero adversarial assertions**. It is no longer in the gap table: it now stands at 277 invocations with 21 adversarial assertions in its blocks. The example was retired by the work it was written to demand, which is the outcome the section wanted. The head of the table today is P|A_Define — the third member of the same eleven-times-defined policy family as P|A_AddIMP, and the one the section above does *not* let off: P|A_AddIMP has a bidirectional pin the instrument cannot see, and P|A_Define has 58 invocations and no assertion of any kind.

---

6. What the round found

DEFECT-LEDGER.md compiles **131 contract defects** for the REPL round, counted separately from the adversarial round for a stated reason: 131 is *compiled from project records*, while every red-team entry was *measured exploit-first*. **Merging them would put two standards of evidence behind one number.** The largest classes:

class	n
Guard reachability — mute, shadowed, dead	36
Pricing — preview versus charge	17
Client-facing diagnostics	14
Pricing — mispriced or unbilled operations	13
State machine, permanently-locked value	12
Arithmetic, lists, empty input	10

The two largest classes are **pricing** and guard **reachability** — not guard **absence**. The round's own summary of that distribution is the sentence worth carrying: *the guards in this system are present and they hold; what fails is the arithmetic around them, and the order in which things happen.*

Four findings illustrate what kind of testing produced them.

A vault that stopped paying and could not be restarted from any entrypoint. STOAICO::XI_CollectFor minted an account's urSTOA unconditionally and guarded only the *delivery* against a zero amount. A zero-amount mint is refused by UEV_Amount, and an account's entitlement is zeroed by its own first collect — so from its **second** distribution round onward the transaction aborted while real rewards were owed. All three exits share that core: self-collect aborted, the admin flush shared the same function, and a new round could not open while the unclaimed count was non-zero. The proof of the fix is 309.474016486404 wSTOA reaching a staker who could not previously be paid.

A defect no .repl could ever have caught. NOSFERATU::A_Fix01 addressed Legendary 1 100 where its twin addresses 1 70 — double-covering thirty positions and demanding a 100-row metadata list from a ladder built entirely of ≤70 rungs, where 70 is the per-transaction gas budget. The mint plan is **literals inside one-line wrappers**: a REPL can execute a rung but cannot read a literal, and nothing executed that family at all. Static property, static instrument — _ladder.py, now fatal in the gate.

A total failure that deleted its own evidence. AOZ::UR_NonFungible projected "sf-asset" — the *SemiFungible* column — from the NonFungibles table. Pact does not reject an unknown projected column; it returns {} and fails one call later as Key "sf-asset" not found in object: {}, which reads like a missing **row**. Because it failed for every input, the function was never usable, so no test was ever written against it: **the defect and the evidence for it**

disappeared together. The registry was write-only through its public interface. Swept afterwards: zero further instances, and the sweep was mutation-tested against the known one before its zero was trusted.

An operator instruction that was exactly backwards. GOV|MIGRATE requires the Global Administrative Pause to be **on**, and told the operator it must be **off**. One word; the logic was always right; anyone following the message would disarm the pause and retry forever.

Instruments built during the round

Each was built the moment a defect revealed a class, and **each was mutation-tested against the defect that motivated it** — the bug is reintroduced, the tool must report it, the bug is reverted:

tool	the class it makes impossible to reintroduce
_ladder.py	a minter batch ladder that mis-tiles its collection
_colproj.py	a projecting read naming a column its own table lacks
_info_measured.py	an INFO_ preview *named* by a test but never *measured* against a charge
_infostoa.py	a preview claiming STOA-free whose execution tree reaches STOA|C_Collect

A clean report from an unverified scanner is indistinguishable from a broken scanner.

---

7. Where the verification has holes

This is the section the chapter exists for.

7.1 The coverage instruments are snapshots, not gates

[VERIFIED by command]. Fifteen tools are run by the gate as fatal --check subprocesses (§3(b)). The instruments that produce this chapter's *coverage numbers* are not among them, because they have no --check to run:

instrument	the number it owns	gate-enforced?
_scale_report.py	G6 function reach (84%, 699 unreached)	**no**
_enforce_coverage.py	G2 guard pinning (live unpinned = 0)	**no**
_info_measured.py	the Phase-1.2 headline (428 / 441)	**no**
_expectfail.py	assertion strength (1,296 of 1,297 sites message-checked)	**no**
_docclaims.py	G5 documented claims	**no**
_cheapseam.py	the zero-fixture guard seam (exhausted)	**no**
_test_ledger.py	G1 (448 / 448; §5 for why it read 447 until 2026-09-18)	**no**
_infostoa.py	previews falsely claiming STOA-free (0)	**no**

CORRECTED 2026-09-18. Five of these rows carried figures that have since moved — they read **708 unreached**, **live unpinned = 1**, **401 / 401**, **1,190 message-checked** and **448 / 448** — and the table's point survives every correction intact, because it is the correction: **not one of these eight numbers is re-derived by the gate, so all five could drift without anything going red, and all five did.** Re-checked at the same time and unchanged:

none of the eight has gained a --check, and _gate.py names none of them as a subprocess, so the answer column still reads “no” eight times. _test_ledger.py’s figure moved for a reason the other four did not share — the tool is now returning a wrong answer (§5), which is precisely the failure an un-gated instrument cannot report about itself.

That is not an indictment — several of these print a census rather than assert a threshold, and there is nothing obvious for them to fail on. But it means the project’s own rule, written in the gate’s own comments, is **not satisfied for any of the six coverage gates**:

> *A number quoted in an audit document as evidence of a repair must be one the gate RE-DERIVES on every run. Otherwise it is a claim about the past, and the repair it certifies can be undone without anything going red.*

_conformance.py and _heavy.py were exactly here on 2026-09-13, and were wired in on 2026-09-14 once someone noticed their zeros were hand-measured. The same argument applies unchanged to the eight above. **Deleting every ignis-need assertion from launchpad-groundtruth.repl tomorrow would turn nothing red**, and the published figure would still read 428 / 441.

7.2 _figuresync locks the narrative to the stats file, not to the tree

_figuresync.py --check is fatal and is a genuinely good control: it prevents the narrative audit documents from carrying a figure the generated statistics do not support, deliberately scoped to **labelled table rows** so that frozen historical figures written in prose are not rewritten.

But its source of truth is the **committed** REPL_SUITE_STATS.md, which is regenerated by hand (_suite_stats.py, which the gate does not run). So a stale stats file propagates *consistently*, and consistency is what the check measures.

[VERIFIED by command] — the same counting rule _suite_stats.py uses, applied at the commit this chapter first measured and at the tree today:

distinct assertions at 1821d73 (2026-09-16, the stats commit this chapter measured)	**5,555** — exactly what the file published then
distinct assertions the stats file publishes now (fb58cba, 2026-09-18)	**6,619**
distinct assertions counted from the tree now	**6,619**

CORRECTED 2026-09-18. This table stopped at two rows and the section concluded that **the generated statistics are 261 assertions stale today, and nothing is red.** They are not stale today. The file was regenerated on 2026-09-18 and its distinct count now reproduces from the tree exactly, which is why the same macro can stand in both of the last two rows. The instance closed; the hazard did not, and the hazard is what the section is about. Regenerating remains a manual act — _suite_stats.py is still not run by the gate — so those two rows agree because somebody typed a command, not because anything would have gone red if nobody had. The figure this chapter quotes is a build-time macro read out of that same file, so the day it goes stale again, so does this sentence.

The contrast with Chapter 13’s _pricesync.py is the whole point: _pricesync **re-runs its generators in memory and diffs**, so its artefacts cannot go stale without the gate failing. _figuresync compares two committed files. *Two controls with the same intent, one closed loop and one open one.*

7.3 Two generators rewrite a tracked artefact on a bare invocation

[VERIFIED by reading] — _suite_stats.py:331 writes ARCHITECTURE/REPL_SUITE_STATS.md and _toolindex.py:46 writes REPL/TOOLS.md, in both cases with no --apply, --write or --check flag to gate it. Running either “to see what it says” rewrites a tracked file.

A third belongs on this list and this chapter did not have it: **_test_ledger.py writes the tracked REPL/_test_ledger.json on a bare invocation** (:183-184), with no flag either. Every figure in §5 comes from running it, so re-deriving this

chapter's own G1 numbers dirties the worktree. That is how it was done, and the file was restored with git checkout -- afterwards.

These are generators and regenerating is the intended action, so this is a much smaller hazard than the five _fvt* tools that rewrote **hand-written contract source** at import and silently deleted 111 lines of schemas. But the repository's own standing rule — *"never run a tool to find out what it does; read its docstring, or check the table in REPL/TOOLS.md"* — is weaker than it looks while two of the tools it names write by default.

REPL/TOOLS.md itself: **[VERIFIED by command]** it indexes **50** scripts and REPL/tools/ holds **53** _*.py files.

CORRECTED 2026-09-18: the second figure read **51**, so the gap this sentence exists to point at was reported as one tool and is three. **[VERIFIED by command]** the three REPL/tools/_*.py absent from the index are _auditbook.py, _booktables.py and _modref.py. _toolindex.py globs every _*.py in the directory and prints "(no docstring)" rather than skipping, so this is not selective omission — it is simply that TOOLS.md has not been regenerated since those three landed, which is the §7.2 open-loop shape again in a second artefact. What makes it worth a sentence rather than a shrug: all three are gate-fatal --check tools (§3(b)), and TOOLS.md is the document CLAUDE.md tells you to read *instead of* running a tool to find out what it does. The index that replaced the dangerous behaviour does not list the newest checks. _suite_stats.py:319 is corrected to :331 above; the write moved, the absence of a flag did not.

7.4 Phase 1.5's own steps, checked against the tree

step, and what it asked for	measured position
1.5.1.1 CI-gate the ground-truth harness; extend it to cover **every** INFO function	the harnesses exist and **are** gate entrypoints (launchpad-groundtruth.repl, triplet-collect-golden.repl, modules/DEFPACT-BILLING.repl, the deb-staleness-* drivers), and 428 of 441 client-facing previews are measured, none unmeasured. **The coverage instrument that says so is still not gated** — re-checked 2026-09-18: _info_measured.py has no --check and _gate.py does not run it. The *denominator* half of this row has closed: the tool used to enumerate its subject from a hardcoded three-file list, which omitted 14 previews of which one was genuinely unmeasured; it now discovers every ClientInfo-returning INFO_ preview across 1_SOVEREIGN/ and 2_CITIZEN/, and that one preview is measured (Chapter 12 §6)
1.5.1.2 repo-wide REPL coverage completion	**substantially done** — G1 at 448 of 448 entrypoints exercised, and since 2026-09-18 the ledger can demonstrate all 448 (§5); zero entrypoints reachable only from an ungated file; **G2's live unpinned worklist now at 0**; G5 closed. **G6 open at 84%, 699 functions unreached**
1.5.1.3 single comprehensive run, one boot, all Pact code	**met by a different mechanism.** The one-boot runner carries 25% of the distinct assertions; the comprehensive run is the 94-way parallel gate

CORRECTED 2026-09-18. Row 1.5.1.1 said *"401 previews are measured"* and that the tool's denominator omitted fourteen; both halves are superseded, and the half that mattered — the instrument is not gated — was re-checked and still holds. Row 1.5.1.2 said *"G1 at 448/448"* and *"G2's live unpinned worklist at 1"* and *"G6 open at 84%, 708 unreached"*. The G2 worklist has reached zero, which is the only one of the three that is a completed piece of work.

7.5 And the finding that qualifies every green assertion in this Part

It comes from the adversarial round, and Part III states it in full, but it belongs here because it bounds what Chapter 15's numbers certify. Four red-team attacks were **refused by the wrong guard** — RT-E-001, RT-H-001, RT-C-001 and RT-E-002 — and RT-H-001 produced two of them, so there are five rows:

attack	the guard that ought to refuse	the guard that actually did
RT-E-001 vault dust sweep	a claimant-set check	a last-collected-round stamp **in a different function**
RT-H-001 swap a token for itself	output-id NOT IN input-ids	the **curve returning exactly zero**
RT-H-001 duplicate swap input	a uniqueness check	a **stable-pool single-input rule**
RT-C-001 treasury-debt wipe	GOV|DPTF_ADMIN	a **solvency check one line above it**
RT-E-002 continue a defpact you did not start	a continuation-authority check	the step **happening to move the starter's own tokens**

Every one of them was green, and would have stayed green through the change that breaks it. The first four have since been repaired. RT-E-002 deliberately has not — giving the defpact layer a continuation-authority check is a design decision about who may drive a multi-step operation, and it would touch every MTX| pact — so it is recorded instead as a standing constraint: *if a step does not move the starter's own assets, it has no caller authentication at all.*

CORRECTED 2026-09-18. This read **"Three of the red team's six clean refusals were by the wrong guard"** above a table of four rows, a three-against-four mismatch inherited verbatim from RED-TEAM-REPORT.md:1657. Both numbers were wrong in a different way. The denominator: the register today records 38 attacks, 18 of them refused, so "six" is the programme's size at an earlier stage and not a live figure. The numerator: RED-TEAM-REPORT.md:892 records a **fourth** wrong-guard refusal, RT-E-002, found on 2026-09-15 and never added here — it is the fifth row above, and the only one still open. The three-versus-four was never an error, only an unexplained one: it counted attacks while the table listed findings, and RT-H-001 yielded two. Deliberately **not** written as "four of 18": the register marks RT-H-001 FIXED rather than REFUSED, because the same attack also found a real defect, so the four are not a subset of the refused count and quoting them as a fraction of it would be false.

> The suite's 29,531 executed assertions — measured at build time; this book did not run the gate — establish that the system behaves as documented. They do not establish that it is defended for the reasons the documentation implies. Those are different claims, and only the first one has a number.

---

8. The methodology traps worth publishing

The round's report keeps a section for the mistakes made while measuring, and three transfer directly to anyone auditing a system like this.

A stale log answered for a run that never happened. A gate launched as cd REPL && … > log from a shell already inside REPL/ never started; a previous session's log at that path ended in GATE GREEN, and a trailing ; echo rc=$? reported the *echo's* success. The stale green hid a real failure for hours. The structural fix is one line, flushed first: the gate now prints GATE RUN STARTED <timestamp> (pid N) before doing anything. **A log without that header was never produced by a run.**

An exclusion list with a wrong reason is where coverage hides. One gate-excluded file was recorded as failing deep inside a probe module. It actually dies on its **first (load)** from a path bug and has never run at all — it was moved into Kursan/ and its REPL-root-relative loads were never fixed. The recorded reason would have sent the next person to debug something never reached. The exclusion now names the real cause and states why the coverage is not lost.

A worklist of “78 unreached `UEV_` guards” was carried across two sessions and was never real. It came from a tool measuring *static call-graph reachability*, whose own output says `calls to a ref- binding this tool could not resolve: 89`. Any guard invoked only as `(ref-M::UEV_X ...)` appears unreached even when a negative test drives it to abort. The tool that owns guard coverage reported live unpinned = 0 the whole time. **Two tools, two questions; the number that was easier to quote answered the wrong one.**

> The general form, which the round states better than this book could: the danger is not only that a count may be wrong, but that **the wrong tool can be authoritative-looking about a question it does not answer.**

---

16. Part III — The red team

> Roadmap §1.6.1. Source records: Audit/records/RED-TEAM-REPORT.md (the attack register), Audit/records/DEFECT-LEDGER.md (the defect record), and REPL/RedTeam/*.repl (the attacks themselves, which run in the gate).

What this round was

Parts I and II document audits that read the code and asked *"is this right?"*. This round asked a different question: **"can I make it do something it should not?"** — and answered it by executing attacks against a live REPL blockchain, not by reasoning about source.

That distinction is the whole point. Four separate times in this programme, a conclusion reached by reading the call chain turned out to be wrong when the operation was actually driven. Pact's let bindings are eager, so a value bound in a Talos wrapper is read **before** the with-capability that follows it — which means the check that actually refuses a bad input is routinely not the one the code reads like it would be. Every finding in this Part was reached by running something.

The register

38 attacks across 11 families. 20 found a defect. None succeeded.

"None succeeded" means no attack achieved its stated goal — no value was moved, no gate was bypassed, no privilege was escalated. It does **not** mean nothing was wrong: **20 of the 38** exposed a real defect on the way to being refused, and those are the substance of this Part.

That sentence read "19 of the 35" for several days after the round grew to 38 attacks, while the bold headline directly above it already said 38 and 20. A summary line and the prose under it are maintained by different reflexes, and only one of them was in the checker's scope. Both are now.

family	attacks	found a defect	refused cleanly
A — economics, MEV, pricing	5	3	2
B — permissionless reach	2	1	1
C — admin impersonation	1	0	1
D — ownership	7	1	6
E — sequencing	3	0	3
F — griefing	2	2	0
G — hostile citizen module	2	0	2
H — input domain	3	3	0
I — gas station	2	1	1
J — conservation of value	3	1	2
K — preview/exec parity	8	8	0

Two families were **invented during the round** rather than planned: **J** (does the protocol's own accounting balance?) and **K** (does the free preview agree with the charged execution?). K is the largest family in the round and every one of its eight attacks found something — which says more about where to look for defects in this kind of system than any of the planned families did.

Coverage against the plan

The round was specified before it began, naming eight attack surfaces. This is what each one actually received — stated as coverage, not as a claim of completeness.

planned surface	families	assessment
capability & auth bypass, composed caps	B, C, D, G	**the deepest** — plus the whole owner-gate programme, Chapter 19
sentinel / collision	H, D	**good** — three input-domain defects, all fixed
preview / execution divergence *(not on the plan)*	K	**the largest family; every one of its eight attacks found a defect**
economic & MEV — front-run, sandwich, ratio extremes	A	**good** — the sandwich attack exists and the AMM's floor now has a witness
arithmetic / rounding / precision	A, J	**adequate** — the share-price boundary and the supply-vs-balances split
defpact / Hydra-slice races	F	**adequate** — both attacks found defects, one of them a live money defect
cross-module boundary abuse	G, B	**adequate**
gas-station exploitation	I	**assessed as thin on an attack count of one — and that assessment was wrong.** modules/DALOS-ADMIN.repl already drives the gas capability through five blocks, including its spending ceiling. What was missing was not the cap but the arm beside it
ordering / reentrancy-like	E	**three attacks, all refused.** One of them documents a *structural absence* rather than a guard: nothing in the multi-transaction layer checks who is driving a recipe; safety today is a property of what each step happens to touch

Two of the nine rows were **not on the plan at all** and were invented during the round: **J** (does the protocol's own accounting balance?) and **K**. K is the largest family and had a 100% hit rate. A planned list of attack surfaces is a hypothesis about where defects live, and this one was wrong about the most productive surface in the system.

One of those "thin" rows was a mistake in the assessment, not in the coverage. Family I was called the round's weakest surface because it contained one attack. The gas capability was in fact already driven through

five blocks of a module suite, including its spending ceiling with a same-form control. **Counting attacks in a red-team family undercounts coverage that lives elsewhere** — and this is the second time in the round that "weakest family" turned out to mean "fewest attacks" rather than "least covered". The genuine gap, once looked for properly, was not the cap but the arm beside it: the cap is one branch of an `enforce-one` whose other branch is the admin guard, so the master keyset has no spending limit at all. Measured, and now pinned.

The remaining row was closed the same way — by counting, not by intuition. Asked which sequencing surface was least covered, the answer came from enumerating every multi-transaction recipe in the codebase and counting the test files that drive each. Ten of eleven had between three and seven. **One had none** — an *admin* entrypoint that push-collects other people's rewards, whose documentation made three claims and had never been executed by a test. Two of the three claims now hold under measurement; the third is untestable with the available fixture and is **stated as untested** rather than implied.

> Both of the round's last two findings came from counting a population rather than inspecting a candidate. Intuition kept pointing at surfaces that were already covered, and the thing with no coverage at all was not the thing that looked suspicious. Family E's thinness is qualified: its second attack establishes that the defpact layer performs **no** driver check, and that every step is currently safe only because it happens to move the starter's own tokens. A future step touching only protocol state would have nothing to demand the starter's key, and that attack would succeed against it without anything else changing.

The headline results

Two live money defects, both found and both fixed.

- **A 38.65% mispricing of OURO.** `URC_OuroPrimordialPrice` computed a weighted pool's price with the weights omitted. The function feeds the on-chain price oracle, the launchpad's payment conversion and the block explorer. Proven by controlled experiment: reserves held constant, the pool's weights varied through the live client path, and the flagged function returned a **bit-identical** value across three different weightings of the very pool it prices.
- **A fee charged before the gate that refuses.** `MTX|C_Issue` with the permissioned flag set commits **2,919.77 IGNIS + 459 STOA** in step 1, then refuses in step 2 at an admin gate the caller can never satisfy. Non-refundable; cancelling cleanly costs a further 53 IGNIS. The single-transaction twin of the same operation refuses for **zero**.

A swap that reported success while moving nothing. The AMM's slippage floor refuses by *returning* a payload rather than raising. One Talos wrapper indexed that payload out of range and raised a bare `Array index out of bounds`; its siblings indexed it *in* range and interpolated the refusal text into a sentence beginning **"Succesfully swapped"**, committed the transaction, and emitted a swap event. Zero in, zero out. The same defect — and the noisy one was the safe one.

A class of capability that was present, reached, and had never refused anybody. This is the largest single thread in the round and has its own chapter. Of the ownership-gated capabilities reachable from a named client operation, **only 19 had ever caused a refusal in any test when the round began** — not because the rest were absent, but because a business rule ran first and answered on their behalf. A gate in that position is indistinguishable from a deleted one, from the outside. By the round's end 90 had been observed to refuse somebody and 69 of those refusals were attributed to the gate itself rather than to something it composes; 98 remain unobserved, and Chapter 19 is about why that is reported as a bound rather than closed.

*This paragraph previously read "**19 had never caused a refusal**", which inverts the figure: 19 is the count that **had** one. The error survived because the sentence is true-sounding in either direction — a small number of gates in an alarming state is exactly what the passage is claiming — and nothing checked it against the table in Chapter 19 that it paraphrases.*

Verification state

At the time of writing, the full gate is **green at 29,531 assertions** (23,912 positive, 5,619 negative) across the whole system — every deploy stage, every scenario suite, every red-team attack, plus the static checks on generated artefacts, tool paths, prefix vocabulary, cross-module member resolution, assertion vacuity, eager-let shadows, and this book's own tables. Wall time ~5-7 minutes. Reproduction: Appendix 1.

> **That number is a snapshot and will move.** It rose from 22,939 to 29,531 during the round documented here. The canonical value is whatever ARCHITECTURE/REPL_SUITE_STATS.md holds, and _figuresync.py --check now verifies that file **against the tree** — it previously verified only that every document agreed with it, which is circular and was green while all of them were wrong together (DEFECT-LEDGER §8.22).

Chapters

01-METHOD.md	How an attack is built here, and the four ways one can be worthless while passing
02-OWNER-GATES.md	The shadowed-gate programme: 167 gates, what it took to witness them
03-DEFECTS.md	The twenty defects, by severity, each with the assertion that would go red
04-INSTRUMENTS.md	The measuring tools — and the defects found *in them*, which were worse

17. How the round was designed

Parts I and II were audits. They read the code and asked whether it was right. This round asked a different question, and the difference in question forced a different method, a different output format, and a different definition of success. This chapter is about those choices — why the round was shaped the way it was, and how it was physically built — before the next three chapters report what it found.

Why run one at all

The two prior rounds had already read every sovereign module. A reasonable objection to a red team at that point is that it can only re-find what careful reading already found.

That objection is wrong for one specific reason, and the reason is a property of Pact rather than of this codebase: **`let` bindings are eager**. A Pact function that opens

```
(let ((parent (UR_Parent id)))
  (with-capability (OWNERSHIP account)
    ...))
```

does not evaluate the capability first. It evaluates `(UR_Parent id)` first, and if that read fails — because the id does not exist, because the row is empty, because a sentinel came back — the transaction dies there, with that read's error, and the ownership gate below it is never reached.

The consequence for an auditor is severe: **the check that actually refuses a bad input is routinely not the check the code reads as though it would be.** You cannot determine which guard fired by reading the call chain. You can only determine it by driving the operation and looking at the message that comes back.

Four separate times in this programme a conclusion reached by reading was overturned by execution. That is the whole justification for the round. Everything in Parts III and IV was reached by running something.

Who the attacker is

A threat model that says "an attacker" and stops has not been written. This round used four distinct attacker classes, because they have genuinely different reach and the interesting defects sit at the boundaries between them:

class	holds	can do	cannot do
Stranger	a funded Ouronet account, nothing else	call any Talos wrapper, sign with their own key	present anybody else's signature
Neighbour	an account plus a legitimate position (LP tokens, a stake, a nonce)	everything a stranger can, plus every operation their own assets authorise	act on assets they do not hold
Citizen author	the ability to deploy a module into `ouronet-ns`	call sovereign entrypoints directly from module code, discard return values, compose capabilities in their own module	acquire a sovereign `GOV|*` capability
Rogue operator	one of the admin keys	everything that key gates	act outside that key's gate, or act in another admin's band

The class matters because using the wrong one is the commonest way to produce a test that looks like a security proof and is not. An attack driven by a stranger against a door that was never reachable by a stranger proves nothing about the door — it proves the stranger could not get to it, which was already known. Chapter 18 calls this *the wrong attacker* and treats it as one of the five ways an attack can be worthless.

The **citizen author** class deserves particular emphasis, because it is the class most systems do not have. Ouronet's namespace is open: anybody may deploy a module alongside the sovereign ones, and that module's code runs with whatever the namespace permits. A citizen module is not a user — it is code, calling code, inside the same chain. Family G exists entirely to ask what that buys an attacker.

How the families were chosen

Eight attack surfaces were named in the round's specification before any attack was written. The point of naming them in advance was to make coverage a checkable claim rather than a feeling: at the end, each named surface either received attacks or visibly did not, and the gaps are stated as gaps.

The eight planned surfaces became families **A** through **I**. Two more families were **invented during the round**, which is worth recording honestly because it says something about where defects in this kind of system actually live:

- **J — ledger conservation.** The question "does the protocol's own accounting balance?" was not on the list. It arose from noticing that a token's `supply` column and the sum of its account balances are maintained by two *separate write paths*, with nothing structural forcing them to agree. That is not an attack surface in the usual sense; nobody signs a transaction called "desynchronise the supply". It is an invariant, and invariants are the thing a red team is best positioned to test because it can drive the system into states a unit test would not construct.
- **K — preview/execution divergence.** Ouronet publishes a free cost preview for every charged operation, so a client can quote a price before committing. Cost parity between preview and execution was already proven for the entire population. **Refusal parity was proven for none of it.** The question "when the operation would fail, does the preview also fail?" had never been asked. Family K asked it eight times and found something every time.

K became the largest family in the round, and its hit rate — 8 of 8 — is the single most useful methodological result here. It did not come from suspicion about any particular function. It came from **counting the population and noticing an entire class of behaviour that had no coverage at all**. Chapter 18's discussion of acceptance criteria is downstream of this.

What counts as a finished attack

An attack was not considered done when it produced a refusal. It was considered done when it produced a refusal **that could only have come from the guard under test**. Those are very different bars, and the gap between them is where most of this round's effort actually went.

The house technique for closing that gap is the **differential pair**: run the operation twice, holding every input constant except the one the guard is supposed to care about, and pin *both* resulting messages. If the guard is deleted, the two messages become identical and the test fails. If only the refusal is pinned, deleting the guard leaves some other check to refuse the same input with a different message — and the test passes while proving nothing.

The specific hazard this defends against has a name in this project: a **shadowed gate**. When a `defcap` both authorises ("are you allowed?") and validates ("is this a sensible request?"), only the first one to run is visible from outside. If the business check runs first and the normal state of the world fails it, then *every* caller — admin and stranger alike — is turned away by the business rule, and the authorisation gate beneath it is never exercised. A test can truthfully report "a non-admin was refused" while the ownership capability it claims to be

testing has been deleted. **A shadowed gate is indistinguishable from an absent one from the outside.** Chapter 19 is the programme that went looking for them.

The exploit-first rule

Where an attack found a defect, the defect was **measured before it was repaired**, by executing the exploit and recording what it actually cost or moved. Only then was the fix written.

This ordering is not ceremony. Three claims in this round would have been published wrong without it:

- The lp-churn deterrent discrepancy was *suspected* at some large multiple. Measured, it was **10.5×** — the defpact door charged 53.00 where the single-transaction door charged 557.03 net for the same operation on the same pool. A fix written from the suspicion would have been correct but the finding would have carried a fabricated number.
- The MTX|C_Issue gate-after-payment defect was measured as **2,919.77 IGNIS + 459.0 STOA committed before the refusal**, plus a further 53.00 to roll back. That figure is the finding; "a fee is charged before the gate" without it is an assertion.
- A suspected **10,000× underpricing** was very nearly filed. Execution showed all three readings were identical at 250.0 — the executor sets one value at one line and re-seeds from the deterrent table twenty lines later. Reading found the first line. Only running found the second.

The rule generalises: **locate by execution, not by reading.**

How it was physically built

The round produces no separate report artefact, and that is deliberate. A red-team report is a document that was true on the day it was written. What this round produces instead is **executable tests that live in the permanent suite**.

Each attack is a block inside a .repl file under REPL/RedTeam/, opening with a structured header:

```
;;<<RT-F-002>> FAMILY: F | STATUS: FIXED
;;HYPOTHESIS: ...what was suspected, stated so it could be wrong
;;METHOD:     ...how the operation was actually driven
;;RESULT:     ...what happened, with measured figures, and what was changed
;;
;;...free prose: why this is a real finding and not a preference, what was
;;   originally believed, which ledger entry records it
```

Three properties follow from that layout, and each was chosen:

1. **The attacks run in the gate.** They are not archived evidence; they are regression tests. A future change that reopens RT-F-002 fails the build. The finding is protected by the same machinery as the fix.
2. **The register is generated, not maintained.** REPL/tools/_redteam.py walks every .repl in the tree, parses these headers, and emits the family counts and the per-attack list. Chapter 21 of this book *is that output*. There is no hand-kept list of attacks that can disagree with the attacks — a category of error this programme found three times in its own instruments and once in its own published figures.
3. **The header format is checked.** A malformed or duplicated header is a gate failure, not a silent omission. This was learned the hard way: quoting another block's tag as ;;<<RT-E-001>> at the start of a line inside a comment caused the register to read it as a real header and the gate to go red. The lesson — that a generated register makes prose about tags dangerous — is itself now in the ledger.

The scan deliberately covers **all** .repl files rather than only REPL/RedTeam/, and reports any attack registered from outside that directory by name. One is: RT-K-008 lives in Stage_02/[6.3]_STOAICO.repl, because it belongs with the suite that sets up the state it needs. The convention is that attacks live in RedTeam/; the tool's job is to make every exception visible rather than to enforce a tidiness that would push a test away from its fixture.

What “none succeeded” means, and what it does not

The headline result is that **no attack achieved its stated goal**. No value was moved that should not have moved, no gate was bypassed, no privilege was escalated, no accounting invariant was broken in a way that survived the transaction.

That is a real result and it is also a narrow one. It does not mean the system was found to be without defect — twenty of the thirty-eight attacks exposed a genuine defect on the way to being refused, two of them involving live money, and those are the substance of the next chapters. It means that in every case the system’s *final* answer was correct, even where the route to that answer charged the caller first, reported the wrong thing, or refused for a reason unrelated to the one the code appeared to give.

It should also be read against its coverage rather than in the abstract. Thirty-eight attacks across eleven families is a bounded effort against a system of this size. The claim is “these thirty-eight did not succeed”, and the register in Chapter 21 exists precisely so that the reader can see the shape of what was tried and judge the shape of what was not.

---

18. Method: building an attack that proves something

A red-team assertion in this codebase looks trivial:

```
(expect-failure "a non-owner cannot move someone else's tokens"
  "Keyset failure (keys-all): [PK_Emma...]"
  (ref-TS01-C1::DPOF|C_Transfer attacker token [1] victim attacker false))
```

It passes. It is green in the gate. And on its own it may prove **nothing at all**. This chapter is about the difference, because that difference is where almost all of this round's effort went, and every rule below was learned by producing the worthless version first.

Failure 1 — Vacuity: the refusal came from somewhere else

The dominant failure mode. A capability is written like this:

```
(defcap DPDC-S|C>RENAME (...)
  (enforce (< (length new-name) 32) "Name too long")   ;; business rule
  (CAP_Owner set-id)                                   ;; the ownership gate
  ...)
```

An attacker who hands it a bad name is refused **by the length rule**. The test is green. The attacker was stopped. And **if you deleted `CAP_Owner` entirely, the test would still pass.**

A gate in that position is called *shadowed*. From outside, a shadowed gate is **indistinguishable from an absent one** — which is the entire problem, because the outside is where tests live.

The remedy is not to reorder the capability. Reordering only moves which check is hidden, and a scripted reorder during this programme silently converted an unconditional admin gate into a conditional one by hoisting a `compose-capability` out of the `if` that guarded it. **The remedy is a fixture that satisfies the earlier checks**, so the gate is the only thing left that can refuse.

The differential pair

The house form, adopted after `DPOF|C>DEBIT`. Hold everything constant except the one input that clears the shadow, and pin **both** messages:

call	amount	refused by	message
shadow	999999.0	the amount-vs-supply fold	Cannot Debit into the Negatives ...
gate	10.0	CAP_EnforceAccountOwnership	Keyset failure (keys-all): [PK_Ancie...]

Same signer, same sender, same nonce. The pair proves **two things at once**: that the shadow is real, and that the fixture escaped it. A lone `expect-failure` on the ownership message proves only that *something* refused.

Failure 2 — Unattributability: the refusal is real but anonymous

`ATS|C>ADD-REWARD-TOKEN` carries **two** ownership checks in sequence — the reward token's, then the pool's. Every token and every pool in the fixtures is owned by the same account, so both checks fail with the *identical* message. An assertion on that message is green, and **survives deleting the second gate**.

No static analysis can see this: both checks are present, both are reached, the test is green. The only remedy is a fixture that **separates** them — here, giving the attacker a token she genuinely owns, so the first gate passes by construction and the message can only be the second.

> This is a third distinct way a gate goes dark, after ordering and after gaps in the reachability map: **two guards that refuse with the same words.**

Failure 3 — No non-vacuity control: you proved a refusal, not a rule

expect-failure proves a call was refused. It does not prove the operation *works* when it should. A wrapper that returned an error unconditionally would satisfy every attack in this round.

So every attack carries a control: **the identical call, with the rightful party signing, must succeed** — and the success is asserted on *state*, not on the returned string. VST-G1b does not check that a message says "succeeded"; it checks that 999,250.0 tokens left one account and arrived at another.

Sometimes a full success is out of reach and the control has to be a **message differential** instead: add exactly one signature and show the refusal moves to a *downstream* check. That proves the gate was satisfied, which is the claim that matters. ATSU-G1 uses three calls that land short of the gate, on it, and past it.

Failure 4 — The wrong attacker

In these fixtures the account ANHD is simultaneously an ordinary user key, a member of the Demiurgoi master keyset, and the owner of most assets under test. An attack signed by ANHD therefore proves nothing: a refusal might be ownership or might be something else, and a *success* might mean "the gate is broken" or "they legitimately own it".

The default rule is to pick an attacker who demonstrably owns nothing relevant, and to **assert that in the test** rather than assume it.

But the rule is a default, not a law, and ATSU|C>COLD_RECOVERY inverts it. There, an ordinary stranger never reaches the gate at all — a broader permission check over five guards turns them away first. The only caller who can reach it is one who passes that broader check while not being the target, and the Demiurgoi master key is exactly that caller. The attack is *"the most privileged key in the system still cannot act on somebody else's behalf"*, and no lesser key gets far enough to ask.

> **Which account should attack is a property of the gate, not a house rule.**

A fifth rule, about what you are allowed to pin

When an operation is refused by something **wrong** — an index fault where a written guard should have spoken — the temptation is to pin the fault text, because it is what happens.

Do not. Pinning a defect makes it the expected behaviour and turns its eventual **repair** red. The correct move is to pin only what is legitimately true (*the call is refused and nothing moves*), record the fault as a defect, and leave the message unpinned until it is right.

This is not hypothetical: RT-A-004 was written that way, the defect it flagged was root-caused and fixed the same week, and the assertion was then rewritten to pin the guard's own words — at which point it also had to start *measuring* that nothing moved, because the fixed call **commits** instead of reverting and transaction rollback no longer guaranteed it.

Failure 5 — The acceptance criterion a defect satisfies

The four failures above are about the *test*. This one is about what the test was asked to prove, and it is the most expensive mistake in the record because **the test was correct and green.**

An earlier audit round found that a reward vault's "last claimant takes the remaining dust" branch tested the wrong condition. It wrote a fix, and it wrote down how the fix would be verified [VERIFIED by reading, ROUND-02-FIXES.md:437]:

> *"…collects →**both the global and member vaults drain to exactly 0.0**. This proves…"*

That is a reasonable criterion. Conservation of value is exactly what you want from a vault. The test was written, it passed, and the finding was closed.

The defect satisfied it. The fix had been built by porting the canonical token-ledger vault and dropping two of its guards, and the surviving branch let a **fully-exited** account collect the whole vault while the rightful sole claimant received 0.0. The vault still drained to exactly zero. Value was still conserved. Every unit that went in came out — to the wrong account.

Found nearly a month later, by a red-team attack asking a different question: not *"does the vault balance?"* but *"who received it?"*

> **"Nothing was lost" and "the right person got it" are different claims, and only one of them was being checked.** A conservation criterion cannot distinguish a correct payout from a theft that happens to balance.

The repaired branch now carries (> deb-user 0.0) — *"still in the claimant set"* — and the source comment records why the predicate is that one: it is the same predicate the collect path itself uses to remove a caller from the set, so the reader and the counter finally agree.

The generalisable rule is not "write better tests". It is:

> **When you write down how a fix will be verified, ask what else would satisfy it.** If a defect can, the criterion is measuring a property of the system rather than the correctness of the change.

What this costs

Roughly: for every line of attack, three or four lines of fixture and control, and a comment explaining why that attacker and that input. The attacks in REPL/RedTeam/ are mostly comment.

That ratio is the finding. A red-team suite that is mostly assertions is probably mostly vacuous.

---

19. The ownership-gate programme

This is the largest single thread of the round, and the one with the least dramatic finding: no capability was **missing**. Every ownership check the architecture called for was present in the source, and every one of them was reachable from a live client path.

What the round established is that **presence is not protection**, and that the difference is invisible to every form of review that does not execute the code.

The question

Ouronet gates asset operations with capabilities of the form:

```
(defcap SOME|C>OPERATION (...)
  ...business checks...
  (CAP_EnforceAccountOwnership account)   ;; or (CAP_Owner asset-id)
  ...)
```

A reviewer reading that file sees the gate and moves on. The question this round asked instead was: **has this capability ever actually turned anybody away?** Not *is it there* — has it **fired**.

For a large fraction of them, the answer was no. And a capability that has never refused anybody is one whose deletion would turn nothing red.

The measurement, and the four times it was wrong

The instrument is REPL/tools/_ownerobs.py. Its history is more instructive than its output, and it is documented here rather than hidden because **every correction moved the number in the direction of more work, not less.**

It began by excluding a third of the tree from its own denominator

The first version mapped each Talos wrapper to the function it called, one hop, and asked which of those functions acquired a gated capability. It reported *“39 of 112 witnessed”*.

The tree contains **185** such capabilities. Seventy-three never entered the denominator — and an excluded capability is not reported as a gap, it is **absent**, which reads as neither.

The excluded third was not a random third. It held the entire token-DEBIT layer — DPTF|C>DEBIT, DPOF|C>DEBIT, DPDC-C|C>SINGLE-DEBIT — and DPTF|C>X-TRANSFER: the gates that stop a stranger moving somebody else’s tokens. DPTF|C>DEBIT is the gate an existing red-team attack was **written to witness**. The tool could not see the thing its own test proved.

The chain broke at a **non-gated intermediary**: C_Transfer acquires DPTF|C>CLASS-1-TRANSFER, which carries no ownership check itself and merely *composes* DPTF|C>X-TRANSFER, which carries the sender check. Filtering to gated capabilities *first* severed the transfer family at exactly that link. Making the walk transitive — following both call edges and compose-capability edges, and filtering at the end rather than at each hop — moved the denominator **112 → 167** and the actionable list **9 → 23**.

> The fourteen extra gates were never closed. They were never **visible**. Several status reports had overstated progress for exactly that reason.

Then it credited gates that no test had targeted

Transitive reachability makes observed an **upper bound**: one refused call credits every gate on its path, though only one of them refused. This was written down as a caveat, and then demonstrated the same day — a new attack driving a non-owner ortofungible transfer changed **no counts**, because the gate it targeted was already credited by some other operation that merely passed through it.

Quantifying that took three attempts, and the first two were wrong in the same direction: toward a tidier number.

attempt	rule	verdict	why it was wrong
binary	attributable only if the crediting op reaches exactly one gate	**1 of 63**	read as *"the observed column is worthless"*; wrong axis
dilution	rank by how many gates the crediting op reaches	30 of 63 well-evidenced	ranks a test by blast radius, not by target — it filed a regression test written *specifically* for a gate as near-worthless because that gate's operation happens to touch ten others
depth	how many hops from the operation the test actually called	**63 attributed at depth 0**	—

Depth also had to stop returning the *first* crediting test found: DPTF|C>DEBIT was being reported at depth 3 via an unrelated fee-withdrawal op while the attack written for it credited it at depth 0. **A metric that ranks evidence must not pick its evidence by file iteration order.**

The depth split then paid for itself immediately. Six gates sat in the *observed* column, looking covered, credited from four hops away, with no test having ever targeted them — including DPOF|C>BULK-TRANSFER and DPDC-T|C>BULK-TRANSFER, each **the plural door beside a witnessed singular one**. None was on the actionable list. Without the split, none would have been written.

Then it generated impossible work

Some gates cannot be witnessed by any client-surface test: those composed only by another capability, or acquired solely through an XE_ forward-module entrypoint, which by this codebase's conventions is called by another *module* and never by a client. The outer gate refuses first, by design.

Splitting those out prevented seven impossible tasks. **And then the split itself was wrong twice**, in opposite directions — first treating compose-capability as a wall rather than an edge (which would have retired the live DEBIT layer as unreachable), then testing four of the **eight** client prefixes the architecture documents, which filed an entire batch-operation band as unreachable although its entrypoints are client recipes by definition.

> A prefix filter that disagrees with the documented prefix table fails silently and **always toward less work** — the direction nobody audits, because a shrinking worklist reads as progress.

And it had a blind spot the codebase's own style rule would trigger

The detector required the expected refusal message to appear *lexically inside* the assertion form. Hoisting a repeated message literal into the enclosing let — which changes nothing about what the test asserts, and which this codebase's style guide **actively encourages** — made an already-witnessed gate report as never observed.

Found by controlled mutation, not by reading. Fixed by widening the search to enclosing binding lists, *asymmetrically*: only the message search widened, because widening the operation-name search too would have re-opened the over-crediting hole. **Trading a blind spot for a false positive is not an improvement; it only moves which column lies.**

Where it ended

	at the round's start	at the round's end
ownership-gated capabilities in the tree	185	185
…reachable from a named client operation	112 *(as then measured)*	**167**
…never observed to refuse anybody	93	**98**
…with a test that caused them to refuse	19	**90**
…attributed at depth 0 (the test targeted *this* gate)	—	**69**
shadowed **and** never witnessed — **the worklist**	23	**0**

The worklist is empty. Four capabilities remain unwitnessed and all four are *structurally inner* — three forward-module entrypoints called only by another module, and one reached solely through a both-internal-and-external function. No client-surface test can attribute a refusal to those: the outer gate refuses first, by design. They are named rather than counted as a gap.

The 18 capabilities outside the denominator are named rather than buried, and the exclusion has since been **verified rather than assumed**. **14 are in the legacy `00_DPMF` module**; the rest are four special-band capabilities in STOAICO, DPDC and RPS.

00_DPMF's exclusion was the open question, since it carries most of the 18. Measured: **zero** modref bindings to its interface anywhere outside itself, **zero** ref-DPMF:: call sites, **zero** Talos wrappers reaching it — and it **is** deployed, at 128,156 gas.

> It is deployed and nothing **can** call it. Not "nothing does" — there is no binding through which any module could reach it, so it is unreachable from the only supported client path. Witnessing a gate on a module no caller can reach would test nothing, which makes those 14 a **correct exclusion rather than a gap**. The module's deploy-slot cost is real and is a separate question, belonging to the redeploy phase.

The sharpest attribution proof came last

The final band produced the cleanest separation technique in the programme, and it is worth stating because it generalises.

DPDC-C|C>REGISTER-NONCES is guarded on the holder of a **transferable role**, not on the collection owner. In the fixtures those are the same account — so the refusal names a key that is true of *either* reading, and attributes nothing. The two candidate explanations are indistinguishable.

The remedy was not a better assertion. It was to **move the role**: the owner legitimately transfers it to a third party, leaving the owner konto untouched, and then the owner — **still the owner** — is refused, naming the new role-holder's key.

An owner gate cannot refuse the owner. One legitimate operation separates two readings that no amount of assertion-writing could.

What the gates turned out to be guarding

Witnessing forces you to find out what an operation actually does, which repeatedly turned out to be more than its name suggested.

- **VST|C>REPURPOSE-TRUE-FUNGIBLE** reads as a bookkeeping operation. Driven successfully it wipes a holder's **entire balance** — 999,250 tokens in the fixture — and re-mints it to an address the caller names, with the emptied account never consenting. It authorises on the *frozen variant's* owner while operating on the *base* token: coherent, since confiscation is a power of the freeze authority rather than the token's owner, but it means the key that matters is never the holder's.
- **VST|X>REPURPOSE-ORTO-FUNGIBLE**: the attacker chosen was the victim's own asset holder. She holds the nonces outright and is still refused, because **holding the asset is not authority over it** — repurposing decommissions a nonce and mints a replacement, an issuance power wearing a transfer's clothes.
- **DPOF|C>DEBIT** could only be reached through a partial-transfer path that requires a token property the obvious fixture did not have. The first attempt died at that property check — a green assertion that said nothing whatever about ownership.

The one field that cannot be gated, and why we are saying so

Every finding above is a gate that was missing, misplaced or shadowed — all of them fixable, and all of them fixed. This section is the opposite: a property the chain **cannot** enforce, stated plainly because an audit that only lists solved problems is not telling you where the edges are.

What the field is

A DALOS account row carries a public field. It is **not** the account's Kadena signing key. The Kadena key is 64 hex characters and lives inside the account's guard; public is a **574–576 character** string over a 50-symbol alphabet with a 9G. / 9H. version prefix — the public half of an Ouronet keypair of its own, against which the client verifies **Schnorr signatures** made with the account's private half. A wrong string does not weaken that verification; it breaks it.

It is written once, at deployment, by whoever calls the deploy entrypoint. Nothing reads it back for validation. The only path that can change it afterwards is DALOS::A_UpdatePublicKey, which is GOV|DALOS_ADMIN-only **by design** — there is deliberately no client path, because the field is account identity rather than account preference.

Why the chain cannot check it

Three independent routes were examined, and each is closed:

1. **Verify a signature under the claimed key.** Pact can enforce Kadena ed25519 and WebAuthn signatures and nothing else. There is no native that verifies a signature under a caller-supplied foreign keypair, so the account cannot be made to prove it holds the private half.
2. **Derive it from the guard.** create-principal does reach a key — this codebase already uses it in UC_GuardProtocol to classify guards as k: / w: / r: / u: / c: / m: / p:. But the key it reaches is the **Kadena** key. public is a different keypair with no arithmetic relationship to it, so the principal tells you nothing about the field.
3. **Derive it from the account name.** Ouronet account names are 162 characters, Ѻ. or Σ. plus a 160-glyph body, and GLYPH|UEV_DalosAccount validates exactly that. The name is not a hash or an encoding of public; the two are independent strings. There is no relationship to check.

So the property *"the public on this account is the public half of a keypair its owner holds"* is **unenforceable on this blockchain**. Not difficult — unavailable.

What that means in practice

Through the Ouronet UI the field is correct by construction: the client generates the keypair and supplies the public half, and the user never types it. The exposure is the **console**, or any hand-built transaction: the deploy entrypoints accept any string, so an account can be created carrying a key nobody holds, or — the sharper case — a **well-formed key belonging to somebody else**, which would publish a recipient the account's owner cannot read.

The loss is borne by the account itself in the first case and by its correspondents in the second. No funds are reachable through it: `public` gates nothing on-chain, holds no authority, and is consulted by no capability — the Schnorr verification it serves happens off-chain, in the client. It is a *correctness* property of account identity, not an authorisation one — which is why the system stays safe without it and merely becomes wrong for that account.

The remedy exists and is the admin override. An account created with a bad key is repaired by `A_UpdatePublicKey`, and since that is the only writer, the repair is as authoritative as the original write. **This is accepted, not unresolved** — the owner's position, and the audit's: enforcement is impossible, detection is off-chain, and the correction path is present and exclusive.

The part that is NOT closed, and is offered as an open recommendation

The three routes above rule out proving **ownership** of the key. They do not rule out checking its **shape** — and the shape is regular: a fixed prefix, a narrow length band, a 50-symbol alphabet. The codebase already performs precisely this class of check on the field sitting beside it, in the same insert: `GLYPH|UEV_DalosAccount` enforces the account name's length, prefix, separator and charset. **`public` is the one account field admitted with no shape check at all.**

Adding one would not make the key provably the owner's — that remains impossible. It would collapse the accidental case (a truncated paste, a wrong field, an empty string, a Kadena key pasted where an Ouronet key belongs) from *silently permanent* to *refused at the door*, leaving only the deliberate substitution of a well-formed foreign key — which is self-harming in the ordinary case and, being deliberate, is the part no validation could have stopped anyway.

Recorded here as a recommendation rather than a change: it narrows the window, it does not close it, and the distinction is the whole point of this section.

---

20. The defects, and what was done

Every defect below was found by driving a live operation, was fixed, and carries the assertion tag that goes red if the fix is reverted. The tags run in the gate.

Ordered by what a user would lose, not by the order they were found.

Twenty of the thirty-eight attacks found a defect, and all twenty are below. One further defect — the OURO mispricing that opens Tier 1 — was found by the audit rounds rather than by an attack, and is included here because it was the most expensive single finding in the whole programme and belongs with the money.

*This chapter enumerated eighteen while its title said nineteen and the register said twenty, for several days, and the discrepancy was found by a machine and not by a reader. `_booktables.py` now checks that every attack the register marks FIXED is named somewhere in this chapter — a check written because the earlier version compared only the summary **table** against the register, and a table can agree with the register while the prose beneath it enumerates a different set.*

Tier 1 — Money

A 38.65% mispricing of OURO, feeding the oracle, the launchpad and the explorer

`URC_OuroPrimordialPrice` priced a **weighted** three-token pool with a flat `value / supply` ratio — the weights omitted entirely. Its sibling `URC_SingleOuroWorthWSTOA` had been repaired months earlier to perform a real weighted-pool swap; this one kept the pre-repair expression, and the two were called **three lines apart** in the launchpad's price block.

Proven by controlled experiment, not by reading. Reserves held constant, `stoa-pid` fixed, the pool's weights moved through the live client path:

pool weights [SSTOA, OURO, WSTOA]	flagged function	weight-aware sibling	error
[0.4, 0.4, 0.2]	0.092000677826921560374870	0.119981820629800000000000	−23.3%
[0.2, 0.6, 0.2]	**bit-identical**	0.179963642484000000000000	−48.9%
[0.3, 0.5, 0.2] — **genesis**	**bit-identical**	0.149973488864900000000000	**−38.65%**

The flagged output does not move by a single digit across three weightings of the very pool it prices. The −38.65% at genesis weights independently reproduces the ~38% gap measured when the sibling was repaired.

Blast radius: the on-chain OURO price oracle writes this value whenever it deviates by >0.1%, and that path is enabled at genesis; the launchpad converts dollar-denominated sale prices through it; the block explorer derives several published figures from it.

Fixed by delegating to the weight-aware path — no new arithmetic. **Pinned by `SWP-G27`**, which recomputes the *removed* formula inline and asserts it **disagrees** on live reserves. That makes the mutation test permanent: every run re-proves both that the guard discriminates and that the original defect was real, rather than resting on a measurement somebody took once.

> The repair changed **no existing assertion**. A 38% change to the token behind the oracle moved nothing in a suite of ~23,000 assertions — which is exactly how it shipped, and exactly how its repair could have regressed unnoticed.

A fee charged before the gate that refuses — RT-F-002

MTX|C_Issue with the permissioned flag set:

step	what happens
0	shape validation — **no authorisation check of any kind**; the permissioned flag actually *skips* a limit check, so the privileged path is the **laxer** one
1	**commits 2,919.77 IGNIS + 459.0 STOA**
2	admin gate → refused: MTX-SWP Ownership not verified

No refund, no cancel. Rolling back costs a further 53 IGNIS; abandoning leaves the transaction open forever, which is structural — Pact has no scheduled execution.

The contrast that makes it a defect: the single-transaction twin of the same operation refuses the same caller and charges **0 IGNIS / 0 STOA**, because it hoists its conditional admin gate above validation, exactly as this codebase's own authorisation-precedes-validation ruling requires. *Same logical operation, two live client doors, and only one charges you for a refusal* — and the flag that selects the expensive door is an undocumented boolean whose documentation never mentions it.

Three controls were required before this could be called a defect: a **differential** (the same drive with the flag off fails with a *different* message, so the refusal is attributable to the admin gate), a **non-vacuity** (the operation completes when the admin key signs), and the **cost contrast** above.

Fixed by hoisting the admin gate above the collection. **Pinned by RT-F-002.**

> This defect also **falsified something already written in the project's own defect ledger**, which had recorded this operation as the *specification* — "validation before money". That is true with the flag off and false with it on: *shape* validation precedes the money, *authorisation* follows it. The operation cited as the counter-example was carrying the unrepaired case.

A deterrent charged on one door and not the other — RT-A-001

Adding standard liquidity was reachable through **two** live client paths, and only one charged the lp-churn deterrent. A deterrent with a free door is not a deterrent.

A share price that could be gamed at the rounding boundary — RT-A-003

An autostake pool's index is a share price, floor(resident-sum / rbt-supply, p), and the deposit path inverts it. The two floor operations do not commute, leaving an exploitable edge.

Tier 2 — The protocol lying about what it did

A swap that reported success while moving nothing — RT-A-005

The AMM's slippage floor — the line the source itself calls *"the real protection this whole check exists for"* — **refuses by returning**, not by raising. It returns a one-element payload carrying the refusal text; the success arm returns four elements. Only the client wrapper turns that into anything a caller sees, and the wrappers did it differently:

door	index taken	on a refusal	result
smart swap	(at 3 out)	out of range on a 1-element list	Array index out of bounds. Length (1), Index (3)
single / multi swap	(at 0 out)	**valid** on a 1-element list	*silent*

door	index taken	on a refusal	result

Measured verbatim on shipped code:

> "Succesfully swapped input(s) to Expected Output of 2366.80… out of Slippage bounds min of 990000.0 - max of 1010000.0 OURO-…"

A sentence that **begins by claiming success and ends by reporting refusal**. The transaction commits. The swap event has already fired — it sits on the capability acquisition, ahead of the floor check — so an off-chain indexer sees a swap event and a success string for a swap that never happened. Zero in, zero out.

> **The same defect, and the louder one was the safer one.** The index fault destroyed the guard's message but stopped the transaction. The silent one let it through. The noisy variant was found first *only because it was noisy*; the dangerous one was found by asking what its siblings did.

The two repairs differ because the two payloads do: the smart-swap door can test length, the single/multi doors cannot — both payloads are one element — so they test the **type** (success is a decimal, refusal a string).

A documented field that did not hold what it said — RT-D-002

A collectable's nonce-holder field is documented as *"stores the account holding the nonce"*. It does not. Anything reading it as authoritative is reading a value that can be stale.

The preview and the execution disagreeing about refusal — RT-K-001 … RT-K-008

The largest family in the round: eight attacks, eight defects, on a surface nobody had swept.

Ouronet offers a free INFO_ preview for every priced operation. Cost parity — *does the preview quote what the operation charges?* — was proven for all 428 previews. **Refusal parity — does the preview refuse what the operation refuses? — was proven for none.**

Where a preview re-derives state rather than sharing the execution path's readers, the two can disagree, and they did: on bad inputs the execution refused cleanly while the preview died on a raw array-index fault, across **seven separate module families** — autostake, true fungibles, ortofungibles, collectables, swap pools, the citizen launchpad, and the acquisition-pool readers.

The launchpad case is the sharpest because its separating input is not a malformed id: the sale is a live object and the amount is simply one no sale can honour. A caller checking the preview before committing gets a crash where the operation would have given them a reason.

The eighth, RT-K-008, was found differently from the other seven and the difference is the methodological point of the whole family. The first seven came from suspicion about a surface. The eighth came from **counting the population**: INFO_Collect was the one client-facing cost preview in the entire tree named by no test at all, and it was identified by enumerating all of them and subtracting, not by anyone noticing it. A sweep that stops when the suspicious cases are exhausted stops before it reaches the case nobody suspected.

Tier 3 — Reach, sentinels and accounting

id	defect
RT-B-001	The inter-module gate, described as passing only for a caller that *is* a registered module, did not enforce that — undermining the claim that the orchestrator is the only supported path
RT-H-001	The capability guarding the **no-slippage** swap path

id	defect
	— the one a caller reaches by passing the `-1.0` sentinel — did not constrain what it should
RT-H-002	Slippage was unbounded **in both directions**; the documented `<= 50` limit was UI policy, not a contract rule, so a forged bound could bypass the check entirely
RT-H-003	A reader returns the `"|"` sentinel when an autostake pair has no hot reward token — **nine of fifteen live pairs** were in that state — and callers treated the sentinel as an id
RT-I-001	The gas station whitelists by reading the transaction's top-level forms; one case pinned a **single** form, so appending others changed what was actually executed under a matched whitelist
RT-J-001	A true fungible's `supply` and the sum of its account balances are written by **separate paths**, so the protocol's own accounting could diverge from itself
RT-F-001	The multi-transaction liquidity recipe collected its **entire deterrent in step 0**, then validated in step 1 that the pool had not moved since it quoted. A caller whose pool shifted between the two — which anybody else's transaction can cause — paid in full for an operation that then refused. The griefing shape is that the *attacker* controls whether the *victim's* already-paid transaction can complete

21. The full attack register

This chapter is **generated** from the attack files by REPL/tools/_auditbook.py. Every entry below is the header block of a test that runs in the gate – the same text the attack carries in source, not a summary written afterwards. If an attack is edited, retired or added, this chapter changes with it; it cannot describe a round that is no longer the one being run.

38 attacks across 11 families. 20 found a defect and it was fixed; 18 were refused by the system as designed. None succeeded.

Read *refused* carefully. It means the attack was turned away — which is the system working — and it is a weaker result than *fixed*, not a stronger one: a refusal proves the door held against one attempt, while a fix proves something was actually wrong and is now not. Several refusals below were nonetheless expensive to obtain, because establishing that a refusal came from the gate under test, rather than from an unrelated check upstream of it, is the hard part of this discipline. Chapter 18 is about that problem.

family	attacks	fixed	refused
A — Arithmetic and value	5	3	2
B — Permissionless reach	2	1	1
C — Admin impersonation	1	0	1
D — Ownership bypass	7	1	6
E — Sequencing and state	3	0	3
F — Griefing and denial of service	2	2	0
G — Hostile citizen module	2	0	2
H — Input domain	3	3	0
I — Gas-station payable surface	2	1	1
J — Ledger conservation	3	1	2
K — Preview/execution divergence	8	8	0
total	**38**	**20**	**18**

Family A — Arithmetic and value

RT-A-001 · FIXED

Runs in REPL/RedTeam/[RT-A]_Economics.repl

What was suspected. adding standard liquidity is reachable through TWO live Talos client paths, and one of them does not charge the lp-churn deterrent. If so, the deterrent is optional: a liquidity provider pays it only if they pick the more expensive door.

How it was driven. read the flat leg each route charges. The defpact's step 0 bills NOTHING except its flat fee – it quotes the pool, yields, and collects – so the patron's IGNIS delta across the starting transaction isolates that leg exactly. The single-tx route's deterrent is read from the price table it bills through.

What happened. CONFIRMED, then FIXED. Measured exploit-first: the defpact door charged 53.0 (a flat literal 100.0 at this patron's 0.53 discount) where the single-tx door's deterrent is 1051.0 raw / 557.03 net – a 10.5x saving on the same operation on the same pool. Repaired in 20_MTX-SWP.pact; the assertions below now pin that both doors read one price table. The same gap was fixed in the Frozen and Sleeping twins at the same time.

> WHY THIS IS A REAL FINDING AND NOT A PRICING PREFERENCE. lp-churn is a DETERRENT: its purpose is to make frequent add/remove cycling expensive, which protects the remaining liquidity providers from having the pool churned around them. A deterrent that can be declined is not a deterrent. And the cheaper door is the one the gas station subsidises. TS01-C3::SWP|C_AddLiquidity -> SWPLC -> UC_IgnisPrice "…" "lp-churn" = 1051.0 TS01-CP::SWP|C_AddStandardLiquidity -> MTX-SWP -> a literal = 100.0 Both are P|UEV_IMC-gated Talos clients, so both are the supported path; this is not a matter of calling core directly. lp-churn appears ZERO times in 20_MTX-SWP.pact. The earlier repair that put every add-liquidity op onto UC_IgnisPrice landed in 18_SWPLC.pact only, and nothing compared the two modules afterwards – the suite measured each route against ITS OWN preview, and both agreed with themselves. THAT IS THE GENERAL LESSON, and it is worth the paper: a preview-versus-charge check, however rigorous, cannot see this class. It asks "does this route quote what it charges", and both routes pass. The question nobody was asking is "do two routes to the same operation charge the same". ==== RT-A-001 - route around the lp-churn deterrent by picking the other Talos client · 01 · env-sigs (caps) ====

RT-A-002 · REFUSED

Runs in REPL/RedTeam/[RT-A]_Economics.repl

What was suspected. this looks exactly like RT-A-001 from the outside, which is why it was chased. MTX|C_AddLiquidity takes asymmetric-collection and gaseous-collection, and the CORE picks them per door – the client only picks the door: C_AddStandardLiquidity -> true true (asymmetry tax ON) C_AddIcedLiquidity -> false true (asymmetry tax OFF) C_AddGlacialLiquidity -> false false (neither tax) All three branches then call the IDENTICAL function with IDENTICAL arguments – ref-SWPL::XE_STOA-PID|AddLiquidity account swpair asymmetric-collection gaseous-collection stoa-pid ld clad – differing only in which capability is acquired. And inside SWPL the flag gates nothing but TAX COLLECTION. So: same operation, two prices, caller picks. That is RT-A-001's shape exactly. Worse, UEV_AddChilledLiquidity enforces "Chilled Liquidity can only be added when asymtric liquidity exists" – so THE TWO DOORS THAT REQUIRE ASYMMETRY ARE EXACTLY THE TWO THAT DO NOT TAX IT.

How it was driven. drive the SAME asymmetric input through both doors and compare the IGNIS charged, then look for what the cheaper door takes instead.

What happened. REFUSED. The tax is not skipped, it is EXCHANGED FOR A LOCK. Step 2 of the pact reads (if (not asymmetric-collection) (ref-VST::C_Freeze SWP|SC_NAME account lp-id secondary) …) – so the doors that skip the asymmetry tax hand back FROZEN LP, locked in VST, while the Standard door hands back free LP. The provider pays in liquidity instead of IGNIS. A coherent product trade-off, not a bypass.

> WHY THIS BLOCK EXISTS ANYWAY. The two consequences of one flag – "no tax" and "LP is frozen" – live FORTY LINES AND ONE DEFPACT STEP APART, in different capabilities. The price and the thing paid for it are not visible together anywhere. If a future change ever removed or conditioned that freeze, the bypass would become real and nothing would notice, because no test asserts the two are coupled. This one does. ==== RT-A-002 - the door that requires asymmetry is the one that does not tax it · 01 · env-sigs (caps) ====

RT-A-003 · FIXED

Runs in REPL/RedTeam/[RT-A]_Economics.repl

What was suspected. an ATS pool's index is a SHARE PRICE – floor(resident-sum / rbt-supply, p) – and URC_RBT inverts it to price a deposit: floor(rt-amount / index, p-rbt). This was entered looking for the classic vault-inflation attack (tiny genesis supply, then donate to inflate the price against later depositors). Audit finding #11M already bounded the KickStart RATIO to [0.1, 100.0] for exactly that reason, and the RBT's 24-decimal precision makes the remaining rounding loss (index x 10^-24 per coil) uneconomic to chase. But the same expression has a SINGULARITY the bound does not cover: index = 0, which is a reachable live state – any pool whose reward-bearing token has supply MINTED OUTSIDE THE POOL reads resident-sum 0 against a positive rbt-supply. Five such pools exist at deploy (the AOZ primal assets).

How it was driven. measure the state first rather than assume it, then drive the real client entrypoint TS01-C2::ATS|C_Coil into it, and pin the refusal BY MESSAGE. The control is the SIBLING op: ATS|C_Fuel guards this identical state deliberately and says so.

What happened. CONFIRMED, then FIXED. The inflation attack itself is REFUSED on economics – at 24 decimals the per-coil rounding loss is index x 10^-24, so moving a real pool would take ~10^24 times its own supply in donated tokens. The SINGULARITY was live: ATS|C_Coil into a zero-index pool died with "Arithmetic exception: div by zero, decimal", through the real Talos client, on a pool that exists at deploy. The same call on the sibling op was refused cleanly by name. Repaired in 08_ATS.pact by enforcing (> index 0.0) inside URC_RBT – the function that actually divides, and the one the INFO previews share, so a quote for an impossible coil now refuses in the same words instead of throwing. A healthy pool still prices normally.

> WHY THE SIBLING IS THE WHOLE POINT. ATSU|C>FUEL carries (enforce (>= index 0.1) "Fueling requires an ATS-Pair Index of at least 0.1") and its comment block is unusually careful – it was REWORDED on 2026-09-13 precisely so a caller whose index is 0.05 is not told something untrue about their own pair. That much thought went into the message of one guard, while ATSU|C>COIL – the op an ordinary user actually reaches – has NO index check at all and dies inside the arithmetic. The guard was not missing because the state was unknown. It was missing on the door nobody looked at.

RT-A-004 · REFUSED

Runs in REPL/RedTeam/[RT-A]_Economics.repl

What was suspected. UDC_SpawnSmartSwapSlippageBounds quotes an expected output from the reserves AS THEY ARE WHEN IT RUNS, and the bound is then carried into CC_SmartSwapWithSlippage as plain data. If nothing re-checks it against live reserves at EXECUTION time, any trade landing in between settles the quoted swap at the worse rate – the back half of a sandwich, whose cost to the attacker is an ordinary trade.

How it was driven. the quote is bound EAGERLY in the let, so it is taken before anything else in the block and is genuinely stale by the time the body runs. A swap in the same direction then moves the price. The quoted swap is then executed carrying the pre-move bound. The mover here is the same account, for fixture reasons – the guard under test is indifferent to WHO moved the price, only to whether execution re-reads the reserves.

What happened. REFUSED. UC_SlippageMinMax derives the floor from the caller's OWN quoted expected output, and 19_SWPU.pact checks (>= feeless-final min) against live reserves at execution – the source calls it "the real protection this whole check exists for".

> WHY THIS BLOCK EXISTS AT ALL. That floor had NO witness before 2026-09-17. The only tests naming "out of Slippage bounds" are the #26M pair in [6.2+3]_DPTF-SWP_Issuance-Only.repl, and BOTH assert the message must NOT appear – positive slippage is deliberately allowed, matching every major AMM. So the most important economic guard in the AMM was asserted only in the direction that proves it stays quiet. Deleting (>= feeless-final min) would have turned nothing in this tree red.

RT-A-005 · FIXED

Runs in REPL/RedTeam/[RT-A]_Economics.repl

What was suspected. a swap refused by the slippage floor should tell the caller it was refused. Found while root-causing G-46: the smart-swap door faulted LOUDLY on a refused swap (Array index out of bounds), which raised the question of what the single- and multi-swap doors – carrying the identical soft-fail payload – do instead.

How it was driven. drive SWP|C_SingleSwapWithSlippage with an unreachable expected output (1,000,000) so the floor must refuse, then read the returned string and both trader balances. Not an adversarial input: this is the ORDINARY refusal path, and the finding is what the protocol SAYS when it refuses.

What happened. FIXED. On shipped code it answered “Succesfully swapped input(s) to ”, committed, and emitted a swap event, having moved nothing.

The Defect. SWPU’s slippage floor SOFT-FAILS – it RETURNS a one-element cumulator carrying the exceed-message instead of raising. On the smart-swap door the success payload has four elements, so (at 3 out) on a refusal raised Array index out of bounds (G-46, fixed). On THESE doors the success payload is also ONE element – [o-id-netto], a decimal – so (at 0 out) did not fault. It silently interpolated the refusal text into a sentence beginning “Succesfully swapped”.

Why Length Cannot Fix It. on this door BOTH payloads are one element. The refusal is a string, the success a decimal, so the TYPE is the only discriminator – which is why the repair here differs from G-46’s (= (length out) 4) even though the cause is shared.

> Measured on shipped code before the fix, verbatim: “Succesfully swapped input(s) to Expected Output of 2366.80... out of Slippage bounds min of 990000.0 - max of 1010000.0 OURO-98c486052a51” The transaction COMMITS. The swap @event has already fired – it sits on the with-capability, ahead of the floor check – so an off-chain indexer sees a swap event and a success string for a swap that never happened. Zero in, zero out. THE ASSERTION THAT IS THE REGRESSION is the second one. Before the fix (contains "Succesfully" result) was TRUE on a swap that moved nothing. Everything else here would have passed on the defective code.

Family B — Permissionless reach

RT-B-001 · FIXED

Runs in REPL/RedTeam/[RT-B]_PermissionlessReach.repl

What was suspected. P|UEV_IMC is described as an INTER-MODULE gate – it should pass only for a caller that IS a registered module, which is what makes “Talos is the only supported client path” true. But the genesis sequence registers the MASTER KEYSET itself as a DALOS policy. If that is live, a key holder satisfies the gate without being a module at all, reaches DALOS core ops directly, and – because IGNIS is collected in Talos and this route never enters it – pays NOTHING.

How it was driven. call DALOS::A_SetIgnisSourcePrice straight at core, twice: once signed by an ordinary user, once by the master key. Measure the price AND the IGNIS charged.

What happened. CONFIRMED as measured – the master key executed the op and was charged 0.0000 – and then CORRECTED on review, which changed what the finding IS rather than whether it was real.

On The Status Field. this reads FIXED, and it read SUCCEEDED when first measured. The field takes one value and the honest one is the CURRENT state; the history is kept in full below, including the part where this report described the finding more broadly than the evidence supported.

What This Block Asserts Now. the attack, re-run, and refused. The bound the original finding established still holds and is still worth stating – the same key could never operate the collector against a third party, because IGNIS::XE_CollectIgnis enforces the payer’s own ownership. *Skipping your own toll needs nobody else’s signature;*

charging someone else's does. The reachable damage was unbilled ADMIN activity, never drained users. Now it is neither. ==== RT-B-001 - reach a sovereign admin op without Talos, and pay nothing · 01 · env-sigs (caps) ====

> THE CORRECTION, and it is the part worth publishing. The registration that produced this lives at REPL/Stage_01/[2.1]_Dalos.repl (a TEST FIXTURE) and NOWHERE else in the repository – in particular, in no genesis payload. The DEPLOYED system never had this exception. This block was reporting a live hole in CLAUDE.md's "only supported client path" sentence; there was no hole. There was something else, and it is not obviously less serious: An audit harness that grants itself a privilege the real system withholds will certify behaviour nobody can reach. It was load-bearing, too. Five guard-type assertions in [2.1] called ref-DALOS::C_Rotate* directly while being LABELLED with the Talos entrypoint names. They passed only because of this registration, and removing it made them – and only them, out of 21,732 assertions – go red. They now run through Talos at [6.12]_DALOS-ADMIN.repl <>. FIXED 2026-09-14 on the owner's ruling: "master key passes as module is correct. RotateStoa must pass through talos, if it doesn't it's a bug." The MECHANISM is correct and untouched – a registered module guard should satisfy P|UEV_IMC, and every real module still registers through P|A_AddIMP at deploy. What is gone is the harness registering a KEY as though it were a module.

RT-B-002 · REFUSED

Runs in REPL/RedTeam/[RT-B]_PermissionlessReach.repl

What was suspected. Ouronet encodes PRIVILEGE IN A TOKEN-ID PREFIX. At least three places read the first two characters of an id and change behaviour on the result:

How it was driven. issue through the ONLY supported client path (Talos) with a barred ticker, and pin the refusal BY MESSAGE. Then issue an otherwise identical PLAIN ticker and pin ITS failure by message too – it must fail SOMEWHERE ELSE, and later, or the first result proves nothing about the bar.

What happened. REFUSED, but by ONE HARDCODED BOOLEAN, repeated. Every issuance family gates the special charset behind an iz-special flag, and every client-reachable wrapper passes it FALSE and does not expose it: TS02-C1::DPSF|C_Issue and TS02-C2::DPNF|C_Issue pass a literal false; DPTF::C_Issue and DPOF::C_Issue build (make-list l1 false) internally and take no such argument at all. The barred ticker dies at UEV_NameOrTicker; the plain one sails past it and dies much later, paying.

> 05_DPTF.pact DPTF|C>X_TOGGLE-TRANSFER-ROLE ft in ["F|" "R|"] SKIPS two validations 03_DPDC-C.pact URCi_RegisterCollectablesPrice ft = "E|" price DIVIDED BY 1000 07_DPDC-T.pact URC_TotalTransferPrice ft = "E|" per-nonce price /1000 An id is UDC_Makeid(ticker) = ticker + "-" + block-hash, so the prefix is simply the first two characters of a CALLER-SUPPLIED TICKER. CT_SPECIAL is ["|" "-" "^"] and UEV_NameOrTicker applies NO positional rule – a special character is legal at index 0. So the question is whether a caller can name a token E|... and buy the thousandfold discount that C_IssueShareholderCollection charges a $100 equity premium for. WHY THIS IS WORTH A TEST DESPITE BEING REFUSED. The prefixes are load-bearing in three modules and the only thing standing between a user and one of them is a literal false written out four times, with nothing central enforcing it. A new issue wrapper that plumbed iz-special through to its caller – the obvious thing to do if someone ever wants user-defined LP-style names – would open all three privileges at once, in three modules none of which mention iz-special. The refusal below is pinned by message so that wrapper cannot land quietly.

Family C — Admin impersonation

RT-C-001 · REFUSED

Runs in REPL/RedTeam/[RT-C]_AdminImpersonation.repl

What was suspected. TS01-A carries 30 admin wrappers, and they do NOT gate uniformly. Eleven acquire P|ADMINISTRATIVE-SUMMONER, which composes GOV|TS01-A_ADMIN and is a real gate. FIFTEEN acquire only P|TS – and

(defcap P|TS () true) grants itself. For those fifteen the Talos layer contributes NO authorisation at all; the only thing standing between a stranger and the operation is whatever the CORE function enforces. That is precisely the shape that made ORBR|A_Fuel exploitable, so it is worth driving rather than reasoning about.

How it was driven. call eight of the fifteen as an ordinary user, chosen for blast radius – pausing the chain, moving the OURO source price, flipping gas collection, repricing usage, wiping treasury debt, setting a branding flag. Record WHICH message refuses each.

What happened. all eight refused. Seven by an admin-shaped guard, named below. The eighth is different and is the reason this block is worth more than a row of green ticks.

> THE RESULT THAT MATTERS IS THE ONE THAT ALMOST PASSED FOR SUCCESS. DPTF|A_WipeTreasuryDebt is refused by “Cannot Wipe Positive Treasury Balance” – a BUSINESS guard, not an authorisation one. Reading GOV|WIPE_ALL-TREASURY-DEBT explains why: (enforce (< treasury-supply 0.0) “Cannot Wipe Positive Treasury Balance”) ;; line 402 (compose-capability (GOV|DPTF_ADMIN)) ;; line 403 The admin gate IS there, one line below – but it is UNREACHABLE while the treasury is solvent, which is its normal state. So for this operation the suite can say “a non-admin was refused” without ever having exercised the admin check at all. Had GOV|DPTF_ADMIN been missing from that cap, this test would still be green. That is why the assertion below is deliberately worded as the business guard rather than as an admin refusal: writing it as “the admin gate refused” would state something this fixture has not shown. The honest score for this block is SEVEN admin refusals and one shadowed gate, and the shadowed one is logged as a follow-up rather than counted as a pass. A note on why try is banned here. An earlier pass ran these eight through (try "REFUSED" ...) and reported 8/8 refused – which is TRUE and useless. The distinction above is invisible without the message, and the shadowed gate would have gone unnoticed.

Family D — Ownership bypass

RT-D-001 · REFUSED

Runs in REPL/RedTeam/[RT-D]_Ownership.repl

What was suspected. DPTF|C_Transfer (patron id sender receiver amount method) takes the payer and the asset owner as two separate arguments. If the path proves only the PATRON – the one role the caller definitely holds, since they are paying – then naming someone else as sender drains them while the attacker foots the (tiny) gas bill.

How it was driven. the attacker is the patron AND the receiver, the victim is the sender, and ONLY the attacker signs. Then the same call is repeated with the victim signing, to prove the refusal is about ownership and not about the shape of the call.

What happened. refused at CAP_EnforceAccountOwnership on the SENDER, naming the victim’s key. Sponsorship still works when the owner signs, so the two roles are genuinely separate rather than collapsed into “whoever pays”.

RT-D-002 · FIXED

Runs in REPL/RedTeam/[RT-D2]_Ownership-Collectables.repl

What was suspected. DPDC|NonceElement.nonce-holder is documented as “Stores the holding the Nonce”. It does not. XE_U|NonceHolder writes OI|UC_ShortAccount(account) = (take 5) + "..." + (take -3) – ELEVEN characters, of which the first two are the fixed 0. prefix, leaving SIX characters of entropy. UEV_NonceQuantityInclusion – the possession gate inside DPDC-C|C>SINGLE-DEBIT, which guards every NFT debit, burn and transfer – decides possession by comparing that abbreviation. If two valid accounts can share it, the gate cannot tell them apart, and the one that does NOT hold the NFT passes the check that exists to stop it.

How it was driven. construct a second account that is glyph-valid, exactly 162 characters, DISTINCT from the holder, and shares its first three and last three body characters. No grinding is involved and none is needed:

GLYPH|UEV_DalosAccount enforces length, the o/Σ prefix, the . separator and the DALOS|CHARSET – and NOTHING ELSE. The account string is not derived from the guard, so it is chosen freely by whoever deploys it. Then call the gate directly with three accounts: the true holder, an unrelated account, and the collider.

What happened. CONFIRMED, then FIXED. Measured exploit-first, all three arms in one run: the true holder PASSES, an unrelated account is REFUSED – so the gate is neither always-true nor always-false – and the collider, a distinct glyph-valid 162-character account, PASSES. Repaired in 02_DPDC.pact by enforcing amount <= nonce-supply on the NFT branch, where nonce-supply is the AccountSupplies read keyed by the COMPLETE account string and was already bound, unused, at the top of the function. The abbreviation compare is kept, with its own message, because it also carries the "nonce is active" meaning – an inactivated NFT stores BAR there.

> WHY THE patron/account SPLIT DOES NOT SAVE THIS. DPDC-C|C>SINGLE-DEBIT gates with CAP_EnforceAccountOwnership account – so the caller must genuinely own the account they name. An attacker naming their OWN colliding account satisfies that completely. The second gate, the one that is supposed to establish that this account holds THIS NFT, is the abbreviation compare. And DALOS|C_DeployStandardAccount is a permissionless CLIENT wrapper on TS01-C1, whose STOA fee is itself toggle-conditional – so the colliding account costs nothing to obtain. WHAT THE FUNCTION ALREADY HAD. UEV_NonceQuantityInclusion opens by binding (nonce-supply (UR_AccountNonceSupply account id son nonce)) – a read keyed by the FULL account string. The SFT branch enforces against it. The NFT branch BINDS IT AND NEVER USES IT. The fix is not new machinery; it is spending the value the function already computed.

RT-D-003 · REFUSED

Runs in REPL/RedTeam/[RT-D]_Ownership.repl

What was suspected. RT-D-001 attacks DPTF|C_Transfer, whose defcap chain reaches DPTF|C>X-TRANSFER and opens on CAP_EnforceAccountOwnership sender. The MULTI variant does not use that chain: DPTF|C>MULTI-TRANSFER reproduces X-TRANSFER's checklist per id – transferability, pause state, both freeze states, move-role – and is missing exactly one line of it, the sender ownership check. It enforces ownership of the RECEIVER instead, and only when method is true and the receiver is a smart account. So the same theft should go through the plural door.

How it was driven. identical to RT-D-001 – attacker is patron AND receiver, victim is sender, only the attacker signs – but through DPTF|C_MultiTransfer with a one-element list, the shape that passes UEV_IzUnique and the length guard so nothing refuses early.

What happened. REFUSED, and the reason is worth recording because the defcap really is missing the line. The check is not absent, it is one layer DOWN: the debit leg calls DPTF::XB_DebitTrueFungible, which acquires DPTF|C>DEBIT, which carries CAP_EnforceAccountOwnership account. Every debit path in the module inherits it, which is why the plural door is shut despite its own defcap not shutting it.

> WHY THIS TEST EXISTS AT ALL. DPTF|C>DEBIT's ownership check was, before this block, pinned by NOTHING. RT-D-001 exercises X-TRANSFER's OWN CAP_EnforceAccountOwnership sender, a different call site, so deleting the one inside DPTF|C>DEBIT would not have turned a single assertion red – while being the sole thing standing between a stranger and every multi-transfer in the tree. That is the shape CLAUDE.md's authorise-first ruling is about, arrived at from the other side: not a gate in the wrong order, but a gate with no witness.

RT-D-004 · REFUSED

Runs in REPL/RedTeam/[RT-D2]_Ownership-Collectables.repl

What was suspected. RT-D-001 and RT-D-003 shut the two TRUE-FUNGIBLE doors. The collectable family has its own, and it does not share their code: DPNF|C_TransferNonce reaches DPDC-T|C>TRANSFER, not DPTF|C>X-TRANSFER or DPTF|C>DEBIT. So the question has to be asked again rather than inferred – the same patron/owner split, on the one family where possession is tracked per NONCE instead of per balance.

How it was driven. the attacker is patron AND receiver, the victim is sender, only the attacker signs. The nonce is one RT-D-002 already proved the victim holds – it passes that block's possession gate as the TRUE holder – so possession cannot be what refuses this, and ownership is the only thing left.

What happened. REFUSED at the owner gate, naming the victim's key. ==== RT-D-004 - the same theft, one collectable nonce · 01 · env-sigs (caps) ==== ==========

RT-D-005 · REFUSED

Runs in REPL/RedTeam/[RT-D2]_Ownership-Collectables.repl

What was suspected. VCT|C>TRUE-FUNGIBLE-VACATE gates the TF batch-vacate recipe (CCp_BatchVacateTrueFungible / CCp_BatchDrainTrueFungible) on CAP_VctVacatePoolOwner. That call sits AFTER a four-conjunct business enforce ("Invalid TF vacate cap input"), and no test has ever reached past it: every negative in the suite trips the business rule, and a business-rule refusal is byte-identical whether the ownership line is present or DELETED. So the claim under test is not "vacate is protected" but "the protection can be SEEN protecting" – a stranger who supplies a perfectly valid batch must still be refused, BY OWNERSHIP, by name.

How it was driven. satisfy all four earlier conjuncts by construction and assert each one true before attacking, so the refusal cannot be the business rule. EMMA is the attacker and the batch names HER OWN leg (owner EMMA, beneficiary EMMA, 40.0 OURO – left live by [6.2.4]), so no third party's authority is needed for any part of the request except the pool's. Then the same call is repeated with one extra signature.

What happened. REFUSED, and attributably: Smart DALOS Account <konto> Ownership could not be verified!. TfStakePool's owner konto is the OURO token's konto, which is the Talos/gas SMART account – so this gate speaks the SMART-account half of CAP_EnforceAccountOwnership, not a keyset failure (DEFECT-LEDGER 7.2h defect 4: a detector that knew only the keyset half read every smart-owned gate as unobserved). Non-vacuity: the identical call with ANHD – the smart account's SOVEREIGN – also signing succeeds and vacates the leg.

> WHAT ALREADY EXISTED, stated precisely, because "no test has ever reached this" is the kind of claim this project has been wrong about before. TWO things were already green and NEITHER of them covers these three capabilities: * [6.2.5]_AQP-VCT.repl <> drives a non-owner at CC_FullVacate, C_AbortVacate and C_EnablePoolStake. Those caps call CAP_VctVacatePoolOwner FIRST, before any business rule – they are the correctly-ordered siblings, and they are exactly the ones that were easy to witness. The three BATCH caps were left alone. * modules/AQP.repl <> calls CAP_VctVacatePoolOwner DIRECTLY, unsigned. That witnesses the DEFUN. It says nothing about whether any capability ever reaches it: a defcap whose business enforce always fires first would leave AQP-F16 green forever. The gate and the route to the gate are two different claims, and only the first had evidence. So ...e below is NOT the interesting assertion – it is the attribution instrument. ...c and ...d are the new facts: the ROUTE through a batch capability actually arrives at the gate. WHY EMMA AND NOT ANHD. ANHD is the Demiurgoi master key AND the sovereign of the account that owns this pool, so an "attack" signed by ANHD would pass the gate and prove nothing. EMMA is a STAKER here and nothing else: she has value in the pool and still may not run the pool-owner recipe. URC_AqpOwnerKonto is asserted below to be neither her nor anything she controls. SHAPE, since it decides how these may be driven: CCp_ is a HYDRA recipe, not a defpact, and this one is a FED SLICE – it takes explicit owner/beneficiary/amount arrays rather than a page size, and computes no window from stored progress. Per CLAUDE.md's recipe axes that makes it order-independent; the attack therefore needs no prior batch to have run.

RT-D-006 · REFUSED

Runs in REPL/RedTeam/[RT-D2]_Ownership-Collectables.repl

What was suspected. same shape, different family and different code. VCT|C>ORTO-FUNGIBLE-VACATE-BATCH guards CCp_BatchVacateOrtoFungible / CCp_BatchDrainOrtoFungible and puts CAP_VctVacatePoolOwner after a FOUR-conjunct enforce ("Invalid OF vacate batch"). Nothing in the suite has ever driven it with a non-owner, so deleting the ownership

line would turn no assertion red while being the only thing between a stranger and every staker's position in an OF pool.

How it was driven. EMMA stakes her OWN MVST nonce 2 into OfStakePool (fixture block above), which ANHD owns, then tries to vacate exactly that leg. Every earlier conjunct is satisfied and asserted. Note the amounts are NOT supplied by the caller here – the recipe resolves them from the live tracker (URC_ResolveOfDecimalAmountsFromTracker, an EAGER let that runs BEFORE the with-capability), so the "no partial vacate" conjunct is true by construction and cannot be what refuses.

What happened. REFUSED, naming the VICTIM: Keyset failure (keys-all): [PK_Ancie...] while the attacker signs PK_Emma. Non-vacuity: one added signature and the same call vacates.

> WHY THE LEG IS THE ATTACKER'S OWN, and why that matters more than it looks. If the batch named ANHD's leg, then adding ANHD's signature in the control would be ambiguous – it could be authorising the leg rather than the pool. Naming EMMA's own leg removes that reading entirely: the ONLY thing ANHD's signature can be doing is satisfying CAP_VctVacatePoolOwner. It also states the rule being tested in its sharpest form – VACATE IS A POOL-OWNER OPERATION, so a staker may not use it even to withdraw her own stake.

RT-D-007 · REFUSED

Runs in REPL/RedTeam/[RT-D2]_Ownership-Collectables.repl

What was suspected. the third of the three, and the one with the longest shadow. VCT|C>COLLECTABLE-VACATE-BATCH guards CCp_BatchVacateCollectables / CCp_BatchDrainCollectable for BOTH collectable branches (son=true DPSF, son=false DPNF) and puts CAP_VctVacatePoolOwner after a FIVE-conjunct enforce ("Invalid collectable vacate batch"). Five business conjuncts is five different ways for an attack to be turned away before the gate is reached, and none of the suite's negatives gets past them.

How it was driven. the attacker is ANHD – deliberately the strongest key available, the Demiurgoi master keyset member, AND the owner of the very nonce in the batch. The pool is NfSyncPool, which LUMY owns. So the request is impeccable in every respect except the one under test, and the attacker's own authority is maximal.

What happened. REFUSED, naming LUMY: Keyset failure (keys-all): [PK_Lumy...] while the attacker signs PK_AncientHodler. The admin key does not override pool ownership. Non-vacuity: adding PK_Lumy – and changing nothing else – vacates.

> AND A DIFFERENTIAL, because "the message names the victim" is only worth asserting if the message is DERIVED. The SAME defcap is driven a second time on SfStakePool (son=true, DPSF, owned by ANHD) with EMMA attacking, and names PK_Ancie instead. One capability, two pools, two different keys in the refusal: the sentence tracks URC_AqpOwnerKonto and is not a constant. WHY ANHD IS THE RIGHT ATTACKER HERE AND THE WRONG ONE IN RT-D-005/006. An attack signed by a key that already owns the thing proves nothing – ANHD owns OfStakePool and is sovereign of TfStakePool's owner, so there he is useless. NfSyncPool is LUMY's, and that makes ANHD the most informative possible attacker: if the gate can refuse the master key, it refuses anybody.

Family E — Sequencing and state

RT-E-001 · REFUSED

Runs in REPL/RedTeam/[RT-E]_Sequencing.repl

What was suspected. STOAICO pays the LAST unclaimed staker the whole remaining vault rather than their computed share – a dust sweep, so nothing is stranded. "Last" is decided by , which is set to ONLY at inject. If a contribution recorded mid-round moves nzs-count without moving unclaimed-count, a fresh account owed

nothing should be able to collect while the counter still reads 1, and take the entire remaining supply out from under the staker it belongs to.

How it was driven. drive the suite to its end state (one staker collected, one outstanding, the whole 690.53 owed to her), then have the admin record a legitimate late contribution of 1.0 v-USD for a NEW participant, and try to collect as that participant.

What happened. the counters DO diverge and the reader DOES offer the whole vault to an account owed zero – but the collect is refused one layer down. Attack fails. Detail below.

> THE HALF THAT IS TRUE, and it is worth pinning on its own. After the late stake: unclaimed-count = 1 nzs-count = 3 <- they have diverged, exactly as predicted URC_AvailableRewards(newcomer) = 0.000000000000 <- owed nothing URC_ClaimableRewards(newcomer) = 690.525983513596 <- offered the ENTIRE vault So URC_ClaimableRewards will hand the whole remaining supply to an account with no claim on it. That value is real and it is wrong; it is simply not reachable through the collect path. WHAT STOPS IT, and it is deliberate rather than lucky. A_Stake's insert for a new contributor stamps with the CURRENT distribution-round: ;;#1C: new contributor starts at the CURRENT distribution-round, so a (mis-ordered) ;; post-inject stake is not eligible for the already-injected round. (UDC_UserData 0.0 0 (UR_Global6) 0.0 (UR_Global11) account) and STOAICO|REDEEM-CONTRIBUTION enforces (< last-collected-round distribution-round). A newcomer is therefore born already-collected for the round they joined. Someone modelled this exact attack and closed it at the eligibility layer. WHY THIS IS STILL WORTH A BLOCK RATHER THAN A SHRUG. The theft is prevented by ONE stamp, in a different function from the one that computes the dangerous number, with no assertion previously connecting them. URC_ClaimableRewards is also read by presentation paths, where it will state a figure the account cannot have. If that stamp is ever changed – a new contributor starting at round 0, say, which reads like a harmless initialisation – this becomes a live theft primitive with no other guard in the way. The assertions below pin BOTH halves, so the day the stamp moves, the first two go red and name what broke.

RT-E-002 · REFUSED

Runs in REPL/RedTeam/[RT-E]_Sequencing.repl

What was suspected. a defpact's steps are separate transactions, so the signer set can CHANGE between them. 20_MTX-SWP.pact contains ZERO occurrences of CAP_EnforceAccountOwnership or CAP_Owner – the MTX pacts never check who is driving them, and delegate ownership to the core ops the steps call. If step 1 does not re-establish the starter's authority, a STRANGER can continue someone else's add-liquidity pact and so CHOOSE THE MOMENT it executes. That matters because RT-F-001 established what step 1 does when the pool has moved: it takes the rollback branch and the starter loses the initiation fee. An attacker who can time the continuation can swap against the pool first and then continue the victim's pact into its own failure branch – turning RT-F-001's accident into a directed grief.

How it was driven. start the pact as ANHD, then REPLACE the signer set with EMMA's alone and continue. One transaction, so the continuation resolves against the pact started here and no pact-id has to be guessed. The question is not only success or failure but WHICH guard answers – a refusal from the wrong guard is still a latent defect.

What happened. REFUSED, on BOTH pact families – and the manner is the finding. The stranger's continuation dies on a plain Keyset failure (keys-all): [PK_Ancie...], naming the STARTER's key. Nothing in the defpact layer asked who was driving: the refusal comes from a DOWNSTREAM core op that happens to touch the starter's account, so the protection is a property of what the step DOES, not of the pact.

> That is safe today and structurally thin. Every MTX step currently moves the starter's tokens, so every one demands their key. A future step that touched only protocol state – a sweep, a recompute, a flag flip – would have nothing to demand it, and this attack would succeed against that step without anything else changing. The attack is worth keeping precisely because it does not test a guard; it tests

an ABSENCE, and the absence is still there. NON-VACUITY is asserted in RT-E-002b: the same pact, continued by the account that started it, proceeds. A step that refused everyone would satisfy the refusal above and prove nothing – RT-C-001's lesson.

RT-E-003 · REFUSED

Runs in REPL/RedTeam/[RT-E]_Sequencing.repl

What was suspected. Ap_FlushUncollectedSlice is an ADMIN Hydra recipe that push-collects other people's rewards. Its @doc makes three claims and NONE of them was tested – the function is driven by ZERO tests anywhere in the tree, the only one of the eleven Cp_/CCp_/Ap_ recipes with no coverage at all. The claims: value goes to each ACCOUNT ("not to the admin, not burned"), the slice is order-independent, and "re-runs are idempotent". A recipe is by construction multi-transaction, so a replay is not a hypothetical – it is what a retrying operator does.

How it was driven. measure the straggler's balance, the ADMIN's balance and the round stamp; run the slice; measure again; run the IDENTICAL slice a second time; measure again. The admin's balance is read at every step because "delivers to the account" and "delivers to whoever ran it" produce the same success string.

What happened. REFUSED – all three claims hold. The account gains, the admin does not move, and the replay is a genuine no-op. One reporting defect found, below.

> FOUND BY COUNTING, NOT BY SUSPICION. The sequencing family was the round's thinnest and the question "which surface is least covered?" was answered by enumerating every Hydra recipe in the tree and counting the test files that drive each. Ten had between three and seven. This one had zero – an admin entrypoint that moves other people's money, with no test.

Family F — Griefing and denial of service

RT-F-001 · FIXED

Runs in REPL/RedTeam/[RT-F]_Griefing.repl

What was suspected. MTX|C_AddLiquidity collects its whole deterrent in STEP 0, then validates in STEP 1 that the pool has not moved since it quoted. PoolState includes the pool's TOKEN SUPPLIES, which every swap changes. So any stranger's ordinary swap should permanently invalidate an in-flight add, AFTER its fee is already paid and with no refund – a denial of service whose cost to the attacker is a normal trade.

How it was driven. the victim starts the pact and the fee is measured. An unrelated account then swaps in the same pool. The victim's step 1 is then attempted, and the fee is re-measured.

What happened. confirmed. Fee 557.03 taken at step 0; one swap by a stranger; step 1 refuses permanently against that quote; the fee is NOT returned.

On The Status Field. FIXED means the owner's 2026-09-14 ruling shipped and is measured – step 0 now takes only the initiation slice, so what a stranger's swap can destroy fell from 557.03 to 53.00 net. It does NOT mean the fee is refunded on the failed step; it is not, and the block below says so in the assertion that bounds the residual loss. The ruling was to SPLIT the fee, not to refund it.

And The Fourth Defpact In The Same File Is The Specification. MTX|C_Issue validates in step 0 and collects in step 1 — validation before money, same file, same author. When one member of a family of identical ops is built the other way round, the odd one out is the design. ==== RT-F-001 - destroy a stranger's add-liquidity fee with one ordinary swap · 01 · env-sigs (caps) ====

> THE GUARD ITSELF IS CORRECT AND MUST STAY. Executing a quote against a pool that has since moved is how a liquidity provider silently gets a worse ratio than the one they agreed to. SWPX-10 in modules/SWP.repl already pins that the guard fires. What NOTHING pinned is what it costs the victim when it does. THE DEFECT IS THE ORDER OF FEE AND VALIDATION, not the validation. Money

moves in step 0; the condition that decides whether the operation can happen at all is checked in step 1. Between them sits the open chain. On a pool with any trading activity the add is not merely griefable — it is unreliable by construction, because PoolState equality is exact and includes supplies. AND IT IS WORSE BECAUSE OF <>. That repair raised this step-0 fee from a flat 100.0 to the real 1051.0 raw deterrent — correctly, since the cheap door was letting providers decline the churn deterrent entirely. But the same change multiplied the griefing payoff by ten: what an attacker destroys with one swap went from 53.0 to 557.03 net. Closing an economic bypass widened a denial-of-service. That trade is worth making — an optional deterrent is worse than an expensive one — but it must be recorded rather than discovered later, and it is the reason this block sits directly after RT-A-001 in the report. RECOMMENDED, NOT APPLIED: collect the deterrent in the step that SUCCEEDS, or refund it on the rollback path. Both change when money moves inside a defpact, which is a design decision and not a transcription fix, so it is left for a ruling. The block pins present behaviour meanwhile. THE RULING COVERS THREE OPS, NOT ONE (established 2026-09-16 by asking the same question of every defpact in the tree). MTX|C_AddFrozenLiquidity and MTX|C_AddSleepingLiquidity collect the same URCi_AddLiquidityInitiation in step 0 and carry the SAME step-1 guard with the same message, character for character. Everything above applies to them verbatim; neither is pinned, because nobody in this chain holds the frozen LP twin and the LP has no sleeping link, so pinning them needs fixtures staged first. See DEFECT-LEDGER 7.3.

RT-F-002 · FIXED

Runs in REPL/RedTeam/[RT-F]_Griefing.repl

What was suspected. MTX|C_Issue's admin gate lived only in STEP 3. Step 1 validated shape, step 2 irreversibly collected IGNIS + STOA, and only then did step 3 ask whether the caller was allowed to do this at all. A non-Demiurgoi calling the permissioned door (p=true) would pay in full and be refused afterwards, with no refund and no cancel – while the SINGLE-TRANSACTION twin refuses the identical operation for nothing.

How it was driven. drive the Talos starter with p=true signed by a non-Demiurgoi. p is an undocumented raw bool on the wrapper's signature, so this is an ordinary mistake, not an exotic one.

What happened. FIXED 2026-09-17. Measured before the fix, by execution: step 2 committed 2,919.77 IGNIS + 459.0 STOA and step 3 then refused with "MTX-SWP Ownership not verified"; rolling back cost a further 53.00 IGNIS. The conditional admin gate is now hoisted into STEP 1, ahead of UEV_Issue and therefore ahead of all money, mirroring SWPI|C>ISSUE, which the 2026-09-14 authorise-first sweep had already repaired. The whole (if p ...) form moved, not the bare acquire: only a PERMISSIONED issuance needs the key, and unwrapping the branch would lock out every ordinary issuance.

ORIGINALLY FILED AS M11 AND CLOSED AS UNREACHABLE. The verdict rested on "MTX-SWP has zero Talos wiring"; 05_TS01-P.pact:221-330 wires all six starters. DEFECT-LEDGER 8.4.

Family G — Hostile citizen module

RT-G-001 · REFUSED

Runs in REPL/RedTeam/[RT-G]_HostileCitizen.repl

What was suspected. a module deployed in an open namespace can call a sovereign C_ entrypoint directly, discard the returned OutputCumulator, and so perform a real state change that nobody was billed for – because IGNIS is collected in Talos and this path never enters it.

How it was driven. deploy a real module in user, call TFT::C_Transfer and DPTF::C_Mint from inside it, and check WHICH guard refuses. A refusal from the wrong place would prove nothing.

What happened. refused, by the intended guard, in both cases: P|UEV_IMC -> U|G::UEV_Any over the registered inter-module policy guards -> "None of the guards passed".

WHY THE BOUNDARY HOLDS, mechanically. P|UEV_IMC passes only if the caller holds one of the capability guards registered in DALOS's policy list, and those are create-capability-guards over capabilities that only the owning module's own code can bring into scope. A foreign module cannot acquire another module's capability in Pact, so it cannot satisfy the gate no matter what it signs. Registration itself is closed too: DALOS::P|A_AddIMP requires GOV|DALOS_ADMIN, so an attacker cannot add their own guard to the list. A WHOLE-SURFACE SCAN BACKS THIS UP rather than leaving it at two samples. Every C_/CC_ implementation across the sovereign core carries P|UEV_IMC. UPDATED 2026-09-20: this used to read "289 of 290", with IGNIS::C_TransferDalosFuel as the single exception – safe by DELEGATION to Stoa's coin rather than by a gate. The IGNIS restructure removed the exception: that function is now XB_MoveDalosFuel, a gated XB_, so the surface has no hole to argue about. <> is kept, and now proves BOTH layers instead of relying on the lower one. ==== RT-G-001b - the adversary attacks the sovereign core · 01 · env-sigs (caps) ==== ==========

RT-G-002 · REFUSED

Runs in REPL/RedTeam/[RT-G]_HostileCitizen.repl

What was suspected. IGNIS::XB_MoveDalosFuel is the primitive every STOA-collection path funnels through. Its body is a bare coin::transfer executor executee amount with the sender taken straight from the argument. If it can be reached and the coin layer satisfied, a foreign module moves native STOA out of any account it names – including the Ouronet gas station, which would stop the chain paying for anything.

How it was driven. push through BOTH layers in turn. (1) call it from the hostile module, which is where a real adversary stands; then, to prove the lower layer independently rather than leave it shadowed, (2) drop to the coin contract DIRECTLY – signing and installing coin.TRANSFER for an account the attacker does not own – and (3) the same against an ordinary user's account. Messages checked at every step.

What happened. refused at every step, by two INDEPENDENT layers, either of which suffices.

THIS ENTRYPOINT USED TO HAVE NO GATE, and the argument for that was reasonable: it is a PRIMITIVE, every XE_CollectStoa* path funnels through it, and gating it looked like gating the collector against itself. The IGNIS restructure of 2026-09-20 showed that was a false choice – the collectors compose P|MOD|CALLER on the way in, so the primitive can carry P|UEV_IMC and still be reachable by the code that legitimately needs it. Attempt (1) is now refused there, before the coin layer is ever consulted. WHICH IS EXACTLY WHY ATTEMPTS (2) AND (3) NO LONGER GO THROUGH THE HOSTILE MODULE. A guard that runs first hides every guard behind it, and a hidden guard is indistinguishable from a deleted one – CLAUDE.md's shadowing ruling. Routing the lower-layer attack straight at coin.transfer keeps the sovereign-account property under test instead of accidentally retiring it. THE LOWER LAYER, ON ITS OWN. Managed capability not installed sounds like a defence and is not: an attacker simply installs it, and attempt (2) does. What stops (2) is that the gas station's STOA account is a c: PRINCIPAL whose guard is (create-capability-guard (DALOS|NATIVE-AUTOMATIC)), and coin.pact:144's (enforce-guard (UR_Guard sender)) therefore demands a CAPABILITY, not a signature. No key can satisfy it; only DALOS's own code bringing that capability into scope can. Capability not acquired: CapabilityGuard {name: ouronet-ns.DALOS.DALOS|NATIVE-AUTOMATIC ...} That is the property worth publishing: the sovereign smart accounts are not protected by secrecy or by a signature that could leak, but by a guard that is unsatisfiable from outside the module. An attacker holding EVERY private key in the system still cannot move the gas station's funds. ==== RT-G-002 - drain the gas station through the STOA-moving primitive · 01 · env-sigs (caps) ====

Family H — Input domain

RT-H-001 · FIXED

Runs in REPL/RedTeam/[RT-H]_InputDomain.repl

What was suspected. SWPU|X>SWAP – the capability guarding the NO-SLIPPAGE swap path, which is the one a caller reaches by passing the -1.0 sentinel – validates the output is on the pool, that swapping is enabled, that input-ids and input-amounts are the same LENGTH, and then loops over the inputs checking each is on the pool with a valid amount. Every one of those is a per-ITEM property. Nothing checked a property of the SET. So two things should be constructible: output-id also appearing in input-ids (swap a token for itself), and the same id appearing twice (pay one token, be credited for two).

How it was driven. drive both through the live Talos client on a real pool, and record WHICH guard refuses. The identity of the refusing guard is the entire result here.

What happened. both were constructible, and both were refused – but by guards that were NOT about the set. Fixed 2026-09-14 on the owner's ruling; this block now pins the real rules.

> WHAT USED TO REFUSE THE SELF-SWAP, and it was not a validator. output-id in input-ids passed every check in SWPU|X>SWAP: it is on the pool, its amount is positive, the lengths match. The swap proceeded INTO the curve math, which computed an output of exactly ZERO – adding and removing the same token on a constant-function curve nets to nothing – and the transfer of that zero was then refused by DPTF::UEV_Amount's (> amount 0.0). The transaction was rejected by a ZERO-AMOUNT TRANSFER RULE, three modules away from the mistake, and only because the arithmetic happened to land on exactly 0.0. That was the finding. The system was not protected from self-swaps by a rule forbidding them; it was protected by the curve returning zero. Any future change that made a self-swap return a non-zero positive amount – a different curve, an amplifier, a fee rebate, a rounding direction – would have turned this into value creation with no guard in the way. WHAT USED TO REFUSE THE DUPLICATE was also not a uniqueness rule: "Only a single Input can be used in Stable Swap" is a POOL-TYPE rule. It happened to make duplicates unconstructible on a STABLE pool because stable pools take one input at all. It said nothing about weighted pools, which accept several inputs by design – so the duplicate case was pinned only for the stable shape and the weighted shape was an open follow-up. THE FIX. SWPU|X>SWAP now states both rules directly, as properties of the SET: (enforce (not (contains output-id input-ids)) "Output Token … cannot also be an Input …") (ref-U|LST::UEV_IzUnique input-ids) Both now refuse IN THE CAPABILITY, before any debit and before the curve is evaluated. That is the difference this block measures: not merely that the attacks fail, but that they fail EARLY and for a reason that names them. The old defences remain in place behind these; they are simply no longer the thing standing between a malformed set and the pool. ==== RT-H-001 - swap a token for itself, and swap a token twice · 01 · env-sigs (caps) ==== =====

RT-H-002 · FIXED

Runs in REPL/RedTeam/[RT-H]_InputDomain.repl

What was suspected. this is Stage 0's V-01, the last of its three first-rank targets – "slippage unbounded in BOTH directions; the @doc's'<= 50' is UI policy the chain never runs". UDC_SpawnSmartSwapSlippageBounds enforces (or (= slippage -1.0) (and (> slippage 0.0) (<= slippage 50.0))), and its own @doc says it is "Called by the UI to generate the slippage-bounds object before submitting the Smart Swap transaction". But the CLIENT SURFACE takes the built OBJECT: SWP|CC_SmartSwapWithSlippage (patron account input-id input-amount output-id slippage-bounds). Talos then does (slippage:decimal (at "slippage-percent" slippage-bounds)) – it reads the number back OUT of the caller's object. So if nothing downstream re-checks it, the ceiling is advisory: an integrator who hand-builds the object is not obliged to call the constructor at all.

How it was driven. assert the constructor DOES refuse an out-of-range value, then feed a hand-built object carrying that same value to the consumer that actually derives the bounds, UC_SlippageMinMax, and see what it computes. Pure compute on both sides, so the result is arithmetic rather than circumstance.

What happened. the ceiling is not enforced anywhere the value is USED. UEV_SwapData validates token sets, lengths and output-not-in-inputs and says nothing about slippage; a grep for enforce beside slippage in 19_SWPU.pact returns ONE hit and it is a doc comment. Above 100% the derived min goes NEGATIVE, which makes the bound a no-op – every possible output passes. That is what the <= 50 ceiling exists to prevent.

RT-H-003 · FIXED

Runs in REPL/RedTeam/[RT-H]_InputDomain.repl

What was suspected. UR_HotRewardBearingToken returns the BAR sentinel ("|") when an ATS pair has no Hot-RBT – nine of the fifteen live pairs are in that state. C_HotRecovery binds h-rbt from it and, in the SAME eager let group, does (UR_NoncesUsed h-rbt) – a DPOF properties read keyed by "|". The bindings are evaluated BEFORE with-capability, so no guard in ATS|C>HOT_RECOVERY can run first.

How it was driven. drive the real client entrypoint at a pair with no Hot-RBT and pin the refusal by message. Then CONSTRUCT the harder state – the pool owner switches hot recovery ON while the pair still has no Hot-RBT, which ATS|S>SWITCH-HOT-RECOVERY permits because it checks only CAP_Owner and the previous toggle value – and drive it again.

What happened. CONFIRMED, then FIXED. Both calls died with No value found in table ouronet-ns.DPOF_DPOF|T|Properties for key: | – through the real Talos client, on a pair that exists at deploy, and in the owner-constructed state too. Repaired in 10_ATSU.pact by hoisting with-capability above the binding group (the C_Recover repair, re-applied) and adding a Hot-RBT presence check to ATS|C>HOT_RECOVERY, because the toggle and the Hot-RBT are independent. The hoist is visible in the test itself: block 01 now needs a signature, because once the capability runs first CAP_EnforceAccountOwnership answers before any business rule – authorisation before validation, as ruled.

> THIS EXACT BUG WAS FIXED TWENTY LINES BELOW, THREE DAYS EARLIER. C_Recover carries a comment block explaining that its capability was moved ABOVE its let on 2026-09-12 because "for a token that is not reward-bearing UR_RewardBearingToken returns the BAR sentinel, so the next binding looked up ATS pair | and died with No value found in table ouronet-ns.ATS_ATS|Pairs for key: |." Same sentinel, same eager-let ordering, same error shape, different table – and the sibling immediately above it was left alone. The repair here is the one that was already written once. ==== RT-H-003 - the BAR sentinel as a table key, again · 01 · env-sigs (caps) ==== ==========

Family I — Gas-station payable surface

RT-I-001 · FIXED

Runs in REPL/RedTeam/[RT-I]_GasStation.repl

What was suspected. the Ouronet gas station whitelists by reading the transaction's top-level forms out of exec-code. Case 1 pins a SINGLE form to (coin.C_, (ouronet-ns.TS, (ouronet-ns.DSP or (ouronet-ns.STOAICO.; Case 2 pins BOTH of two forms to (coin.C_. Case 3 accepts (>= n 3) and then inspects indices 0, 1 and 2 ONLY. If an attacker supplies the three-form prefix Case 3 wants and APPENDS a fourth form, that form is never looked at – and Ouronet pays the gas for whatever it is. The appended code still has to satisfy its own guards, so this is not theft; it is FREE EXECUTION, which is the thing a gas station exists to ration.

How it was driven. drive DALOS.GAS_PAYER directly with env-data supplying exec-code, exactly as chainweb supplies it during buy-gas – the idiom modules/DALOS-ADMIN.repl <> establishes. ONE TRANSACTION PER CASE, because test-capability grants the cap for the remainder of the transaction. Then PRICE the door by measuring a real IGNIS::XE_CollectIgnis of UDC_CustomCodeCumulator, for a standard patron and for a gassless one.

What happened. BOTH FINDINGS ARE NOW CLOSED (2026-09-20). This block used to read "SUCCEEDED … a plain DESIGN-INTENT VIOLATION". That verdict covered TWO separate things, and they were resolved separately – one by a code fix, one by the owner stating that it was never a finding at all. Leaving the old verdict in place would have this file asserting two vulnerabilities the system no longer has.

> [1] THE GASLESS-PATRON EXEMPTION – WAS REAL, IS FIXED. The owner's rule: only ONE hardcoded smart account may be a gasless payer. The code at the time read iz-gassles-patron off the smart-contract flag, which XI_DeploySmartAccount sets unconditionally – so EVERY smart account was

exempt. 02_IGNIS.pact:1901 now reads: (iz-gassles-patron:bool (= patron (ref-DALOS::GOV|DALOS|SC_NAME))) – a single hardcoded account, exactly as specified. The narrowing was measured first: all 37 genuine patron slots that pass a smart-account constant are in 03_DSP+.pact, so nothing outside the daily minter relied on the wide exemption. [2] CASE 3's UNINSPECTED FOURTH FORM – WAS NEVER A VIOLATION. Owner, 2026-09-20: this is the PRODUCT. The caller pays 25 IGNIS and the gas station sponsors a custom let block of arbitrary size. The whitelist inspects forms 0-2 and deliberately does not read the fourth, because the fourth is what the 25 IGNIS BUYS. It is rationed execution, not free execution – the door has a price. The paid door is now DPTF|C_Transmute of the IGNIS DPTF (see that function's @doc for why 25, and why it costs no IGNIS), NOT a direct XE_CollectIgnis call: the collector became an IMC-gated X_ function in the same pass and can no longer be reached from exec-code at all. OWNER: "an IGNIS gas payer account can only be a STANDARD account. A smart account can never be a gassless payer, EXCEPT one single account HARDCODED into the code, to allow admin-based gassless IGNIS transactions – the Ouroboros daily minter uses such a gassless patron. No other smart account should have this property." THE CODE IMPLEMENTS NO SUCH HARDCODING. iz-gassles-patron is (DALOS::UR_AccountType patron) = the smart-contract flag, which XI_DeploySmartAccount sets to true UNCONDITIONALLY. "gassles" appears exactly TWICE in all of 1_SOVEREIGN and both are those two lines. So the exemption's population is not one account – it is SEVEN system smart accounts created by [4.0]_Sovereign-Executor, PLUS every user smart account, because DALOS|C_DeploySmartAccount is a CLIENT wrapper (TS01-C1:311), not an admin one. Measured on KC.BJ – an ordinary user account on a user keyset – at RT-I-001h. FIX SHAPE (the account is the owner's to name): iz-gassles-patron should compare the patron against the ONE designated account, not read the smart flag. The original framing below still holds and is the mechanism: the defect is NOT the unvalidated tail – that is the custom-code door working as designed, with the fee as its bound. The defect is that THE FEE CAN BE DECLINED. Every link measured below: 1. Case 3 inspects forms 0, 1, 2 only; (>= n 3) bounds nothing above. [RT-I-001c] 2. The COUNT of appended forms is unbounded. [RT-I-001d] 3. Form 2 need only START with "(let" – its body is unread. [RT-I-001e] 4. The door is priced at a FLAT (* 5.0 tier-biggest): 25 raw, 13.25 after a top-tier discount – independent of what the tail does. [RT-I-001f] 5. A GASSLESS (smart) account pays 0.0. IGNIS::XE_CollectIgnis collects only (if (and (!= ignis-sum 0.0) (not iz-gassles-patron)) ...). [RT-I-001g] 6. Smart accounts are PERMISSIONLESS: DALOS|C_DeploySmartAccount is a CLIENT wrapper on TS01-C1:311, not an admin one, priced in STOA at acct-smart 1000 deter = 100 STOA (~$10) – and that fee is itself conditional on UR_AccountCreationStoa, so it is ZERO when the account-creation toggle is off. So: pay once (or nothing), then submit unlimited transactions of the shape namespace + IGNIS.XE_CollectIgnis(CustomCodeCumulator) + (let ...) + <arbitrary code>, paying NO IGNIS, with the gas station funding up to DALOS|GAS-BUDGET = 2,000,000 gas units of NATIVE STOA each time. (There is no gas station for IGNIS – only for native STOA.) THIS IS RT-A-001's SHAPE IN THE GAS STATION: a deterrent that can be declined. There the second door skipped lp-churn; here the gassless exemption skips the custom-code fee. The appended code still satisfies its own guards, so it is not theft – it is FREE EXECUTION, which is the one thing a gas station exists to ration. SCOPE OF WHAT IS PROVEN. Links 1-5 are measured in this file; link 6 is read off the Talos surface. The COMPOSITION is not executed end to end, because the REPL cannot simulate chainweb's buy-gas phase – test-capability acquires GAS_PAYER exactly as buy-gas would, which is what makes 1-3 real, but no REPL can show the KDA actually leaving the station. That last step is an on-chain smoke test, and it is the only part of this finding that is inference rather than measurement. ONE TRANSACTION PER CASE, AND THAT IS NOT COSMETIC. The first draft of this attack put all five cases in ONE begin-tx and reported a clean success. It was vacuous: test-capability GRANTS the capability for the remainder of the transaction, so cases 2-5 never re-evaluated the defcap body at all – they were reading a cap already in scope from case 1. The CONTROL is what exposed it: a single foreign form, which DALOS-G2c proves is refused, came back as "expected failure, got result: ()". Without that control this file would have claimed a gas-station hole it had not demonstrated. Capability scope is per-transaction; so is each case below. BASELINE. Without this the refusals below prove nothing – a Case 3 that rejects everything is not a hole, it is a closed door. ==== RT-I-001a - BASELINE: the intended Case 3 shape is funded · 01 · env-sigs (caps) ==== ======

RT-I-002 · REFUSED

Runs in REPL/RedTeam/[RT-I]_GasStation.repl

What was suspected. GAS_PAYER's spending cap is not a plain enforce – it is one arm of an enforce-one whose FIRST arm is (enforce-guard GOV|MD_DALOS). An enforce-one passes if ANY arm passes, so the Demiurgoi master keyset does not merely have a higher allowance: it has NO allowance check at all. If that is so, the gas station's only spending bound is absent for its most powerful key, and nothing in the tree says so.

How it was driven. hold EVERYTHING constant – the same payable form, the same 100,000,000 gas at a 1,000x price – and change ONLY the signer set. A differential of exactly one variable, because a refusal and an admission under different inputs would prove nothing about the guard.

What happened. REFUSED for a stranger, ADMITTED for the master keyset. No privilege was escalated – this is the documented shape of the enforce-one, not a bypass of it – but the PROPERTY had never been witnessed, and it is worth witnessing: the station's spending cap is the difference between subsidising a client call and funding an arbitrarily expensive transaction, and for one keyset that difference does not exist. It matters the day the master key is driven by automation rather than by a person.

> WHY THIS BLOCK EXISTS AT ALL. Family I was assessed as the round's thinnest surface on an attack COUNT – one attack. That assessment was wrong in the way counts usually are: modules/DALOS-ADMIN.repl already drives GAS_PAYER through five blocks, including the notional ceiling at <> and two dead guards at <>. Counting attacks in a family undercounts coverage that lives in a module suite. What was genuinely missing was not the cap – it was the ARM BESIDE the cap.

Family J — Ledger conservation

RT-J-001 · FIXED

Runs in REPL/RedTeam/[RT-J]_Conservation.repl

What was suspected. a DPTF's supply and the sum of its account balances are maintained by SEPARATE write paths – XBv_UpdateSupply writes the supply column of DPTF|PropertiesTable, while balances are written by XI_UpdateBalance, which for the two CORE tokens (OUROBOROS and IGNIS) dispatches into a table owned by a DIFFERENT MODULE, DALOS. Nothing anywhere compares the two. If any client path moves one without the other, the ledger drifts silently and permanently, and no existing test can see it: every conservation assertion in the suite is LOCAL and HAND-NAMED ("op+emma+lumy = 5000").

How it was driven. sweep, do not reason. Enumerate every token with DPTF.UR_P-KEYS and every balance row with DPTF.UR_KEYS, attribute each row to its token by FULL-ID prefix (two token ids contain the separator – F|VST-..., R|OURO-... – so splitting on the first bar misattributes them), and compare UR_Supply against the summed balances. The two CORE tokens are not row-enumerable at all: DALOS exposes no key list for DALOS|AccountTable, only a COUNT. So their holders are supplied as a candidate set and the sweep is made non-vacuous by proving that set exhaustive AGAINST that count.

What happened. CONFIRMED, then FIXED. Measured exploit-first across 296 tokens and 739 balance rows: 295 tokens conserved to the last decimal and ONE did not – OURO, the protocol's own money, with supply 8.0 BELOW the sum of all balances. Bisected to a single transaction and then to a single call, DPTF|C_ClearDispo. Its step 6 zeroes a negative OURO balance by calling DALOS::XB_UpdateBalance DIRECTLY – the only one-sided use of that writer in the tree – so the balance rises by the cleared dispo while supply does not. Repaired in 09_TFT.pact by pairing that write with XBv_UpdateSupply ouro-id ouro-amount true. After the fix all 296 tokens conserve.

> WHY THE TWO NON-VACUITY GUARDS ARE NOT DECORATION. This sweep is an argument of the form "I looked everywhere and found nothing", which is worthless unless "everywhere" is proven. Both guards fired for real during construction: (a) the first run reported OURO conserved and GAS off

by 9302.0225. Both were WRONG – the sweep was summing DALOS balances over only the accounts that happened to own a DPTF placeholder row, 6 of 24. The partition guard says every row is attributed; the account guard says every ACCOUNT is swept. Only the second one could catch that. (b) loading [6.3]_SWP raised the account count from 24 to 26 – it deploys two more. The account guard went red immediately. Without it the sweep would have quietly gone back to under-counting, and reported a DIFFERENT wrong number with equal confidence.

RT-J-002 · REFUSED

Runs in REPL/RedTeam/[RT-J]_Conservation.repl

What was suspected. DPOF (ortofungible) keeps the SAME quantity in three places – Properties.supply, a per-account total-account-supply in DPOF|T|AccountRoles, and a per-nonce supply in DPOF|T|Nonces where each nonce records its own holder. The nonce table is the LEDGER; total-account-supply is a CACHED AGGREGATE of it. A cached aggregate beside its own source of truth is the classic place for drift, and no test had ever compared them – RT-J-001's sweep covers DPTF only, and DPOF's writers are a different family.

How it was driven. enumerate ids (UR_P-KEYS), account rows (UR_KEYS) and nonce rows (UR_N-KEYS), attribute by FULL-ID prefix – DPOF ids contain the separator far more often than DPTF's do (V|OURO-, Z|VST-, Z|P|OURO-BUSD|LP-), so splitting on the first bar is useless here – and cross-check all three levels against each other.

What happened. REFUSED. 16 ortofungibles, 34 account rows, 80 nonce rows: every level agrees exactly, including the aggregate-vs-ledger check, which is the one that had never been run.

> THE FIRST RUN LOOKED LIKE FOUR DEFECTS AND WAS FOUR MISREADINGS OF ONE CONVENTION. Summing nonce supplies naively gave Z|VST short by 31.0, DDKOSON by 4.0, V|OURO by 1.0, and Z|OURO a NEGATIVE total of -1.0 against a supply of 0.0. Every one of those deltas turned out to equal that token's UR_NoncesExcluded exactly, because an excluded nonce is TOMBSTONED, not deleted: supply = -1.0 and holder = BAR -1.0 is a marker, not a balance. So the assertions below do not merely skip tombstones – that would hide the convention rather than test it. They pin it: the tombstone COUNT must equal UR_NoncesExcluded, no nonce may carry any other negative value, and supply = -1.0 must hold if and only if holder = BAR. MUTATION-TESTED, because an all-green sweep is exactly the shape a vacuous one has. TWO controls were needed, and finding that out was the point: 1] sentinel -1.0 -> -2.0 in this block: <>, <> and <> go red. 2] that control left <> GREEN – and the reason matters. 002e selects a nonce by its HOLDER, and a tombstone's holder is BAR, which is not an account, so no account ever picks a tombstone up no matter what the sentinel is. The aggregate-vs-ledger check is therefore independent of the tombstone convention, and needed its own control: flipping the holder match to != reddens <> and NOTHING ELSE. One perturbation reddening four assertions would have PROVEN LESS, not more – it would have meant the four were one assertion wearing four labels.

RT-J-003 · REFUSED

Runs in REPL/RedTeam/[RT-J2]_Conservation-Collectables.repl

What was suspected. DPDC stores possession in DPxF|T|AccountSupplies, keyed +BAR++BAR+, beside a per-nonce nonce-supply on the nonce row itself. Two tables, two writers, nothing comparing them – the shape that produced RT-J-001's live defect in DPTF.

How it was driven. enumerate every holdings row with URH_AS-Keys for BOTH sons, split each key on BAR (ids themselves contain bars – E|DH-... – so the middle segments are rejoined, never split on the first bar), and for each collection sweep EVERY nonce 1..NoncesUsed rather than only the nonces that happen to appear in a holdings row. A nonce carrying supply with NO holder is precisely the leak worth finding, and it is invisible to a sweep driven by holdings alone.

What happened. REFUSED. DPSF conserves exactly across every nonce. DPNF is consistent under its OWN rule, which is not conservation: supply is the constant 1, an ACTIVE nonce has exactly one full-account holder, and an INACTIVE one has none.

THE MISREADING THIS FILE EXISTS TO PREVENT. Summed naively, DPNF looks broken: six CNF nonces and one TSFN nonce report supply 1 against zero holdings. They are not leaks. nonce-supply for an NFT is documented as “Always 1 … even when burned or wiped” (01_DPDC-UDC.pact:55) and nonce-holder = BAR means the nonce is INACTIVATED. So <> pins the constant itself: if nonce-supply ever becomes a real quantity for NFTs, this goes red and whoever made that change is told that the sweep above now means something different. <> IS ALSO RT-D-002’s SAFETY PREMISE. That fix added a full-account possession check to UEV_NonceQuantityInclusion on the strength of one measurement – that AccountSupplies is reliably maintained for NFTs. Here that measurement becomes a standing assertion instead of a memory. MUTATION-TESTED, one control PER ASSERTION. RT-J-002 established why that matters: a single perturbation reddening several assertions proves they are one assertion wearing several labels, and a single perturbation that leaves one green proves nothing about it at all. Each of these reddens EXACTLY its own line and nothing else: 003a count rows with parts > 2 instead of < 3 003b compare DPSF supply against (held + 1) 003c flip the holder short-form match to != 003d flag inactive nonces with n = 0 instead of n != 0 003e assert the NFT supply constant is 2 instead of 1

Family K — Preview/execution divergence

RT-K-001 · FIXED

Runs in REPL/RedTeam/[RT-K]_PreviewParity.repl

What was suspected. cost parity is proven for all 401 previews; REFUSAL parity is proven for none. Where a preview re-derives state instead of sharing the op’s readers, the two can disagree – and an ATS pair with a zero index and no Hot-RBT (five exist at deploy) is exactly the input that separates them.

How it was driven. drive INFO_ATS| and TS01-C2::ATS|C_ at the SAME arguments and require the same message from both. Non-vacuity last: the same preview on a HEALTHY pair must still return a quote, or these lines only prove that previews throw.

What happened. CONFIRMED, then FIXED. THREE of seven ATS ops diverged, in three different ways: Fuel’s preview QUOTED a success the op refuses; ColdRecovery’s threw a raw table error naming a composite key and the caller’s full account; DirectRecovery’s threw an arithmetic exception. HotRecovery’s refused for a different (though true) reason than the op. Only Coil and Curl already agreed – because their guard lives in a reader BOTH paths share, which is the whole lesson. All seven now refuse in identical words, and a healthy pair still quotes normally.

WHAT WAS FOUND, of seven ops swept: Coil, Curl parity already – their guard lives in URC_RBT, which BOTH paths share. Fuel the preview returned a FULL QUOTE, ignis-need 0.53, with the post-text “Succesfully fueled 100.0 BI-… into ATS-Pair Bisthanium-…” for an op C_Fuel refuses outright. A confident prediction of success for something that cannot happen. ColdRecovery the preview threw No value found in table ouronet-ns.ATS_ATS|Ledger for key: <pool>|<the caller's full 162-char account> – a raw table error that also prints the account back at them. DirectRecovery the preview threw Arithmetic exception: div by zero, decimal. HotRecovery both refused cleanly but for DIFFERENT reasons – the preview named the missing Hot-RBT, the op named the toggle. Both true; still a divergence. AND THE BUG HAD THREE LAYERS, which is the part worth remembering. DirectRecovery carried it in the exec (fixed first), again in URCi_DirectRecovery (its own eager let), and AGAIN in INFO_ATS|DirectRecovery, whose wrapper re-derives release-amounts before it ever calls the cost reader. Fixing the first two left the quote still throwing. THE REPAIRS ARE ALL “SHARE THE GUARD”, NEVER “COPY THE MESSAGE”: * UEV_FuelableIndex – new module-local helper, called by ATSU|C>FUEL *and* URCi_Fuel, so the message exists once; * the Cold/Direct/Hot previews call ref-ATS::UEV_*RecoveryState – the very function their capabilities call; * URCv_RTSplitAmounts gets the zero-index guard once, covering all TEN of its call sites; * INFO_ATS|DirectRecovery binds its cost reader FIRST, so the op’s gate runs before the wrapper’s own derivations. ==== RT-K-001 - does the quote refuse where the op refuses · 02 · let / invocation ==== =========

RT-K-002 · FIXED

Runs in REPL/RedTeam/[RT-K]_PreviewParity.repl

What was suspected. RT-K-001 swept ATS. The same question of DPTF – the most-used family in the system.

How it was driven. drive preview and exec at a token that DOES NOT EXIST, and separately at an amount the holder cannot afford. Require the same message in the first case; in the second, record what each does and decide nothing.

What happened. CONFIRMED and FIXED for the structural case; the transient case is RECORDED, not changed. INFO_DPTF|Burn and INFO_DPTF|Mint returned full quotes narrating "Succesfully burned/minted 1.0 NOSUCHTOKEN-98c486052a51 …" for a token that has never existed, while their execs refused with UEV_id's own message. Repaired by calling UEV_id inside URCi_Burn / URCi_Mint – the readers BOTH paths share, so one wording serves both. The affordability divergence is pinned as-is under <> for the owner to rule on.

> THE LINE THIS BLOCK DRAWS, because getting it wrong would mean bolting validation onto 401 previews on my own authority: STRUCTURAL impossibility – the token does not exist, the pool index is zero, there is no Hot-RBT. Nothing the caller does in this block makes the op available. A quote here is simply WRONG, and INFO_DPTF|Burn was returning the post-text "Succesfully burned 1.0 NOSUCHTOKEN-98c486052a51 on Account …" – narrating success for a token that has never existed. FIXED: URCi_Burn / URCi_Mint now call UEV_id, the same check the exec uses, so both refuse in one wording. TRANSIENT affordability – the balance is too low, the role is missing. These change between the quote and the submission, and that is the normal life of a quote. INFO_DPTF|Transfer quotes 99,999,999 BUSD against a balance of 8,929,990 and says "Succesfully transfered …". That is defensible AS A COST QUOTE: post-text reads as the success TEMPLATE rather than a prediction, and a UI may well want the price before the user has funded anything. It is also arguably a dry-run failure. THAT IS AN OWNER DECISION, not mine, so <> pins the CURRENT behaviour rather than changing it – documented, and impossible to alter silently. ALSO OBSERVED, and left alone deliberately: driven at a non-existent id, DPTF|C_Transfer and its preview BOTH give the raw No value found in table ouronet-ns.DPTF_DPTF|PropertiesTable for key: – they are in PARITY, so it is not a family-K divergence, but neither carries the UEV_id check that C_Burn and C_Mint both have. That is the sibling-rule shape from RT-H-003 on a hotter path, and it is recorded rather than patched here because it changes a refusal MESSAGE on the most frequently called op in the system, which is exactly the kind of change that wants its own pass. ==== RT-K-002 - the same question of the most-used family, and where the line is · 02 · let / invocation ====

RT-K-003 · FIXED

Runs in REPL/RedTeam/[RT-K]_PreviewParity.repl

What was suspected. third family swept. Six DPOF ops driven at a non-existent id, preview against exec.

How it was driven. same as RT-K-001/002 – one input, both paths, compare the message.

What happened. CONFIRMED, then FIXED – one divergence of six, and five clean negatives. The DeployAccount preview quoted a token that does not exist; its exec died on a raw VerumRoles read DESPITE ALREADY CALLING UEV_id, which sat in the let body under a binding that reads that very table. Repaired by hoisting UEV_id above the binding group (the third time this exact shape has been repaired) and by calling the same function from the INFO wrapper. A real ortofungible still quotes normally.

Third Occurrence Of One Shape. C_Recover (fixed 2026-09-12), C_HotRecovery (RT-H-003), and here. A validation placed in a let body cannot protect a read placed in that let's bindings. The guard being PRESENT is what makes it hard to see – grep finds it, and the reviewer moves on.

> WHAT THE SWEEP RETURNED, and the negative half is most of it: RotateOwnership, TogglePause, AddQuantity, Burn, Mint IN PARITY. Both paths give the SAME raw No value found in table ouronet-

ns.DPOF_DPOF|T|Properties for key: .... Not a family-K divergence – the quote is exactly as wrong as the op, which is all family K asks. The shared raw WORDING is a separate, H-class gap and is recorded as known-open in DEFECT-LEDGER 7.3, together with the same gap on DPTF|C_Transfer. <> pins the PARITY, deliberately not the wording: it must keep holding if and when that wording is improved on both sides. DeployAccount DIVERGED. The preview returned a full quote narrating "Succesfully deployed a New DPOF Account for DPOF NOSUCHOFT-98c486052a51 …" – the wrapper only ever FORMATS , so nothing in it touched the token. AND THE EXEC SIDE WAS THE MORE INTERESTING HALF. C_DeployAccount ALREADY CALLED UEV_id. It sat in the let BODY, three lines under a binding group whose second entry is (create-role-account (UR_Verum4 id)) – a read of DPOF|T|VerumRoles. Pact evaluates every binding in a group before the body, so the check written for exactly this input could never run, and the op died with No value found in table ouronet-ns.DPOF_DPOF|T|VerumRoles for key: <id>. ==== RT-K-003 - the ortofungible family, and a guard that existed but could not run · 02 · let / invocation ====

RT-K-004 · FIXED

Runs in REPL/RedTeam/[RT-D2]_Ownership-Collectables.repl

What was suspected. fourth family swept for refusal parity. Five DPDC ops, preview against exec, driven at a collection id that does not exist.

How it was driven. as RT-K-001/002/003 – one input, both paths, compare the message.

What happened. CONFIRMED, then FIXED – one divergence of five, and it exposed an EXEC diagnosis bug as well. The preview died on a raw properties read; the exec answered cleanly but about a ROLE ON A COLLECTION IT HAD NOT ESTABLISHED EXISTS. UEV_id – which DPDC already has – now runs FIRST in DPDC-MNG|C>BURN-NFT and is called by the preview, so both say "DPNF ID does not exist". A real collection still quotes normally.

The Sweep. three of five in parity (DPNF/DPSF TogglePause, DPNF ToggleBurnRole – both paths give the same raw properties-table read). One mismatch of internals, recorded not fixed: DPNF MoveCreateRole's preview names DPNF|T|Properties while its exec names DPNF|T|VerumRoles – both refuse, both rawly, and they disagree only about WHICH table they failed to read. That is the same shared-raw-wording gap recorded for DPTF|C_Transfer in DEFECT-LEDGER 7.3.

AND ONE REAL DIVERGENCE, which also turned out to be an EXEC diagnosis bug: preview raw No value found in table ouronet-ns.DPDC_DPNF|T|Properties for key: … exec clean, but WRONG ABOUT WHY: "NFT Burn Role for NOSUCHCOL-98c486052a51 on Account … must be set to true for exec" The op answered about a ROLE ON A COLLECTION IT HAD NOT ESTABLISHED EXISTS. That message is true and useless: there is no collection to hold a role on. DPDC-MNG|C>BURN-NFT checked the role before existence, so UEV_id – which DPDC already has, and which yields "DPNF ID does not exist" – never got the chance to say the useful thing. WHY THE PREVIEW GOT ONLY THE STRUCTURAL CHECK, and not the role check its exec also runs: family K's rule from RT-K-002 is that a preview validates what the caller CANNOT change (the collection does not exist) and not what they can (a role they may be granted between quote and submission). Mirroring the role check would have bought message parity at the cost of turning 401 cost quotes into dry runs, which was decided against. ==== RT-K-004 - the collectable family: a role reported on a collection that does not exist · 02 · let / invocation ====

RT-K-005 · FIXED

Runs in REPL/RedTeam/[RT-K]_PreviewParity.repl

What was suspected. fifth family. SWP's previews are the most arithmetic-heavy in the system, so they re-derive more pool state than any other family's – the condition under which RT-K-001's divergences appeared.

How it was driven. first the cheap separating input, a swpair id that does not exist: FIVE ops swept (ChangeOwnership, ToggleAddLiquidity, ToggleSwapCapability, UpdateFee, RemoveLiquidity) and ALL FIVE

were already in parity, both paths giving the same raw SWP|Pairs read. A clean negative. Then the structural one that matters here: the CHILLED doors on a pair whose frozen-LP functionality is off.

What happened. CONFIRMED, then FIXED. The bad-id sweep found FIVE clean negatives. The structural one found the defect: both chilled previews died on DPTF ID | does not exist – the BAR sentinel reaching a token-existence check – for a pair whose real problem is that frozen LP is off, and whose exec does not say so until STEP 1, after step 0 has taken the initiation fee. Both previews now refuse with the door's own wording, before any money moves. The STANDARD door still quotes the same pair and amounts.

> WHY THIS ONE IS WORTH MORE THAN A TIDIER MESSAGE. SWP|C_AddIcedLiquidity is a DEFPACT. UEV_AddChilledLiquidity refuses the door on a pair without frozen LP – but it refuses in STEP 1, and RT-A-002 already measured that STEP 0 SUCCEEDS AND COLLECTS THE INITIATION FEE. So the quote is the only chance a caller has to learn the door is shut BEFORE paying to find out. And what the quote said instead was DPTF ID | does not exist. With frozen LP off, the frozen- token getter returns the BAR sentinel, which reached a token-existence check – a true statement that names the SEPARATOR as if it were a token, about a token the caller never mentioned. Third family in which the BAR sentinel has surfaced as a user-facing message (RT-H-003, RT-K-003). ONLY THE FROZEN-LP HALF IS CHECKED in the preview. UEV_AddChilledLiquidity's other enforce depends on the amounts, which the caller controls and can change between quote and submission – transient, and family K validates what the caller CANNOT change (RT-K-002). ==== RT-K-005 - the swap family: a quote that could not warn about a door it knew was shut · 02 · let / invocation ====

RT-K-006 · FIXED

Runs in REPL/RedTeam/[RT-K2]_PreviewParity-Launchpad.repl

What was suspected. the citizen launchpad previews were never swept. Their separating input is not a bad id – the sale is a live object – but an AMOUNT no sale can honour.

How it was driven. the sale's own gate amount <= remaining-supply is deliberately NOT used here: the remaining supply changes as other buyers act, which makes it TRANSIENT, and family K validates what the caller cannot change (RT-K-002). An amount of zero or below is invalid whatever the chain does, so it is structural, and it is the right probe.

What happened. CONFIRMED, then FIXED, in both directions. Buying ZERO was QUOTED by the preview and refused by the op; buying -5 was refused by both but in DIFFERENT WORDS, the preview borrowing "Deposit amount must be non-negative" from a royalty helper it happened to reach first. The exec's inline enforce is now UEV_DepositDollarAmount, called by DEMIPAD|C>DEPOSIT and by URCi_Deposit – so ONE definition serves every sale on the launchpad, demonstrated on StoicPay as well as Spark. A sane amount still quotes.

> WHAT WAS FOUND, in both directions at once: buy 0 the PREVIEW QUOTED IT. The exec refuses "Invalid Dollar Amount for Deposit". buy -5 both refused – with DIFFERENT MESSAGES. The exec gave its own; the preview gave "Deposit amount must be non-negative", which belongs to UCv_ComputeDepositRoyalty and was reached incidentally on the way to a price. A refusal borrowed from whichever helper happened to notice first is not the op's answer, it is a coincidence. THE FIX POINT IS WHY THIS ONE IS WORTH ITS RUNTIME. Every launchpad sale – Spark, Snakes, Custodians, StoicPay – prices its purchase through DEMIPAD::URCi_Deposit, and every one of them executes through DEMIPAD|C>DEPOSIT. The check lived INLINE in the capability, so the reader had no way to share it. Extracted to UEV_DepositDollarAmount and called by both, ONE definition now serves every sale on the launchpad. ==== RT-K-006 - the launchpad quote, at an amount no sale can honour · 02 · let / invocation ====

RT-K-007 · FIXED

Runs in REPL/RedTeam/[RT-D2]_Ownership-Collectables.repl

What was suspected. seventh surface – the AQP readers, the most derived cost logic in the codebase.

How it was driven. three ops driven at an id that does not exist (RevokeAnchor, RotateScoreOwnership, ControlScore). The last two were already in parity. RevokeAnchor was not, and chasing it reached something better than a preview defect.

What happened. CONFIRMED, then FIXED on the exec side, and the preview half is RECORDED with its reason. The exec's refusal for a non-existent anchor moved from `No value found in table ouronet-ns.AQP-ANK_ANK|T|Anchor for key: <id>` to `Anchor <id> must be alive for operation` – the sentence UEV_LiveAnchor was written to say and could never reach. Two of the three ops swept were already in parity.

> `UEV_LiveAnchor` EXISTS AND IS WRITTEN FOR EXACTLY THIS INPUT: (enforce iz-anchor-active (format "Anchor {} must be alive for operation" [anchor-id])) It could not deliver that sentence. `iz-anchor-active` comes from `UR_ANK|State`, which was a bare `read` – so for an anchor that does not exist the raw `No value found in table ouronet-ns.AQP-ANK_ANK|T|Anchor for key: <id>` fired one line earlier, every time. The validator was reachable for anchors that exist and unreachable for the only input that needed it. THIRD INSTANCE OF ONE SHAPE, and the first where the obstruction was a READER rather than an eager `let`: C_Recover (2026-09-12) and C_HotRecovery (RT-H-003) had their capability below the bindings; C_DeployAccount (RT-K-003) had its check below the bindings; here the check is in the right place and the value it reads raises before it can be tested. Same lesson from a new direction: A GUARD BEING PRESENT IS NOT A GUARD BEING REACHABLE, and `grep` cannot tell the two apart. Fixed by defaulting `UR_ANK|State` – an anchor that does not exist is not active, which is what both of its callers mean by the question (the other, an anchor filter, already screens BAR and wants false for anything not alive). Same shape as DPTF's `UEV_id`, which defaults to -1.0 for exactly this reason. THE PREVIEW HALF IS RECORDED, NOT FIXED, and DEFECT-LEDGER 7.3 carries the reason: `INFO_AQP-ANK|RevokeAnchor` still quotes a revoke of an anchor that never existed, post-text "Anchor revoked." Guarding it is a two-line change, and it breaks `[6.5]_AQP-INFO.repl` – a DELIBERATELY FIXTURE-FREE cost-shape suite that passes arbitrary ids ("ANK-x", "DPTF-x", "BC-x") to all 83 AQP readers on the sound principle that AQP prices are ARGUMENT-INDEPENDENT. Making AQP previews validate ids would require anchor, score and boost-class fixtures for every one of them. That is a real piece of work with a real design question inside it, and it is not something to smuggle in behind a one-line commit. ==== RT-K-007 - a third guard that existed and could not run · 02 · let / invocation ==== =======

RT-K-008 · FIXED

Runs in `REPL/Stage_02/[6.3]_STOAICO.repl`

What was suspected. `INFO_Collect` was the ONE client-facing cost preview in the tree named by no test at all – found not by suspicion but by counting the population, after the coverage tool that reports on previews turned out to scan three HARDCODED files and to have reported "401 of 401 measured, 0 never named" about a set it had defined to exclude the gaps. An unmeasured preview is a number a client is shown and nobody checks.

How it was driven. read the preview's `ignis-need`, then drive the real collect through the citizen Talos and measure the patron's IGNIS balance across it. Preview against charge, one transaction, nothing else moving.

What happened. FIXED – and the gap was not small. Preview said **88.0** (mint leg 87.0 + transfer leg 1.0) for an operation that charged **1.0**.

> WHY, AND WHY IT IS THE NORMAL CASE RATHER THAN AN EDGE. `XI_CollectFor` used to mint urSTOA unconditionally; the 2026-09-14 vault-deadlock fix moved that mint INSIDE a `(!= urSTOA-supply 0.0)` guard, because a zero-amount mint is refused and the vault deadlocked. That fix repaired the EXECUTOR and left `URCi_Collect` – the reader the preview wraps – with its mint still outside the guard. And the collect that zeroes an account's urSTOA is its FIRST one, so the two paths disagreed on EVERY SUBSEQUENT ROUND. The disagreeing case is the ordinary one. The fix mirrors the executor's branch into the reader. This block asserts the state that makes it the interesting case, so a fixture drifting to non-zero urSTOA turns the PRECONDITION red rather than quietly making the parity check trivial.

---

22. Part IV — The instruments

An audit is only as good as the things it measures with. This chapter records the defects found in the **measuring tools** rather than in the contracts, and it is here for a specific reason: several of them were more consequential than the contract defects in Chapter 20, because a wrong instrument does not fail — **it reports.**

Every item below was found during this round, and every one had been reporting confidently before it was found.

The pattern

Fifteen instrument defects, and the majority are one shape:

> **A measurement that silently excludes part of its own subject, and therefore reports `clean` or a ratio about a population it never examined.**

An excluded item is not reported as a gap. It is **absent**, which reads as neither a problem nor a success. That is strictly worse than a known gap, because a known gap gets worked and an invisible one does not.

The coverage instrument reported ratios without their exclusions — twice

`_ownerobs.py` spent days reporting *"39 of 112 ownership gates witnessed"*. The tree contains **185.** Seventy-three were outside the denominator, including the entire token-DEBIT layer — the gates that stop a stranger moving somebody else's tokens. Chapter 19 tells that story in full.

The second time was subtler and is the one worth generalising. The tool's own documentation already catalogued this error **in other tools**. It recurred in the tool written to audit it.

The tool index was wrong about a quarter of the tools it indexed

The repository documentation names `TOOLS.md` as the safe way to learn what a tool does — explicitly, *"never run a tool to find out what it does; read its docstring, or check the table"*. That guidance exists because a loop that imported five source-rewriting tools *"just to prove they load"* had silently deleted 111 lines of schemas from a contract.

The index was wrong about **13 of 49 entries**. The documented alternative to the dangerous action was itself unreliable.

Five tools reported on populations they had defined to exclude the gaps

This is the dominant failure mode in the whole record, and it is worth naming precisely because each instance looked different and none looked like a bug.

tool	enumerated	missed	what it reported
path checker	3 tool directories	a 4th, holding a tool that	`clean`

tool	enumerated	missed	what it reported
		rewrote contract sources by default	
owner-gate mapper	4 of the 8 documented client prefixes	an entire batch-operation family	a shrinking worklist
preview coverage	**3 hardcoded files**	14 previews, one of them never tested at all	**428 of 441, 1 gaps**
stats generator	2 log file *extensions*	a run written with a third	a report from **the previous day**
attack register	one directory	an attack that had to live beside its fixtures	a total of 37 where 38 existed

Five tools. Five hand-maintained lists. **No list can report its own incompleteness**, so each reported confidently about a population it had never seen in full — and the preview tool reported a *perfect score*, which is worse than a gap, because a perfect score ends the enquiry.

The remedy in every case was the same: **discover the population, then check the list against it.** Where discovery is impossible — a figure that needs a live run the tool cannot perform — the honest move is to say the tool cannot see it, not to check a number it cannot derive.

Two checks that enforced what they should have questioned

Worse than a tool that misses something is a tool that **locks in the error**.

- The **price-sheet generator** computed its headline by summing three of its four categories, dropping the rows priced in one currency rather than another — publishing **431** where the sheet listed **442**. And the artefact checker *required the narrative to quote that total*. **The gate enforced the undercount and would have gone red on anyone correcting it.**
- The **figure checker** verified that every narrative document agreed with a generated stats file, and never that the stats file agreed with the tree. Perfectly circular: **every figure in every document matched, and all of them were wrong together** — by 275 assertions and counting. Closing the loop moved four published figures at once, the first movement in weeks.

> A consistency check between N documents and one source proves the N documents consistent. It says nothing about the source — and a stale source reads *more* convincingly than a correct one, because everything agrees.

The path checker scanned three of the four tool directories

`_toolpaths.py` exists to catch tools that reference paths which no longer resolve — a real incident, in which moving the tool directory killed eleven tools that died at *import*, so nothing that diffed their output could see it.

It enumerated three tool directories. There are four. The fourth held a tool that **rewrites contract sources and wrote by default** — the exact inversion of the repository's own rule that source-rewriting tools must require an explicit `--apply` flag — while being invisible to the checker.

The comment above that enumeration had been added earlier, specifically to warn against under-enumeration. **It under-enumerated.**

> That is the argument for discovery over enumeration in one line: a hardcoded list **cannot report its own incompleteness**, so it says `clean` about what it never opened. The fix was not to add the fourth directory — it was to make the checker report any directory holding tools that its list does not cover, and to verify that report fires by planting one.

Three metrics that were wrong in the direction of a tidier number

Chapter 19 records the binary → dilution → depth progression in full. The generalisable part:

> **A single number chosen to summarise a distribution is a claim about that distribution.** The binary scored "1 of 63" and read as *the observed column is worthless*; it was technically true and substantively false. It was only caught because the distribution was printed before the summary was published.

Two further corrections in the same instrument went in **opposite** directions within an hour — first discarding real work by treating a capability-composition edge as a wall, then recovering it but still filing an entire band as unreachable because the filter matched four of the **eight** documented client prefixes. The first error shrank the worklist by more than half.

> A worklist that shrinks reads as progress, which is why errors in that direction are the ones that survive review.

Two mistakes made while checking for mistakes

These are recorded because they are the most instructive items in the chapter.

A truncated shell pipeline nearly produced a false finding against the audit record. A search for the regression test protecting a fixed defect was piped through `head -6` and showed nothing — making a previous audit's *"adversarially proven"* look like an unretained claim. The tests exist, at two named lines. A confident wrong answer from a pipeline artefact, and the second such artefact in two days.

A mutation test reported success without having applied its mutation. While verifying a fix to the coverage instrument's blind spot, a literal was patched, the tool re-run, and the result recorded as *"10 → 10, unchanged"* — from a patch that had replaced **zero occurrences**. The slicing matched nothing.

> **A mutation test that reports "no change" is indistinguishable from a mutation that never happened.** The remedy is a fixture where non-application is impossible: the verification is now three synthetic snippets — inline, hoisted, and an unrelated binding that must **not** credit — where the negative control is what keeps the fix honest.

This one is worth stating plainly: it happened one screen after writing a negative control into somebody else's test, in the instrument built to audit exactly this class of error, by the person auditing it.

Two process defects

Source-rewriting tools that ran on a bare invocation. Two REPL formatters rewrote the tree without `--apply`, and because one of them also performs the other's work, running both duplicated every banner it inserted — **188 files, 16,457 insertions** during a routine tool census. Nothing was lost because the tree was committed. *"The tree was committed" is not a safety property.*

`git add -A` in a tree with concurrent work. Committing while other work was in flight swept other agents' in-progress scratch state into commits that had nothing to do with it — including deliberately-failing probes, which made two test suites red **by construction** under commit messages describing something else entirely. Both were repaired by later commits. *"The next commit fixed it"* is the same sentence as *"the tree was committed"*, and it is not a safety property either.

It also **misattributes authorship**, which matters more than it sounds: this entire audit's value rests on the record of who established what, and with what evidence.

What was added

instrument	what it protects
_ownerobs.py	which ownership gates have actually refused somebody, at what evidence depth
_modref.py	cross-module calls to members that **do not exist** — invisible to every other check, because the language resolves them dynamically and only raises if the branch is taken
_toolpaths.py *(extended)*	now discovers tool directories rather than listing them, and says so when it finds one it does not cover
_redteam.py *(extended)*	the attack register and the defect ledger must agree; a defect recorded in only one is one the audit undercounts

The _modref.py result is worth stating because it is a **clean** one: of the calls it flags, the class that is genuinely dead — a call to a member defined nowhere — numbers **13, every one of them inside the legacy module** the architecture marks as frozen. **Live code: zero.** That zero is now gated rather than remembered.

> It also refutes a finding. A reviewer flagged a single instance of the *other* class — a member that exists but is not declared on the interface — as a coupling defect. Counting the population first showed **106 live instances**, one member accounting for 57. That is the convention, stated as such in the architecture documentation. **A single instance cannot tell you whether it is a defect or a dialect.** Fixing it would have made the tree less consistent and reported a practice as a bug.

Turning the instrument on the book

(Added 2026-09-18, when this book was consolidated into a single volume.)

Every failure mode in this chapter is about a measurement that was wrong while looking right. The obvious next question is whether this book — which is nothing but measurements — has the same problem. It was asked directly: a full re-derivation pass over all nineteen chapter sources, checking every quantitative claim against the tree rather than against the other chapters.

Result: 124 stale figures, against roughly 516 that still held.

Not one had been written carelessly. Every one was correct on the day it was typed and the tree moved underneath it — the assertion total, the preview count, a dozen file:NNN citations whose line numbers had drifted, an entire section documenting a defect that had since been fixed. Two chapters came back completely clean.

Three things follow, and they are the reason this section exists rather than a quiet round of edits.

A book about stale figures had 124 stale figures. Knowing the failure mode confers no immunity from it. The chapters that were *most* careful — the ones that footnote their method and name their denominator — were not meaningfully less stale than the rest, because carefulness is a property of the moment of writing and staleness is a property of elapsed time.

Hand-correcting 124 numbers would have fixed nothing. It resets the clock and changes no mechanism. So the volatile figures are no longer written in the chapters at all: a source now writes 29,531 or 428, and the assembler substitutes the number it **measures at build time** by running the tool that owns it. About eight seconds per build. A figure that cannot be measured is a build failure rather than a fallback to the last known

value — a default would be a stale figure with extra steps. Twenty-two figures are wired this way, and that class of drift is now closed rather than corrected.

One of the 124 was not stale, and how it failed is the chapter's own lesson. The sweep reported that Chapter 7's closing claim was false: the chapter cites SWP|TX 015b as the live witness for finding #72C, and the file that holds it — Stage_01/[6.2+3]_DPTF-SWP_Issuance-Only.repl — is in _gate.py's EXCLUDED list. The reasoning is sound and the premise is true. The conclusion is wrong.

EXCLUDED is a list of files not run **as entrypoints**, for the stated reason that this one is an *alternative* to [6.2]+[6.3] and would duplicate their work. It says nothing about reachability. Stage01_Tester.repl loads the file at line 42, and thirteen files load Stage01_Tester.repl, of which eight are gate entrypoints. Running one of them and grepping the output settles it in two minutes:

```
"--- [SWP|TX 015b - #72C Regression: Stable-Swap Inverse Newton Domain Guard · 02 · let / invocation] ---"
"Expect: success #72C regression — in-domain inverse quote (output=0.5x reserve) is positive"
```

The witness runs, several times per gate run. The chapter's claim stands.

This is the book's own rule — **locate by execution, not by reading** — failing in the one place it would be most embarrassing: an audit *of* the book, by a reader who had the rule available and reached a confident, well-argued, false conclusion from a list. Exclusion is not unreachability. A name absent from a runner's entrypoints is not a name absent from the run, and only running it can tell you which.

---

23. Part V — The canon sweep

> Source records: OuronetInformational/HANDOFFS/HANDOFF-executor-canon-sweep.md (the canon, the 46-module worklist and eighteen recorded lessons), Audit/AUDIT-V2-DELTA.md (a per-module entry for all 46), and OuronetInformational/StoicSyntax-Prefixes.md §2.2 (the rule itself).

The question this round asked

Parts I and II asked *"is this right?"*. Part III asked *"can I make it do something it should not?"*. This Part asked a third question, and it is the one that turns out to be hardest to answer by reading:

> **"Who did this?"**

Not *may* they — that is authorisation, and the red team spent a round on it. **Who.** Ouronet is a gas-station system: a **patron** pays for a transaction that somebody else performs. So every client entrypoint has at least two accounts in it whether or not it names them, and before this round the great majority named only one.

The canon states the fix in one line:

> Every A_/AA_/C_/CC_ entrypoint takes **patron first, executor second, executee third** when one exists. Position is canon. P| policy functions are exempt.

- **patron** — who *pays*.
- **executor** — who *acts*. The account the ledger should name when asked who did this.
- **executee** — who is *acted upon*. Validated, sometimes not consulted at all, never the actor.

Position, not merely presence. Pact arguments are positional, so an executor sitting fourth is non-conforming, and correcting a signature **rewrites every call site that reaches it** — in the modules, in the Talos wrappers, and in every REPL fixture.

The numbers

46 modules. 776 entrypoints. 0 remaining. Three days, 57 commits, 2026-09-20 to 2026-09-22.

measure	at the start	at the end
A_/C_ entrypoints conforming to the canon	302*	**776**
…with the executor's ownership PROVEN	—	**756 of 756**
entrypoint signatures changed vs. the pre-sweep baseline	—	**694** in 58 files
assertions executed per full gate run	25,234	**29,531**
distinct assertions written	5,903	**6,619**
unregistered patron slots	unmeasured	**0**

* *That number was wrong, and the error is the first thing this Part has to report.*

The plan was wrong by 63% on its first day

The refactor was scoped against _bandplan.py, whose entrypoint filter read, in effect:

```
re.match(r"^(A|AA|C|CC)_" + "|" + r"\|(A|AA|C|CC)_", n)     # the broken form
```

re.match anchors the **whole** pattern at position 0, so the second alternative could only ever fire on a name that *starts* with a bar. Every Talos entrypoint — ATS|A_KickStart, DPTF|C_Issue, all 482 of them — was invisible. The tool reported **302 done, 474 remaining** against an actual **776**, and the programme was scoped, planned and reported against that number for its entire first day.

> **Talos is the only supported client path in this system.** The blind spot was not a corner of the surface; it was the client surface.

What found it was a *second* tool, _executorplan.py, written specifically to re-derive the same worklist by a different route. That is the reason it exists, and it is the first entry in this Part's method: **when a number decides the scope of the work, compute it twice.**

What this Part contains

chapter	what it covers
Chapter 24	The four shapes an entrypoint can have, and how to tell them apart
Chapter 25	What the sweep found — three live defects, two wrong published prices, and an authorisation surface measured entrypoint by entrypoint
Chapter 26	The instruments, and the eighteen lessons the round paid for

What the system gained

Four things, and only the first is what the round set out to do.

1. The ledger can answer "who". Every A_/C_ names the acting account in a fixed position, and that account's ownership is proven on every path. The @event a capability emits now carries an actor that the transaction had to sign for, rather than one derived inside a capability and never written down — or, in 322 entrypoints, never named at all.

2. Sponsorship became possible where it had been silently forbidden. Separating *pays* from *acts* is not only an audit-trail question. Before the separation, several entrypoints enforced (= patron owner) — forcing the **gas payer** to **be** the asset owner. No sponsor could pay for a vault owner's operation, and the gasless patron was unusable across that whole surface. Those enforces were the *symptom* of one parameter doing two jobs; the canon removes the need for them.

3. Talos wrappers became mechanical. With a fixed (patron, executor, executee, …) prefix, what a Talos wrapper must do is determined by the canon rather than decided per function: a C_ passes the caller's patron through, an A_ supplies GASLESS-PATRON itself. That is why a call-site checker can be gate-fatal at all — there is now one correct shape to check against.

4. The authorisation surface is written down. For the first time, AUTH-SURFACE.md records per entrypoint every ownership enforce reachable through its capability chain, and the gate refuses any commit in which an entrypoint's set **shrinks**. "We did not weaken anything" stopped being a claim and became a check — which is how the sixty newly-gated entrypoints in Chapter 25 were counted rather than asserted.

Why a renaming exercise found defects at all

It should not have. Renaming a parameter changes nothing a user can observe. The reason it found three live defects, two wrong published prices and sixty entrypoints that gained an ownership gate is that **the canon forces one question at every single entrypoint, including the ones nobody suspected**:

> *Which account does this function's authority actually prove — and is it the one the signature names?*

Answering that 776 times is not a renaming exercise. It is an audit with a fixed question, applied exhaustively, where the renaming is merely the artefact left behind. Three of those 776 answers were *“neither — it proves nothing at all.”*

---

24. The canon, and the four shapes a proof can take

Three roles, one transfer

The rule is easiest to see on the operation that needs all three:

```
(C_Transfer patron sender receiver id amount method)
;;          ^     ^     ^
;;          |     |     executee -- acted upon
;;          |     executor -- acts
;;          patron -- pays
```

role	what it is	what must be true of it
patron	the account that PAYS	ownership enforced **always, directly**, via the gas-collection path
executor	the account that ACTS	ownership enforced **always** — directly, or indirectly by a route the function's own `@doc` names
executee	the account acted UPON	enforced **conditionally**, and the conditions must be named in the `@doc`

The executee's conditionality is not a weakening. A receiver needs an ownership check only when the transfer is *method*-style and the receiver is a smart Ouronet account; for an ordinary credit, demanding the receiver's signature would make it impossible to pay anyone who is not online. The canon's demand is that whichever rule applies is **written down in the function**, not inferred from the capability three modules away.

Position is canon, not just presence

Pact arguments are positional. An `executor` sitting fourth is non-conforming even though every check on it passes, because the next caller — reading the signature rather than the body — will put the wrong account in the slot. The practical consequence dominated the whole round:

> **Correcting a signature rewrites every call site that reaches it.**

Not only the direct ones. A core module's `C_` is reached through a Talos wrapper, that wrapper is reached from citizen modules and from REPL fixtures, and a single reorder in Stage 1 propagated into 172 call sites in one turn, 86 in another, 31 in a third.

And there is a specific hazard in that: **Pact checks modref call arity at RUNTIME, not at load.** A call left one argument short compiles, deploys, and partially applies into a *closure*. Worse, an `expect-failure` **absorbs** it: the test fails as expected, green, for the wrong reason. This round found six such call sites, two of them inside `expect-failure`s that had looked green for weeks. That is why `_callarity.py` is gate-fatal rather than a report.

Why the executor is unconditional — the attribution rule

Stated by the owner mid-round, after it was asked once too often:

> **An unenforced executor is WORSE than none.**

A parameter nobody checks is one the caller chooses freely. An entrypoint that takes an `executor` and never proves it does not record who acted; it records *whoever the caller felt like naming* — and the `@event` it emits can implicate an account that was nowhere near the transaction. A missing executor is visibly missing. A decorative one **looks like attribution**, which is the property that makes it worse than the gap it appears to fill.

Admin operations are where this goes wrong, and for an understandable reason: the GOV|*_ADMIN keyset already opened the door, so the executor *feels* like a label — and a label is what it becomes. Found in exactly that shape on the second day, in three DPTF treasury admin ops that took an executor and never mentioned it again.

The four shapes of proof

_executorenforced.py accepts exactly four, and refuses everything else:

verdict	what it means
DIRECT	CAP_EnforceAccountOwnership / UEV_Executor* / CAP_Owner runs on the executor, in the function or in a capability it acquires (two levels of compose-capability are followed)
FORWARDED	the executor is handed to another module's entrypoint **in the executor position** — the Talos-wrapper and cross-core shape
INDIRECT	registered in the tool **with the route**, which must *also* appear in the function's own @doc
SELF-PROVING	account creation, where the executor is the account being created and proves itself with the guard it supplies

Two of these deserve their reasoning spelled out.

FORWARDED is position-aware on purpose. It matches (ref-TFT::C_Transfer patron executor receiver ...) and deliberately does **not** match (ref-TFT::C_Transfer patron sender executor ...) — in the second, executor is sitting in the executee slot, and a callee proving its *own* executor proves nothing about this one. The check passed one function for a day on the strength of a CAP_Owner two arguments away, which is exactly the conflation the position rule exists to stop.

INDIRECT is not a waiver. Registering a route in the tool does not silence it: the function must still *say* the route in its @doc, and the tool re-reads the @doc on every run. So the justification lives beside the code, and deleting the sentence breaks the build. This is the canon's *"the path MUST be named"* clause, made mechanical.

SELF-PROVING is the base case of the attribution rule, resolved by the owner after being raised as an open question. In the four deploy entrypoints the executor is the account *being created*, so its ownership cannot be read from a table — there is no row yet. It does not need to be: the **guard** the account will be governed by is enforced in the same transaction, which is the same proof and the same key, arriving as an argument because there is nowhere else it could come from. The claim is load-bearing and is pinned by a negative test: a deploy whose guard the caller does not hold is refused, and refused *before* the format checks — so if that list element were ever dropped, account creation would become unauthenticated and the registry entry would silently become false.

Four reasons a real proof is invisible

The round's most repeated surprise: a function reports UNPROVEN, and the proof is *right there*. Four distinct shapes produce that, and the correct response to every one of them is to **register the route and write it in the @doc** — never to add a second enforce to satisfy a tool.

1. **Proven in place.** The capability binds (= executor owner-konto) and then enforces ownership of owner-konto. The matcher looks for the enforce applied to executor; here it is applied to the name executor was just proven equal to. Correct code, invisible shape. Eight AQP score entrypoints and six DSA entrypoints are this.

2. **A same-module hop.** The executor is forwarded into an XI_ in the *same* file. FORWARDED scans for a cross-module ref-X:: call by design — a foreign module is what does the proving — so an internal hop has to be traced by a human and written down.
3. **A defpact step.** The entrypoint acquires its capability and then starts a defpact; the call carrying the executor lives in a *step*, so the entrypoint's own body contains no forwarding call at all.
4. **An intra-Talos sibling.** A thin alias delegating to another wrapper in the same Talos file — the same internal-hop shape, one layer further out.

Sometimes the answer is no executor

Not every entrypoint has an actor. Two anchor-sync repairs were given **none**, and that is the finding rather than a gap. The test that settles it, in order:

1. Is any account on the path ownership-checked? If not, ask **why** before adding one.
2. Is the op idempotent truth-restoration? These recompute a ratio from actual balances — every outcome is the correct one.
3. Who pays? The patron. A caller can only make someone else's data correct **at their own cost**.
4. Would a signature requirement remove a legitimate path? Yes — the party who *notices* a stale anchor is usually whoever issued it, not the beneficiary.

All four, and the verdict is **EXECUTORLESS: the account in the signature is an executee.** Six entrypoints ended there, registered with their reasoning attached, and the two *different* rationales kept visibly separate — permissionless repair in one case, authority-is-a-guard in the other. A shared register with one blurred justification would have been the easier artefact and a worse one.

Where Talos diverges

	core module signature	Talos wrapper signature
C_	(patron executor …)	passes the **caller's** patron — sponsorship preserved
A_	(patron executor …)	**no patron** — the wrapper supplies GASLESS-PATRON itself

An A_ is gasless **because its ordinary collection is served by the one account the IGNIS collector exempts**, not because collection is skipped. The path is preserved, which matters: a skipped path cannot be regression-tested, and an exempted account can.

The intermediate state nobody can see

One consequence of sweeping a dependency graph module by module has no clean solution, only an honest one. While module *N* is being swept, a caller in module *N+5* has no patron to thread, so slot 0 carries whatever account initiates the call — client, culler, a service-account constant. That is **correct today and wrong after the caller's turn**, and it is invisible: the arity is right, the value is unused by every swept callee, and no assertion can reach it.

So it is written down. _patronslots.py carries every such site with its evidence, distinguishing **permanent** (the patronless OUROBOROS family, where patron = executor is the truth) from **provisional** (re-point at that module's turn), and the gate is fatal on an **unregistered** one. The register ended the round at nineteen: fifteen permanent, four provisional, all four in one module, and those four *measured* inert — both callees use patron zero times in their bodies once the signature and doc string are stripped. An inert wrong value is the most patient kind, and the only thing that keeps it from becoming a live one is that somebody wrote it down.

---

25. What the sweep found

Three live defects, two wrong published prices, one module that no instrument was looking at, and an authorisation surface that had to be measured twice before it told the truth. None of these was the object of the exercise. Every one of them was found by asking the same question at an entrypoint nobody suspected: *which account does this prove, and is it the one the signature names?*

Defect 1 — a revoke that checked no account at all

ANK|C>REVOKE-BOOST-CLASS. The capability validated exactly two things: that the BoostClass was **empty**, and that it was **active**. It checked **no account**. Any account reachable through Talos could revoke any empty BoostClass that was not theirs.

What makes this one instructive is the shape of the mistake. A boost class **owner** field had been added earlier in the audit programme, as the fix for a different hole — attaching an anchor to somebody else's class. That fix landed on the **attach** path and enforced class-owner there. The **revoke** path was never carried over. So the module had the field, had the enforcement, and had a door that used neither.

> A partially-applied fix is the most convincing kind of absence: the field exists, the enforcement exists, and the grep that would find the gap succeeds.

The repair enforces both halves — that the executor *is* the recorded owner, and that the transaction is *signed* for that account:

```
(defun UEV_ExecutorIzClassOwner (executor:string boost-class-id:string)
  (let ((co:string (at "class-owner" (UR_BC|Data boost-class-id))))
    (enforce (= executor co) "... is not the BoostClass's owner ...")
    (ref-DALOS::CAP_EnforceAccountOwnership co)))
```

Both are needed. The equality alone would let anyone *name* the owner; the signature alone would prove the caller owns *some* account, not this class.

Defect 2 — an admin keyset says MAY, never WHO

Four 00_Demipad admin operations — register an asset to the launchpad, define its price, toggle it open for business, toggle retrieval — ran under a GOV|*_ADMIN keyset and took **no account argument at all**. So the events they emitted named nobody, and the audit trail for a launchpad price change was the keyset that permitted it rather than the administrator who made it.

This is the general failure the canon exists to catch, and it hides behind a true statement: *"only an admin can call this."* True, and irrelevant to the question being asked. **Authority and attribution are two different gates.** A keyset held by several people answers *may they*; it cannot answer *which one*.

It was confirmed by measurement rather than by reading. With the new guard disabled, A_DefinePrice **succeeded**, returning:

```
"Asset TSFS-... price succesfully updated with the Price Object {"pid": 1.0}"
```

An administrator could change a launchpad price and have the event name somebody else. All four now take patron and executor and run CAP_EnforceAccountOwnership on the executor **before** the admin capability — authorisation first, then attribution, in that order.

The regression test for this one has an unusual shape worth recording: it is a **rollback-tx**, not a commit-tx, because the subject **mutates when the guard is absent**. A test that proves a guard is missing by successfully performing

the forbidden act has to leave no trace, or the next assertion is reading a state the suite created by attacking itself.

Defect 3 — dead code that published a wrong price

DPDC-T::C_RepurposeCollectable and DPDC-F::C_RepurposeCollectableFragments each opened with a let binding an entire duplicated cost model — owner, the small and medium price legs, the sum of amounts, the computed price, the trigger — and then **read none of it**. Dead bindings, of the kind the sweep learned to expect: the derived actor is so obviously the subject of the operation that somebody bound it by reflex, and the signature had nowhere to put it.

This one was not merely dead. The IGNIS price sheet derives a function's price by reading the tier legs its body **mentions**, and the dead block mentioned **both** tiers where the live code charges one — (if son s m) picks exactly one of them. So the published price sheet quoted a floor of **5** for an operation whose real floor is **2** for a semi-fungible and **3** for a non-fungible. Deleting the dead block corrected the published figure.

> Dead code is usually harmless because nothing reads it. **A generator reads everything.**

Two prices that rested on a word

The same generator produced two more corrections, and together they make a rule.

A comment classified a price. The sheet's SCALES test — does this operation's cost grow with the size of its input? — matched the literal phrase per-nonce, and it runs over text that **includes @doc prose**. C_RepurposeCollectable's doc says *"per-nonce"*; C_RepurposeCollectableFragments's says *"per-fragment"*. Identical price shape, different classification, decided by a hyphenated word in a sentence. Replaced with structural patterns that match the **form** that actually scales — (dec (length ...)), a fold (+) 0.0, a multiplication by a counted quantity. The published tally moved from *185 exact · 135 floor* to *183 · 137*: two operations that had been quoted an exact price they could not honour.

And documenting a function raised its price. The deterrence worksheet counts a function's cost legs — inserts, updates, reads, scans, cross-module hops — from its source text, transitively. It counted them in ;; comments and inside @doc strings too. Measured the day it was found: **24 ref-x:: tokens inside @doc strings and 25 inside ;; comments**, forty-nine phantom cross-module hops, plus phantom reads and scans, inflating the design figures for **29 published rows**. The trigger was as ordinary as it gets: eight functions gained an @doc explaining how their executor is proven, and the act of explaining it made them more expensive.

Both corrections are the same rule, and it is now written into the generator beside the fix:

> **A cost leg must be read from a FORM, and a form only exists in code.**

With one deliberate exception, kept visible in the same comment: the price *classifier* still reads prose on purpose, because per-nonce in a doc string is evidence about the **shape** of a price rather than a **leg** of it.

The module nobody was looking at

On the last day, with the sweep reported complete and every instrument green, a routine cross-check compared _executorenforced.py --swept against the same tool run over the whole tree.

```
--swept:      756 proven, 0 UNPROVEN
whole tree:   758 proven, 8 UNPROVEN
```

The eight were all in one module, 20_MTX-SWP. The cause was one character. --swept derives its module list from the worklist's ticked rows — deliberately, because a hardcoded list had already gone stale twice in this programme — and the pattern it used required the row's index column to be a **number**. One ticked row carries an em-dash there instead, because that module had no turn of its own: its signatures were carried in by *other* modules' turns. So the derivation dropped it, --swept judged **45** modules while the worklist plainly said **46**, and reported a confident 0 UNPROVEN.

All eight were the defpact-step shape, and all eight turned out to be genuinely proven — every route bottoms out in a **debit of the executor's own tokens**, which is the strongest form this proof takes, since a caller naming an account it does not own cannot pay. So the finding is not that eight entrypoints were unguarded. The finding is that **for two days, a tool reported a verified zero over a set that silently excluded a module.**

Three things came out of it:

1. The eight routes are now registered and named in each function's own @doc.
2. The tick parser accepts any index token. *A tick is the definition of swept; what sits in the column beside it is decoration.*
3. **The check is now in the gate, over the whole tree.** The sweep is complete and the tree is clean at **766 proven, 0 UNPROVEN**, so there is no longer a reason to gate on a subset — and a gate over everything cannot be narrowed by a list going stale.

This is the fifth time in this programme that a tool's correctness depended on a list that changes as the work proceeds. The first four were fixed by **deriving** the list. This one *was* derived, and the derivation had the hole — which is the next lesson in the sequence rather than a repetition of the last one.

The authorisation surface, measured

ARCHITECTURE/AUTH-SURFACE.md records, for every sovereign client entrypoint, the set of accounts whose ownership is enforced **anywhere in its capability chain**. The gate requires that set to only ever **grow**. It is the instrument that turns "we did not weaken anything" from a claim into a check.

Reading its headline across the round is a lesson in its own right, because the headline goes the **wrong way**:

	at the start	at the end
entrypoints scanned	1,104	1,193
reaching at least one ownership enforce	813	856
reaching **none**	291	**337**

Forty-six *more* entrypoints reach no ownership enforce at the end of a round whose entire purpose was to reduce that number. Taken at face value the round made things worse. It did not, and the only way to know that is to diff the surface **entrypoint by entrypoint** rather than compare its totals:

	count
entrypoints present in **both** measurements	1,075
of those, **gained** an ownership gate	**60**
of those, **lost** one	**0**
of those, changed their enforced set at all	110
appeared during the window	118 — of which **116** are one admin pair added to 58 modules
disappeared during the window	29 — 21 archived, 6 re-prefixed to protected, 2 renamed

813 − 18 + 1 + 60 − 0 = 856, which is the whole of the movement accounted for. The 116 newcomers are P|A_SetIMP / P|A_RemoveIMP, the inter-module policy recovery hatch, added to every module by a separate piece of work inside the same three days. They are GOV|*_ADMIN-gated and P| policy functions, exempt from the canon by rule, and they enforce no *account* ownership **by construction**. They are the entire reason the “reaching none” column rose.

> **A total is a summary of two populations, and a summary cannot tell you that one of them changed.** The number that matters is 60 gained, 0 lost — and it is only visible per entrypoint.

The sixty are worth naming by family, because they are the round’s actual security product: thirteen DALOS admin operations and their Talos wrappers; five SWP pool-parameter admins; five PYTHIA and two CODEX operations; the four DEMIPAD launchpad admins from Defect 2; the ANK revoke from Defect 1; two launchpad price setters in citizen modules; and the FVT multiplet family issuer. Almost all of them gained the same pair — account:string, executor — which is what an admin operation looks like once it has to say **who**.

What an auditor has to re-verify

Because the client surface is positional, this round **moved every adversarial call site in the previous edition’s attack register**. Against the pre-sweep baseline, **694 entrypoint signatures changed across 58 files**.

The consequence is sharper than it sounds:

> An attack that still passes without being re-pointed is passing on an **arity error** rather than on the guard it names.

A short modref call partially applies into a closure, so the expect-failure around it fails as expected — green — for entirely the wrong reason. This happened **six times** during the round, twice inside expect-failures that had looked green for weeks. It is why Audit/AUDIT-V2-DELTA.md carries a per-module entry written *at the time the module was swept*, naming which assertions are invalidated, which findings are re-framed, and which new gates need an adversarial test that did not exist before. That judgement cannot be derived from a diff, and writing it afterwards from memory across 46 modules is how an audit becomes fiction.

---

26. Method: how the round was run

How the need was found

Not by a tool, and not by a threat model. By the owner reading one function.

ATS|A_KickStart takes an account called kickstarter. It is the account that funds and starts an autostake pool — the account that *acts*. Elsewhere the same role is called account, owner-konto, client, injector, beneficiary-id, recoverer, owner-account, curler, coiler, fueler, coiler-vester, curler-vester. Thirteen bespoke names for one idea, and because every name was locally sensible, nothing in the codebase looked wrong at any single site.

That is the shape of the whole finding. **A naming inconsistency is invisible from inside any one module**, and it stops being cosmetic the moment you ask the system a question that spans modules: *who performed this operation?* The system could not answer, because the answer was in a different place, under a different name, in every module — and in some it was nowhere at all.

So the canon was written down first, as owner rulings, and only then applied. The reference implementation was committed before the sweep began, to be copied rather than re-derived 776 times.

The classification that could not be guessed

The first useful thing the worklist tool did was refuse to treat the 719 outstanding entrypoints as one job. They split three ways, and the split is not visible from a signature's shape:

class	count	what the second parameter is
RENAME	115	an **account** under a bespoke name — account (53), owner-konto (13), client (10), injector (8), …
ADD	322	an **entity id** — id, ats, swpair, pool-id, fvt-id. There is no executor at all; it is derived inside a capability
PATRON	282	the *first* parameter is not patron

> **Assuming "second parameter = executor" is wrong for 322 of them.**

The 322 are the interesting population, and they have a name: **"authority proven, actor unrecorded."** The capability reads the entity, derives its owner, and enforces ownership of that derived account. The authority is real. The *actor* is never a parameter, so it is never in the event, never in the signature, and unavailable to anything asking who did this. Fixing one of these is not a rename — it is adding a binder and proving the parameter equals the derived account.

Deploy order, and why the cascade is the point

The worklist ran in **deploy order** — first module of Stage 1 to the last of Stage 2 — not smallest-first. A module's call sites live in the modules deployed *after* it, so sweeping forward means every module is already fixed before anything that calls it is touched. Smallest-first would revisit the same call sites repeatedly.

The effect compounds, and the last four modules measured it: of the 180 entrypoints across the four Stage-2 Talos modules, **176 were already DONE when their turns arrived**. One module's row reads *"64 of 65 arrived by cascade, 1 by its own turn."* The ordering did four fifths of the work for free.

It also produces a rule that only becomes obvious in the middle: **the turn that clears a provisional patron slot is the turn that gives its CALLER a patron.** A helper cannot thread an account that does not yet exist

upstream of it. Two such helpers sat provisional for nine modules and cleared in a single edit when their caller's turn finally came.

The instruments

Nine tools carried the round; five are gate-fatal.

instrument	what it answers	gate
_executorplan.py	*What is left?* Classifies every entrypoint DONE / RENAME / ADD / PATRON. **Ground truth** — never a remembered number	—
_executorenforced.py	*Is every executor actually PROVEN?* DIRECT / FORWARDED / INDIRECT / SELF-PROVING, everything else a finding	**fatal**
_authsurface.py	*Did any entrypoint stop enforcing something it used to?* Per-entrypoint ownership sets, may only grow	**fatal**
_patronslots.py	*Is slot 0 the right KIND of value?* Registry of every non-patron patron	**fatal**
_callarity.py	*Does every call site pass the right NUMBER of arguments?*	**fatal**
_docstrings.py	*Will the module load at all?* Malformed \ continuations	**fatal**
_modulecomplete.py	*Is this module's turn finished?* Seven obligations, the seventh being _executorenforced	—
_deadbind.py	*Where did somebody bind the actor and have nowhere to put it?*	—
_auditdelta.py	*Has every changed module written down what the audit must re-verify?*	**fatal**

Two of these are worth a sentence each on **why they are fatal rather than reports**.

_callarity.py, because Pact checks modref call arity at **runtime**: a short call compiles, deploys, and partially applies into a closure — and an expect-failure around it goes green for the wrong reason. A caller that no test exercises is precisely the one that will not be found by running anything.

_patronslots.py, because the thing it catches is **unreachable by assertion**. During the sweep a caller in a not-yet-swept module threads whatever account initiates the call into slot 0. The arity is right, the value is unused by every swept callee, and no test can observe it. The gate is fatal only on an **unregistered** one — the registry is the artefact, and the gate's job is to refuse a site nobody wrote down.

Eighteen lessons, and the seven that transfer

The handoff carries eighteen lettered lessons, one per module that taught something. Most are local. These seven are not.

1. When a number decides the scope of the work, compute it twice. The plan was wrong by 63% on its first day because one regex was anchored wrongly, and Talos — the only client path — was invisible to it. A second tool, written to re-derive the same list by another route, is what found it.

2. Derive the list — then check the derivation. Four tools in this programme carried a hand-maintained list that could not report its own incompleteness. Each was fixed by deriving the list from an artefact that is already the single source of truth. Then a *derived* list silently dropped a module because its index column held an em-dash instead of a number. Deriving is the right answer and it is not the end of the answer.

3. The implementation is the LAST match, never the first. Interfaces are embedded inline at the top of every file, so a regex for `(defun NAME` finds the **stub**. In one module the discriminator between finding the stub and finding the body was a **trailing space** — one function's stub had one and its sibling's did not, so the same pattern matched the body for one and the stub for the other. The rule is mechanical: take `hits[-1]`.

4. A grep window is not a read. A sixty-line `sed` window showed only shape checks, and the conclusion written from it — *"this entrypoint checks no account at all"* — was false; the ownership enforce sat fourteen lines past the bottom of the window. A redundant guard and a wrong `@doc` were added on the strength of it. What corrected it was the **existing test suite**, which already pinned the refusal the new claim said was missing.

5. Run the guard test without the guard. Twice in this round a fix was credited with closing a hole that was already closed somewhere else. The only thing that settles it is to **disable the new guard and run the suite**: if the test still passes, the guard is not what it is passing on. One such assertion survived — relabelled honestly as a property test rather than a discriminating one, because a test that cannot fail is a green light wired to nothing.

6. A rename is only safe inside the extent you have actually reasoned about. A file-wide substitution of `beneficiary-id → executee` changed 80 occurrences; the correct number was **15**. The word means one thing in a repair and another in a stake, and no regex can tell them apart. Reverted by writing back the committed blob, then redone scoped.

7. A proof the matcher cannot see is still a proof. The correct response to a false UNPROVEN is to register the route and write it in the function's `@doc` — **never** to add a second enforce to satisfy a tool. A guard added to make a report go green is a guard nobody chose, in a place nobody reasoned about, and it will be read later as evidence that somebody did.

Three ways the round broke itself

Recorded because they were all cheap to cause and expensive to find.

An arity-preserving reorder is not idempotent. Swapping two arguments of the same type is invisible to every checker in the gate — the arity matches, the types match, the module loads, the tests pass or fail for unrelated reasons. Running such a pass twice restores the original order and looks like a no-op. The passes were therefore written to read arity **from the source** rather than declare it, run exactly once, and then be parked with a `.DONE` suffix so they cannot be run again by reflex.

Removing the last binding deletes the `let`. Four of eight dead-binding removals left `(let ( ) ...)`, which Pact rejects at load. Fifty-five suites reported BROKEN with **zero** assertions — and a module-completeness check cheerfully reported 7/7 on a file that did not load, which is why the cheapest check in the gate now runs first.

An unescaped `"` inside an `@doc` closes the string, and the rest of the function parses as code. Three occurrences, all caught statically before any suite ran.

What the round is worth

At the end: 776 of 776 entrypoints conforming, every executor proven, 60 entrypoints enforcing an ownership gate they did not enforce before and none enforcing less, the authorisation surface written down per entrypoint and gate-protected against shrinking, and a register of every place where a value is knowingly provisional.

The three defects are the visible return. The durable one is smaller and harder to point at: the system can now be **asked** who performed an operation, and the answer comes from the operation's own arguments rather than from a capability three modules away. Every instrument above exists to keep that true after the people who did this have forgotten the details.

---

27. Appendix A — Reproducing everything

Every result in this book was produced by running something, and every one can be re-run. This appendix is the instruction set — set-up, the commands, the expected output, and a figure-by-figure index of which command produces which number in this book.

What you need

Pact	**5.4.1**. The suite uses Pact 5 semantics throughout; Pact 4 will not load it.
Python	3.9 or newer for the tooling (developed against 3.14). No third-party packages are required for the gate.
Cores	any. 16 were used for every timing in this book. See Chapter 3 for how wall time scales — below 16 it is linear, above ~21 it stops improving.
RAM	a `pact` run peaks well under 1 GB; the reference machine had 62 GB and never approached it.
pandoc	only to rebuild this book's `.docx` (3.7 used). Not needed for anything else.

The `pact` binary is the one thing that reliably goes wrong. It is commonly installed to `~/.local/bin`, which is not on a non-interactive `PATH`. The gate resolves this itself and accepts an override, but a bare `pact Z.repl` from a script will fail in a way that looks like a suite failure and is not:

```
export PACT=~/.local/bin/pact      # or put ~/.local/bin on PATH
python3 REPL/tools/_gate.py
```

Relative `(load ...)` paths inside the suite are written from `REPL/`, so anything invoking `pact` directly must run from there. The gate `chdir`s itself and does not care.

The one command that matters

```
python3 REPL/tools/_gate.py
```

This is the gate. It resolves its own paths, so it runs from any directory. It executes the full suite — every deploy stage, every scenario, every red-team attack — and then runs the static checks that guard the generated artefacts, the tool paths, the function-prefix vocabulary, cross-module member resolution, and the assertion-vacuity scan.

Expect **five to seven minutes** on 16 cores. The last three lines are what matters:

```
wall 318.5s   executed 25035 assertions (20042 positive, 4993 negative)

GATE GREEN
```

Anything other than `GATE GREEN` on the final line is a failure, including `GATE FAILED` with zero failing assertions — that is a **static check** refusing, and the reason is printed above it.

A green gate is the only claim this book makes about the code as a whole. Everything else is a claim about a specific operation, and each of those names the assertion that proves it.

Confirming deploy-readiness specifically

Loading the whole deploy chain, in order, is a narrower and faster question than the gate:

```
cd REPL && pact Z.repl ; echo "exit=$?"
```

Deploy-ready means **exit 0 with an empty stderr**. Do not grep the output for the word "error" and conclude from it: this suite contains negative tests whose *names* contain the word, and a grep for it returns 14 hits on a perfectly clean run.

Running one piece

The full gate takes minutes. For iterating on a single module there are standalone testers that boot a fresh environment and run only that module's suites:

```
cd REPL && pact modules/<MODULE>.repl       # DPTF, DPOF, SWP, ATS, VST, AQP, DPDC, ...
cd REPL && pact "RedTeam/[RT-A]_Economics.repl"   # one red-team family, self-loading
```

Each red-team file loads its own prerequisites, so any attack family can be run alone.

A warning that is not boilerplate

Do not run a tool to find out what it does. Several tools in this repository rewrite contract sources. They now refuse without an explicit `--apply`, but that protection exists *because* a loop that imported five of them — merely to prove they loaded — silently deleted 111 lines of schemas from a live contract. Read the docstring, or `REPL/TOOLS.md`.

Note also that `REPL/TOOLS.md` was itself found to be wrong about 13 of 49 entries during this round (Chapter 22). It is generated now. Regenerate before trusting it.

Reproducing the two money defects

Both were established by controlled experiment, and both experiments are reproducible from outside the repository — no repo file needs to be created or modified. Write a probe to `/tmp`, load the pipeline by absolute path, and run it from the `REPL/` directory:

```
cd REPL && pact /tmp/your_probe.repl
```

The OURO mispricing. Hold the primordial pool's reserves constant, move its weights through the live client path, and read both the flagged function and its weight-aware sibling after each change. The flagged one returns a bit-identical value across all three weightings; the sibling tracks the weight ratio exactly. The permanent form of this experiment is assertion `SWP-G27`, which recomputes the *removed* formula inline and asserts it disagrees — so the mutation test runs on every gate.

The fee-before-gate defect. Drive the permissioned issuance defpact with each step in its own committed transaction and the pact id pinned, reading the patron's IGNIS and STOA balances between steps. Step 1 commits; step 2 refuses. Then run the same drive with the flag off and observe a *different* refusal message — that differential is what makes the refusal attributable to the admin gate rather than to anything downstream. Pinned as `RT-F-002`.

Reproducing a specific figure from this book

Every quantitative claim in the book comes from one of these. Run the command, compare the number.

figure	command
assertions executed, positive/negative split	`python3 REPL/tools/_gate.py`
distinct assertions written, suite size, lines by area	`cd REPL && python3 _suite_stats.py`
client-facing cost previews, and how many are measured	`python3 REPL/tools/_info_measured.py`
`enforce` sites pinned by message, live vs dead modules	`python3 REPL/tools/_enforce_coverage.py`
owner-gate observed / attributed / never-observed	`python3 REPL/tools/_ownerobs.py`
the attack register, family counts, per-attack status	`python3 REPL/tools/_redteam.py`

figure	command
priced operations, per-function price rows	python3 REPL/tools/_ignis_price_sheet.py
module line counts against the size bands	python3 REPL/tools/_scale_report.py
assertions that cannot fail	python3 REPL/tools/_vacuous.py
negative assertions accepting any error	python3 REPL/tools/_expectfail.py
cross-module calls to members defined nowhere	python3 REPL/tools/_modref.py

If a figure in this book does not reproduce, **the book is wrong and the tree is right.** Three times during this programme a published figure went stale while every document agreed with every other document, because they had all been copied from one original — so cross-checking the prose detects nothing and only re-derivation does.

Rebuilding this book

The book is a generated artefact, and its staleness is a gate failure like any other.

```
python3 REPL/tools/_auditbook.py          # rebuild the .md from its chapter sources
python3 REPL/tools/_auditbook.py --docx   # ...and the .docx, via pandoc
python3 REPL/tools/_auditbook.py --check  # exit 1 if the committed .md has drifted
```

Chapter sources live under `Audit/book/`; the order, the numbering and the version string live in `REPL/tools/_auditbook.py`. Chapter 21 has no source file at all — it is emitted from the `;;<<RT-*>>` headers in the attack files, so editing an attack edits the book.

Reading the coverage numbers

```
python3 REPL/tools/_ownerobs.py           # the current owner-gate position
python3 REPL/tools/_ownerobs.py --census     # its own denominator and what it excludes
python3 REPL/tools/_ownerobs.py --weak       # gates credited only by proximity
python3 REPL/tools/_ownerobs.py --selftest   # does the detector actually detect?
```

Read `--census` before reading the headline. The headline is conditioned on a capability being reachable from a named client operation, and a reader not told how many fail that condition cannot distinguish *"90 of 167 witnessed"* from *"90 of everything"*. That distinction is not academic — Chapter 19 records the days this instrument spent reporting a ratio whose denominator excluded a third of the tree, including the layer that stops token theft.

Two properties of the output are stated in the tool itself and bear repeating:

- **`observed` is an upper bound.** Reachability is not proof: a single refused call credits every gate on its path. The `depth 0` figure is the honest one — it counts gates the calling test actually targeted.
- **`never observed` is therefore a lower bound**, which is why it is the column that drives work. **Use the observed column to decide nothing.** A test written specifically to close a gap once moved no counts at all, because the gap had already been credited transitively.

Checking the audit's own bookkeeping

```
python3 REPL/tools/_redteam.py --check   # the attack register and the defect ledger must agree
python3 REPL/tools/_pricesync.py --check # the generated price artefacts match their generator
python3 REPL/tools/_toolpaths.py --check # every tool path resolves; reports uncovered tool dirs
python3 REPL/tools/_modref.py --check    # no cross-module call to a member that does not exist
```

The first of these exists because a defect recorded in only one of the two records is one the audit **undercounts**, and that is a failure mode with no symptom. All four are fatal inside the gate.

28. Appendix B — Verification state

This appendix states, in one place, what is verified about this codebase as of the version on the title page — and, for each claim, the command that re-derives it. Nothing here is asserted without a way to check it, because a figure in an audit document that cannot be re-derived is a figure that will be wrong eventually and silently.

Three separate times in this programme a published number was found to have gone stale while every document agreed with every other document. That is the failure this appendix is built against: the figures below are re-derivable from the tree, not copied between files.

The gate

python3 REPL/tools/_gate.py is the gate. It runs 92 entrypoints across 321 reachable files on 16 workers, takes roughly five to seven minutes, and is the only thing entitled to the word “green”.

```
wall 318.5s   executed 25,035 assertions (20,034 positive, 4,991 negative)
GATE GREEN
```

The distinction between the two counts matters. A **positive** assertion (expect) says an operation produced a specific value. A **negative** assertion (expect-failure) says an operation was refused — and, where the refusal is pinned by *message*, says which guard refused it. Roughly one assertion in five in this suite is a negative one, which is what a security-oriented suite should look like; a suite that only proves things work cannot detect a deleted gate.

The gate also runs a set of static checks that are **fatal**, not advisory. A tree that passes every assertion but fails one of these does not go green:

check	what it enforces
_pricesync --check	the generated price sheet matches the price tables in source
_figuresync --check	the figures in ARCHITECTURE/*.md match the tree
_toolpaths --check	every hard-coded path literal in every tool still resolves
_redteam	every attack header parses; the register matches the ledger
_booktables --check	this book’s headline tables sum to their own totals
_auditbook --check	this book is not stale against its chapter sources
orphan check	every asserting file is reachable from a gate entrypoint

The orphan check is the least obvious and the most valuable: a test file that nothing loads is a test file that passes forever. It found files in this tree.

Deployment readiness

Confirmed by execution, not inspection:

check	command	result
full deploy chain loads	cd REPL && pact Z.repl	exit 0, **stderr 0 bytes**
no interface left at its live version	_ifacebump --check	0
no interface present at two versions in the tree	as above	0

check	command	result
every qualified module{X} / object{X.Y} names a real interface	as above	0 missing
every implements names a real interface	as above	0 missing[1]
full gate after the bump	_gate.py	GREEN, 25,035 — unchanged from before it

[1] One implements names stoa-ns.gas-payer-v1, which is external to this repo and legitimately not in the tree.

The last row is the one that carries the weight. An interface bump touching **3,333 references across 116 files** that leaves the assertion count *exactly* where it was is a bump that changed names and nothing else. Had it changed behaviour, the count would have moved.

Seven interfaces were bumped from V2 to V3: AcquisitionAnchorsV2, AcquisitionPoolsV2, AcquisitionScoresV2, AcquisitionVacateV2, AqpMtxV2, DsaV2, IgnisCollectorV2. The number is seven and not eleven because the first attempt to derive it parsed the highest version mentioned *anywhere* in the version record, including prose discussing hypothetical future bumps. Parsing only the live column of the table cut it from eleven to seven. That error is recorded here because it is the kind that produces a bump which loads fine and is wrong.

Coverage worklists, all at zero

Each of these began the programme at a non-zero number and was driven to zero. Each is measured by a tool that can be re-run; none is a judgement.

worklist	what an entry means	now
owner gates, testable and never observed	a client path acquires this ownership gate, and no test targets it	**0**
live unpinned enforce sites	a refusal nothing pins by message, in a live module	**0** of 790
cost previews not measured	a client-facing preview whose quote no test compares to the charge	**0** of 414
client entrypoints untested	a C_/A_ surface named in no live .repl	**0** of 452
modref class-B	a call to an interface member that is defined nowhere	**0** live
vacuous assertions	an assertion that passes regardless of the code under test	**0**
module coverage	a module with no suite of its own	**88 / 88**

Two of these rows carry deliberate, stated exclusions, and the exclusions are the honest part:

- **Unpinned enforces**: 24 unpinned sites remain in 00_DPMF.pact, which is superseded by DPOF and deployed only for provenance. They are excluded as dead, and the tool names the module rather than silently shrinking its own denominator — which an earlier version of it did.
- **Owner gates**: the observed figure is **85**, of which **66** are attributed at depth 0 — that is, the refusal was proven to come from the called operation's *own* gate rather than from a gate it composes. Eighty-two gates were never observed at all. Chapter 19 is about why that gap is reported as a bound rather than

closed, and why an earlier version of this same measurement was wrong four times, twice in each direction.

Scale

Measured from the tree, not from memory:

Pact modules (1_SOVEREIGN + 2_CITIZEN)	93
test files (REPL/**/*.repl)	275
gate entrypoints	92
files reachable from the gate	321
assertions executed per full run	25,035
client-facing cost previews	414
priced operations in the generated price sheet	442
red-team attacks	38

What is not verified

An audit book that lists only what it proved is a sales document. The following are known, deliberate limits of the work in this volume:

- **The attacks are a bounded sample.** Thirty-eight attacks across eleven families is what was run. No claim is made that the space is covered; Chapter 21 shows its shape so the reader can judge the shape of what is missing.
- **Owner-gate attribution is a bound, not a proof.** Eighty-five gates were *observed* to refuse someone. Sixty-six of those refusals were attributed to the gate itself. The remaining eighty-two gates have never been seen to refuse anybody, and the honest statement is that nothing is known about them from the outside — not that they are broken.
- **One instrument's alarm path is not self-demonstrated.** _eagerlet.py --check gates the count of un-annotated eager-let shadows. Three attempts to demonstrate its alarm with a synthetic shadow failed to trigger it. That limitation is written into the tool's own docstring rather than smoothed over, and it means the tool's *green* is weaker evidence than the others here.
- **Nothing here is a statement about the deployed chain.** Every figure is about the tree. The interface versions recorded as "live" are a snapshot of on-chain state taken before the bump, and are explicitly not refreshed from the tree, because the tree is now ahead of the chain.
- **Economic design is out of scope.** This programme tested whether the system does what it says. Whether what it says is a good idea — whether a deterrent is set at the right level, whether a share price should round the way it does — is a question for a different document.